

Estructuras de Datos y Algoritmos

Include CD-ROM



Prentice
Hall

Roberto Hernández • Juan Carlos Lázaro
Raquel Dormido • Salvador Ros

ESTRUCTURAS DE DATOS Y ALGORITMOS

**ROBERTO HERNÁNDEZ
JUAN CARLOS LÁZARO
RAQUEL DORMIDO
SALVADOR ROS**

**Departamento de Informática y Automática
UNIVERSIDAD NACIONAL DE EDUCACIÓN A DISTANCIA**

PRENTICE HALL

**Madrid • México • Santafé de Bogotá • Buenos Aires • Caracas • Lima • Montevideo • San Juan
San José • Santiago • São Paulo • White Plains**

datos de catalogación bibliográfica

ESTRUCTURAS DE DATOS Y ALGORITMOS
Hernández, R.; Lázaro, J. C.; Dormido, R.; Ros, S.

PEARSON EDUCACIÓN, S. A., Madrid, 2001

ISBN: 84-205-2980-X
Materia: Informática. 681.3

Formato 170 × 240

Páginas: 296

ESTRUCTURAS DE DATOS Y ALGORITMOS
Hernández, R.; Lázaro, J. C.; Dormido, R.; Ros, S.

No está permitida la reproducción total o parcial de esta obra
ni su tratamiento o transmisión por cualquier medio o método,
sin autorización escrita de la Editorial.

DERECHOS RESERVADOS

© 2001 respecto a la edición en español por:
PEARSON EDUCACIÓN, S. A.
Núñez de Balboa, 120
28006 MADRID

ISBN: 84-205-2980-X
Depósito legal: M. 33.933-2000

PRENTICE HALL es un sello editorial autorizado de PEARSON EDUCACIÓN, S. A.

Editor: Andrés Otero
Asistente editorial: Ana Isabel García
Impreso por:

IMPRESO EN ESPAÑA - PRINTED IN SPAIN

Este libro ha sido impreso con papel y tintas ecológicos

Índice general

Prólogo ix

1	<i>Introducción y conceptos fundamentales</i>	1
1.1	Estructuras de datos	1
1.2	Estructuras de datos y tipos de datos abstractos	3
1.2.1	Concepto	3
1.2.2	Tipos de datos básicos	7
1.2.3	Tipos de datos abstractos: Puntero y Listas enlazadas	12
1.2.4	Listas enlazadas	17
	Inserción por la cabeza	21
	Inserción por el final	23
	Suprimir por la cabeza	25
	Suprimir por el final	27
	Inserción según un criterio de orden	28
	Imprimir una lista enlazada	34
1.2.5	TDA Lista implementado con listas enlazadas	35
1.3	Algoritmos	40
1.4	Conclusiones	43
2	<i>Clasificación en memoria principal</i>	45
2.1	Introducción	45
2.2	Métodos directos de clasificación	48
2.2.1	Clasificación por inserción directa	48
	Análisis de la clasificación por inserción directa	50

2.2.2	Clasificación por inserción binaria	53
	Análisis de la clasificación por inserción binaria	55
2.2.3	Clasificación por selección directa	57
	Análisis de la clasificación por selección directa	59
2.2.4	Clasificación por intercambio directo	62
	Análisis de la clasificación por intercambio directo	64
2.2.5	Clasificación por sacudida o vibración	66
	Análisis de la clasificación por vibración	68
2.2.6	Comparación de los métodos directos de clasificación	69
2.3	Métodos avanzados de clasificación	70
2.3.1	Inserción por incremento decreciente	70
	Análisis de la clasificación Shell	72
2.3.2	Clasificación por montón	75
	Análisis de la clasificación por montón	81
2.3.3	Clasificación por partición	82
	Análisis de la clasificación por partición	86
	Cálculo del k -ésimo mayor elemento	89
2.3.4	Comparación de los métodos avanzados de clasificación	90
2.4	Conclusiones	91

3***Clasificación en memoria secundaria*****93**

3.1	Introducción	93
3.2	clasificación externa basada en mezcla	95
3.2.1	Mezcla directa	95
	Análisis de la mezcla directa	101
3.2.2	Mezcla natural	101
	Análisis de la mezcla natural	109
3.2.3	Mezcla balanceada múltiple	109
	Consideraciones sobre la implementación	114
	Análisis de la mezcla balanceada múltiple	116
3.2.4	Clasificación polifásica	116
	Consideraciones sobre la implementación	122
3.3	Archivos indexados	129
3.4	Tablas de dispersión (Hashing)	131
3.4.1	Funciones Hash	134
3.4.2	Manejo de desbordamiento o sobrecarga	136
	Análisis de las tablas hash	139
3.5	Conclusiones	141

4 *Tipos de datos abstractos dinámicos lineales* 143

4.1	Introducción	143
4.2	Ejemplos de TDA dinámicos lineales	144
4.3	Pilas	155
4.3.1	Implementación de pilas mediante arreglos	157
4.3.2	Implementación de pilas mediante listas enlazadas	160
4.4	Colas	162
4.4.1	Implementación de colas mediante arreglos	164
4.4.2	Implementación de colas mediante listas enlazadas	174
4.5	Ejemplos de aplicación	179
4.5.1	Evaluación de la parentización de expresiones	179
4.5.2	Notación de expresiones prefija y postfija	182
4.5.3	Búsqueda de infinitivos en un texto	187
4.6	Consideraciones generales	189
4.7	Conclusiones	191

5 *Tipos de datos abstractos dinámicos no lineales: árboles* 193

5.1	Introducción	193
5.2	Conceptos y definiciones	194
5.2.1	Árboles perfectamente balanceados	200
5.3	Árboles binarios ordenados según el recorrido	204
5.3.1	Árboles de expresión	205
5.4	Árboles de búsqueda binarios	207
5.4.1	Concepto y definiciones	207
5.4.2	Procedimientos sobre árboles binarios de búsqueda	209
5.4.3	Análisis de la búsqueda y de la inserción en árboles binarios de búsqueda	214
5.5	Árboles de búsqueda balanceados	218
5.5.1	Concepto	218
5.5.2	Inserción en árboles AVL	219
	Inserción de un nuevo nodo por la izquierda de un subárbol	220
	Inserción de un nuevo nodo por la derecha de un subárbol	227
	Algoritmo de inserción	227
	Ejemplos	229
5.5.3	Eliminación en árboles AVL	234
5.5.4	Análisis de árboles AVL	239
5.6	Árboles con varias condiciones mutuamente excluyentes	241
5.7	Conclusiones	242

6	<i>Árboles avanzados</i>	245
6.1	Introducción	245
6.2	Árboles B	246
6.2.1	Búsqueda	247
6.2.2	Inserción	248
	Ejemplo de inserción en un árbol B	249
6.2.3	Eliminación	252
6.3	Árboles B binarios	256
6.4	Árboles B binarios simétricos (BBS)	259
6.4.1	Ejemplos de crecimiento de árboles BBS	263
6.5	Conclusiones	265

7	<i>Programación orientada a objetos</i>	267
7.1	Introducción	267
7.2	Clases y objetos	268
7.3	Herencia y paso de mensajes	269
7.4	Enlace dinámico y Polimorfismo	271
7.5	Conclusiones	273

Bibliografía 275

Índice analítico 277

Prólogo

El objetivo básico de este texto es establecer los principios esenciales de los fundamentos de la disciplina conocida como Estructuras de Datos y Algoritmos. Dentro de las recomendaciones dadas por las asociaciones internacionales de reconocido prestigio, tales como IEEE y ACM, sobre curriculum en Informática, constituye por sí misma una de las nueve áreas de interés: Arquitectura, Inteligencia artificial y robótica, Bases de datos y recuperación de información, Comunicación hombre-máquina, Cálculo numérico y simbólico, Sistemas operativos, lenguajes de programación, metodología e ingeniería del software y Algoritmos y Estructuras de Datos.

El término Informática se empleará aquí para hacer referencia a la ciencia dedicada al estudio del tratamiento sistemático de la información, entendida ésta como una secuencia de caracteres de un alfabeto dado. En latinoamérica está más extendido el término computación y en este texto deben entenderse como sinónimos.

En general se engloba bajo el término Algoritmos y Estructuras de Datos el estudio de los algoritmos y su utilización en la resolución de problemas, incluido el análisis de problemas, el diseño de algoritmos y su implementación, evaluación y verificación. El término Estructuras de Datos hace referencia a métodos de organizar, almacenar, ordenar y buscar cantidades significativas de datos. Evidentemente la amplitud de los contenidos que se incluyen en este área, así como su profundidad, es enorme.

Por otro lado los conceptos, las técnicas y las bases fundamentales que constituyen esta disciplina están claramente establecidos desde principios de la década de los 70, y evidentemente existe un gran número de excelentes textos que los presentan. Sin embargo, este texto se ha diseñado con el fin de ser utilizado como texto base de la asignatura "Estructuras de Datos y Algoritmos" que se imparte en las titulaciones de Ingeniería Técnica en Informática de Sistemas e Ingeniería Técnica en Informática de Gestión, de la Escuela Universitaria de Informática de la Universidad Nacional de Educación a Distancia. Por ello la vocación de este texto es primordial-

mente pedagógica, desarrollándose con el fin de facilitar el estudio y el autoaprendizaje. De este modo todos los conceptos, algoritmos, tipos de datos abstractos, etc., se presentan ilustrados con ejemplos. Dichos ejemplos son descritos paso a paso y con detalle, con el fin de minimizar el esfuerzo de comprensión.

En el mismo sentido los contenidos se han estructurado de manera que sean evaluables, separando lo fundamental de lo accesorio, lo formativo de lo informativo. Así por ejemplo, los algoritmos de clasificación se presentan de manera que los detalles de implementación, que requieren un esfuerzo especial ya que los procedimientos pueden ser relativamente largos, no interfieran con los conceptos básicos que definen el algoritmo. El hecho de presentar las implementaciones completamente desarrolladas responde a la necesidad, que todos tenemos, de disponer del "programa" final para comprobar su "funcionamiento". En cualquier caso lo fundamental, tal y como se insiste a lo largo de todo el texto, son las ideas básicas y los algoritmos, no los programas. Por ello, y con el fin de que el alumno no "pierda el tiempo" en escribir los programas, se incluyen en CDROM las fuentes tanto en Pascal como en Modula. Esto no quiere decir que sólo los conocedores de estos lenguajes de alto nivel pueden comprender este texto. De hecho todos los contenidos del mismo se presentan insistiendo claramente en la diferencia entre algoritmo y programa, Tipo de Datos Abstracto y su implementación. Por tanto, los conceptos básicos son generales e independientes de las implementaciones utilizadas así como del lenguaje utilizado para realizarlas.

Los Autores.

Madrid, Junio de 2000.

CAPÍTULO 1

Introducción y conceptos fundamentales

1.1 ESTRUCTURAS DE DATOS

En general puede entenderse que todo sistema informático puede realizar dos tareas básicas: cálculos o gestión de información. Deliberadamente se ha prescindido de la expresión tratamiento de información por ser un aspecto más general que incluye al cálculo como caso particular. En una tarea de cálculo, función que explica el término de origen anglosajón *computador* (computer), el objetivo fundamental será la resolución de problemas numéricos tal que a partir de un conjunto de datos dados se obtengan los resultados requeridos; mientras que en la tarea de gestión de información, descrita por el vocablo que procede del francés *ordenador* (ordonateur) se almacenan datos con el fin de organizarlos y recuperarlos cuando sean precisos.

Dependiendo del ámbito específico de aplicación, se utilizará predominantemente una u otra capacidad, o ambas. Así por ejemplo, el cálculo predomina en la actividad científica y técnica de manera que el computador se ha convertido en una herramienta habitual en cualquier centro de investigación y desarrollo. Por otro lado, el tratamiento de información prevalece en las aplicaciones de gestión mediante las cuales los sistemas informáticos han irrumpido en la sociedad desde hace algunos años, aplicándose a labores que van desde la elaboración del censo hasta el inventario de un pequeño comercio. Más recientemente, el computador

forma parte de la vida cotidiana para un sinnúmero de tareas personales, desde la consulta de enciclopedias multimedia a la comunicación telemática a través de Internet.

En cualquier caso, independientemente de la tarea específica a la que vaya a ser dedicado un sistema informático, siempre se va a tener la necesidad de almacenar y manipular datos. Para ello se realizarán programas, es decir, una secuencia de instrucciones que indica las acciones que han de ser ejecutadas por el sistema programable.

La metodología de diseño de un programa ha ido evolucionando a lo largo de los años. En un principio era de hecho un arte, en el que la habilidad del programador para utilizar las posibilidades que le ofrecían los primeros lenguajes de programación era una componente fundamental. Además, el código de un programa medianamente complejo solía ser laborioso y difícil de seguir debido a que la 'herramienta' comúnmente utilizada era la conocida sentencia 'go to', que obligaba a una planificación de acciones ciertamente intrincada. Desde entonces el arte de programar ha ido conceptualizándose, desarrollando técnicas para una programación sistemática y bien organizada. Así surgió la programación estructurada en la que las tareas se dividen y agrupan en funciones o procedimientos de manera que por un lado se atiende a los detalles de programación de éstos y posteriormente se utilizan en el programa principal que los requiera.

La programación estructurada, también llamada programación orientada a procedimientos, es bien conocida en la actualidad por todo aquel que se haya iniciado en programación con cualquier lenguaje de alto nivel, y constituye un claro ejemplo de *abstracción*. Los detalles de una tarea específica se encuentran en una función que posteriormente será vista como algo que tiene unas entradas y ofrece unas salidas determinadas. De este modo, puede observarse que la abstracción permite una programación más sencilla y sistemática, siendo la operación subyacente a todos los conceptos que serán presentados a lo largo del texto.

En sentido estricto, abstraer es la operación mediante la cual la inteligencia llega a formar conocimiento conceptual común a un conjunto de entidades, separando de ellos los datos contingentes e individuales para atender a lo que los constituye esencialmente. Es decir: aislar mentalmente o considerar por separado las cualidades de un objeto. En general, cuando un estudiante de cualquier disciplina, escucha que debe abstraer unos conocimientos dados, suele pensar que es una tarea difícil, poco tangible y nada práctica. Esto suele ser debido a que se confunde con 'imaginar algo inexistente'. Abstraer debe entenderse como 'extraer y agrupar', es decir, descubrir las cualidades comunes y englobarlas bajo un mismo concepto. Esta capacidad de la mente se pone de manifiesto constantemente en la vida cotidiana; de hecho, el lenguaje es el exponente máximo de la abstracción. En la programación, la acción de abstraer se realiza constantemente y a todos los niveles. Incluso cuando se trata con ceros y unos se están ignorando las características propias de cada uno, tales como los valores reales de las tensiones que los representan internamente en la máquina y su nivel de ruido.

Atendiendo a estas consideraciones la metodología de programación ha ido avanzando en el proceso de abstracción. En la programación orientada a procedimientos este hecho es claramente reflejado por la expresión del profesor N. Wirth:

$$\text{Estructuras de datos} + \text{Algoritmos} = \text{Programas}$$

A lo largo de este capítulo serán analizados los conceptos fundamentales de las dos columnas básicas en las que se basa la programación: las *estructuras de datos* y los *algoritmos*. Ambos están íntimamente relacionados, hecho que se manifestará a lo largo de todo el texto.

Finalmente debe destacarse el hecho de que en el último nivel de abstracción aparece una nueva metodología conocida como programación orientada a objetos, la cual, parafraseando la expresión del profesor N. Wirth, podría expresarse como:

$$\text{Tipos de datos abstractos} \cup \text{Algoritmos} = \text{Objeto}$$

A esta metodología se dedica el último capítulo a modo de introducción.

1.2 ESTRUCTURAS DE DATOS Y TIPOS DE DATOS ABSTRACTOS

1.2.1 CONCEPTO

Aunque los conceptos englobados en los términos ‘estructuras de datos’ y ‘tipos de datos abstractos’ son sencillos, en ocasiones puede producirse cierta confusión debido a que en algunos textos puede encontrarse el término ‘estructuras de datos’ referido a lo que en otros son los ‘tipos de datos abstractos’, y en gran número de ellos se utilizan los términos ‘tipo de datos’ y ‘tipo de datos abstracto’ indistintamente. Con el fin de precisar los términos, en principio se distinguirá entre ‘tipo’, ‘tipo de datos abstracto’ y ‘estructura de datos’.

Definición 1.1: Un tipo de datos es una colección de valores.

Definición 1.2: Un Tipo de Datos Abstracto (TDA) es un tipo de datos definido de forma única mediante un tipo y un conjunto dado de operaciones definidas sobre el tipo.

Definición 1.3: Una estructura de datos es la implementación física de un tipo de datos abstracto.

A la vista de estas definiciones cabe preguntarse qué son los enteros, caracteres, etc., conocidos en programación básica y generalmente predefinidos en los lenguajes de alto nivel. Por ejemplo, el tipo Booleano está formado por los dos valores

True y False pero no es un tipo de datos abstracto ya que sobre él no se han definido las operaciones para manipularlo. Sin embargo, el tipo Booleano junto con los operadores booleanos (AND, OR, NOT) constituye un tipo de datos abstracto. Del mismo modo los enteros son un tipo formado por el conjunto de valores denominados por las matemáticas ‘enteros’. Este tipo junto con las operaciones definidas sobre ellos forman un tipo de datos abstractos.

Es importante considerar estas definiciones y tenerlas en mente a lo largo de todo el texto, no olvidando nunca que las operaciones que manipulan los objetos están incluidas en la especificación de un tipo de datos abstracto y que no interviene ninguna consideración sobre la implementación. Ésta es expresa y conscientemente ocultada, hecho que es conocido como encapsulación de datos (Figura 1.1 a). El hecho de que se oculte la información no significa que sea inaccesible. Se trata de disponer únicamente de lo necesario para realizar las operaciones, funciones de acceso y variables de entrada y salida. Esto se realiza una vez asegurado que la implementación es correcta y adecuada.

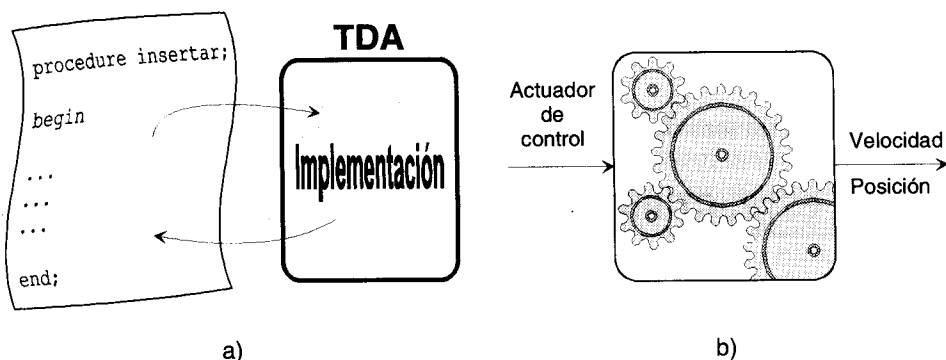


Figura 1.1 a) Encapsulación de datos b) Motor.

El encapsulado, consecuencia misma del proceso de abstracción, se utiliza en todos los ámbitos de la ciencia y de la ingeniería. Por ejemplo, se utilizan diagramas de bloques caracterizados por las entradas y salidas de los sistemas físicos. De este modo un *motor* puede representarse por la entrada proveniente del actuador de control, y la salida (velocidad y/o posición) tal y como se ilustra en la Figura 1.1.b, sin atender a los detalles físicos del mismo (escobillas o no, inducido, eléctrico o de combustión, etc.). Del mismo modo, esta idea se utiliza en otros ámbitos de la informática, como por ejemplo en la técnica de monitorización en el desarrollo de sistemas operativos.

Los conocedores del C estándar y de los primeros compiladores de este lenguaje recordarán que el tipo de datos abstracto Boolean no era un tipo predefinido, mientras que en Pascal por ejemplo si lo era. Pero de este hecho es un error deducir que el tipo de datos Boolean no existe en C estándar. Lo cierto es que no está imple-

mentado en los tipos básicos, hecho superable sencillamente definiendo un tipo con los valores `True` y `False` y asociándole los operadores `AND`, `OR` y `NOT`. Otro ejemplo característico es el de los enteros. Como es bien sabido, todo computador presenta limitaciones para la representación de los números enteros de manera que el rango de posibles valores que se adaptan a este tipo dependen del computador. En general la implementación de un entero utiliza una palabra para su almacenamiento. Así, en un IBM PC con palabras de 16 bits se utilizan dos bytes para almacenar cada entero, con lo que el rango de posibles valores va desde -32768 hasta +32767. Pero otras máquinas ofrecen rangos diferentes. Los clásicos DEC VAX y los procesadores de la familia i86 de Intel (a partir del 80386) utilizan 32 bits, por ejemplo. Es más, en lenguaje C es posible implementar los enteros mediante diversas formas, aumentando o disminuyendo el rango de valores. Así, si el entero mediante la declaración `int` está representado por 2 bytes, con la declaración `long int` se almacena en 4 bytes; si `int` almacena en 4 bytes, la declaración `short int` los almacenará en 2 bytes. Es decir, las implementaciones de algo tan sencillo como son los enteros pueden ser distintas. Pero lo que es importante resaltar es que el tipo de datos abstractos denominado ‘enteros’ es siempre el mismo. Si en un caso dado la implementación con el rango de 2 bytes es insuficiente será preciso cambiarla, pero esto no afecta al tipo de datos abstracto ni al uso que se haya hecho de él a lo largo del programa elaborado. Lo mismo sucede con otras estructuras más complejas que se presentarán en temas posteriores. Por ejemplo, supóngase que en un programa se utiliza una pila, y en su momento el diseñador decidió utilizar una implementación estática mediante un arreglo de 100 elementos. Pasado el tiempo se observa que es necesario un tamaño mayor. En ese momento basta con atender a los detalles de la implementación de la estructura, aumentando el número de elementos del arreglo o cambiando a una pila implementada dinámicamente. Estas modificaciones no requieren un gran esfuerzo ya que el tipo de datos abstracto es el mismo, y por tanto, la pila y su funcionalidad siguen siendo las mismas.

Los anteriores comentarios no implican que las implementaciones sean poco importantes o irrelevantes. De hecho son fundamentales y la elección de una u otra implementación para un TDA dado deberá analizarse cuidadosamente. Lo importante es entender la diferencia entre un TDA y su implementación, y considerar cada uno de ellos en su momento. De esta manera se facilita el proceso de diseño así como sus posibles modificaciones y mejoras.

Como puede observarse de las consideraciones anteriores, el término ‘estructura de datos’ hace referencia, según la Definición 1.3, a la implementación del TDA. En este sentido podría entenderse que los numerosos textos titulados *Estructuras de datos y algoritmos* hacen referencia a implementaciones de TDA y a algoritmos. Sin embargo, al manejar cualquiera de ellos, inmediatamente se precisan los términos, introduciendo los conceptos de TDA y estableciendo su significado. El término ‘estructura de datos’ se reserva para las representaciones físicas de los datos.

Con todo lo dicho, la selección de un TDA para resolver un problema dado deberá realizarse analizando en primer lugar el problema para determinar las restric-

ciones y especificaciones que debe satisfacer la solución, seguidamente se deberán determinar las operaciones básicas a realizar y por último se seleccionará la más adecuada. Evidentemente la más adecuada vendrá dada en función de alguna característica relevante, para lo cual habrá que establecer algún parámetro objetivo que proporcione una medida de su coste o de sus beneficios. A lo largo de todo el texto se presentarán numerosos TDA, y una vez comprendidos tanto su concepto como su funcionalidad, serán analizados con detalle.

Desgraciadamente no existe un único TDA capaz de ser el mejor en todos los casos, y todos ellos presentan inconvenientes y ventajas que los hacen adecuados o no para resolver un problema específico. Así por ejemplo, se presentarán los árboles de búsqueda. Una vez analizados se obtiene la conclusión de que en el peor de los casos degenerarán en una lista lineal, perdiendo sus ventajas y haciendo que la búsqueda sea muy costosa. Buscando una solución mejor se introducen los árboles AVL, se analizan y se concluye que en el peor de los casos la operación de búsqueda tiene un coste menor. Sin embargo, esto no significa que sean los mejores; en una aplicación dada es posible que nunca se dé el peor de los casos y por tanto, puede que los árboles de búsqueda sean una mejor solución para ese problema en concreto.

El coste del TDA no viene determinado por un único factor sino que deberá ser analizado desde diversos puntos de vista. En general deberán considerarse siempre dos aspectos. El primero, en relación con el almacenamiento que será establecido por los datos y la cantidad de ellos que deberán ser manejados. El segundo aspecto, igualmente importante, será el coste de las operaciones básicas. En el caso de los árboles, por ejemplo, serán los costes de las operaciones de inserción, búsqueda y eliminación. En general estas operaciones se realizan mediante algoritmos, con lo que su análisis se realizará utilizando las técnicas de análisis de algoritmos. Como puede observarse, estos dos aspectos consideran las dos restricciones fundamentales que deben tenerse en cuenta en el diseño de todo programa: las limitaciones en el espacio de almacenamiento y el tiempo de ejecución necesario para realizar una tarea.

En definitiva, debido al proceso de abstracción, el diseño deberá realizarse siguiendo tres pasos fundamentales:

1. Análisis de datos y operaciones.
2. Elección del Tipo de Datos Abstracto.
3. Elección de la implementación.

En los capítulos siguientes se presentarán diversos TDA básicos y bien conocidos. En todos ellos se presentará el concepto que incorporan, su utilidad y su coste. Sin embargo, no sólo son interesantes con el fin de conocerlos para utilizarlos si fuese necesario, sino que fundamentalmente son útiles como ejemplo de metodología de diseño de TDA. De este modo, si para un problema específico no es útil ninguno de los TDA básicos, se podrá definir y realizar un TDA siguiendo la misma metodología.

Para finalizar esta sección un breve comentario sobre terminología. En la mayoría de la literatura se muestra una diferencia entre los términos ‘tipo de datos’

y ‘tipo de datos abstracto’ [Shaffer, 1997], [Heileman, 1998]. Cuando esto sucede, el concepto de TDA sigue siendo el dado en la Definición 1.3 mientras que se hace referencia a ‘tipo de datos’ como una descripción matemática y formal de un tipo y un conjunto de operaciones. En otros textos [Dale, 1989] se utiliza el término ‘tipo de datos’ como TDA. En cualquier caso, desde el punto de vista de la metodología de programación el concepto fundamental que debe considerarse siempre y comprender sin ambigüedades es el de TDA.

Con el fin de tener una perspectiva de conjunto, en el siguiente apartado se realiza una breve presentación de los tipos de datos conocidos de programación básica, pero presentados como TDA.

1.2.2 TIPOS DE DATOS BÁSICOS

En un primer acercamiento a la programación, independientemente del lenguaje utilizado, es habitual entender que los tipos de datos son herramientas o utilidades para almacenar los datos necesarios para resolver el problema. Así, se suele entender que los enteros se almacenan en variables tipo Entero (int en C, Integer en Pascal o Modula2, ...) , los caracteres en variables tipo carácter (char en C, en Pascal o Modula2,...). Es decir, en este contexto no suele atenderse a la diferencia entre tipo y TDA. Sin embargo, todos ellos son de hecho TDA. De este modo las definiciones de los tipos de datos estándar son las siguientes:

Definición 1.4: El TDA Entero tiene como tipo el conjunto de números enteros definido por las matemáticas $\{-1, -2, \dots, \infty\} \cup \{0, 1, 2, \dots, \infty\}$, y como operaciones la suma, la resta, la multiplicación y la división entera.

Definición 1.5: El TDA Real tiene como tipo el conjunto de números reales definido por las matemáticas y como operaciones la suma, la resta, la multiplicación y la división.

Definición 1.6: El TDA Booleano tiene como tipo el conjunto de valores booleanos {True, False} y como operaciones las definidas por el álgebra de Boole (AND, OR, NOT).

Definición 1.7: El TDA Carácter tiene como tipo el conjunto de caracteres definido por un alfabeto dado y como operaciones todos los operadores relacionales (<, >, =, ≥, ≤, =, <>).

En el apartado anterior ya se realizaron algunos comentarios sobre la implementación de los TDA Entero y Booleano. Del mismo modo el TDA Real tiene en su implementación diferentes soluciones, en función de la representación que se realice de ellos, que depende de diversos factores. En coma flotante debe considerarse la representación de la mantisa (complemento a 2, magnitud-signo,...), la del exponente (polarizado, no polarizado,...), la localización de ambos, etc. Estas implementaciones están normalizadas. Por ejemplo, el Institute of Electrical and Electronics Engineers (IEEE) ha propuesto la norma IEEE 754-1985 para la representación de números en coma flotante para las operaciones aritméticas en mini y microcom-

putadores, en la que se contemplan tres formatos básicos, denominados de simple, doble y cuádruple precisión (véase [Dormido, 2000]).

Análogamente el TDA Carácter se implementará de diferentes formas dependiendo del alfabeto utilizado. En general, todos los alfabetos vendrán caracterizados por una secuencia de cotejo, es decir, por la secuencia ordenada de los caracteres que constituyen el conjunto. En todos los alfabetos, como es natural, las letras se encuentran ordenadas entre sí, al igual que los dígitos. Así 'a' < 'b' < 'c'..., y '1' < '2' < '3'. Los tres conjuntos de caracteres más conocidos son el CDC-Científico, el EBCDIC (*Extended Binary Coded Decimal Interchange Code*) y el ASCII (*American Standard Code for Information Interchange*). Las Tablas 1.1, 1.2 y 1.3 muestran las secuencias de cotejo de los tres.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	E.B.	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Tabla 1.1 Tabla de caracteres ASCII. El número correspondiente de la secuencia de cotejo, se forma con dos cifras hexadecimales, siendo la primera cifra, la que aparece en la primera columna y la segunda la que aparece en la primera fila. E.B. indica el carácter espacio en blanco.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX		PT			GE				FF	CR		
1	DLE	SBA	EUA	1C		NL				EM			DUP	SF	FM	ITB
2							ETB	ESC						ENQ		
3			SYN				EOT						RA	NAK		
4	E.B.												ø	.	<	(
5	&												!	\$	*	)
6	-	/												,	%	_
7													:	#	@	'
8		a	b	c	d	e	f	g	h	i						
9		j	k	l	m	n	o	p	q	r						
A		~	s	t	u	v	w	x	y	z						
B																
C	{	A	B	C	D	E	F	G	H	I						
D	}	J	K	L	M	N	O	P	Q	R						
E	\		S	T	U	V	W	X	Y	Z						
F	0	1	2	3	4	5	6	7	8	9						

Tabla 1.2 Tabla de caracteres EBCDIC. La celda con E.B. indica el espacio en blanco.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	:	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	P	Q	R	S	T	U	V	W	X	Y	Z	0	1	2	3	4
2	5	6	7	8	9	+	-	*	/	(	)	\$	=	E.B.	,	.
3	≡	[	]	%	≠	Γ	√	^	↑	↓	<	>	≤	≥	¬	;

Tabla 1.3 Secuencia de cotejo CDC-Científico. La celda con E.B. indica el espacio en blanco.

En estas tablas, las celdas con más de un carácter indican caracteres de control que aparecen representados por su correspondiente mnemotécnico. Como puede observarse en el código ASCII los números van antes que las letras, mientras que en los otros dos códigos van después. Las minúsculas van antes que las mayúsculas en EBCDIC, después en ASCII, y en CDC-Científico no forman parte del conjunto de valores.

Los tipos de datos definidos hasta ahora se conocen como tipos de datos simples o atómicos debido a que no son divisibles, es decir, no tienen componentes que puedan ser accedidos independientemente. Todos ellos excepto los reales son tipos ordenados, y a todos los tipos ordenados se les puede aplicar los operadores relacionales. En este contexto se definen los ‘tipos de datos escalares’ de la siguiente forma:

Definición 1.8: Se denominan *tipos de datos escalares* a aquellos en los que el conjunto de valores está ordenado y cada valor es atómico.

Ejemplos de ellos son los tipos de datos Carácter, Entero, Real y Booleano. Son escalares ya que todos ellos son de tipo atómico y están ordenados (los enteros y reales según indican las matemáticas, los caracteres según su secuencia de cotejo y los booleanos tal que False es menor que True).

Algunos tipos escalares presentan una propiedad adicional, la ordinalidad.

Definición 1.9: Se denominan *tipos de datos ordinales* a aquellos en los que cada valor tiene un único predecesor (excepto el primero), y un único sucesor (excepto el último).

Los tipos Carácter, Entero y Booleano son ordinales, pero como es bien conocido de matemáticas el tipo Real no lo es. Los lenguajes de alto nivel suelen presentar tres funciones adicionales asociadas a los tipos de datos ordinales: una que devuelve la posición de un elemento (Ord), otra que devuelve el predecesor (Pred), y otra que devuelve el sucesor (Succ).

Es bien sabido de la matemática, que los números reales no forman un conjunto ordenado al no ser numerable. Es bien cierto que a los números reales se les puede aplicar los operadores relacionales pero no se puede determinar su predecesor o su sucesor de forma única. Pensemos por ejemplo en los números 5.1 y 5.2; está claro que 5.1<5.2 y podríamos concluir erróneamente que 5.1 es el predecesor de 5.2 y este último el sucesor de 5.1. Sin embargo si consideramos dos cifras decimales, el

predecesor de 5.20 ($=5.2$) será 5.19 y el sucesor del 5.10 ($=5.1$) será el 5.11. Queda claro pues que para un número real no se puede establecer un único predecesor y un único sucesor ya que estas operaciones dependen de la precisión con que se trabaje y no del número en sí. Obsérvese que esto no afecta a los operadores relacionales y por tanto, se sigue cumpliendo que: $5.1=5.10<5.20=5.2$. El problema viene porque entre cada par de números reales que se escoja, siempre hay un número infinito de ellos.

En el contexto de la iniciación a la programación estructurada mediante cualquier lenguaje de programación, los tipos de datos definidos hasta ahora, suelen denominarse tipos de datos predefinidos o tipos primitivos estándar debido a que la mayoría de los lenguajes de alto nivel los incorporan en sus normas básicas. Sin embargo, debe observarse que incluso a este nivel iniciático surge la necesidad de definir tipos para estructurar los datos adecuadamente. En este contexto suelen denominarse ‘tipos de datos definidos por el usuario’, siendo los característicos de la metodología de programación estructurada los tipos: conjunto, arreglo y registro. Dado que son divisibles en componentes que pueden ser accedidas individualmente serán tipos de datos *compuestos*.

La mayoría de estos tipos de datos son colecciones de componentes organizadas mediante la forma de acceso a cada una de las componentes individuales. A este tipo de datos se le denomina *tipo de datos estructurado*.

En general, las operaciones asociadas a los tipos estructurados son las de *almacenar y recuperar* sus componentes individuales. Cuando todos los elementos de la colección son de un mismo tipo, denominado tipo base, se dice que el tipo es homogéneo mientras que cuando no lo son se dice que es heterogéneo.

Las siguientes definiciones precisan los TDA compuestos básicos.

Definición 1.10: El TDA Conjunto es una colección de elementos tratados con las operaciones unión, intersección y diferencia de conjuntos.

Definición 1.11: El TDA Arreglo es una colección homogénea de longitud fija tal que cada una de sus componentes pueden ser accedidas individualmente mediante uno (unidimensional) o varios (multidimensional) índices, que serán de tipo ordinal, y que indican la posición de la componente dentro de la colección.

Definición 1.12: El TDA Registro es un tipo de datos heterogéneo compuesto por un número fijo de componentes denominadas campos a las que se accede mediante un selector de campo (expresión formada por el nombre y un operador de selección como por ejemplo: ‘.’).

Como puede verse el TDA Conjunto no es estructurado ya que no está organizado mediante el modo de acceso a sus componentes individuales, mientras que el Arreglo y el Registro si lo son. Es importante observar el anidamiento existente entre los TDA. Los registros pueden tener como campos tipos de datos simples o arreglos o incluso otros registros. Cada uno de los campos será tratado como indica la definición del TDA al que pertenecen.

Los TDA básicos suelen ser los más frecuentemente utilizados, pero además cada problema puede requerir la definición de varios TDA propios. Así por ejemplo, en un problema matemático puede definirse un TDA matriz de la siguiente forma:

Definición 1.13: El TDA matriz se define como aquel que tiene como tipo el conjunto de matrices definido por las matemáticas (disposiciones rectangulares de elementos organizados en filas y columnas), y los operadores asociados a las mismas: *Obtener_elemento* (mediante la indexación por fila y columna), *Asignar_elemento*, *Sumar*, *Restar*, *Negar*, *Producto_escalar*, *Producto_matricial*, *Determinante*, *Inversa* y *Transponer*.

A lo largo de este texto se irán presentando los TDA más utilizados en las aplicaciones informáticas. Todos ellos vendrán dados por la necesidad de organizar y recuperar la información almacenada. Ejemplo básico de ello es el TDA lista, definido de la siguiente forma:

Definición 1.14: El TDA lista (o Secuencia) es una colección homogénea de datos, ordenados según su posición en ella, tal que cada elemento tiene un predecesor (excepto el primero) y un sucesor (excepto el último), y los operadores asociados sobre ellas son:

- *Insertar*: inserta un elemento, x , en una posición p , de la lista, pasando los elementos de la posición p y siguientes a la posición inmediatamente posterior.
- *Localizar*: localiza la posición p en la que se encuentra un elemento dado x .
- *Recuperar*: encuentra el elemento x que está en la posición p .
- *Suprimir*: elimina de la lista el elemento de la posición p .
- *Suprimir_dato*: elimina de la lista cualquier aparición del elemento x .
- *Anula*: ocasiona que la lista se vacíe.
- *Primero (Fin)*: proporciona el primer (último) elemento de la lista.
- *Imprimir*: imprime todos los elementos de la lista en su orden.

Obsérvese que algunos de estos operadores básicos se podrían realizar utilizando un subconjunto del resto de operadores. Así por ejemplo, *Suprimir_dato* podría realizarse con el operador *Localizar*, que proporciona la posición p en la que se encuentra un elemento dado x y seguidamente con *Suprimir*, que elimina de la lista el elemento de la posición p . Sin embargo, se incorpora a los operadores asociados dada su frecuencia de utilización y debido a que su implementación particularizada mejorará el coste de la misma. Esta estrategia no es nueva, recuérdese que los Enteros tienen definido el operador multiplicación, pese a que podría ser realizado por iteración del operador suma. Sin embargo la multiplicación es una operación muy frecuente y existen implementaciones específicas que mejoran la suma iterada.

En general el término ‘lista’ se utiliza cuando el TDA se implementa sobre memoria principal mientras que ‘secuencia’ suele reservarse para implementaciones sobre memoria secundaria. Evidentemente, según su implementación, las operaciones básicas asociadas serán de una forma u otra.

Como puede observarse, la lista es una estructura dinámica desde el punto de vista lógico, ya que su longitud dependerá del número de elementos que tenga, aumentará al insertar y se reducirá al suprimir.

El TDA lista puede implementarse de formas muy diferentes. La más inmediata a partir de los TDA presentados hasta ahora es mediante arreglos. Así, la lista, que es dinámica independientemente de su implementación, se realiza mediante una implementación estática. Sin embargo también es posible implementarla dinámicamente, mediante TDA basados en punteros.

En general, los tipos de datos presentados hasta ahora, tanto simples como compuestos, son bien conocidos por aquellos iniciados en las técnicas básicas de programación. No ocurre lo mismo con otros dos TDA fundamentales: la Secuencia y los Punteros. El primero de ellos debido a que como claro ejemplo de abstracción suele confundirse con alguna de sus implementaciones (ficheros o archivos, buffers de memoria, ...), mientras que el segundo suele estar poco dominado por considerarse un aspecto 'avanzado' de programación básica. Por ello se tratan de forma algo más detallada en las siguientes secciones.

1.2.3 TIPOS DE DATOS ABSTRACTOS: PUNTERO Y LISTAS ENLAZADAS

En principio los punteros son materia conocida de las nociones básicas de programación. En esta sección se recuerdan los conceptos esenciales de los mismos dado que su manejo será fundamental para todas las estructuras de datos dinámicas presentadas en el texto.

En principio un puntero es un tipo de datos simple cuyo valor es la dirección de una variable de otro tipo, denominada variable referenciada. La Figura 1.2 muestra la representación gráfica característica de estos tipos.

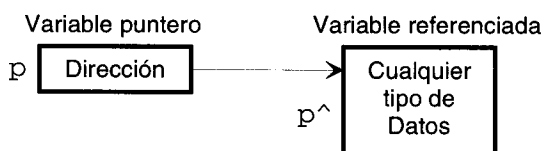


Figura 1.2 Concepto de punteros y variables referenciadas.

La utilidad de estos tipos radica en que permiten realizar una reserva dinámica de memoria. Para comprender este concepto es importante recordar cómo trabaja un programa compilado. Recuérdese que en cualquier programa de alto nivel lo primero que debe realizarse es la declaración de constantes (Const), tipos (Type) y variables (Var) que el programa va a necesitar. Cuando se ejecuta el programa lo primero que se hace es reservar memoria para todas estas variables. El sistema operativo es quien se encarga de gestionar la memoria primaria, y lleva el control de la memoria que está reservada porque es necesaria para los programas que se están ejecutando y la que el sistema tiene libre. A estas variables se las denomina *variables*.

bles estáticas, ya que ‘existirán’, en el sentido de que disponen de memoria reservada, a lo largo de toda la ejecución del programa.

Las variables referenciadas son *variables dinámicas*, es decir, se crean en tiempo de ejecución. Recuérdese que ‘crear’ una variable significa reservar espacio en memoria para dicha variable y asignar una etiqueta a ese espacio, es decir, tomar la memoria libre del sistema necesaria e indicar que está ocupada por dicha variable. Para las variables estáticas, esta indicación se realiza en la sección de declaraciones de forma que en tiempo de ejecución la memoria está reservada siempre, se utilice o no. Sin embargo, la memoria requerida por las variables dinámicas se reserva, como memoria ocupada, durante la ejecución del programa y sólo cuando es necesaria. Cuando no hace falta se libera o ‘destruye’, es decir, se indica que es memoria libre y el sistema podrá reutilizarla para otras necesidades.

En la mayoría de los computadores el valor de un puntero es un número entero, debido a que las posiciones de memoria van desde cero hasta el tamaño de la memoria menos uno. Sin embargo, este valor no es de tipo entero sino una dirección. Es más, en algunos tipos de máquinas, la memoria es accedida a través de páginas o segmentos y la dirección puede tener una estructura compleja formada por un selector o indicador de segmento y un desplazamiento dentro del segmento, alejándose considerablemente del concepto de tipo entero. El contenido de la variable referenciada puede ser cualquier tipo de dato: registro, otro puntero, etc. (Figura 1.2).

La declaración de los tipos puntero se realiza en la sección TYPE. En Modula2, por ejemplo, la sintaxis es la siguiente:

TYPE

```
Tipo_puntero = POINTER TO Tipo_variable_referenciada;
```

donde *Tipo_variable_referenciada* es el tipo de datos al que pertenecerán las variables referenciadas. La declaración de la variable puntero se realiza en la sección VAR, y su sintaxis en Modula2 es:

VAR

```
variable_puntero : Tipo_puntero;
```

La variable referenciada es accedida con el puntero mediante el operador apuntador, cuyo símbolo en Modula2 es el acento circunflejo: ‘^’.

Como ya se ha explicado detalladamente, la memoria necesaria para la variable puntero es reservada y ocupada de la misma forma que las variables estáticas. Sin embargo, la variable referenciada (por ejemplo un registro con varios campos) no existe hasta que no ha sido creada. Para crearla en Modula2 se dispone del procedimiento *Allocate*, cuya sintaxis es:

```
Allocate(variable_puntero, SIZE(Tipo_variable_referenciada));
```

Este procedimiento realiza dos acciones. En primer lugar toma la memoria libre necesaria para la variable referenciada. Esto lo realiza mediante la función `SIZE` que determina el tamaño de la variable referenciada con el fin de reservar únicamente la cantidad de memoria imprescindible. En segundo lugar asigna la dirección de memoria que ha reservado al puntero, `variable_puntero`. Tras este procedimiento la variable referenciada existe en el sentido de que hay memoria asignada para ella y además puede ser accedida por el puntero.

Una vez que una variable referenciada ya no va a ser utilizada por el programa, será destruida o liberada. El procedimiento en Modula2 que realiza esta tarea es `Deallocate`, cuya sintaxis es:

```
Deallocate(variable_puntero, SIZE(Tipo_referenciada));
```

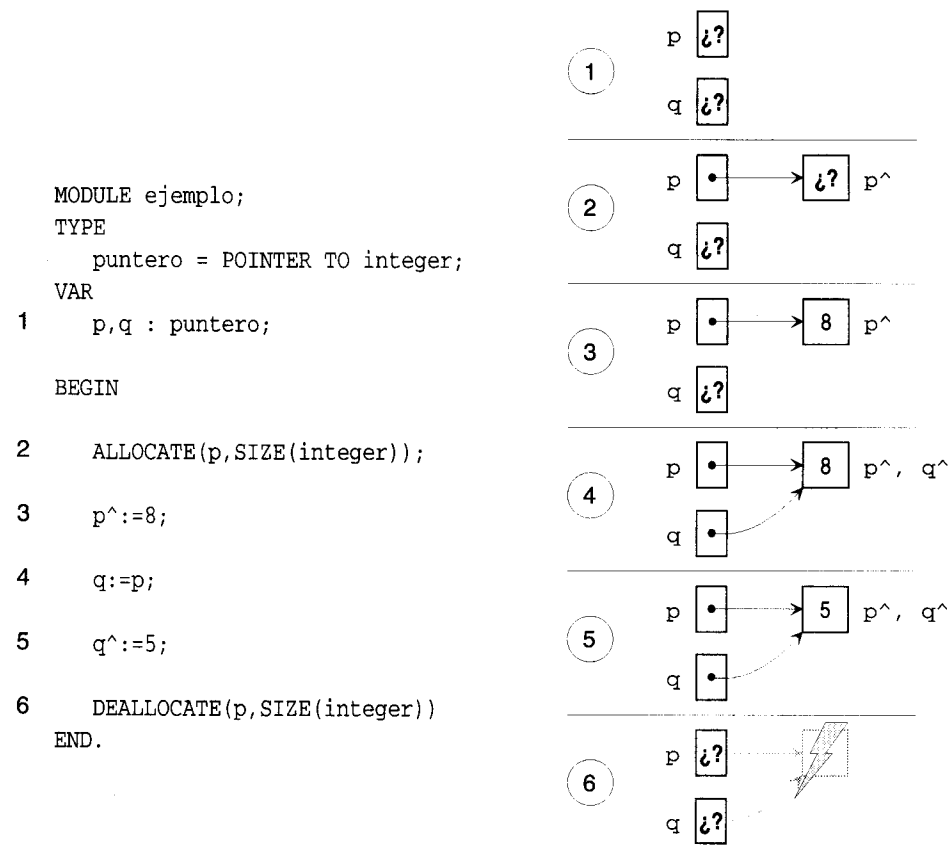


Figura 1.3 Ejemplo de programa mostrando las operaciones básicas con punteros.

Los comentarios anteriores muestran las características del tipo puntero, pero para poder establecerlo como TDA es imprescindible analizar las operaciones asociadas a los mismos. Éstas son la *asignación* y la *comparación*.

Recuérdese que los valores de los punteros son las direcciones a las que apuntan y por tanto, estas operaciones afectan a dichas direcciones. La Figura 1.3 ilustra estas operaciones y las anteriores consideraciones. En primer lugar se declara un tipo puntero a entero, de manera que la variable referenciada será de tipo entero. Seguidamente se declaran dos variables puntero de este tipo, p y q . Con ello el valor de estas variables no se asigna de ninguna manera, por lo que su contenido debe considerarse desconocido. En este sentido su comportamiento es el mismo que cualquier otra variable estática.

Cuando durante la ejecución del programa se requiera utilizar las variables referenciadas, éstas se crean mediante las sentencias: `Allocate(p, SIZE(integer))` y `Allocate(q, SIZE(integer))` (o también: `New(p)` y `New(q)`). Entonces cada puntero tendrá como valor la dirección de su variable referenciada de forma que apunta hacia ella. En esta situación las variables referenciadas recién creadas no tienen ningún valor (están sin inicializar). Para asignarlo se accede a la variable referenciada mediante el nombre del puntero que la apunta y el operador de acceso '^', realizándose la asignación mediante su operador, ':='. Debe resaltarse el hecho de que esta asignación no es la operación asociada al TDA puntero, sino que es simplemente la asignación de valores a variables. El hecho de que éstas sean variables referenciadas no implica diferencia alguna. En la Figura 1.3 se muestra la asignación del valor 8 a la variable referenciada ($p^{\wedge}=8$).

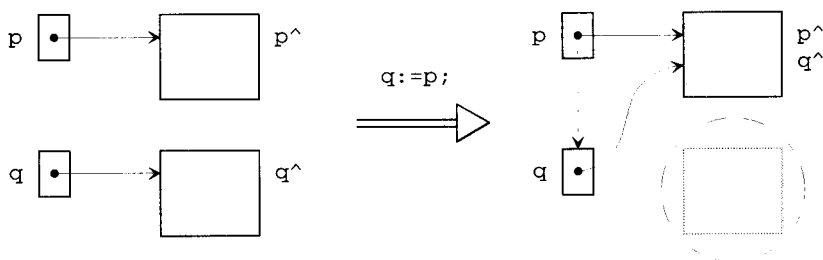


Figura 1.4 Pérdida de memoria debido a una incorrecta asignación entre punteros.

La asignación de punteros consiste en la asignación de valores puntero, es decir direcciones, entre variables puntero. Así $q:=p$ tiene como resultado que el valor de q sea la dirección almacenada en p , es decir, q apunta a donde apunta p . Debe tenerse en cuenta que la asignación incorrecta de punteros puede tener un efecto muy negativo. Este efecto consiste en dejar una variable referenciada sin tener acceso a ella, tal y como se ilustra en la asignación de punteros de la Figura 1.4. Obsérvese cómo la variable que estaba apuntada por q queda sin apuntar por nadie (señalada con un círculo punteado en la Figura 1.4). Por tanto, se está ocupando memoria que no podrá ser reutilizada. Por ello, antes de asignar punteros debe tenerse en cuenta que si la

variable referenciada va a ser necesitada posteriormente tiene que estar apuntada por algún puntero, y si no lo va a ser entonces debe ser liberada, es decir, debe ser devuelta a la memoria libre del sistema. Esto se realiza en Modula2 mediante los procedimientos Deallocate O Dispose.

Del mismo modo, la asignación entre variables referenciadas no es más que una asignación entre variables del tipo de la misma. Así, según se muestra en la Figura 1.5, la asignación $q^{\wedge} := p^{\wedge}$, asignaría el valor de $p^{\wedge}$ (25), a $q^{\wedge}$, en el supuesto de que se hubiesen creado las correspondientes variables referenciadas.

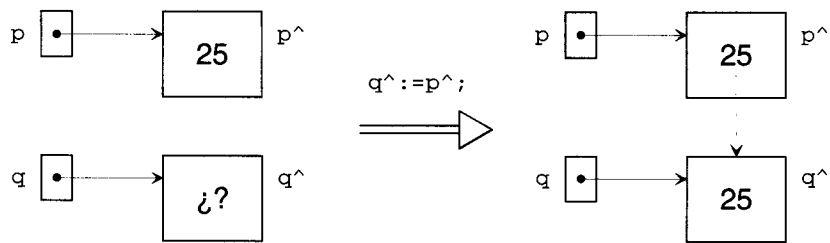


Figura 1.5 Asignación entre variables referenciadas.

La segunda operación asociada a los punteros es la comparación. Al comparar se está comprobando si sus direcciones son iguales o no, es decir, si apuntan o no a la misma dirección de memoria. En la Figura 1.6 se muestra cómo cuando apuntan a diferentes direcciones el resultado de la comparación es Falso (aunque los contenidos de las variables referenciadas sean idénticos), mientras que es Verdadero si apuntan a la misma dirección.

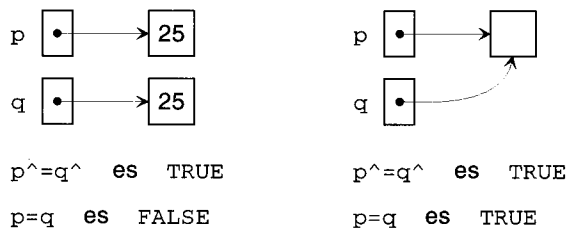


Figura 1.6 Operaciones de comparación entre punteros y variables referenciadas.

Es importante determinar cuándo un puntero no señala a ninguna variable referenciada. Para ello, en Modula2 por ejemplo, se dispone de la palabra clave NIL que representa una constante y en otros lenguajes se dispone de constantes similares. Así, la asignación de esta constante a un puntero indica que éste no apunta a nada. Su representación gráfica se muestra en la Figura 1.7. Esta constante tiene la particularidad de ser universal y poder ser asignada a cualquier tipo de puntero, independientemente del tipo de variable referenciada al que apunte.

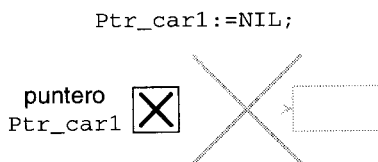


Figura 1.7 Puntero que no apunta a ninguna variable referenciada.

Atendiendo a estas consideraciones el TDA puntero puede definirse de la siguiente manera:

Definición 1.15: El TDA puntero es un tipo de datos simple cuyos valores son direcciones de memoria, y sus operadores asociados son la asignación de punteros y la comparación de punteros.

Por último, debe tenerse en cuenta el ahorro de almacenamiento que supone la utilización de punteros. La reserva de memoria realizada estáticamente sólo afecta a las variables puntero mientras que las referenciadas se crean y se destruyen de manera que sólo se reserva memoria cuando es necesitada. En este sentido, el ejemplo de la Figura 1.3 no es en absoluto ilustrativo de este hecho, y debe considerarse un ejemplo académico con el único propósito de ilustrar de forma sencilla las operaciones básicas y los operadores asociados a este TDA. En efecto, obsérvese que estáticamente se reservan dos punteros, y si cada puntero requiere cuatro bytes (como sucede habitualmente en las máquinas de 32 bits), ocupan lo mismo que podrían requerir cuatro variables de tipo entero en una máquina que representa los enteros con 16 bits. Por tanto, al crear las variables referenciadas se estarían utilizando un total de doce bytes, ocho reservados estáticamente para los punteros y cuatro dinámicamente para las variables referenciadas, cuando con reservar cuatro de forma estática sería suficiente para almacenar los dos enteros deseados. Sin embargo el ahorro de memoria y su apropiada utilización es evidente cuando la memoria a utilizar por la variable referenciada es superior a la necesaria para almacenar un puntero, o una misma memoria debe ser utilizada para distintos fines a lo largo de la ejecución de un programa; o se utiliza, como se va a describir en el siguiente apartado con las listas enlazadas, para la creación de estructuras de datos dinámicas.

Los punteros son la base de construcción de todas las denominadas estructuras dinámicas, como los árboles que se presentarán en este texto, o las listas enlazadas. Seguidamente se recuerdan los aspectos fundamentales de estas últimas.

1.2.4 LISTAS ENLAZADAS

Para almacenar de una forma organizada un conjunto de datos homogéneos se dispone de los arreglos. Esta estructura, como se dijo, es estática y está incorporada en los lenguajes de alto nivel. En este apartado se presentan las listas enlazadas. Su finalidad, como en el caso de los arreglos, es almacenar organizadamente datos del mismo tipo.

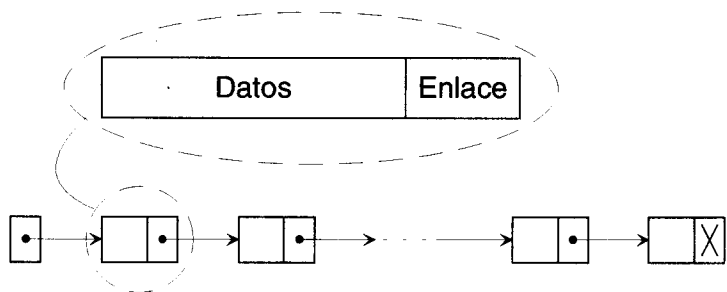


Figura 1.8 Representación esquemática de una lista enlazada y de uno cualquiera de sus nodos. El campo destinado a almacenar la información (Datos) es normalmente de un tamaño mucho mayor que el campo enlace.

Las listas enlazadas son tipos de datos dinámicos que se construyen con nodos. Un *nodo* es un registro con dos campos, uno de ellos contiene las componentes y se le denominará ‘datos’ (o data) y el otro es un valor que señala al siguiente nodo y se le denominará enlace (o next). El campo datos debe entenderse conceptualmente, es decir, en una implementación concreta pueden ser varios los campos que se consideren como datos. Por ejemplo, podrían ser los campos: Nombre, Apellidos, Dirección, DNI, etc.

La definición del TDA lista enlazada es la siguiente:

Definición 1.16: El TDA lista enlazada es una colección de nodos ordenada según su posición, tal que cada uno de ellos es accedido mediante el campo enlace del nodo anterior.

La Figura 1.8 ilustra el concepto de lista enlazada.

Haciendo uso de los punteros, es decir, definiendo los valores del campo Enlace del tipo nodo como punteros, las listas enlazadas se implementan aprovechando todas las ventajas de la asignación dinámica de memoria. Ésta es, de hecho, la forma habitual y más eficaz de implementar la lista enlazada (lo que explica que la representación gráfica de una lista enlazada se realice en términos de direccionamientos dinámicos, tal y como muestra la Figura 1.8).

Sin embargo, ésta no es la única implementación posible. Obsérvese que las listas enlazadas pueden implementarse estáticamente, sin más que definir el tipo del arreglo como nodos y utilizar los índices en el campo enlace para determinar el siguiente nodo. La Figura 1.9 muestra un ejemplo de este tipo de implementación de lista enlazada.

En esta figura, cada nodo está almacenado en una posición del arreglo y está formado por un registro con dos campos, al igual que la implementación dinámica basada en punteros. Por una parte está el campo de datos que contiene la información almacenada en la lista, y por otra el campo enlace, con la diferencia de que ahora en lugar de almacenar un puntero, es un valor ordinal que apunta a la posición del arreglo en la que está el siguiente elemento.

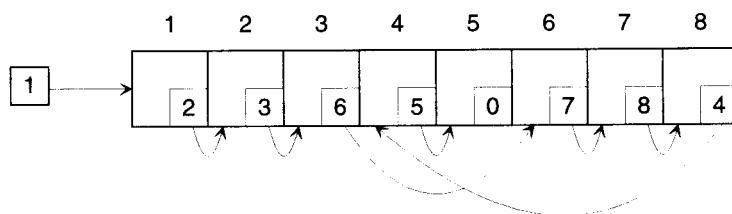


Figura 1.9 Implementación estática de una lista enlazada mediante un arreglo de registros.

Aunque en principio la gestión de una lista implementada mediante un arreglo pueda parecer más sencilla que una con punteros, no es cierto. Téngase en cuenta que en el caso de los arreglos siempre hay que mantener una lista enlazada con los elementos vacíos, de forma que cuando sea necesario añadir un nuevo elemento, se tome de la lista de nodos disponibles y se añada a la lista. Y a la inversa, cuando haya que eliminar un nodo de la lista habrá que devolverlo a la lista de disponibles, con lo que el trabajo es siempre el doble, puesto que hay que mantener dos listas sobre el mismo arreglo. En el caso de la lista con punteros, esta tarea la realiza el sistema automáticamente.

En este punto hay que marcar la distinción entre un tipo de datos abstracto y la naturaleza de su implementación. Ambos conceptos pueden ser considerados de forma estática o dinámica. Una variable de tipo 'array' (arreglo) es una estructura típicamente estática, pero puede almacenarse en memoria de forma estática (declarada en una zona de declaración de variables) o de forma dinámica (mediante un puntero). Lo mismo sucede, por ejemplo, con una pila o una lista, que son por naturaleza estructuras típicamente dinámicas pero que pueden implementarse con asignación de memoria estática (dentro de un array) o con asignación de memoria dinámica (basada en punteros). La Figura 1.10 muestra varios ejemplos con las cuatro posibilidades según se combinen estructuras de datos estáticas o dinámicas con asignación de memoria estática o dinámica.

		TDA	
		Estáticas	Dinámicas
Asignación de memoria	Estática	integer real record array	pilas, colas, etc. con arrays
	Dinámica	^integer ^real ^record ^array	pilas, colas, lista, árboles, etc. con punteros

Figura 1.10 Ejemplos de tipos de datos y de sus posibles implementaciones.

Comparando las definiciones de los TDA Lista y Lista enlazada es evidente que una forma natural de implementar una Lista en memoria primaria es mediante una Lista enlazada, y así se hace habitualmente. Sin embargo, esto no debe llevar a confundir ambos conceptos. El TDA Lista puede implementarse perfectamente mediante arreglos. De hecho, en antiguos lenguajes de programación, tales como Algol o el antiguo FORTRAN, no se disponía de implementaciones para punteros, hecho que no impedía que se implementasen listas (mediante cursores, véase detalles en [Aho, 1988], por ejemplo).

A la vista de estas consideraciones es importante reflexionar sobre las características de los TDA Arreglo, Lista enlazada y Lista. Obsérvese en primer lugar que los arreglos y las listas tienen como tipo componente cualquiera, mientras que las listas enlazadas tienen como componentes nodos. Los arreglos son estructuras estáticas, mientras que las otras dos son dinámicas. El arreglo accede a sus elementos, de forma aleatoria, mediante una indexación de los mismos, mientras que la lista y la lista enlazada lo realizan secuencialmente; y la lista enlazada necesariamente mediante el campo Enlace. La Figura 1.11 muestra una comparativa de las características de estos TDA.

	Acceso	Tipo de datos
Arreglo	Aleatorio	Cualquiera
Lista	Secuencial	Cualquiera
Lista enlazada	Secuencial	Nodos

Figura 1.11 Comparativa de los TDA Arreglo, Lista y Lista enlazada.

Seguidamente se presenta la implementación dinámica de la lista enlazada. Como se ha dicho, las variables dinámicas pueden crearse y liberarse durante la ejecución del programa.

Haciendo uso de las variables dinámicas podrán construirse estructuras de datos con un número de elementos que varía durante la ejecución del programa. El campo de enlace será un puntero y los nodos son variables referenciadas. Con esto, la declaración de tipos para los nodos en Modula2 es:

```
TYPE Ptr_Nodo = POINTER TO Nodo;
   Nodo = RECORD
       datos : Tipo_datos
       enlace : Ptr_Nodo;
   END;
```

Como se ha dicho, una lista enlazada es un conjunto organizado de componentes en los que el orden se establece mediante el campo enlace de cada nodo. Para acceder a la lista se tiene un puntero al primer nodo que se denomina puntero externo. La declaración de la lista se realizará mediante la del puntero externo, que por economía del lenguaje se le suele denominar 'lista'. En Modula2 sería:

```
VAR lista : Ptr_Nodo;
```

Para ganar claridad, también puede definirse un nuevo tipo Tlista y declarar las variables de este nuevo tipo:

```
TYPE  
  Tlista = Ptr_Nodo;  
VAR  
  lista : Tlista;
```

En cualquier caso, sobre este TDA se puede realizar cualquier función necesaria gestionando adecuadamente los punteros. Seguidamente se presentan algunas de las funciones más elementales.

Inserción por la cabeza

Al ser ésta la primera operación se va a comentar detenidamente para ilustrar el manejo de punteros. Las siguientes operaciones se presentarán de forma resumida.

Para insertar un nuevo elemento en la cabeza de una lista ya existente se deberá crear un nuevo nodo, seguidamente habrá que introducir el nuevo dato a almacenar en el campo de datos y finalmente realizar los enlaces adecuados para recolocar los nodos en la lista. Es decir:

1. Crear el nuevo nodo (reservar memoria).
2. Almacenar el dato en el campo correspondiente (datos).
3. Como este nuevo nodo va a ser el primero, su campo enlace deberá apuntar al hasta ahora primer nodo.
4. Por último el enlace externo (lista), no perteneciente a ningún nodo y que apunta al primer nodo de la lista, deberá apuntar ahora al nuevo nodo.

El manejo de punteros correspondiente a esta secuencia de pasos se ilustra en la Figura 1.12. Los números indican el orden en el que debe efectuarse cada enlace, y el nodo rodeado por un círculo punteado es el nuevo nodo. Obsérvese que el orden de las acciones es fundamental. Si primero se realizase la asignación del puntero externo al nuevo nodo, cuando se desease enlazar este nuevo nodo con la lista no se podría, pues su dirección estaba en el puntero lista. La lista se ha perdido. La gestión de punteros debe realizarse de forma cuidadosa. Podría pensarse en almacenar primero todas las direcciones que posiblemente se podrán necesitar, y no prestar

mucha atención al orden de los enlaces. Sin embargo, esta ‘solución’ no es aceptable (aunque ‘funcione’), primero porque se utilizan variables innecesarias, segundo porque la lista se manejaría de forma ineficaz y tercero porque se complica la legibilidad del código. El Programa 1.1 muestra el código necesario para la inserción por la cabeza de una lista enlazada.

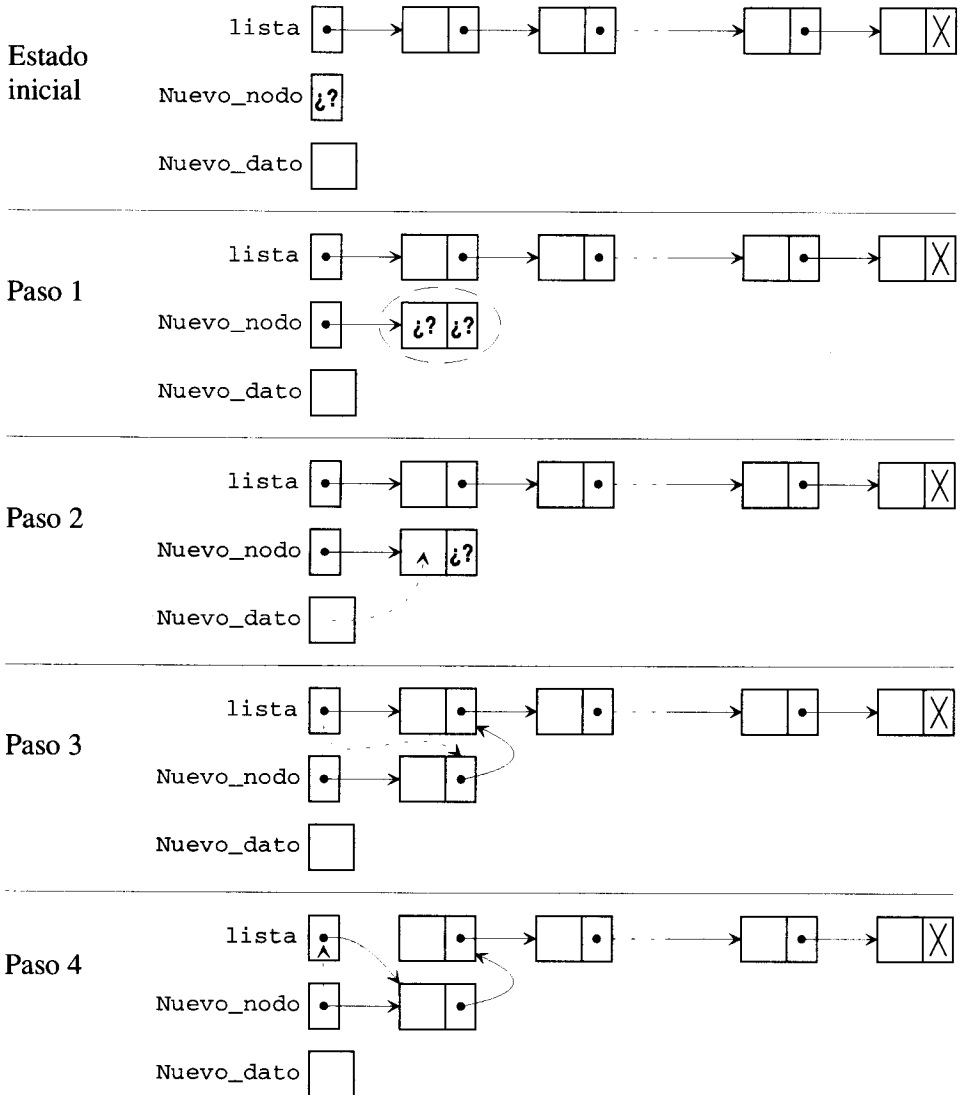


Figura 1.12 Pasos necesarios para realizar una inserción por la cabeza. Las flechas de trazo continuo indican punteros, las de trazo discontinuo indican trasvase de información correspondiente a una sentencia de asignación.

Programa 1.1 Inserción por la cabeza de una lista enlazada.

```
PROCEDURE Insertar_cabeza(VAR lista:Ptr_Nodo;nuevo_dato:Tipo_datos);
VAR
    Nuevo_nodo : Ptr_Nodo;
BEGIN
    ALLOCATE(Nuevo_nodo,SIZE(Nodo));
1     Nuevo_nodo^.datos := nuevo_dato;
2     Nuevo_nodo^.enlace := lista;
3     lista := Nuevo_nodo;
4 END Insertar_cabeza;
```

Inserción por el final

Para insertar por el final deberemos llegar hasta él y hacer la inserción. En primer lugar se crea un nuevo nodo, se introducen los nuevos datos en el campo de datos y como a partir de ahora será el último elemento se asigna NIL a su campo enlace. Para llegar hasta el final se hace uso de un puntero auxiliar (*Actual*) que irá recorriendo la lista hasta que localice el final. Para hacer esto podemos elegir una estructura iterativa *While*, antes de la cual habrá que inicializar adecuadamente el puntero temporal (*Actual*) para que apunte al primer elemento de la lista (*Actual:=lista*). Dentro del bucle lo único que hay que hacer es *avanzar* este puntero (*Actual*) para que recorra toda la lista pasando de un elemento al siguiente (*Actual:= Actual^.enlace*). Cuando *Actual* apunte al último nodo, *Actual^.enlace* será NIL. Es entonces cuando hay que salir de la iteración, con lo que la condición del bucle será *Actual^.enlace<>NIL* y el último nodo estaría siendo apuntado por *Actual*.

Por último, se realiza el enlace entre el último elemento de la lista y el nuevo nodo (*Actual^.enlace:=Nuevo_nodo*).

No obstante, la condición del bucle es válida únicamente para una lista no vacía, por lo que habrá que comprobar este hecho antes de hacer una llamada a este procedimiento. Si la lista está vacía, el puntero *Actual* se inicializaría a NIL, y por tanto, la condición del bucle estaría basada en una variable referenciada inexistente. Sin embargo, el caso de inserción por el final de una lista enlazada vacía es el mismo que el de inserción por el principio, problema que ya está resuelto.

En la Figura 1.13 se muestran los pasos necesarios para una inserción por el final. Como en el resto de las secuencias de figuras que se muestran en este capítulo, cada una muestra la situación de la estructura después de ejecutar el paso correspondiente que está marcado con un número a la izquierda del código asociado al algoritmo. En esta Figura 1.13, el paso 4 indica la situación de los punteros después de salir del bucle *while* que aparece en el Programa 1.2. El paso etiquetado en las figuras con el número 0 indica el estado inicial antes de comenzar la ejecución de la inserción o eliminación, por lo que no aparece en el código correspondiente.

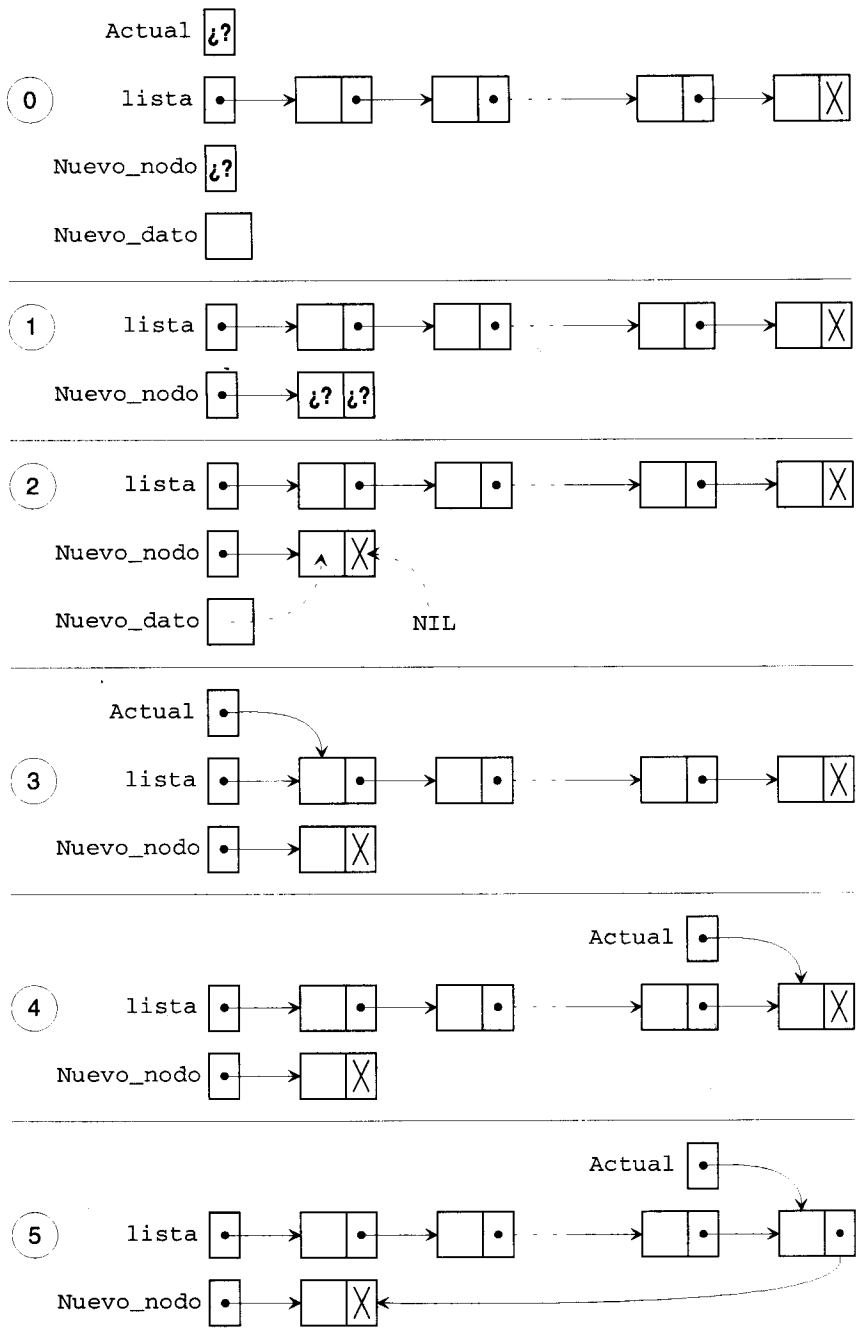


Figura 1.13 Inserción por el final de una lista enlazada.

Programa 1.2 Inserción por el final de una lista enlazada.

```

PROCEDURE Insertar_final(VAR lista:Ptr_Nodo; nuevo_dato:Tipo_datos);
VAR
    Nuevo_nodo, Actual : Ptr_Nodo;
BEGIN
    IF lista=nil THEN Insertar_cabeza(lista,nuevo_dato) ELSE
Paso 1      ALLOCATE(Nuevo_nodo,SIZE(Nodo));
2          Nuevo_nodo^.datos := nuevo_dato;
2          Nuevo_nodo^.enlace := NIL;
3          Actual := lista;
4          WHILE Actual^.enlace <> NIL DO
                Actual := Actual^.enlace;
                END;
                Actual^.enlace := Nuevo_nodo
5          END
END Insertar_final;

```

Suprimir por la cabeza

La eliminación del elemento del principio de la lista consta de cuatro pasos:

1. En primer lugar se almacena la dirección de este primer elemento, apuntado por la variable *lista*, en un puntero auxiliar (*Aux*), *Aux:=lista*;
2. En segundo lugar, habrá que asignar al parámetro formal *dato*, el valor almacenado en el elemento a eliminar.
3. Seguidamente se realiza la asignación de la lista al nuevo primer elemento (*lista:=lista^.enlace*).
4. Por último, se libera la memoria del nodo que ya no pertenece a la lista.

Debe tenerse muy en cuenta que el no realizar este último paso, es decir, suprimir únicamente mediante la asignación *lista:=lista^.enlace*, es un error muy grave. Estaría ocupándose memoria no necesaria y a lo largo de la ejecución del programa el sistema podría quedarse sin memoria libre pese a que en realidad no se utiliza la memoria que se está ocupando. Recuérdese los comentarios realizados previamente y que se ilustraban en la Figura 1.4. En el caso de las listas enlazadas el problema es especialmente grave pues puede resultar reiterativo debido a las numerosas inserciones y eliminaciones que normalmente se realizan en este tipo de estructuras.

Obsérvese también que el puntero auxiliar es imprescindible para liberar finalmente el nodo. El código necesario para realizar la supresión por la cabeza se muestra en el Programa 1.3 y la Figura 1.14 ilustra los pasos necesarios.

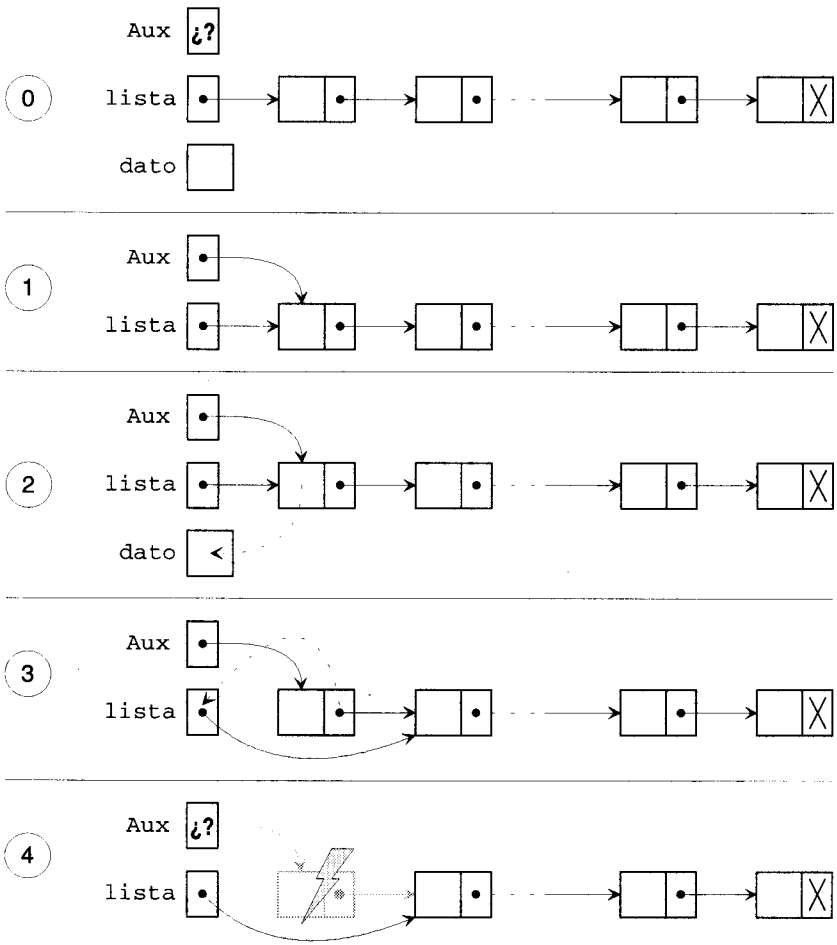


Figura 1.14 Supresión por la cabeza de una lista enlazada.

Programa 1.3 Supresión por la cabeza de una lista enlazada.

```
PROCEDURE Suprimir_cabeza(VAR lista: Ptr_Nodo;VAR dato: Tipo_datos);
VAR
    Aux : Ptr_Nodo;
BEGIN
    Paso 1    Aux := lista;
    2        dato := lista^.datos;
    3        lista:=lista^.enlace;
    4        DEALLOCATE(Aux,SIZE(Nodo));
END Suprimir_cabeza;
```

Suprimir por el final

La supresión por el final requiere lógicamente localizar la última posición de la lista y liberar el último nodo. Pero además, será imprescindible asignar el valor NIL al penúltimo elemento de la misma, que tras la eliminación pasará a ser el último, ya que de no ser así la lista no tendría final y el último enlace estaría apuntando a una dirección, de memoria desconocida (y posiblemente utilizada para otro cometido, como por ejemplo otra variable), con lo que los recorridos sobre ella no tendrían fin.

Para ello se utiliza un puntero auxiliar (Actual) que apuntará al final del recorrido (al final del bucle) al último elemento, y otro puntero (Ant) que quedará apuntando al anterior. Con una estructura iterativa While la condición de salida del bucle será `Actual^.enlace <> NIL`. Por último, se realiza la eliminación del último nodo y se asigna NIL al nuevo último nodo (`Ant^.enlace := NIL`).

El Programa 1.4 muestra el código necesario para la supresión por el final de una lista enlazada implementado en Modula2. La Figura 1.15 muestra la secuencia de pasos correspondiente.

Las acciones necesarias para la supresión por el final de una lista enlazada pueden resumirse en los siguientes pasos:

- Inicialización de los punteros.
- Recorrido de la lista hasta alcanzar el final.
- Recuperación del dato almacenado en el último nodo.
- Eliminación del nodo y marcado del nuevo último nodo.

Programa 1.4 Supresión por el final de una lista enlazada.

```

PROCEDURE Suprimir_final(VAR lista: Ptr_Nodo; VAR dato: Tipo_datos);
VAR
  Actual, Anterior : Ptr_Nodo;
BEGIN
  Paso 1   Anterior := lista;
          1   Actual:=lista;
          2   WHILE Actual^.enlace <> NIL DO
                Anterior := Actual;
                Actual := Actual^.enlace
              END;
          3   dato := Actual^.datos;
          4   Anterior^.enlace := NIL;
              IF Anterior=Actual THEN (*si sólo hay un nodo*)
                lista:=NIL
              END
          5   DEALLOCATE(Actual, SIZE(Nodo))
END Suprimir_final;
```

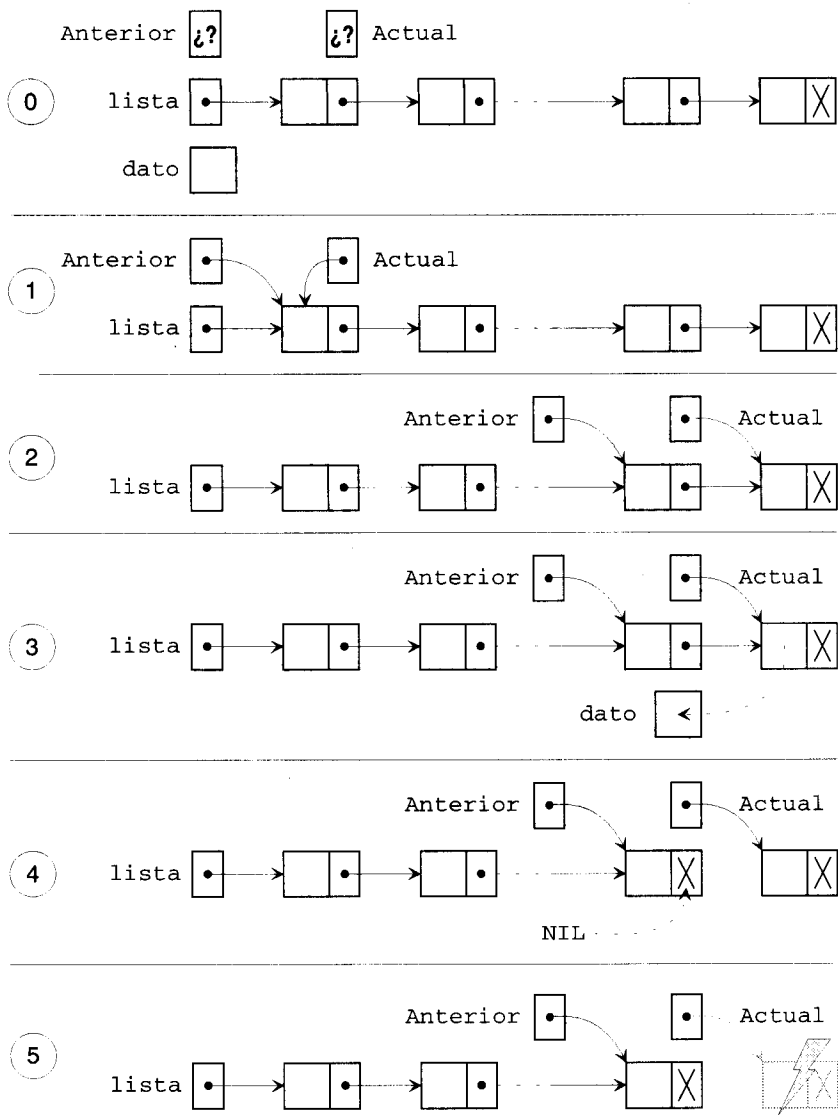


Figura 1.15 Supresión por el final de una lista enlazada en el caso general de que tenga más de un nodo.

Inserción según un criterio de orden

Otra función típica sobre listas enlazadas es insertar en el medio. Para ello habrá que hacerlo según un criterio. Supóngase que la lista se utiliza para almacenar números enteros (Tipo_datos=INTEGER) y que la lista está ordenada en sentido creciente. El algoritmo que se describe a continuación, inserta un nuevo número entero en su lugar y la Figura 1.16 muestra los pasos necesarios.

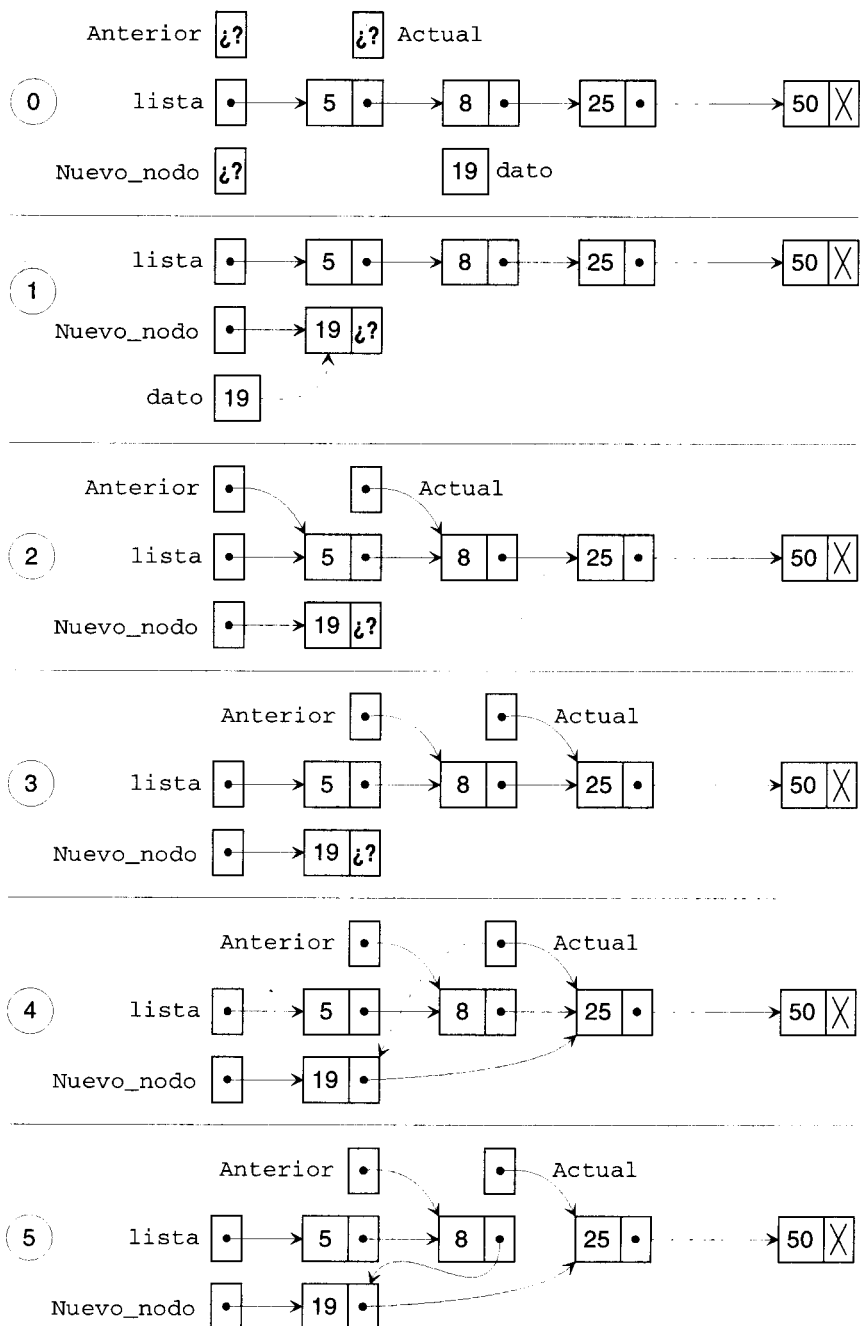


Figura 1.16 Inserción en orden en una lista enlazada delante de Actual.

En general, deben considerarse tres situaciones posibles: que la posición a insertar sea la primera, la última o una entre éstas. Localizada la situación en la que es necesario insertar al principio ($\text{dato} < \text{lista}^{\text{.datos}}$) puede utilizarse el procedimiento `Insertar_cabeza()` ya explicado anteriormente. Una situación idéntica se presenta cuando la lista no contiene ningún elemento. Hay que tener en cuenta que la condición anterior no es válida para una lista vacía, por lo que esta última situación ($\text{lista} = \text{NIL}$) deberá ser comprobada en primer lugar dando lugar a dos sentencias `IF _ THEN` anidadas. No obstante, algunos compiladores permiten la evaluación abreviada de expresiones booleanas (con 'cortocircuito'), de tal forma que si antes de completar la evaluación de la expresión ya se puede concluir que será verdadera o falsa, no continúan evaluando el resto de la misma. Esta característica puede aprovecharse en nuestro caso para combinar las dos sentencias `IF _ THEN` en una sola:

```
IF (lista=NIL) OR (dato<lista^.datos) THEN ...
```

Suponiendo evaluación abreviada de expresiones lógicas en esta expresión, si la lista está vacía la primera parte de la misma será verdadera independientemente de cómo sea la segunda y el programa no comprobará la segunda condición. De esta forma, siempre que se acceda a la evaluación de la segunda parte se tendrá la garantía de que la lista no está vacía y por tanto, $\text{lista}^{\text{.datos}}$ es una referencia válida.

En el caso general en el que se encuentra en la lista un elemento mayor que el que queremos añadir, la inserción del dato deberá realizarse por delante de él. Esta situación se ilustra en la Figura 1.16, en la que `Actual` apunta al nodo que contiene el valor 25 delante del cual se insertará el nuevo dato (de valor 19 en la figura). El procedimiento en `Modula2` se detalla en el Programa 1.5. En primer lugar se almacena el dato en el campo `datos` del nuevo nodo ($\text{Nuevo_nodo}^{\text{.datos}} := \text{dato}$) y se hace que el campo `enlace` del nuevo nodo apunte al elemento siguiente al nuevo ($\text{Nuevo_nodo}^{\text{.enlace}} := \text{Actual}$), tal y como se muestra en el paso 4 de la Figura 1.16. Por último, hay que conseguir que el nodo que contiene el elemento anterior (valor 8) apunte al nuevo nodo ($\text{Anterior}^{\text{.enlace}} := \text{Nuevo_nodo}$) como aparece en el paso 5 de la Figura 1.16. Hay que tener en cuenta que al realizar la inserción debemos acceder simultáneamente a dos nodos consecutivos, por lo que necesitamos dos punteros, de forma similar a como se hizo con la supresión por el final: `Actual` que indica el nodo al que se está apuntando a lo largo del recorrido y `Anterior` que apuntará al nodo anterior al que apunta `Actual`.

La inserción por el final habrá que realizarla si no se encuentra a lo largo de toda la lista un elemento mayor que el que se desea insertar. Para determinar esta situación se recorre la lista hasta que se encuentre, y si no es así se indica mediante una variable booleana (`encontrado`). El programa 1.5 es válido independientemente de la posición en la que haya que insertar el nuevo elemento.

Programa 1.5 Inserción en orden en una lista enlazada, (delante de Actual).

```

PROCEDURE Insertar_orden_delante (VAR lista: Ptr_Nodo; dato : Tipo_datos);
VAR
    Encontrado: BOOLEAN;
    Actual,Nuevo_nodo,Anterior: Ptr_Nodo;
BEGIN
    IF (lista=NIL)OR(dato<lista^.datos) THEN  (* Inserción al principio *)
        Insertar_cabeza(lista,dato)
    ELSE
1       ALLOCATE(Nuevo_nodo,SIZE(Nodo));
1       Nuevo_nodo^.datos:=dato;
        (* Busca el punto de insercion*)
2       Anterior:=lista;
2       Actual := lista^.enlace;
        Encontrado := FALSE;
3       WHILE (Actual<>NIL)AND(NOT Encontrado) DO
            IF dato>Actual^.datos THEN
                Anterior:=Actual;
                Actual:=Actual^.enlace
            ELSE
                Encontrado:=TRUE;
            END;
        END;
        (*Insertar el nuevo nodo*)
4       Nuevo_nodo^.enlace:=Actual;
5       Anterior^.enlace:=Nuevo_nodo
    END;
END Insertar_orden_delante;

```

Este procedimiento también podría haberse realizado insertando el nuevo nodo por detrás del puntero Actual, tal y como se muestra en las Figuras 1.17 y 1.18. Para conseguir esto, se avanza sobre la lista de la misma forma que en los casos anteriores, pero ahora al encontrar el elemento mayor (si existe) se realiza la inserción dentro del mismo bucle. Esto se hace en tres pasos, el primero consiste en copiar el contenido del nodo referenciado por el puntero de recorrido (Actual[^]) en el nuevo nodo (paso 4 de la Figura 1.17). A continuación, el enlace del nodo al que apunta Actual se hace que apunte al nuevo nodo (Actual[^].enlace:=Nuevo_nodo), con lo que la inserción se produce efectivamente detrás del puntero Actual. Por último, se asigna el valor del nuevo dato al campo datos del nodo apuntado por Actual (Nuevo_nodo[^].datos:=dato). El paso 5 de la Figura 1.17 ilustra estas dos últimas acciones y el Programa 1.6 muestra el código correspondiente.

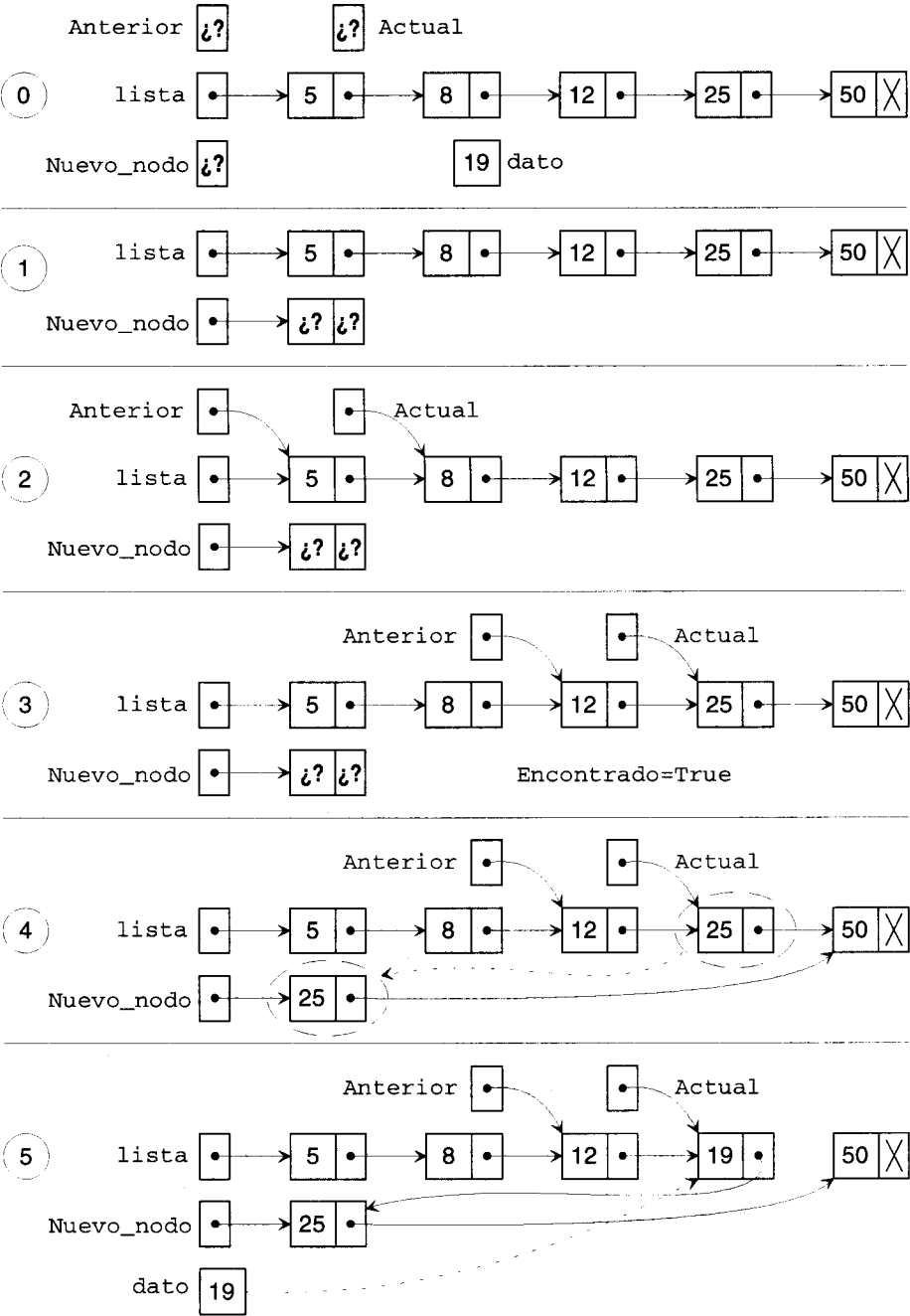


Figura 1.17 Inserción en orden en una lista enlazada (detrás de Actual).

Programa 1.6 Inserción en una lista enlazada según un criterio (detrás de Actual).

```

PROCEDURE Insertar_orden_detras (VAR lista: Ptr_Nodo; dato : Tipo_datos);
VAR
    Encontrado : BOOLEAN;
    Actual, Nuevo_nodo, Anterior : Ptr_Nodo;
BEGIN
    IF (lista=NIL) OR (dato<lista^.datos) THEN (* Insercion al principio *)
        Insertar_cabeza(lista,dato)
    ELSE
        Paso 1    ALLOCATE(Nuevo_nodo,SIZE(Nodo));
        2         Anterior:=lista;
        2         Actual:=lista^.enlace;
                 Encontrado:=FALSE;
        3         WHILE (Actual<>NIL) AND (NOT Encontrado) DO (* Ins. en medio *)
                    IF dato > Actual^.datos THEN
                        Anterior:=Actual;
                        Actual:=Actual^.enlace
                    ELSE
        4             Nuevo_nodo^:=Actual^;
        5             Actual^.enlace:=Nuevo_nodo;
        5             Actual^.datos:=dato;
                        Encontrado:=TRUE;
                    END;
                END;
                IF NOT Encontrado THEN (* Insercion al final *)
        6             Nuevo_nodo^.datos:=dato;
        6             Nuevo_nodo^.enlace:=NIL;
        7             Anterior^.enlace:=Nuevo_nodo;
                END
            END
        END Insertar_orden_detras;

```

Sin embargo, la solución que se acaba de comentar no es válida para el caso en el que todos los elementos son menores que el nuevo, ya que el bucle terminará sin que se haya realizado la inserción, puesto que en ninguna de las iteraciones habrá dejado de cumplirse la condición `dato>Actual^.datos` que es la que realiza la inserción comentada en el párrafo anterior. Por este motivo, esta situación hay que tratarla aparte, fuera del bucle, lo que se ilustra en los pasos 6 y 7 de la Figura 1.18.

Hay que tener en cuenta que esta figura no es una continuación de la anterior, sino una alternativa a los pasos 4 y 5 de la misma. Este hecho queda claramente de manifiesto en el código correspondiente que se muestra en el Programa 1.6. En este programa se muestra que si se ejecutan los pasos 4 y 5 dentro del bucle la variable booleana `Encontrado` toma el valor `True` lo que impide que se ejecuten los pasos 6 y 7 externos al bucle.

Análogamente puede desarrollarse un procedimiento para suprimir según un criterio dado.

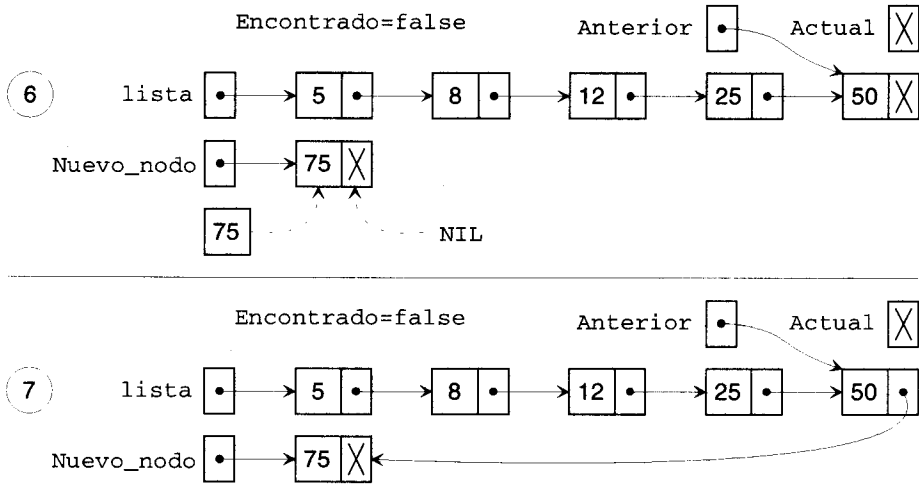


Figura 1.18 Inserción en orden en una lista enlazada por detrás del puntero de recorrido, en el caso en que el nuevo elemento sea mayor que todos los de la lista.

Imprimir una lista enlazada

Evidentemente imprimir los datos de la lista enlazada será una tarea habitual. Esto puede realizarse en orden, es decir, de manera que aparezcan los datos de principio a fin de la lista tal y como muestra el Programa 1.7

Programa 1.7 Imprimir una lista enlazada en orden.

```
PROCEDURE Imprimir_lista(lista : Ptr_Nodo);
VAR
  Aux : Ptr_Nodo;
BEGIN
  Aux := lista;
  WHILE Aux <> NIL DO
    WriteInt(Aux^.datos,7);
    Aux := Aux^.enlace;
  END;
END Imprimir_lista;
```

Sin embargo, también es posible imprimirla en orden inverso recorriendo una sola vez la lista mediante una implementación recursiva, tal y como se muestra en el Programa 1.8.

Programa 1.8 Imprimir una lista enlazada en orden inverso.

```

PROCEDURE Imprimir_contrario (lista: Ptr_Nodo);
BEGIN
  IF lista <> NIL THEN
    Imprimir_contrario(lista^.enlace);
    WriteInt(lista^.datos,6);
  END;
END Imprimir_contrario;

```

Las operaciones sobre listas enlazadas mostradas en este apartado tienen como fin fundamental ilustrar el manejo de los punteros, pero por supuesto no son las únicas posibles. Cualquier operación que fuese requerida para la solución de un problema se realizará manejando los punteros de forma análoga a la aquí presentada.

1.2.5 TDA LISTA IMPLEMENTADO CON LISTAS ENLAZADAS

Por último, es evidente que cuando el TDA lista se implementa con listas enlazadas, sus operadores asociados se implementan análogamente. Seguidamente se presentan todos ellos. Es importante analizarlos detalladamente como ejercicio de programación con listas enlazadas.

Programa 1.9 Inserción en una lista en una posición p.

```

PROCEDURE Insertar (VAR L: Ptr_Nodo; x : Tipo_datos;p: INTEGER);
(* Inserta un elemento, x, en una posicion p de L, pasando los elementos
  de la posicion p y siguientes a la posicion inmediatamente posterior *)
VAR
  Anterior, Actual,Nuevo_nodo: Ptr_Nodo;
  cont: INTEGER;
BEGIN
  IF p=1 THEN (*Inserta en la primera posicion*)
    ALLOCATE(Nuevo_nodo,SIZE(Nodo));
    Nuevo_nodo^.datos:=x;
    Nuevo_nodo^.enlace:=L;
    L:=Nuevo_nodo
  ELSE (* localiza la posicion p de insercion *)
    Anterior:=NIL;
    Actual:=L;
    cont:=1;
    WHILE (Actual#NIL)&(cont#p) DO
      Anterior:=Actual;
      Actual:=Actual^.enlace;
      cont:=cont+1
    END;
    IF p<cont+1 THEN (* p menor que el numero de elementos de L *)

```

```

        ALLOCATE (Nuevo_nodo, SIZE(Nodo));
        Nuevo_nodo^.datos:=x;
        Anterior^.enlace:=Nuevo_nodo;
    IF (Actual=NIL) OR ((Actual=L) & (cont=p-1)) THEN
        (* insertar en la ultima posicion *)
        Nuevo_nodo^.enlace:=NIL
    ELSIF cont=p THEN          (*insertar en posicion intermedia*)
        Nuevo_nodo^.enlace:=Actual;
    END
END
END
END Inserir;

```

Programa 1.10 Localización de la posición de un determinado elemento pasado como parámetro.

```

PROCEDURE Localizar (L: Ptr_Nodo; x : Tipo_datos):INTEGER;
(* Localiza la posicion en la que se encuentra un elemento dado x,
   si no lo encuentra devuelve 0 *)
VAR
    Actual,Anterior: Ptr_Nodo;
    Encontrado: BOOLEAN;
    p: INTEGER;
BEGIN
    Anterior:=NIL;
    Actual:=L;
    p:=0;
    Encontrado:=FALSE;
    WHILE NOT Encontrado & (Actual#NIL) DO
        IF Actual^.datos=x THEN
            Encontrado:=TRUE;
            p:=p+1
        ELSE
            Anterior:=Actual;
            Actual:=Actual^.enlace;
            p:=p+1;
        END;
    END;
    IF NOT Encontrado THEN p:=0;  (*Elemento no encontrado*)
    ELSE Actual:=Actual^.enlace;
    END;
    RETURN p
END Localizar;

```

Programa 1.11 Obtención de la posición de un elemento dado como parámetro.

```

PROCEDURE Recuperar (L: Ptr_Nodo; VAR x : Tipo_datos;p: INTEGER;
VAR encontrado:BOOLEAN);
(* Encuentra el elemento x que esta en la posicion p, si la posicion p es
mayor que el numero de elementos de L, devuelve encontrado a FALSE*)
VAR
    Actual: Ptr_Nodo;
    cont:INTEGER;
BEGIN
    encontrado:=FALSE;
    Actual:=L;
    cont:=1;
    WHILE (Actual#NIL)&(cont<p) DO
        Actual:=Actual^.enlace;
        cont:=cont+1
    END;
    IF (cont=p)&(Actual#NIL) THEN
        x:=Actual^.datos;
        encontrado:=TRUE;
    END
END Recuperar;

```

Programa 1.12 Eliminación del elemento de una posición dada.

```

PROCEDURE Suprimir (VAR L: Ptr_Nodo; p: INTEGER);
(*elimina de L el elemento de la posicion p*)
VAR
    Anterior,Actual: Ptr_Nodo;
    cont:INTEGER;
BEGIN
    Anterior:=NIL;
    Actual:=L;
    cont:=1;
    (*Localizar la posicion p*)
    WHILE (Actual^.enlace#NIL)&(cont<p) DO
        Anterior:=Actual;
        Actual:=Actual^.enlace;
        cont:=cont+1
    END;
    IF cont=p THEN      (*Eliminar*)
        IF cont=1 THEN  (*eliminar el primer elemento*)
            L:=Actual^.enlace

```

```

        ELSIF Actual^.enlace=NIL THEN (*eliminar el ultimo*)
            Anterior^.enlace:=NIL
        ELSE (*intermedio*)
            Anterior^.enlace:= Actual^.enlace;
        END;
        DEALLOCATE(Actual,SIZE(Nodo))
    END
END Suprimir;

```

Programa 1.13 Eliminación de todas las apariciones de un determinado valor.

```

PROCEDURE Suprimir_dato (VAR L: Ptr_Nodo; x: Tipo_datos);
(*elimina de L todas las apariciones del elemento x*)
VAR
    Anterior,Actual,Aux: Ptr_Nodo;
    Encontrado: BOOLEAN;
BEGIN
    Anterior:=NIL;
    Actual:=L;
    WHILE Actual#NIL DO (*busqueda de todas las apariciones*)
        Encontrado:=FALSE;
        WHILE NOT Encontrado & (Actual#NIL) DO
            IF Actual^.datos=x THEN Encontrado:=TRUE
            ELSE
                Anterior:=Actual;
                Actual:=Actual^.enlace;
            END;
        END;
        IF Encontrado THEN (*Eliminar*)
            IF Actual=L THEN (*eliminar el primer elemento*)
                L:=Actual^.enlace;
                Aux:=L;
            ELSIF (Actual^.enlace=NIL)&(Anterior#NIL) THEN
                (* eliminar el ultimo *)
                Anterior^.enlace:=NIL;
                Aux:=NIL;
            ELSE (*intermedio*)
                Anterior^.enlace:= Actual^.enlace;
                Aux:=Actual^.enlace;
            END;
            DEALLOCATE(Actual,SIZE(Nodo));
            Actual:=Aux;
        END;
    END;
END Suprimir_dato;

```

Programa 1.14 Vaciado completo de la lista.

```

PROCEDURE Anula (VAR L: Ptr_Nodo);
(*ocasiona que L se vacie*)
VAR
    Actual: Ptr_Nodo;
BEGIN
    IF L#NIL THEN
        Actual:=L;
        WHILE Actual^.enlace#NIL DO
            L:=Actual^.enlace;
            DEALLOCATE(Actual, SIZE(Nodo));
            Actual:=L;
        END;
        DEALLOCATE(L, SIZE(Nodo));
        L:=NIL;
    END;
END Anula;

```

Programa 1.15 Obtención del primero y último elementos de la lista.

```

PROCEDURE Primero (L: Ptr_Nodo): Tipo_datos;
(*proporciona el primer elemento de L*)
BEGIN
    IF L#NIL THEN RETURN L^.datos
    END;
END Primero;

(*****)

PROCEDURE Fin (L: Ptr_Nodo): Tipo_datos;
(*proporciona el ultimo elemento de L*)
VAR
    Anterior, Actual: Ptr_Nodo;
BEGIN
    Anterior:=NIL;
    Actual:=L;
    WHILE Actual#NIL DO
        Anterior:=Actual;
        Actual:=Anterior^.enlace
    END;
    IF Anterior#NIL THEN RETURN Anterior^.datos
    END;
END Fin;

```

Obsérvese cómo, tal y como se indicó en la definición del TDA lista, la incorporación del operador `Suprimir_dato(L,x)` es muy adecuada ya que es una operación muy frecuente y si se realizase mediante los operadores `Localizar(L,x,p)` y `Suprimir(L,p)` habría que recorrer dos veces la lista, duplicando su costo innecesariamente.

1.3

ALGORITMOS

No es objeto de este texto realizar una presentación exhaustiva y detallada de la teoría general de algoritmos. Las técnicas básicas de análisis y diseño de algoritmos, análisis de invariantes, métodos de verificación de la corrección del algoritmo así como cálculo del coste computacional se tratan detalladamente en numerosos textos [Peña, 1998] y se suponen conocidas. Sin embargo a lo largo del texto se presentarán numerosos algoritmos asociados a los distintos tipos de datos abstractos, así como sus implementaciones en funciones y procedimientos. Por ello, en esta sección se presentan los conceptos básicos necesarios para una buena comprensión de los razonamientos detallados acerca de cada uno de ellos.

En primer lugar, es importante recordar que no se debe confundir algoritmo con programa. Los primeros son muy anteriores a los últimos. Los griegos propusieron un gran número de algoritmos, y sin embargo no realizaron ningún programa, que se sepa. Un ejemplo clásico es el algoritmo de Euclides para calcular el máximo común divisor de dos números naturales, que hoy en día suele utilizarse para realizar un ejercicio simple de programación.

El término algoritmo fue acuñado por *Muhammad ibnMūsā al-Jwārizmi*, matemático árabe de principios del siglo IX, como el conjunto de símbolos y de procedimientos de los cálculos matemáticos. Sin embargo, en la actualidad el término algoritmo debe entenderse en sentido amplio, tal y como indica la siguiente definición.

Definición 1.17: Un algoritmo es un método o proceso seguido para resolver un problema.

En este contexto un programa es una implementación del algoritmo para su ejecución en un computador.

Las propiedades básicas que debe satisfacer todo algoritmo son las siguientes:

- En primer lugar el algoritmo debe ser correcto, es decir, debe solucionar el problema en todos los casos posibles.
- Además, debe estar compuesto por una serie dada de pasos y para cada uno de ellos no debe existir ambigüedad en la definición de cuál es el siguiente.
- El número de pasos debe ser finito, y por supuesto, el algoritmo debe terminar.

En relación con este último aspecto debe tenerse en cuenta que existen algoritmos que terminan y otros que no. Así por ejemplo, el algoritmo de Euclides para calcular el máximo común divisor de dos números naturales presenta el resultado en

un número finito de pasos, y sin embargo el algoritmo para la expansión de la raíz cuadrada de un número natural en fracciones decimales no garantiza que termine completamente en un número finito de pasos, ya que el resultado puede ser irracional. Sin embargo, en estos casos se entiende que el algoritmo termina introduciendo el concepto de precisión requerida o solución suficientemente terminada. La teoría general de algoritmos presenta las técnicas necesarias para asegurar todas estas propiedades fundamentales.

En general, existen diferentes algoritmos para resolver un mismo problema. Evidentemente esto permite elegir el 'mejor'. Sin embargo, como se verá a lo largo del texto, un mismo problema puede ser mejor resuelto por un algoritmo para unos datos determinados, mientras que con otros datos ese algoritmo puede ser muy costoso. De ahí la importancia del análisis del costo computacional, que indicará cuándo un algoritmo es mejor o peor que otro, permitiendo la elección más adecuada.

Son dos los factores fundamentales a considerar en el costo de un algoritmo: el tiempo de ejecución y el espacio de almacenamiento requerido para implementarlo.

La importancia del primero de ellos es muy evidente. El algoritmo deberá en primer lugar finalizar obteniéndose la solución deseada en un tiempo adecuado según los requerimientos del problema. Y en segundo lugar, es útil para comparar distintos algoritmos aplicables para solucionar el mismo problema. Como es bien sabido, la medida del tiempo de ejecución no se realiza en términos estrictos de tiempo, ya que este factor depende del computador utilizado, el lenguaje, el compilador, etc., es decir, depende de múltiples factores además del propio algoritmo. Además, un dato como que 'el algoritmo tarda cuatro semanas en un computador personal' no es muy aclaratorio. ¿Cuánto tardaría en un supercomputador de última generación? La respuesta no sería otra que 'mucho menos'. Por ello se estiman los costes en términos de parámetros invariantes del algoritmo, de manera que lo caractericen de forma generalizada.

El coste de los algoritmos se calcula en función de las operaciones características más frecuentes de los mismos. Así por ejemplo, en los algoritmos relacionados con el problema de clasificación las operaciones características más frecuentes son las comparaciones y los movimientos entre los elementos a clasificar. Sin embargo, no debe entenderse que todos los algoritmos se miden en estos términos. Así por ejemplo, los algoritmos utilizados en ciencia e ingeniería, incomprensibles lógicamente para aquellos que no están formados en una de estas disciplinas específicas, no suelen tener como operaciones características ni comparaciones ni movimientos entre elementos, sino sumas, multiplicaciones, etc. Un ejemplo típico, relacionado con el tratamiento digital de señales es la Transformada Rápida de Fourier, FFT. El coste viene dado por el número de operaciones (sumas y multiplicaciones) a realizar con números complejos. Este algoritmo es muy celebrado dentro de las disciplinas que lo utilizan debido a que su coste en cuanto a tiempo de ejecución es muy inferior al del algoritmo convencional, conocido como Transformada Discreta de Fourier, DFT. El análisis de ambos se realiza mediante las técnicas generales de algoritmos. Del análisis del mismo se deduce que la FFT tiene coste del orden

$N \log N$, donde N es la longitud o número de puntos, mientras que el orden de la DFT es de orden N^2 . Como puede observarse el coste es función de la variable de la que depende la operación característica, en este caso el número de muestras (N).

Para algunos algoritmos diferentes valores de los datos de entrada pueden presentar distintos costos. Típicamente éste es el caso de los algoritmos relacionados con las tareas de clasificación. En estos casos se calculan los costes de tres situaciones de interés: caso mejor, peor y promedio. El caso mejor es una cota inferior del coste presentado, mientras que el caso peor es una cota superior. Para calcularlos será necesario analizar el comportamiento del algoritmo en función de los valores de los datos posibles, determinando en que situación se realizan el mínimo y el máximo número de operaciones características más frecuentes, respectivamente. Típicamente un algoritmo de ordenación requerirá más o menos comparaciones y movimientos de los elementos a ordenar según los datos estén de partida completamente ordenados o completamente desordenados. El caso promedio presentará el orden de magnitud del costo en los casos en que los datos vengan ordenados aleatoriamente, donde se supondrá una distribución estadística uniforme. Debe observarse que no es suficiente con conocer el coste en el caso promedio. En un problema determinado podría suceder que los datos a tratar se presentasen habitualmente como en el peor caso, con lo que el coste promedio no sería significativo en absoluto y la elección de un algoritmo con coste mayor en el caso promedio pero menor en el peor caso sería más adecuada. Ejemplos concretos de este hecho se encontrarán en los temas siguientes.

A lo largo del texto se realizarán detalladamente los análisis del costo computacional de los algoritmos siempre que sea necesario, siguiendo las técnicas habituales de análisis de algoritmos, por lo que no se insistirá más en este aspecto. Para una documentación detallada sobre teoría general de algoritmos consúltese [Peña, 1998].

El segundo aspecto básico en el análisis de un algoritmo está relacionado con el espacio requerido para almacenar los datos. Esta cuestión es fundamental. Dados dos algoritmos con similar coste computacional, se elegirá obviamente aquel que requiera un menor almacenamiento. Sin embargo, la economía de almacenamiento es un factor que puede ser prioritario respecto al coste computacional. Así ocurre con los algoritmos de clasificación, por ejemplo. En general se dividen en dos ámbitos de actuación: algoritmos de clasificación sobre estructuras en memoria primaria y algoritmos de clasificación sobre estructuras en memoria secundaria. Los primeros se presentan en el segundo capítulo y los últimos en el tercero. Como se verá, existen algoritmos que se emplean para ordenación sobre memoria secundaria, como por ejemplo la mezcla natural, que presentan mejor coste computacional que algunos de los utilizados para ordenar sobre memoria primaria. Y sin embargo, no suele utilizarse el método de la mezcla natural para clasificar un arreglo. Esto es debido a que para ordenar n elementos se requerirá un arreglo de n elementos con cualquiera de los métodos de ordenación sobre arreglos, mientras que si se implementase la mezcla directa se necesitaría un arreglo de tamaño $2n$. Una implementación de la mezcla directa sobre arreglos puede encontrarse en [Wirth, 1987]. Por

tanto, este método suele descartarse por razones de economía de almacenamiento. Sólo cuando diferentes algoritmos, útiles para resolver un problema dado, presentan requerimientos de espacio de almacenamiento similares, el coste computacional es el factor determinante para elegir entre uno u otro.

4

CONCLUSIONES

En este capítulo se han presentado los conceptos fundamentales relacionados con el ámbito de las Estructuras de Datos y Algoritmos. En primer lugar, se ha mostrado la idoneidad de realizar un proceso de abstracción, de manera que la programación sea más sencilla y sistemática, siendo la operación subyacente a todos los conceptos que serán presentados a lo largo del texto. Así se han precisado los términos ‘estructuras de datos’, ‘tipos de datos abstracto’ y ‘tipos’. Seguidamente se han formulado los TDA básicos y bien conocidos de programación elemental. Los punteros y las listas enlazadas se han detallado especialmente, así como el concepto de asignación dinámica de memoria, dado que su manejo es imprescindible para la implementación dinámica de los TDA dinámicos, tal y como se verá en los capítulos cuatro y siguientes. En este sentido es importante insistir en la diferencia entre el carácter del TDA y su implementación, concepto fundamental y que nunca debe olvidarse.

En los programas de eliminación de un nodo tanto por el inicio como por el final (Programas 1.3 y 1.4) y en las correspondientes figuras que los ilustran (Figuras 1.14 y 1.15), no se ha considerado la posibilidad de que la lista esté vacía. Esto se ha hecho explícitamente para ilustrar el hecho de que existe cierta libertad para implementar las funciones sobre listas enlazadas. Sin embargo, esto no significa que no haya que considerar todos los casos y situaciones posibles. Así, si se decide utilizar la implementación de los Programas 1.3 y 1.4, es imprescindible, que previa a la eliminación del nodo, el programa principal llame a una función que indique si la lista está vacía y obre en consecuencia: eliminar o no, dar un mensaje de error, realizar una acción alternativa, etc.

CAPÍTULO 2

Clasificación en memoria principal

2.1 INTRODUCCIÓN

El problema de clasificación es, de hecho, un paradigma en el ámbito de la programación. Esto se debe fundamentalmente a dos razones. La primera es que una de las tareas de tipo general más ampliamente demandada en cualquier aplicación informática es la clasificación. Tanto es así que, recuérdese, el término ordenador hace referencia a este hecho.

La ordenación de los datos es tan importante debido a que es la forma de organizar la información que permite la búsqueda y recuperación de esta información cuando es solicitada. Supóngase por ejemplo el caso de un profesor de universidad que tiene 2000 alumnos en una asignatura, de los cuales 1000 se presentan al examen. Llegado el momento un alumno solicita la revisión de su examen. Entonces hay que buscarlo, y evidentemente buscar uno entre 1000 requiere su tiempo. Por tanto, es razonable pensar en una forma de organizar estos exámenes con el fin de buscar alguno de ellos rápidamente cuando sea necesario. La manera más inmediata es hacerlo por orden alfabético. Como puede intuirse, la tarea de ordenación será costosa, pero una vez organizados la búsqueda será mucho más eficaz. Podrían establecerse otros criterios de ordenación. Si por ejemplo, esta universidad fuese de ámbito estatal y dispusiese de Centros Asociados en los que los alumnos realizasen exámenes en numerosas provincias, e incluso en el extranjero, la ordenación se podría efectuar por Centros Asociados, reduciendo el problema.

En este capítulo se presentan los algoritmos básicos para la clasificación sobre arreglos. Este TDA se implementa de forma inmediata en memoria primaria, y es aquí donde estos algoritmos tienen su ámbito de aplicación. Su necesidad es tan patente que hoy en día todos los compiladores de lenguajes de alto nivel incorporan funciones en sus librerías básicas que implementan estos algoritmos, e incluso pueden encontrarse incorporadas en programas específicos para el cálculo matemático, como Matlab¹.

La importancia de la tarea de clasificación deberá tenerse en cuenta a lo largo de todo el texto. Esto es debido a que no sólo los algoritmos sobre arreglos presentados en este capítulo tienen que ver con ella, sino que de hecho, todos los TDA presentados en los capítulos posteriores están relacionados de una u otra forma con esta tarea.

En este capítulo se presentan los métodos básicos para la clasificación en memoria principal. Cuando la ordenación se realiza sobre estructuras en memoria primaria se conoce como ordenación *interna*, mientras que cuando se realiza sobre estructuras en memoria secundaria se denomina ordenación *externa*. La ordenación externa será necesaria cuando el número de elementos a ordenar es demasiado elevado en relación con los recursos de memoria principal del sistema.

Antes de nada es conveniente precisar el problema de clasificación.

Definición 2.1: Sea una colección de n elementos, $a_1, a_2, \dots, a_n$, tal que cada elemento a_i tiene una clave asociada, $clave[a_i]$. Se dice que existe una *relación de orden* sobre las claves si para tres valores cualesquiera de la clave, $clave[a_i]$, $clave[a_m]$ y $clave[a_n]$ se satisface que:

1. $clave[a_i] < clave[a_m]$ o $clave[a_i] = clave[a_m]$ o $clave[a_i] > clave[a_m]$
2. Si $clave[a_i] < clave[a_m]$ y $clave[a_m] < clave[a_n]$ entonces $clave[a_i] < clave[a_n]$

Así, el problema de ordenación se define de la siguiente forma:

Definición 2.2: Dada una colección de n elementos, $a_1, a_2, \dots, a_n$, tal que cada elemento a_i tiene una clave asociada, $clave[a_i]$, y entre ellas existe una *relación de orden*, el problema de clasificación consiste en reorganizar la colección de n elementos tal que $clave[a_{p(1)}] < clave[a_{p(2)}] < \dots < clave[a_{p(n)}]$

donde $p(i)$ es la posición i dentro de la colección, y $a_{p(i)}$ es el elemento en la posición i de la colección.

En la nomenclatura de los algoritmos de clasificación dos términos informan sobre las características del mismo. El primero de ellos es la *estabilidad*. Se dice que el algoritmo es estable si el orden relativo de los elementos con llaves iguales permanece inalterado en el proceso de clasificación, es decir, no intercambia elementos

1. MATLAB es una marca registrada de MathWorks

con claves iguales. El segundo es el comportamiento *natural*. Si el mejor caso se da cuando el arreglo está inicialmente ordenado, entonces se dice que el algoritmo tiene un comportamiento natural.

En este capítulo se presentan los métodos básicos para la clasificación en memoria principal. Las colecciones de elementos a clasificar se almacenarán en arreglos, de manera que cualquier elemento puede ser accedido en el mismo tiempo, es decir, con el mismo costo. La definición de tipos y variables será, por tanto, en Modula2:

Definición de tipos:

```
TYPE Tipo_datos = RECORD
    llave : Tipo_llave;
    (*otros componentes*)
END
```

Declaración de variables:

```
VAR a: ARRAY [1..n] OF Tipo_datos;
```

En la Figura 2.1 se muestra el caso general de un arreglo de registros con varios campos donde uno de ellos se utiliza como campo llave.

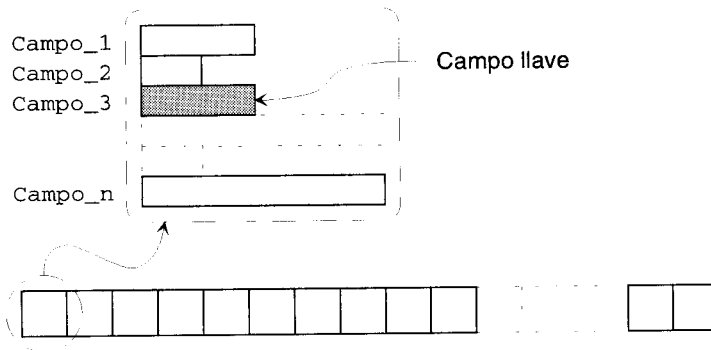


Figura 2.1 Ejemplo de estructura clasificable por un campo llave.

La taxonomía realizada en términos de métodos de clasificación directa y métodos avanzados se debe a que los primeros explotan las ideas más intuitivas (directas) y en general presentan un coste computacional de orden n^2 , mientras que los segundos reducen este coste utilizando ideas y técnicas más elaboradas. Para todos ellos se presenta la idea en que se basa el algoritmo, un ejemplo de aplicación, la implementación del algoritmo en Modula2 y finalmente el análisis del mismo.

A lo largo de los ejemplos de los métodos de clasificación se utilizarán Enteros, ya que presentan una relación de orden bien conocida, y por simplicidad de código no se considerarán otros componentes. Así, la declaración de tipos es la siguiente:

```
TYPE Tipo_datos = Integer;
```

Sin embargo, no debe olvidarse en ningún caso que estos mismos algoritmos son aplicables a cualquier TDA que presente una relación de orden, como por ejemplo los caracteres según la secuencia de cotejo de su implementación, y en el caso más general serán registros con un campo llave de algún tipo ordinal, tal y como se mostró en la Figura 2.1.

2.2 MÉTODOS DIRECTOS DE CLASIFICACIÓN

Como ya se ha indicado en la introducción, los métodos directos de clasificación se basan en ideas sencillas. Básicamente se fundamentan en tres acciones, que dan nombre a cada uno de los métodos fundamentales:

1. *Clasificación por inserción*

La idea en que se basan consiste en insertar un elemento dado en el lugar que le corresponda de una parte ya ordenada del arreglo. En este apartado se tratan los métodos de inserción directa y binaria.

2. *Clasificación por selección*

La idea en que se basa consiste en seleccionar el menor elemento de una parte desordenada del arreglo y colocarlo en la posición del primer elemento no ordenado.

3. *Clasificación por intercambio*

La idea en que se basan consiste en comparar pares de elementos adyacentes e intercambiarlos si están desordenados. Dentro de los métodos de intercambio se incluyen aquí los de intercambio directo o método de la burbuja y el de sacudida que es una variante de éste.

Ninguno de estos métodos resulta el ideal en todas las situaciones sino que cada uno de ellos puede presentar mejoras en algún aspecto, hecho que se describirá detalladamente en cada caso.

2.2.1 CLASIFICACIÓN POR INSERCIÓN DIRECTA

La idea básica es insertar un elemento dado en el lugar que le corresponda de una parte ya ordenada del arreglo. En un primer paso se considera que la parte ordenada está formada por el primer elemento del arreglo. Entonces se elige el elemento de la posición 2 y se inserta en la parte ordenada, de manera que si debe estar en la posición anterior se desplaza el elemento de la posición 1 hacia la derecha, haciéndole sitio, y se inserta en la posición 1. Como puede observarse se requerirá de una variable temporal que almacene el elemento a insertar. Seguidamente se considera la parte del arreglo $a[1] \dots a[2]$ ordenada, se elige el tercer elemento y se siguen las

mismas acciones. Este método se repite hasta que el último elemento, $a[n]$, haya sido tratado. En resumen, el algoritmo puede describirse de la siguiente forma:

- Tomar un elemento en la posición i .
- Buscar su lugar en las posiciones anteriores.
- Mover hacia la derecha los restantes.
- Insertarlo.

La Figura 2.2 ilustra el funcionamiento del método de inserción directa. Partiendo del estado inicial, se supone que el primer elemento está en su sitio (el número 8 en este ejemplo) y que por tanto, la parte de arreglo correspondiente (formada únicamente por este primer elemento) está ordenada. Los elementos ordenados aparecen sombreados en la Figura 2.2. A continuación se toma el siguiente elemento (14) y se busca su posición dentro de la parte ordenada, pero como es mayor que el ocho se inserta en su posición inicial. Se continúa después con el siguiente elemento (5) y se procede de la misma forma: se busca su posición en la parte ordenada y se inserta desplazando el resto de elementos hacia los índices superiores (a la derecha en la figura). En este caso ha sido necesario desplazar los elementos 8 y 14, lo que se indica con una pequeña flecha debajo de los mismos. El proceso continúa de la misma manera hasta que se inserta el último elemento. La Figura 2.2 muestra todos los pasos necesarios y en cada uno de ellos se indica con un círculo el elemento que se inserta y con flechas los elementos que es necesario desplazar para realizar la inserción.

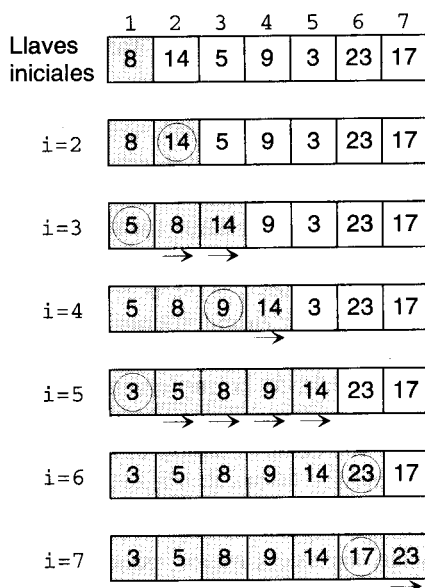


Figura 2.2 Descripción de la clasificación por inserción directa. Con un círculo se indica el elemento que se inserta en cada pasada y las flechitas indican los elementos que ha sido necesario desplazar para realizar la inserción.

El Programa 2.1 implementa en Modula2 este algoritmo. Como puede observarse en esta implementación `a[0]` se utiliza como variable temporal y como centinela en el bucle `WHILE`. Para hacer esto es necesario ampliar el arreglo en un elemento de manera que la variable será `VAR a: ARRAY [0..n] OF Tipo_datos`. Esto obliga a copiar el arreglo y a ordenar sobre este nuevo arreglo, lo que se puede evitar fácilmente si esta variable temporal se considera de forma independiente.

Programa 2.1 Implementación del algoritmo de inserción directa.

```

VAR
  i,j: Tipo_indice;
BEGIN
  FOR i:=2 TO n DO
    a[0]:=a[i]; (* Garantiza la salida del while *)
    j:=i;
    WHILE a[0]<a[j-1]DO
      a[j]:=a[j-1]; (* movimientos *)
      j:=j-1
    END;
    a[j]:=a[0] (* inserción *)
  END
END;
```

Análisis de la clasificación por inserción directa

Las operaciones más frecuentes, características del algoritmo, son la comparación de elementos del arreglo ($a[0] < a[j-1]$), y los movimientos entre elementos del mismo ($a[0] := a[i]$, $a[j] := a[j-1]$ y $a[j] := a[0]$). En ocasiones se confunden los movimientos con las asignaciones. Debe observarse que los movimientos entre elementos del arreglo se realizarán mediante asignaciones en la implementación del algoritmo, pero no todas las asignaciones corresponden a movimientos. Así por ejemplo, las asignaciones $j := j-1$ o $j := i$ no realizan movimiento alguno entre los elementos del arreglo. Evidentemente son operaciones auxiliares necesarias para la correcta implementación pero no caracterizan al algoritmo. En un estudio detallado del programa, es decir, de la implementación del algoritmo, sería necesario analizar su contribución al costo computacional, pero en un análisis asintótico en el que interesa el orden de los costos cuando el número de elementos es elevado, esta aportación es irrelevante.

El análisis del algoritmo requiere el cálculo del número de comparaciones y de movimientos en los casos mejor, peor y promedio. Analicemos en primer lugar a qué situación inicial corresponde cada uno de ellos. El caso mejor será aquel en el que menos comparaciones y movimientos se realicen. Atendiendo a la descripción del algoritmo puede observarse que si el arreglo está ordenado inicialmente entonces no será necesario realizar movimientos entre elementos del arreglo y las comparaciones a realizar serán sólo entre el último elemento de la parte del arreglo

ordenada y el elemento a insertar. Por tanto, el mejor caso se dará cuando el arreglo esté inicialmente ordenado. Del mismo modo, si las llaves iniciales están totalmente desordenadas, es decir, el arreglo está ordenado en sentido contrario (decreciente), se deberán realizar siempre todas las comparaciones ($a[0] < a[j-1]$) para encontrar la posición a insertar, y se tendrán que desplazar todos los elementos de la parte ordenada del arreglo ($a[j] := a[j-1]$). Por tanto, el número de comparaciones y movimientos será el mayor posible. Entonces, el peor caso se dará cuando las llaves iniciales del arreglo estén completamente desordenadas. El caso promedio es evidentemente cuando las llaves del arreglo inicial están ordenadas aleatoriamente.

Siempre que se consideren llaves ordenadas aleatoriamente, se supondrá que la distribución estadística es uniforme. Otras distribuciones estadísticas pueden presentar otras propiedades al verse alterado el cálculo de probabilidades.

Consideraremos a continuación las comparaciones y movimientos para cada uno de los tres casos: mejor, peor y promedio. Así, atendiendo al Programa 2.1 el análisis del algoritmo es el siguiente:

- Comparaciones

a) *Mejor caso*: las llaves iniciales están ordenadas.

En esta situación la comparación ($a[0] < a[j-1]$) en el bucle WHILE sólo se realiza una vez en cada paso. Como el bucle FOR se realiza $n-1$ veces, el número de comparaciones mínimo será:

$$C_{min} = n - 1$$

b) *Peor caso*: las llaves iniciales están completamente desordenadas.

En esta situación en el primer pase ($i=2$) se realizan dos comparaciones: $a[1]$ con $a[0]$ y $a[0]$ con $a[0]$ (acción de centinela). En el segundo pase ($i=3$) se realizan tres comparaciones ($a[2]$, $a[1]$ y $a[0]$ con $a[0]$), en el pase $k-1$ ($i=k$) se realizan k comparaciones y en el paso $n-1$ se realizan n comparaciones. En conclusión, el número de comparaciones máximo será la suma de estas comparaciones en cada paso:

$$C_{max} = 2 + 3 + 4 + \dots + n$$

Esta expresión es una suma aritmética, con lo que de este sencillo razonamiento se puede deducir que el número de comparaciones máximo es de orden n^2 (ya que todas las sumas aritméticas son de orden n^2).

Aplicando la expresión de la suma aritmética se deduce que:

$$C_{max} = 2 + 3 + \dots + n = 1 + 2 + \dots + n - 1 = \sum_{i=1}^n i - 1 = \frac{(n^2 + n - 2)}{2}$$

y por tanto:

$$C_{max} = \frac{(n^2 + n - 2)}{2}$$

c) *Caso promedio*: las llaves iniciales están aleatoriamente ordenadas.

En el caso promedio las llaves del arreglo inicial están ordenadas aleatoriamente. Por ello para realizar el análisis habrá que hacer uso de argumentos probabilísticos atendiendo a la comparación ($a[0] < a[j-1]$). La probabilidad de que dos números enteros cualesquiera estén ordenados es la misma de que estén desordenados, por tanto, la probabilidad de que la comparación tenga como resultado True (o False) es de $1/2$. Por lo que el número de comparaciones promedio será:

$$C_{prom} = \frac{1}{2} C_{max}$$

es decir:

$$C_{prom} = \frac{(n^2 + n - 2)}{4}$$

- Movimientos

a) *Mejor caso*: las llaves están inicialmente ordenadas.

En esta situación nunca se entra en el bucle WHILE, por tanto, el movimiento ($a[j] := a[j-1]$) no se realiza nunca. Sin embargo, el bucle FOR se realiza $n-1$ veces, y dentro de él se realizan dos movimientos ($a[0] := a[i]$ y $a[j] := a[0]$). Por tanto:

$$M_{min} = 2(n-1)$$

b) *Peor caso*: las llaves iniciales están completamente desordenadas.

En este caso además de los $2(n-1)$ movimientos internos al bucle FOR y externos al WHILE, que se harán siempre, se realizará una asignación ($a[j] := a[j-1]$) cada vez que se entre en el WHILE. Como en el peor de los casos se realizan todas las comparaciones, se entrará en el WHILE en todas ellas excepto cuando se compara el centinela y se produce la salida del bucle. Así, para $i=2$ se realiza un movimiento (recuérdese que se efectuaban dos comparaciones), para $i=3$ habrá dos movimientos (tres comparaciones), y así sucesivamente hasta que para $i=n$ se realizan $n-1$ movimientos (n comparaciones). Por tanto:

$$M_{max(while)} = 1 + 2 + \dots + n - 1 = \sum_{i=1}^{n-1} i = \sum_{i=1}^n i - n = \frac{n(n+1)}{2} - n = \frac{n^2 - n}{2}$$

Sumando finalmente los $2(n-1)$ movimientos mínimos se obtiene:

$$M_{max} = \frac{(n^2 + 3n - 4)}{2}$$

c) *Caso promedio*: las llaves iniciales están aleatoriamente ordenadas.

Teniendo en cuenta que cada vez que se ejecute una comparación (excepto la del centinela) se realiza un movimiento en el WHILE, el número de movimientos promedio en el WHILE será:

$$M_{prom(while)} = C_{prom}$$

y sumando los $2(n-1)$ movimientos que se realizan fuera del bucle WHILE se obtiene:

$$M_{prom} = M_{prom(while)} + 2(n-1) = \frac{n^2 + 9n - 10}{4}$$

2.2.2 CLASIFICACIÓN POR INSERCIÓN BINARIA

El método de inserción directa es susceptible de una mejora inmediata en la tarea de búsqueda de la posición en la que debe insertarse el elemento a ordenar dentro de la parte del arreglo ya ordenada. Como puede observarse el método de inserción directa realiza una búsqueda secuencial comenzando en el extremo derecho de la parte del arreglo ya ordenada y comparando todos y cada uno de los elementos hasta encontrar la posición a insertar. Como es bien sabido, esta búsqueda puede realizarse mucho más eficazmente utilizando una búsqueda dicotómica o binaria. Aplicándola a la ordenación por inserción, se toma el elemento que ocupa la posición central de la parte ordenada y se compara con el elemento a insertar. Si éste es mayor se descarta la parte izquierda de la parte ordenada, ya que será mayor que todos ellos, y si es menor se descarta la parte derecha ya que será menor que todos ellos. Aplicando sucesivamente el mismo proceso a las partes elegidas se obtiene la posición en la que se debe insertar.

En resumen, el algoritmo puede describirse de la siguiente forma:

- Tomar un elemento en la posición i .
- Buscar *dicotómicamente* su lugar en las posiciones anteriores.
- Mover hacia la derecha los restantes.
- Insertarlo.

La Figura 2.3 muestra uno de los pasos en un proceso de inserción binaria. En esta figura se muestra en primer lugar el estado inicial del paso decimosexto ($i=17$) en el que hay que insertar el número 15, indicado mediante un pequeño triángulo bajo él. Con fondo sombreado aparece la parte que falta por ordenar. En un primer paso se compara el 15 (elemento a insertar) con el 22 que es el que está en el centro de la parte ordenada ($(L+R) \text{ DIV } 2 = (1+17) \text{ DIV } 2 = 9$). Obsérvese que $R=17$ con lo que el 15 se considera en la parte ordenada. Como es menor se descarta la parte

derecha (elementos 23 a 50) lo que se muestra en la figura con trazo atenuado y se realiza la búsqueda en la izquierda (3 ... 22). Seguidamente se compara con el 13 y como es mayor se realiza la búsqueda en la parte derecha (14 ... 22) descartando el resto de la parte izquierda. Siguiendo con este esquema, al final se obtiene una parte compuesta por un único elemento (17) que indica la posición en que debe insertarse el número.

El Programa 2.2 implementa en Modula2 este algoritmo.

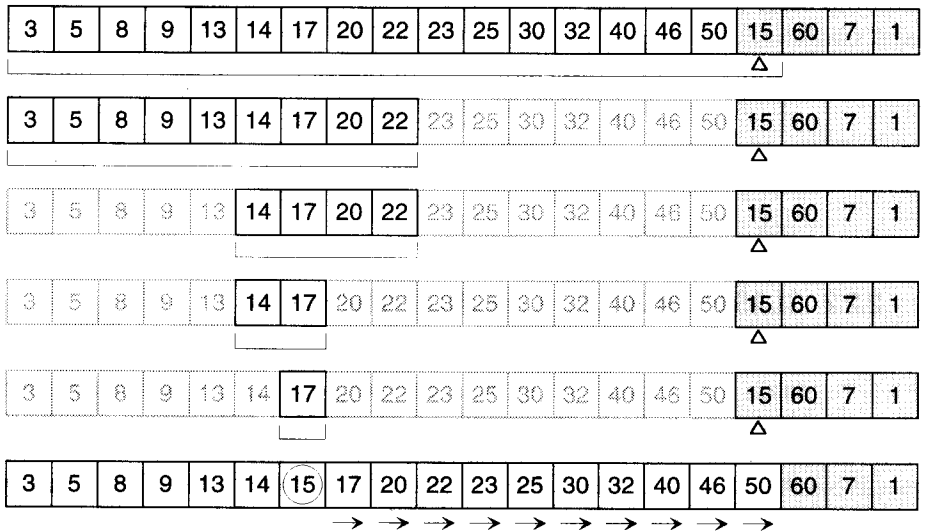


Figura 2.3 Búsqueda dicotómica de la posición para la inserción.

Programa 2.2 Implementación del algoritmo de inserción binaria.

```

VAR
  i,j,m,L,R: Tipo_indice;x: Tipo_datos;
BEGIN
  FOR i:=2 TO n DO
    x:=a[i];
    (* L: índice al elemento de la parte izquierda del
       subarray en el que se realiza la búsqueda en cada paso *)
    L:=1;
    (* R: índice al elemento de la parte derecha del
       subarray en el que se realiza la búsqueda en cada paso *)
    R:=i;
    WHILE L<R DO
      m:=(L+R)DIV 2;

```

```

    IF a[m] <= x THEN
        L := m + 1
    ELSE
        R := m
    END
END;
FOR j := i TO R + 1 BY -1 DO
    a[j] := a[j - 1] (* desplazamiento *)
END;
a[R] := x; (* inserción *)
END
END;

```

Análisis de la clasificación por inserción binaria

El número de movimientos no mejora significativamente respecto al método de inserción directa, y seguirán siendo de orden n^2 , ya que la única modificación realizada tiene que ver con la tarea de búsqueda en la que sólo intervienen las comparaciones.

El número de comparaciones no depende significativamente del orden inicial de los datos a clasificar. Por ello puede calcularse aproximadamente el número de comparaciones de forma general de la siguiente manera. Al realizar la búsqueda binaria se divide una longitud n por la mitad un número determinado de veces. Así:

$$n = 2^{(\text{número de veces que se divide por 2})}$$

Por tanto:

$$N = \log n \quad (2.1)$$

donde N es el número de veces que se divide por 2, (y log es el logaritmo en base 2 como es habitual).

La Figura 2.4 muestra un ejemplo de este hecho. Como puede observarse, un arreglo de $n=16$ requiere cuatro divisiones ($4=\log 16$).

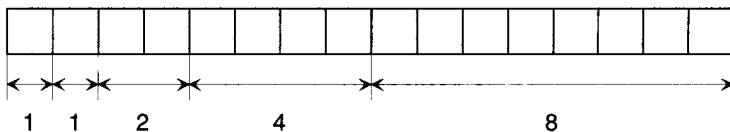


Figura 2.4 Ejemplo de división binaria en un arreglo de 16 elementos.

La inserción se producirá cuando la parte del arreglo en la que hay que buscar tenga longitud 1 ($L=R$). Atendiendo a la Expresión 2.1 para una longitud i se requerirán $\log i$ divisiones hasta obtener la posición a insertar (longitud 1). Sumando para todos los i :

$$C = \sum_{i=1}^n \lceil \log i \rceil$$

y aproximando por la integral se obtiene:

$$C = \int_1^n (\log x) dx \quad (2.2)$$

Para resolver esta integral recuérdese que:

$$\int_1^n (\ln x) dx = x(\ln x - 1) \Big|_1^n$$

Como $\log x = a$ entonces $x = 2^a$. Aplicando el logaritmo neperiano a ambos términos de la igualdad se obtiene $\ln x = a \ln 2$, y por tanto:

$$a = \log x = \frac{\ln x}{\ln 2}$$

que aplicándolo a la integral a resolver (2.2) se obtiene:

$$\int_1^n (\log x) dx = \frac{1}{\ln 2} \int_1^n \ln x dx = \frac{1}{\ln 2} x(\ln x - 1) \Big|_1^n$$

donde teniendo en cuenta que $\frac{1}{\ln 2} = \log e$ se obtiene finalmente:

$$C = n(\log n - \log e) + \log e \quad (2.3)$$

Este número de comparaciones es una aproximación (obsérvese la sustitución del sumatorio por la integral). Esto no es en absoluto un inconveniente ya que, recuérdese, se trata de un análisis asintótico y el orden, $n \log n$, no varía por el hecho de ser un cálculo aproximado.

El número de comparaciones ha sido calculado considerando que no varían dependiendo del orden inicial de los datos. Esto no es del todo cierto debido al

carácter truncador de la división. Obsérvese que en la división por dos se toma como valor el cociente (división entera), despreciando el resto. Evidentemente el resto puede ser cero o uno, y cuando es uno se está truncando el resultado real de la división por 2. Por ello el número de comparaciones calculadas puede ser una más de las calculadas anteriormente. El truncamiento tiene además como consecuencia que una posición de inserción localizada en el extremo inferior se encuentra en promedio antes que una posición de inserción en el extremo superior del arreglo. Por tanto, se encuentran antes las posiciones de los elementos que están inicialmente muy lejos de su posición a insertar. En conclusión el algoritmo tiene un comportamiento no natural ya que el mejor caso se da cuando están desordenados y el peor cuando están ordenados. Sin embargo, estas consideraciones no tienen relevancia extrema dentro del análisis asintótico realizado (se está diferenciando el caso mejor y el peor en términos de una comparación más o menos debido al carácter truncador de la división). En conclusión, la Expresión 2.3 puede considerarse adecuada para todos los casos.

2.2.3 CLASIFICACIÓN POR SELECCIÓN DIRECTA

La idea básica es seleccionar el menor elemento de una parte desordenada del arreglo y colocarlo en la posición del primer elemento no ordenado. En un primer paso se recorre el arreglo hasta encontrar el elemento menor. Para ello se coloca el primer elemento en una variable temporal y se va comparando con los demás elementos del arreglo tal que si se encuentra uno menor se asigna a la variable temporal. Recorrido todo el arreglo, el elemento de la variable temporal (que será el menor) se intercambia con el de la primera posición. Seguidamente se considera únicamente la parte del arreglo no ordenado y se repite el proceso de búsqueda del menor, y así sucesivamente. En conclusión, el algoritmo puede describirse de la siguiente forma:

- Seleccionar el elemento menor de la parte del arreglo no ordenada.
- Colocarlo en la primera posición de la parte no ordenada del arreglo.

La Figura 2.5 ilustra la realización de una clasificación por selección directa. Se muestra en primer lugar la situación inicial de las claves. En la primera pasada debemos recorrer la estructura completamente para localizar el menor de todos los elementos, que en este caso resulta ser el número 3. Una vez localizado, se intercambia con el que ocupa la primera posición. De esta forma hemos colocado el primer elemento (el menor) en su posición correcta (la primera). A continuación, buscamos nuevamente el elemento menor dentro de la parte de arreglo que queda por ordenar. Este número buscado es ahora el 5 y se sustituye por el que está en la segunda posición (el 14) con lo que tenemos ya dos elementos en su posición correcta. Continuando de la misma forma, podemos obtener el arreglo completamente ordenado en $n-1$ pasadas (búsqueda del menor e intercambio).

El Programa 2.3 implementa en Modula2 este algoritmo.

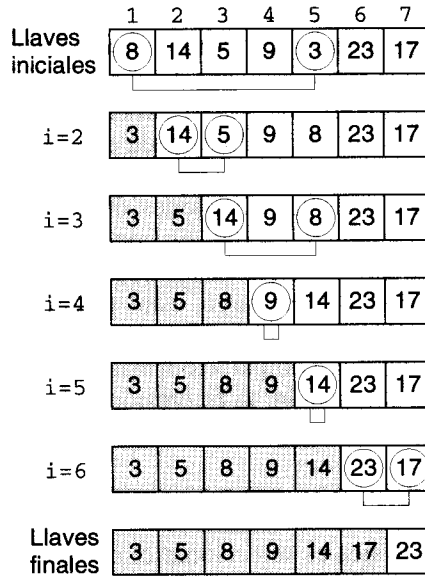


Figura 2.5 Descripción de la clasificación por selección directa. Los elementos sombreados indican la parte ya ordenada. Los elementos a intercambiar en cada etapa (el menor y el primero de la parte desordenada) se indican por círculos unidos por una línea.

Programa 2.3 Implementación del algoritmo de selección directa.

```

VAR
  i,j,k: Tipo_indice; x: Tipo_datos;
BEGIN
  FOR i:=1 TO n-1 DO
    k:=i;
    x:=a[i];          (* selección inicial *)
    FOR j:=i+1 TO n DO  (* búsqueda en secuencia fuente *)
      IF a[j]<x THEN
        k:=j;
        x:=a[k] (* selección *)
      END
    END;
    a[k]:=a[i];      (*intercambio*)
    a[i]:=x;
  END
END;

```

Obsérvese que el método de clasificación por selección directa presenta un tratamiento del arreglo completamente contrario al de inserción directa. En la inserción se toma en cada paso *un* elemento de la parte no ordenada y se recorren *todos* los de la parte ordenada mientras que en la selección se recorren *todos* los de la parte desordenada para intercambiarlo con *uno*, el siguiente a la parte ordenada que es el primero de la parte desordenada.

Análisis de la clasificación por selección directa

Las operaciones más frecuentes, características del algoritmo, son la comparación de elementos del arreglo ($a[j] < x$), y los movimientos entre elementos del arreglo ($x := a[i]$, $x := a[k]$, $a[k] := a[i]$ y $a[i] := x$), y como es habitual el análisis asintótico requiere el cálculo del número de comparaciones y de movimientos en los casos mejor, peor y promedio sin considerar las operaciones auxiliares.

En este caso, es obvio que el número de comparaciones es independiente del orden inicial de las llaves del arreglo, ya que en cualquier caso se realizará la búsqueda del menor elemento en toda la parte no ordenada del arreglo. Sin embargo, para los movimientos sí se distinguen diferentes casos. Cuando el arreglo está completamente ordenado no hay sucesivas elecciones de un menor, es decir, la llave asignada inicialmente a la variable temporal no será sustituida por ningún elemento de la parte no ordenada. Sin embargo, en el caso de que inicialmente los elementos del arreglo estén completamente desordenados (ordenados en orden inverso) todos ellos serán elegidos como el menor, realizándose todas las veces la asignación a la variable temporal. Así el mejor caso para los movimientos se dará cuando el arreglo esté inicialmente ordenado y el peor caso cuando esté completamente desordenado presentando, por tanto, un comportamiento natural.

Atendiendo al Programa 2.3 el análisis del algoritmo es el siguiente:

- Comparaciones

Como se ha comentado, es independiente del orden inicial de las llaves y por tanto su número es el mismo para todos los casos. Esto puede observarse claramente en la implementación del algoritmo (Programa 2.3) en la que las dos estructuras de iteración son bucles FOR. Así, la comparación ($a[j] < x$) se realizará $n-1$ veces para la primera elección del elemento menor, $n-2$ para la segunda elección y así sucesivamente hasta elegir el elemento menor $n-1$, en la que se realizará una comparación. Sumando todas ellas se observa que es una suma aritmética y por tanto, el número de comparaciones será de orden n^2 . Así tenemos:

$$C = 1 + 2 + \dots + n - 1 = \frac{n^2 - n}{2}$$

es decir:

$$C = \frac{n^2 - n}{2}$$

- *Movimientos*

a) *Mejor caso*: las llaves iniciales están ordenadas.

En esta situación nunca se entra en el IF, por tanto, el movimiento $(x:=a[k])$ no se realiza nunca. Sin embargo, el bucle FOR externo se realizará $n-1$ veces, y dentro de él se ejecutan tres movimientos $(x:=a[i], a[k]:=a[i] \text{ y } a[i]:=x)$. Por tanto:

$$C_{min} = 3(n - 1)$$

b) *Peor caso*: las llaves iniciales están completamente desordenadas.

En este caso, además de los $3(n-1)$ movimientos internos al bucle FOR externo, que se harán siempre, se realizará una asignación $(x:=a[k])$ cada vez que se entre en el IF. Sin embargo, esto no quiere decir que se harán todas las elecciones desde $i+1$ hasta n . Obsérvese que al intercambiar la primera posición con el menor (última posición) ambos quedan ya colocados, puesto que al estar inicialmente en orden inverso el que estaba en la primera posición era el mayor. Del mismo modo, el segundo menor estará en la posición $n-1$, y al intercambiarlo con el segundo quedarán ambos colocados ya que éste es el segundo mayor. De esta manera, se van ordenando de dos en dos con lo que el número máximo de ordenaciones de parejas es $n/2$. La Figura 2.6 muestra este hecho.

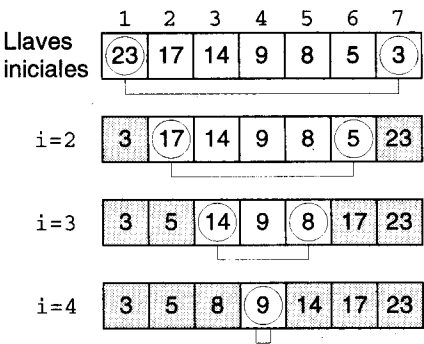


Figura 2.6 Selección directa: análisis de los movimientos en el peor caso.

Según esto, el número máximo de movimientos que se realizan dentro del IF será:

$$M_{max(i\,f)} = 1 + 3 + 5 + \dots + \frac{n}{2}$$

que es una suma aritmética y por tanto, de orden n^2 . Sumando los $3(n-1)$ del FOR externo y aplicando la expresión de la suma aritmética se obtiene:

$$M_{max} = 3(n - 1) + \frac{n^2}{4}$$

c) *Caso promedio*: las claves iniciales están aleatoriamente ordenadas.

En el caso promedio las claves del arreglo inicial están ordenadas aleatoriamente y habrá que utilizar argumentos probabilísticos. El movimiento a analizar es $x := a[k]$. La comparación $a[j] < x$ compara la variable temporal x con todos y cada uno de los elementos, realizándose siempre. Sin embargo, sólo se ejecutará la asignación cuando x sea el menor elemento entre los que se realiza la búsqueda. Así, para que se ejecute el movimiento en un primer paso, el segundo elemento deberá ser menor que el primero. Suponiendo que todos los números enteros son equiprobables, la probabilidad de que un entero sea menor que otro es $1/2$. En un segundo paso, el tercer elemento tendría que ser menor que los dos primeros para ser seleccionado, y la probabilidad de que un entero sea el menor de tres dados es de $1/3$. En un tercer paso el cuarto elemento debería ser el menor de los cuatro primeros, y la probabilidad de que esto suceda es de $1/4$. Por tanto, el número de movimientos promedio en cada pase i será:

$$M_{prom,i} = \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{i}$$

Así:

$$M_{prom,i} = \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{i} = H_i - 1$$

donde H_i es el i -ésimo número armónico. Teniendo en cuenta que

$$H_i = \ln i + \gamma + \frac{1}{2i} - \frac{1}{12i^2} + \dots$$

donde $\gamma = 0.577216\dots$ es la constante de Euler y despreciando los términos de orden $O\left(\frac{1}{n^k}\right)$, $k > 0$, el número de movimientos promedio en cada pase i será

$$M_{prom,i} = \ln i + \gamma + 1$$

Sumando para todos los pases obtenemos:

$$M_{prom} = n(\gamma + 1) + \sum_{i=1}^n \ln i$$

Finalmente, aproximando por la integral:

$$M_{prom} = n(\gamma + 1) + \int_1^n \ln x dx$$

y resolviendo $\int_1^n (\ln x) dx = x(\ln x - 1) \Big|_1^n$ como ya se hizo con la Integral 2.2 se obtiene:

$$M_{prom} = n(\gamma + 1) + x(\ln x - 1) \Big|_1^n \cong n(\ln n + \gamma)$$

En conclusión:

$$M_{prom} \cong n(\ln n + \gamma)$$

Este resultado permite concluir que el método de selección directa es preferible al de la inserción directa ya que su número de movimientos promedio es de orden $\ln n$. Pero si se compara con la inserción binaria cabe preguntarse cuál es más adecuado ya que tiene un coste de orden n^2 para los movimientos pero $\log n$ para las comparaciones, mientras que en la selección directa es de orden n^2 para las comparaciones y $\ln n$ para los movimientos. La decisión en esta situación viene determinada por consideraciones de bajo nivel. En general, la implementación de una comparación tiene un coste inferior al de un movimiento. Por tanto, la selección directa será la opción a elegir en general, aunque la inserción es algo más rápida cuando las llaves se clasifican predominantemente al principio.

En algunas ocasiones puede sorprender que en el análisis del caso promedio se obtenga un coste significativamente menor que en el peor cuando en este último sólo se realizan asignaciones de selección ($x := a[k]$) hasta $n/2$, mientras que en el primero hay que analizar hasta n . Debe observarse que en el caso peor aunque sólo sea hasta $n/2$, se ejecutan todas, mientras que en el promedio aunque es hasta n sólo se realizan según los argumentos probabilísticos. Por ello, el hecho de que sea hasta $n/2$ no implica una reducción sensible del coste computacional.

2.2.4 CLASIFICACIÓN POR INTERCAMBIO DIRECTO

La idea básica es comparar parejas de elementos contiguos e intercambiarlos si es necesario. En un primer paso se recorre el arreglo en sentido ascendente comenzando por el principio, comparando parejas de llaves. Si el elemento anterior es mayor se intercambian, si es menor no, y en ambos casos se sigue avanzando. En conclusión, el algoritmo puede describirse de la siguiente forma:

- Comparar pares de elementos contiguos.
- Intercambiarlos si están desordenados.

La Figura 2.7 muestra cada paso hasta obtener la ordenación final de un arreglo de siete elementos.

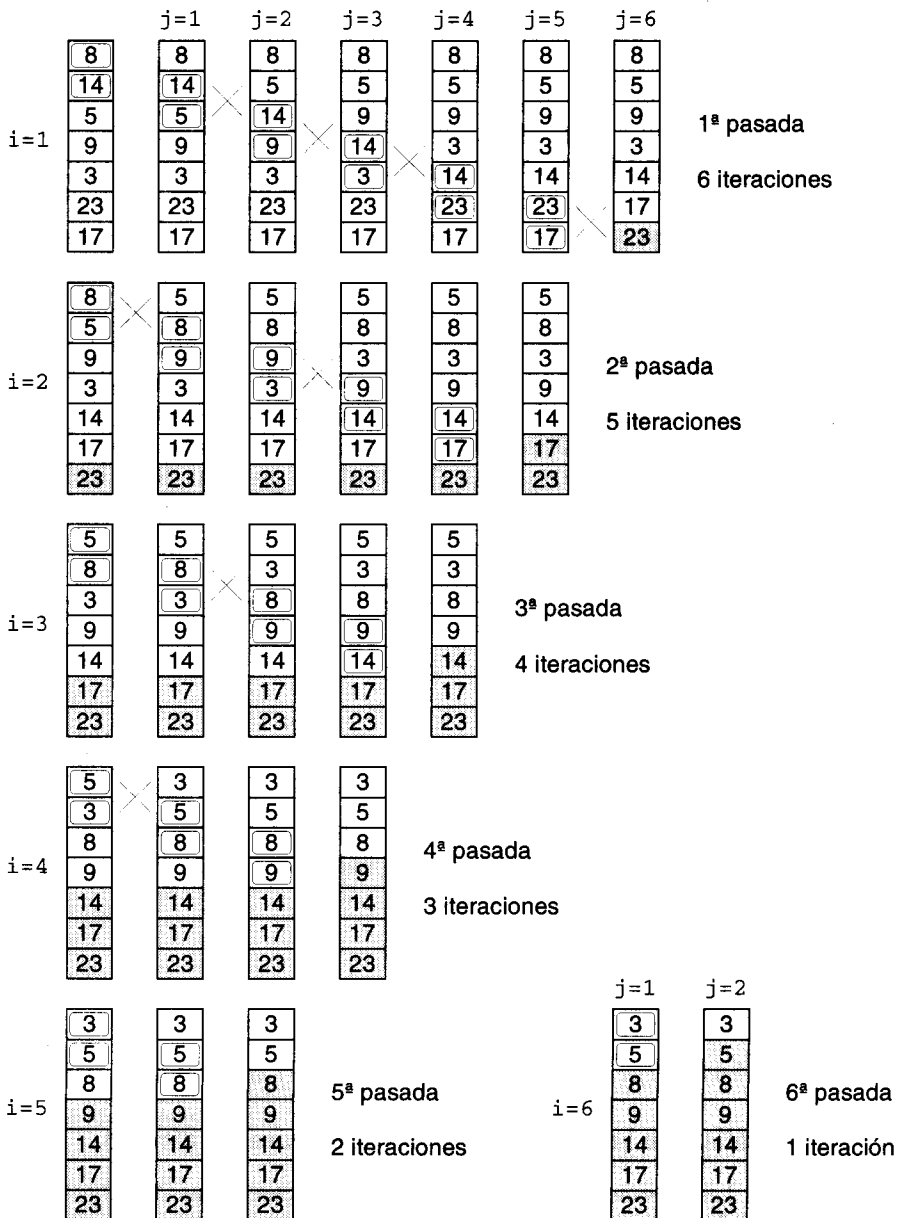


Figura 2.7 Descripción de la clasificación por intercambio directo. Los elementos que se comparan en cada iteración están marcados por un rectángulo redondeado. Las aspas indican intercambios entre una iteración y la siguiente y los elementos sombreados forman la parte ordenada del arreglo en cada momento.

Veamos detalladamente el primer pase. Se elige el $a[1]$ (el 8) y se compara con el siguiente, $a[2]$ (el 14), y como están en orden (el primero es menor que el segundo) se dejan en sus respectivas posiciones. Seguidamente se compara $a[2]$ (el 14) con el siguiente $a[3]$ (el 5) y como entre ellos están desordenados se intercambian. Así sucesivamente se comparan $a[3]$ (nuevamente el 14) con $a[4]$ (el 9) y hay intercambio, $a[4]$ (el 14) con $a[5]$ (el 3) y hay intercambio, $a[5]$ (otra vez el 14) con $a[6]$ (el 23) y no hay intercambio y finalmente $a[6]$ (el 23) con $a[7]$ (el 17) y hay intercambio. La parte superior de la Figura 2.7 muestra este pase con detalle, donde se han marcado los elementos que se comparan en cada iteración y mediante un aspa los intercambios entre iteraciones sucesivas.

El Programa 2.4 implementa en Modula2 este algoritmo, que es conocido como método de la burbuja debido a que en cada pase van ascendiendo (o como en este caso descendiendo) elementos hacia su posición. Obsérvese, por ejemplo el número 14 que ha intervenido en varias comparaciones e intercambios. Si hubiese sido el mayor de todos, no habría parado de ‘burbujear’ hasta llegar al final que sería su posición definitiva. En cada pasada, el método de la burbuja, garantiza que el mayor de los elementos se sitúa en su posición definitiva (la última).

Programa 2.4 Implementación del algoritmo de intercambio directo.

```

VAR
  i, j : Tipo_indice;
  aux : Tipo_datos;
BEGIN
  FOR i:=1 TO n-1 DO
    FOR j:=1 TO n-i DO
      IF a[j]>a[j+1] THEN (* Comparar elementos contiguos *)
        aux:=a[j+1];      (* Intercambiar *)
        a[j+1]:=a[j];
        a[j]:=aux;
      END;
    END;
  END;
END;
```

Análisis de la clasificación por intercambio directo

Las operaciones más frecuentes características del algoritmo son la comparación de elementos del arreglo ($a[j]>a[j+1]$), y los movimientos entre elementos del arreglo ($aux:=a[j+1]$, $a[j+1]:=a[j]$ y $a[j]:=aux$), y como es habitual, el análisis asintótico requiere el cálculo del número de comparaciones y de movimientos en los casos mejor, peor y promedio y no se consideran las operaciones auxiliares.

También en este algoritmo, al igual que en el de selección directa, el número de comparaciones es independiente del orden inicial de las llaves del arreglo ya que

siempre se recorre por completo la parte desordenada del arreglo para comparar las parejas de claves adyacentes. Para los movimientos, por el contrario, sí se distinguen diferentes casos. Cuando el arreglo está completamente ordenado no hay ningún intercambio. Cuando está ordenado en orden inverso se realizará un intercambio con cada comparación, es decir, se realizan todos los intercambios posibles. Así el mejor caso para los movimientos se dará cuando el arreglo esté inicialmente ordenado y el peor caso cuando esté completamente desordenado (ordenado en sentido inverso) presentando, por tanto, un comportamiento natural.

Atendiendo al Programa 2.4 el análisis del algoritmo es el siguiente:

- Comparaciones

Es independientemente del orden inicial de las llaves.

La comparación ($a[j] > a[j+1]$) se realizará $n-1$ veces para el primer pase ($i=2$), $n-2$ en el segundo ($i=3$) y así sucesivamente hasta el último pase en el que realiza una comparación. Sumando todas ellas se observa que es una suma aritmética y por tanto, el número de comparaciones será de orden n^2 . Así:

$$C = 1 + 2 + \dots + n - 1 = \frac{n^2 - n}{2}$$

$$C = \frac{n^2 - n}{2}$$

- Movimientos

a) *Mejor caso*: las llaves están inicialmente ordenadas.

En esta situación nunca se entra en el IF, por tanto el intercambio, formado por tres movimientos ($\text{aux}:=a[j+1]$, $a[j+1]:=a[j]$ y $a[j]:=aux$), no se realiza nunca.

Por tanto:

$$M_{min} = 0$$

b) *Peor caso*: las llaves iniciales están completamente desordenadas.

En este caso se realiza el intercambio siempre que se realiza la comparación.

Por lo que tenemos:

$$M_{max} = 3C$$

$$M_{max} = 3 \frac{(n^2 - n)}{2}$$

c) *Caso promedio*: las llaves iniciales están aleatoriamente ordenadas.

En el caso promedio las llaves del arreglo inicial están ordenadas aleatoriamente y habrá que utilizar argumentos probabilísticos. El intercambio se realizará

dependiendo de que un número entero sea mayor o menor que otro, hecho de probabilidad $1/2$ suponiendo equiprobables los enteros. Así, el número promedio de movimientos es:

$$M_{prom} = \frac{1}{2} M_{max}$$

o lo que es lo mismo:

$$M_{prom} = 3 \frac{(n^2 - n)}{4}$$

En la Figura 2.7 puede observarse que una vez finalizado el cuarto pase ($i=4$) el arreglo está ordenado con lo que los demás pases son innecesarios. Obsérvese que en los pasos restantes ($i=5, 6$) no se produce ningún intercambio más. Esto permite pensar en posibles mejoras. Una primera solución consiste en recordar si durante un pase hay intercambio, de manera que si no lo hay será indicativo de que el arreglo está ordenado y se podrían ignorar el resto de pasadas eliminando las comparaciones correspondientes. También sería posible mejorar el algoritmo recordando la última posición en la que hubo intercambio ya que antes de esa posición el arreglo estará ordenado. Sin embargo, la desventaja más evidente del método de la burbuja es que aunque los elementos con llave grande en posiciones del inicio bajan rápidamente, los elementos con llave pequeña en una posición del final requieren un gran número de pasadas para ser clasificados. Es decir, en el método de la burbuja los elementos bajan bien (varias posiciones cada pase) pero suben mal (una posición en cada pase). Este hecho lo corrige la clasificación por sacudida.

2.2.5 CLASIFICACIÓN POR SACUDIDA O VIBRACIÓN

Este método de clasificación por intercambio mejora el método de la clasificación por burbuja alternando la dirección de pases consecutivos.

La Figura 2.8 muestra un ejemplo de clasificación por vibración.

El primer pase es el mismo que el descrito en el método de la burbuja y recorre el arreglo desde la primera posición hacia el final. El segundo pase comienza en las posiciones finales y se recorre en sentido inverso al anterior. El punto de comienzo ahora tiene en cuenta que la primera pasada ya ha colocado al mayor elemento en su sitio y por tanto, en lugar de comenzar comparando el sexto con el séptimo comienza comparando el quinto con el sexto. De esta manera, la posición $a[5]$ (el 14) se compara con la $a[6]$ (el 17) y no hay intercambio, la $a[4]$ (el 3) con la $a[5]$ (el 14) y tampoco hay intercambio, la $a[3]$ (el 9) con la $a[4]$ (el 3) y deben intercambiarse y así sucesivamente. Los pases impares se harán en sentido descendente y los pases pares en sentido contrario. En cada pasada, la parte de arreglo a ordenar está marcada por los índices L y R que marcan respectivamente el inicio y el final de dicha parte. En cada pasada hacia el final se decrementa el índice de final (R), ya que el último elemento se coloca en su sitio y en cada pasada hacia el inicio se incrementa el índice de inicio (L) por la razón, simétrica de la anterior, de que el menor de

los elementos ha alcanzado su posición definitiva. El algoritmo se para, obviamente, cuando estos índices se cruzan, pues esto indica que el final está antes que el principio y por tanto, no existe parte de arreglo todavía por ordenar.

En el Programa 2.5 se muestra el código necesario para implementar este método de clasificación en Modula2.

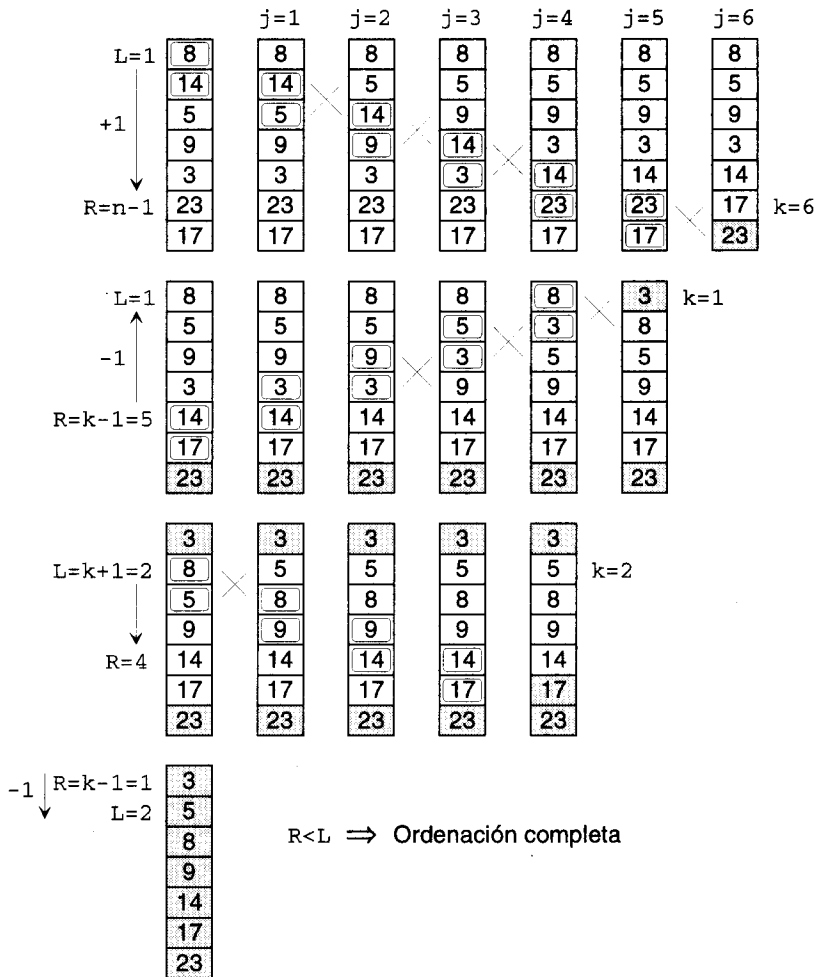


Figura 2.8

Descripción de la clasificación por vibración. Los pasos impares se realizan desde las posiciones iniciales hacia las finales (hacia abajo en la figura) y los pasos pares desde las posiciones finales hacia las iniciales (hacia arriba). Cada pasada hacia abajo sitúa en la última posición el mayor de los elementos y de forma simétrica cada pasada hacia arriba coloca el menor en la primera posición. Por este motivo, cada pasada impar (hacia abajo) decrementa el límite R en una unidad y cada pasada par (hacia arriba) incrementa el límite L en una unidad. De esta forma, llega un momento en que estos índices, que marcan los límites de la parte desordenada del arreglo, se cruzan indicando que no queda parte por ordenar.

Programa 2.5 Implementación del algoritmo del método de ordenación por sacudida o vibración.

```

VAR
  j,L,R,k  : Tipo_indice;
  aux      : Tipo_datos;
BEGIN
  L:=1; R:=n-1; k:=1;
  REPEAT
    FOR j:=L TO R DO
      IF a[j]>a[j+1] THEN
        aux:=a[j+1];
        a[j+1]:=a[j];
        a[j]:=aux;
        k:=j;
      END
    END;
    R:=k-1;
    FOR j:=R TO L BY -1 DO
      IF a[j]>a[j+1] THEN
        aux:=a[j+1];
        a[j+1]:=a[j];
        a[j]:=aux;
        k:=j;
      END
    END;
    L:=k+1;
  UNTIL L>R;
END;

```

Análisis de la clasificación por vibración

La primera observación importante es que las mejoras realizadas sobre la clasificación por burbuja no mejoran el número de movimientos a realizar ya que los intercambios para colocar en su lugar una llave serán los mismos. La mejora se realiza únicamente en el número de comparaciones, eliminando las que se ejecutan de forma redundante. El número mínimo de comparaciones es $C_{min} = n - 1$, el máximo es $C_{max} = n - k_1 \sqrt{n}$ y en el caso promedio C_{prom} es proporcional a $\frac{1}{2}(n^2 - n(k_2 + \ln n))$, siendo k_1 y k_2 constantes de proporcionalidad. Teniendo en cuenta que la implementación de los movimientos tiene un coste mayor que el de las comparaciones puede decirse que la mejora no es significativa.

2.2.6 COMPARACIÓN DE LOS MÉTODOS DIRECTOS DE CLASIFICACIÓN

De los análisis de costes computacionales realizados para cada algoritmo puede establecerse que el coste computacional de los algoritmos de clasificación directa es de orden n^2 . La Tabla 2.1 muestra los costos comparativamente.

		Mínimo	Promedio	Máximo
Inserción	Comparaciones	$n - 1$	$\frac{n^2 + n - 2}{4}$	$\frac{n^2 + n - 2}{2}$
	Movimientos	$3(n - 1)$	$\frac{n^2 + 13n - 14}{4}$	$\frac{n^2 + 5n - 6}{2}$
Selección	Comparaciones	$\frac{n^2 - n}{2}$	$\frac{n^2 - n}{2}$	$\frac{n^2 - n}{2}$
	Movimientos	$3(n - 1)$	$n(\ln n + \gamma)$	$3(n - 1) + \frac{n^2}{4}$
Intercambio	Comparaciones	$\frac{n^2 - n}{2}$	$\frac{n^2 - n}{2}$	$\frac{n^2 - n}{2}$
	Movimientos	0	$3\frac{n^2 - n}{4}$	$3\frac{n^2 - n}{2}$

Tabla 2.1

Tabla comparativa de los métodos directos de clasificación sobre arreglos.

Entre ellos la selección directa será la opción a elegir en general, aunque la inserción es algo más rápida cuando las llaves predominantemente se clasifican al principio y la vibración puede ser mejor cuando el arreglo está inicialmente casi ordenado.

En la siguiente sección se presentan los métodos avanzados de clasificación de arreglos, que mejoran sensiblemente estos resultados aunque, como se verá, no para cualquier tamaño del arreglo.

2.3

MÉTODOS AVANZADOS DE CLASIFICACIÓN

En este apartado se presentan los tres métodos avanzados de clasificación de arreglos básicos:

1. Inserción por incremento decreciente (Shell).
2. Clasificación por montón.
3. Clasificación por partición (Quicksort).

Cada uno de ellos se fundamenta en una estrategia de clasificación directa diferente, mejorándola. Así, la inserción por incremento decreciente se basa en la inserción, la clasificación por montículo de Floyd se basa en la selección y la clasificación por partición en el intercambio.

2.3.1

INSERCIÓN POR INCREMENTO DECRECIENTE

Este método propuesto por D.L. Shell se basa en la ordenación por inserción de los elementos que difieren h_1 posiciones, después de los que difieren h_2 , y así sucesivamente según un conjunto de incrementos decrecientes h_i , hasta ordenar los que difieren en $h_i=1$. Existen numerosos conjuntos de incrementos decrecientes que garantizan la clasificación del arreglo, como por ejemplo: 1, 2, 4, ...; 1, 3, 5, 9, ...; 1, 4, 13, 40, 121, ...; 1, 3, 7, 15, 31, ...; etc. Se comienza en cada caso por el incremento mayor que permita el tamaño del arreglo y se va reduciendo en cada actuación.

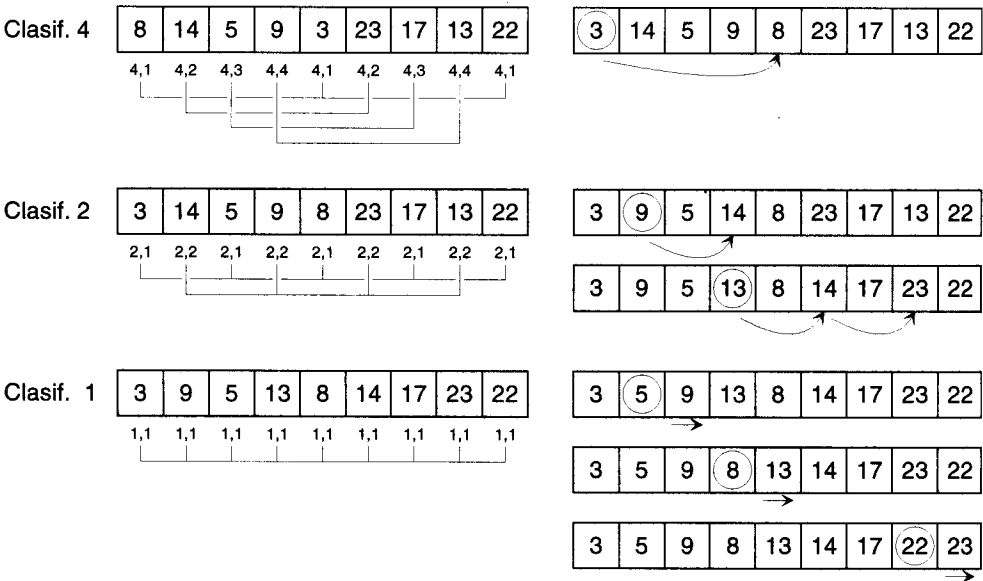


Figura 2.9 Ejemplo de clasificación por el método Shell con incrementos 1, 2 y 4.

La Figura 2.9 muestra un ejemplo de ordenación Shell para un arreglo de nueve elementos con incrementos $h_1=4$, $h_2=2$, $h_3=1$.

En un primer pase se ordenan por inserción directa los elementos que están separados cuatro posiciones. Así deben ordenarse las llaves de las posiciones 1, 5 y 9, con llaves 8, 3 y 22 respectivamente; las posiciones 2 y 6 con llaves 14 y 23; las posiciones 3 y 7 con llaves 5 y 17; y las posiciones 4 y 8 con llaves 9 y 13. Obsérvese que hemos formado cuatro grupos que se ordenan por inserción de forma independiente. Esto se muestra en la figura con las líneas inferiores de enlace y con los pares de índices que aparecen debajo de cada elemento. El primer índice indica el tamaño del incremento y el segundo es el índice del grupo. El número de grupos será siempre el mismo que el incremento escogido. Podemos pensar en estos grupos como si se tratase de arreglos individuales pues en cada clasificación son ordenados por inserción de forma independiente.

Esta ordenación de los elementos separados cuatro posiciones se conoce como *Clasificación-4*. Seguidamente se realiza la ordenación de los elementos que estén separados dos posiciones, con lo que tenemos dos grupos, proceso denominado *Clasificación-2*. De este modo, se ordenan por un lado los elementos en las posiciones 1, 3, 5, 7 y 9 cuyas llaves son 3, 5, 8, 17 y 22; y por otro lado, los elementos en posiciones pares con llaves 14, 9, 23 y 13. Finalmente se realiza una *Clasificación-1*, es decir, se ordenan los separados una posición. En este último paso tenemos un solo grupo que engloba todo el arreglo.

A la hora de hacer la ordenación por inserción, hay que tener en cuenta que los desplazamientos necesarios para insertar un elemento no se harán ahora de una posición a la siguiente adyacente como se hacía en el método de inserción convencional (Figura 2.2 y Programa 2.1). Ahora los incrementos vienen dados por el nivel de clasificación en que nos encontremos y sólo el último paso (*Clasificación-1*) se comporta de la misma forma que el algoritmo convencional de inserción. Esto puede verse en la Figura 2.9 donde durante la *Clasificación-4* fue necesario insertar el número 3 (indicado mediante un círculo) y para ello hubo que desplazar el número 8 cuatro posiciones, pero sin alterar los que quedan en medio (números 14, 5 y 9). Durante la *Clasificación-2* hubo que insertar el 9 para lo que hubo que desplazar el 14 dos posiciones y posteriormente el 13, para lo que se tuvieron que desplazar el 14 y el 23.

Ésta es realmente la ventaja del método Shell, que no es más que una mejora del de inserción directa. De hecho, el último paso (*Clasificación-1*), es una clasificación por inserción directa. Los pasos previos permiten proporcionar al último una situación en la que los elementos no estén muy lejos de su posición final por lo que los desplazamientos necesarios serán menos. Y esto se ha conseguido con desplazamientos a 'gran escala', y no de una posición a la adyacente. De esta forma, el número de desplazamientos necesario es considerablemente menor.

El Programa 2.6 implementa este algoritmo en Modula2, con ordenación por inserción directa para cada clasificación- i . Al igual que se hizo con la implementación del método de inserción directa, el arreglo se ha ampliado para albergar el cen-

tinela, con la diferencia de que ahora debe ampliarse más de una posición, para poder albergar un centinela independiente para cada grupo. De esta forma, si se comienza con una *Clasificación-4*, tendremos cuatro grupos y deberemos ampliar el arreglo en 4 posiciones, con lo que comenzará en el índice -3 en lugar de en 0 como en la inserción directa $[-h_1+1...n]$. No obstante, esto no es estrictamente necesario y como en el método de inserción directa puede utilizarse un arreglo independiente.

Programa 2.6 Implementación del algoritmo de la clasificación Shell.

```

CONST
    t=3;          (*número de ordenaciones por inserción directa*)
VAR
    i,j,k,
    s: Tipo_indice; (* posición del centinela de cada clasificación*)
    m:[1..t];      (*índice que indica el número de ordenación*)
    h:ARRAY[1..t] OF INTEGER; (*vector de incrementos*)
BEGIN
    h[1]:=4; h[2]:=2; h[3]:=1;
    FOR m:=1 TO t DO
        k:=h[m]; s:=-k;
        FOR i:= k+1 TO n DO
            j:=i-k;
            IF s=0 THEN s:=-k END;
            s:=s+1; a[s]:=a[i];          (*Inserción directa*)
            WHILE a[s]<a[j] DO
                a[j+k]:= a[j]; j:=j-k
            END;
            a[j+k]:=a[s]
        END;
    END
END;

```

Análisis de la clasificación Shell

El análisis detallado de este algoritmo requiere una elaboración matemática rigurosa en función de los posibles valores de incrementos decrecientes a elegir, cuyo detalle queda fuera del alcance de este texto. Sin embargo, pueden analizarse algunas secuencias particulares como indicación del coste del algoritmo.

Evidentemente este método clasifica el arreglo ya que la ordenación del último pase, $h_t=1$, lo asegura. De hecho, es obvio que cualquier conjunto de incrementos decrecientes, $h_{i+1}<h_i$, tal que el último es $h_t=1$, ordenará el arreglo. El interés radica en que las clasificaciones parciales con los incrementos decrecientes faciliten la *Clasificación-1*, pero en último extremo la *Clasificación-1* realizaría toda la tarea de ordenación.

El peor caso es de especial interés. Para analizarlo supóngase que n es potencia de 2 y que se encuentran $n/2$ llaves con valores grandes en las posiciones pares y $n/2$ con valores pequeños en las impares. Los incrementos se toman pares, excepto el último ($h_t=1$). Como todos los incrementos excepto el último ($h_t=1$) son pares, cuando se llega a este último pase los $n/2$ números de valores grandes están todavía en las posiciones pares, y los $n/2$ menores en las impares. Antes de este último pase, el i -ésimo menor elemento ($i \leq n/2$) está en la posición $2i-1$. Para colocar en el último pase este elemento en su lugar será necesario moverlo $i-1$ posiciones, y entonces para colocar los $n/2$ elementos menores en sus posiciones se necesitarán como mínimo:

$$\sum_{i=1}^{n/2} i - 1$$

que es una suma aritmética y por tanto, de orden n^2 . Luego en el mejor caso el coste como mínimo es de orden n^2 .

Además, el coste es como máximo también de orden n^2 . En efecto, como se ha explicado en la descripción del algoritmo un pase de incremento h_k consiste en h_k inserciones directas de aproximadamente N/h_k elementos. Como la inserción directa es de orden n^2 el coste total de un pase será de orden $h_k(N/h_k)^2$, y sumando para todos los pases se obtiene de orden:

$$O\left(\sum_{i=1}^t n^2/h_i\right) = O\left(n^2 \sum_{i=1}^t 1/h_i\right)$$

que es una suma geométrica de razón 2 y el término más grande es $h_1=1$. Así:

$$\sum_{i=1}^t 1/h_i < 2$$

Por tanto, el máximo coste en el peor caso es de orden n^2 .

En la elección del ejemplo con incrementos decrecientes múltiplos de 2 evidentemente se observa que en cada pase la *Clasificación- i* clasifica dos grupos ya ordenados por la anterior *Clasificación- $2i$* , es decir, clasifica elementos que ya han sido tratados entre sí. Así cualitativamente se observa que deben evitarse conjuntos de incrementos en los que unos sean múltiplos de otros ya que no aprovechan la clasificación realizada anteriormente.

La elección de un conjunto de incrementos para todos los casos no está absolutamente definida y en todo caso es una tarea difícil. Knuth [Knuth, 1998] reco-

mienda diversas series. Así por ejemplo, una de las secuencias que recomienda es $h_{i+1}=3h_i+1$, $h_1=1$ con $t = \lceil \log_3 n \rceil - 1$ (1, 4, 13, 40, ...). También propone la secuencia (1, 3, 7, 15, 31, $2^k-1, \dots$). Obsérvese que esta secuencia, denominada incrementos de Hibbard, evita los múltiplos de sí mismos, ya que los incrementos consecutivos no tienen factores comunes.

Veamos el máximo coste para esta secuencia.

El máximo coste vendrá determinado por la suma de los costes en cada pase. Al llegar al pase h_k evidentemente ya se han realizado los pases h_{k+1} y h_{k+2} . Dadas dos posiciones, P y $P-i$ ($i \leq P$), si i es múltiplo de h_{k+1} o h_{k+2} entonces antes del pase h_k el elemento en la posición $P-i$ es menor que el de la posición P . Pero además, este hecho también se satisface si i se puede expresar como combinación lineal de h_{k+1} y h_{k+2} . Por otro lado, h_{k+1} y h_{k+2} no tienen divisores comunes. Cuando esto sucede se puede demostrar que todos los enteros mayores que

$$(h_{k+1} - 1)(h_{k+2} - 1) = 8h_k^2 + 4h_k \quad (\text{ya que } h_{k+1} = 2h_k + 1)$$

pueden expresarse como combinación lineal de h_{k+1} y h_{k+2} [Papernov, 1965]. Haciendo uso de este resultado para cada una de las $n-h_k$ posiciones el bucle interno se ejecuta como máximo h_k+4 veces. Por tanto, el coste máximo es de orden nh_k por cada pase. Teniendo en cuenta el hecho de que aproximadamente la mitad de los incrementos satisfacen que $h_k = n^{1/2}$ y suponiendo que t es par, el coste total será:

$$O\left(\sum_{k=1}^{t/2} nh_k + \sum_{k=t/2+1}^t n^2/h_k\right) = O\left(n \sum_{k=1}^{t/2} h_k + n^2 \sum_{k=t/2+1}^t h_k\right)$$

Como ambas sumas son geométricas y $h_{t/2}$ es de orden $\sqrt{n}$, la expresión del máximo coste será de orden:

$$O(nh_{t/2}) = O(n^2/h_{t/2}) = O(n^{3/2})$$

Por tanto, el análisis matemático, ciertamente complicado, demuestra que el peor coste es de orden $n^{1.3}$, con lo que mejora los métodos directos.

El análisis de caso promedio para los incrementos de Hibbard no ha sido aún resuelto. Mediante simulación se piensa que es de orden $n^{5/4}$, pero por ahora no se ha obtenido una demostración rigurosa.

Aunque los incrementos de Hibbard son bien conocidos existen otras secuencias de incrementos que mejoran su coste. Así, Sedgewick [Sedgewick, 1986] propuso varias secuencias para las cuales demostró analíticamente que su coste es de

orden $n^{4/3}$ en el peor caso. Tampoco para estas secuencias se han conseguido los costes del caso promedio, pero se conjetura que son de orden $n^{7/6}$.

Los resultados experimentales muestran que las secuencias de Sedgewick mejoran de forma apreciable la secuencia de Hibbard. La mejor parece ser la secuencia cuyos términos siguen las expresiones:

$$9 \times 4^i - 9 \times 2^i + 1 \quad \text{o} \quad 4^i - 3 \times 2^i + 1: \{1, 5, 19, 41, 109, \dots\}$$

En general, el análisis de la clasificación por incrementos decrecientes es complicado y requiere de resultados matemáticos de teoría combinatoria y teoría de números. Una base teórica introductoria a estas técnicas puede encontrarse en Knuth [Knuth, 1998].

2.3.2 CLASIFICACIÓN POR MONTÓN

En general, un montículo o montón se define como una secuencia de llaves h_i , $i=L, L+1, \dots, R$, tal que mantienen la siguiente relación de orden:

$$h_i \leq h_{2i} \quad \text{y} \quad h_i \leq h_{2i+1} \quad \text{para} \quad i=L, \dots, R/2$$

conocida como definición de Williams. Un montón puede implementarse sobre árboles binarios, aportando un criterio de ordenación. Sin embargo, en este apartado interesa su implementación sobre un arreglo, basada en la que Floyd desarrolló, y su utilización para realizar la clasificación del arreglo 'in situ'. El planteamiento de Floyd es sencillo. Dado un arreglo de n elementos, $h_1 \dots h_n$, los elementos a partir de la mitad del arreglo, $k = (n \text{ DIV } 2) + 1$, hasta el final, forman un montón ($h_k \dots h_n$) puesto que no hay dos índices i, j en esta parte que no satisfagan la definición de montón. Seguidamente se amplía ($h_k \dots h_n$) tomando un elemento por la izquierda y se reconstruye el montón con este nuevo elemento. Una vez construido se toma un elemento de la izquierda y se reconstruye de nuevo, y así sucesivamente hasta que han sido añadidos todos los elementos del arreglo, con lo que éste será un montón.

La Figura 2.10 ilustra la construcción de un montón 'in situ'. En el primer pase la parte del arreglo de posiciones $i = 5 \dots 9$ ($9 \text{ DIV } 2 + 1 = 5$) es un montón ya que no existen posiciones $j=2i$ ni $j=2i+1$; esto se indica en la figura con fondo sombreado. Entonces se incorpora el elemento de la posición 4 (llave 9) al montón, marcándolo con un círculo en la Figura 2.10. Se satisface que $h_4 = 9 \leq h_8 = 13$ y $h_4 = 9 \leq h_9 = 22$, y además el resto del montón ($i = 5 \dots 9$) no ha sufrido modificaciones y sigue satisfaciendo la definición de Williams. Seguidamente se incorpora $h_3=5$ al montón y como se satisface $h_3=5 \leq h_6=23$ y $h_3=5 \leq h_7=17$ es un montón. En el siguiente pase se incorpora $h_2=14$ al montón. En este caso, deja de

cumplirse la definición de Williams ya que $h_2 = 14 > h_4 = 9$ y hay que intercambiarlo con $h_5=3$. Como ahora se han hecho modificaciones hay que comprobar que para $i>2$ se siguen satisfaciendo las condiciones de la definición, tras lo cual se concluye que sigue siendo un montón. Por último, se incorpora $h_1=8$ al montón, intercambiando con $h_2=3$. Comprobando que se verifican las condiciones para $i>1$, se obtiene finalmente el montón. En la Figura 2.10 se muestra con un círculo el elemento a añadir y los elementos con los que se compara.

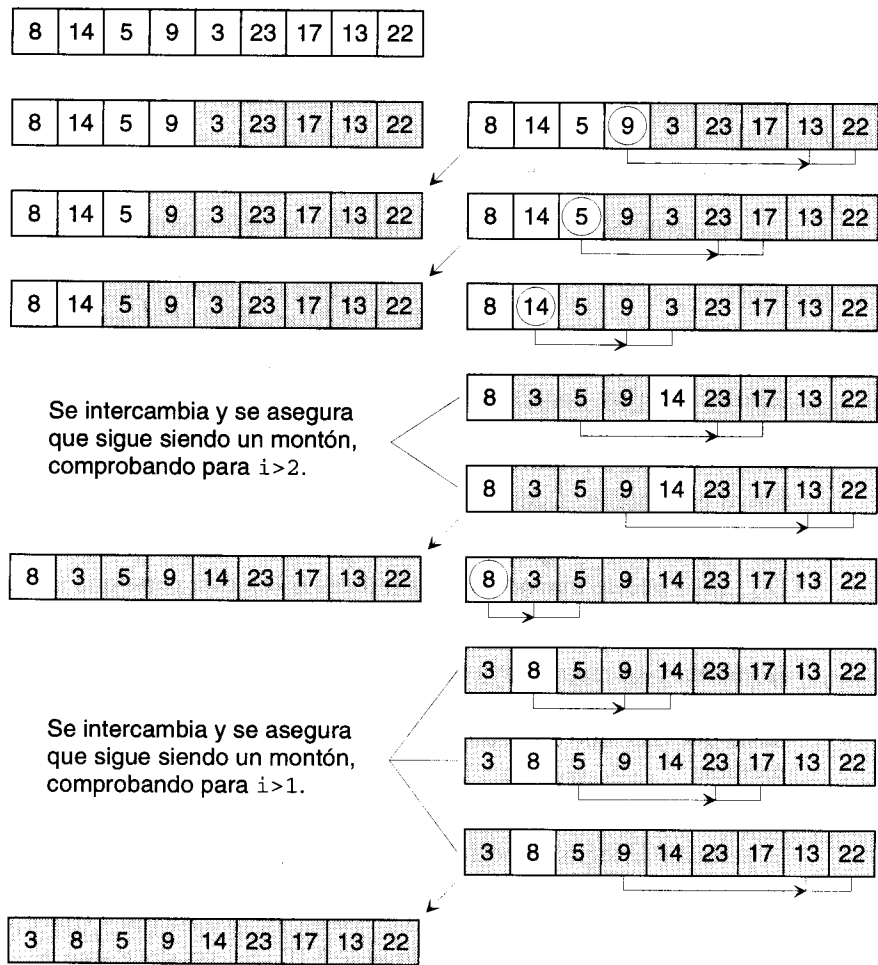


Figura 2.10 Construcción de un montón 'in situ' de Floyd. Cada vez que se añade un elemento al montón hay que comprobar que cumple la definición de Williams y si no, hay que intercambiarlo. Cada vez que se hace un intercambio hay que comprobar el resto del montón para garantizar que no se ha alterado el resto del mismo.

El Programa 2.7 muestra la implementación en Modula2 del procedimiento para incorporar un elemento a un montón, y el Programa 2.8 la iteración necesaria para organizar 'in situ' un arreglo como un montón.

Programa 2.7 Procedimiento para incorporar un elemento a un montón.

```

PROCEDURE amontonar(L,R:Tipo_indice);
  (* R: posición del extremo derecho del montón *)
  (* L: posición del elemento que se esta incorporando al montón *)
VAR i,j:Tipo_indice; x: Tipo_datos;
BEGIN
  i:=L;
  j:=2*L;
  x:=a[L];  (* Elemento que se incorpora al montón *)
  IF (j<R) & (a[j+1]<a[j]) THEN
    j:=j+1
  END;
  (*j: índice del elemento que intercambiará de entre el 2i y el 2i+1*)
  WHILE (j<=R) & (a[j]<x) DO
    a[i]:= a[j];
    a[j]:=x;      (* intercambio *)
i:=j; j:=j+1
j:=2*j; i:=2*i
    IF(j<R) & (a[j+1]<a[j]) THEN
      j:=j+1
    END
  END      (* verificar montón hasta el final *)
END;
END amontonar;

```

Programa 2.8 Organización 'in situ' de un arreglo como un montón.

```

L:=(n DIV 2)+1;
R:=n;
WHILE L>1 DO
  L:=L-1; (*posición del nuevo elemento que se incorpora al montón*)
  amontonar(L,R);
END;

```

Una vez organizado el arreglo como un montón, la clasificación del mismo se obtiene teniendo en cuenta que la cima, el elemento h_1 , es el menor de todos los elementos del montón. Así para clasificar un arreglo de n elementos se retira la cima y se construye un montón con los demás $n-1$ elementos. Su cima será de nuevo el menor de los que quedan, se retira y se vuelve a construir un montón con los $n-2$ restantes, y así sucesivamente. La cuestión a resolver ahora es dónde almacenar las

sucesivas cimas. Si se realizase en otro arreglo se estaría utilizando una memoria de tamaño $2n$, lo que es inaceptable por economía de almacenamiento. Una solución adecuada consiste en intercambiar la cima, primer elemento del arreglo, con el último, y construir un montón con los $n-1$ primeros. Seguidamente se intercambia la cima con el último del nuevo montón, posición $n-1$, y se reconstruye un montón con los $n-2$ primeros, y así sucesivamente. La Figura 2.11 muestra el proceso de clasificación. Se marcan con un círculo las sucesivas cimas y con fondo sombreado los elementos ordenados.

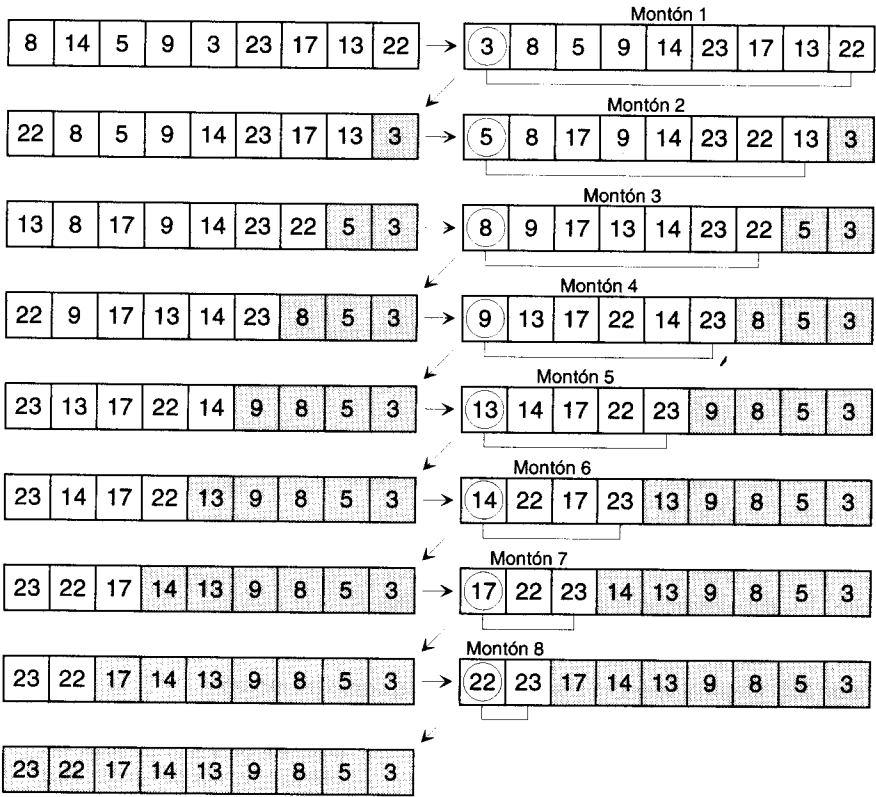


Figura 2.11 Clasificación por montón. La parte sombreada indica los elementos ya ordenados. En la columna de la derecha se muestran con fondo blanco los sucesivos montones y con un círculo las cimas correspondientes.

Obsérvese que en la construcción de cada montón no es suficiente comprobar que $h_1 \leq h_2$ y $h_1 \leq h_3$, sino que debe comprobarse toda la parte del arreglo implicada en dicha construcción, como se ponía de manifiesto en la Figura 2.10 donde después de hacer cada intercambio se comprobaban las condiciones de montón en el resto del arreglo. Aunque para el primer montón que es el que se representaba en esa figura los intercambios no alteraban la condición de montón, para el segundo sí que

se alteran. De esta forma, en la creación del montón 2, que se ilustra en la Figura 2.12, $h_1=22>h_2=8$ y $h_1>h_3=5$, el intercambio entre h_1 y h_3 garantiza que estos tres elementos satisfacen la definición de montón pero no es así para el resto del arreglo. Será, por tanto, necesario considerar toda la parte del arreglo a construir como montón al hacer un intercambio y realizar los intercambios adicionales necesarios si las condiciones de montón hubiesen sido alteradas.

El Programa 2.9, muestra el código en Modula2 para realizar la clasificación 'in situ', y el Programa 2.10 implementa en Modula2 el algoritmo completo de la clasificación por montón.

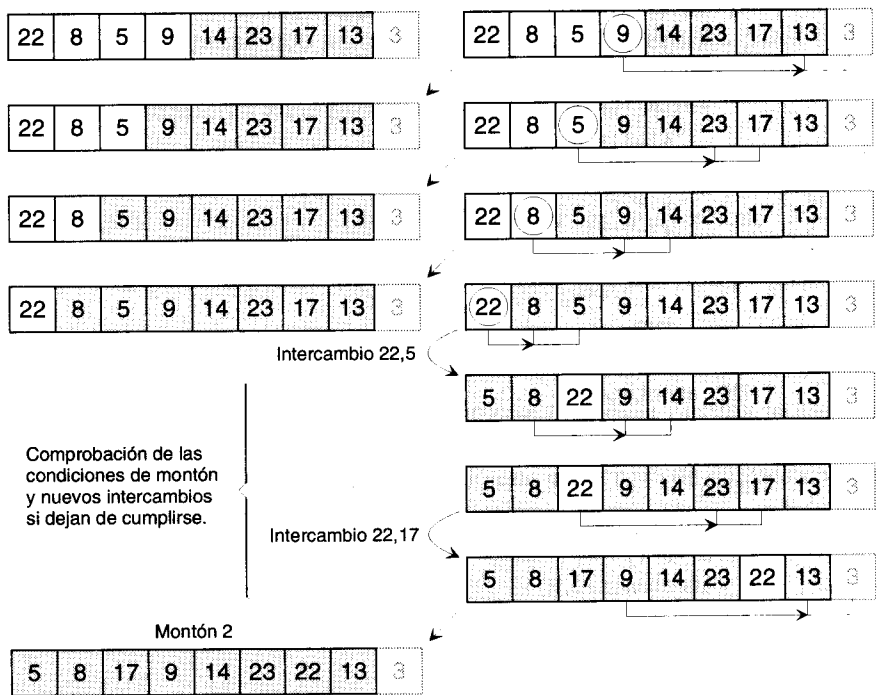


Figura 2.12

Detalle de la construcción del montón 2 de la Figura 2.11. Las celdas sombreadas indican el montón en cada momento y la casilla atenuada, la parte del arreglo ya ordenada.

Programa 2.9

Código en Modula2 para la clasificación 'in situ' por montón.

```

R:=n;
WHILE R>1 DO
  x:= a[1];
  a[1]:=a[R];
  a[R]:=x;
  R:=R-1;
  amontonar(L,R);
END;
```

Programa 2.10 Clasificación por montón.

```

PROCEDURE monton;
VAR
  L,R: Tipo_indice;x: Tipo_datos;
  PROCEDURE amontonar(L,R:Tipo_indice);
  VAR i,j:Tipo_indice; x:Tipo_datos;
  BEGIN
    i:=L;
    j:=2*L;
    x:=a[L];
    IF (j<R) & (a[j+1]<a[j]) THEN
      j:=j+1
    END;
    WHILE (j<=R) & (a[j]<x) DO
      a[i]:= a[j];
      a[j]:=x;
      i:=j;
      j:=2*j;
      IF(j<R) & (a[j+1]<a[j]) THEN
        j:=j+1
      END
    END;
  END amontonar;
BEGIN (*montón*)
  L:=(n DIV 2)+1;
  R:=n;
  WHILE L>1 DO
    L:=L-1;
    amontonar(L,R);
  END;
  WHILE R>1 DO
    x:= a[1];
    a[1]:=a[R];
    a[R]:=x;
    R:=R-1;
    amontonar(L,R);
  END
END monton;

```

Como puede observarse en la Figura 2.11, la clasificación por montón ordena el arreglo en orden inverso. Este hecho no tiene mayor importancia ya que en el arreglo todas las posiciones son accesibles con igual coste, lo que permite imprimir los elementos de manera descendente comenzando por el final sin coste adicional.

Sin embargo, si se deseara que estuviese clasificado en orden ascendente no hay más que definir un montón de forma alternativa tal que:

$$h_i \geq h_{2i} \quad \text{y} \quad h_i \geq h_{2n+1}$$

y realizar las modificaciones asociadas a este hecho en las implementaciones.

Análisis de la clasificación por montón

El peor caso se produce cuando se realiza el máximo número de intercambios en la construcción de los sucesivos montones. En cada pase i , la reconstrucción de un montón requerirá $2\lceil \log i \rceil$ comparaciones como máximo, dos para cada uno de los elementos de la mitad que no es montón. Sumando todos los pases (y aproximando por la integral, tal como se hizo en el análisis de la inserción binaria) el número de comparaciones en el peor caso es $2n \log n - O(n)$.

En el caso promedio el análisis es complicado. Por un lado, se puede demostrar que el número de montones para n elementos es mayor que $\left(\frac{n}{4e}\right)^n$ donde $e=2.7182\dots$ Por otro, se demuestra que únicamente una fracción pequeña de estos montículos (sólo $\left(\frac{n}{16}\right)^n$) tienen un coste menor que $M = n(\log n - \log \log n - 4)$.

Para demostrar este último punto supóngase que en la i -ésima reconstrucción del montón, la cima (que es el elemento intercambiado con el que era el menor en el montón anterior) se desplaza l_i niveles, con lo que el número de comparaciones realizadas será $2l_i$. Así, para todas las reconstrucciones de montón se tendrán $2L$ comparaciones, siendo

$$L = \sum_{i=1}^n l_i$$

Para cada l_i hay 2^{l_i} posiciones en las que la cima puede situarse. Por tanto, para cualquier secuencia $l_1, l_2, l_3, \dots, l_n$, el número de secuencias distintas vendrá dado por:

$$S = 2^{l_1} 2^{l_2} \dots 2^{l_n} = 2^L$$

Como los posibles niveles desplazados, l_i , pueden ser cualquier valor entre 1 y $\lceil \log n \rceil$ habrá como máximo $\log n^n$ secuencias $l_1, l_2, l_3, \dots, l_n$. Por tanto, el número de secuencias distintas con coste M será como máximo el producto de este máximo

número de secuencias $\log n^n$ por el número de secuencias que tienen coste total M , 2^M . Así, el máximo número de secuencias distintas con coste M será:

$$\log n^n 2^M$$

y el número total de montones con coste menor que M será como máximo:

$$\sum_{i=1}^{M-1} \log n^n 2^i < \log n^n 2^M$$

Considerando $M = n(\log n - \log \log n - 4)$ entonces:

$$(\log n)^n 2^M = (\log n)^n 2^{\log n^n} 2^{-\log(\log n)^n} 2^{-4n} = (\log n)^n \frac{n^n}{\log n^n 16^n} = \left(\frac{n}{16}\right)^n$$

con lo que el número de comparaciones promedio vendrá dado por $2M$. Por lo que resulta ser de orden:

$$2n \log n - O(n \log \log n)$$

2.3.3 CLASIFICACIÓN POR PARTICIÓN

Este método desarrollado por C.A.R. Hoare se conoce también como clasificación rápida (Quicksort), lo que es indicativo de su eficacia. Se fundamenta en el método de intercambio directo que, recuérdese, era el peor de los directos. Al igual que todos los métodos avanzados, se basa en el hecho de que cuanto mayor sea la distancia entre los elementos que se intercambian más eficaz será la clasificación. Para ello, se elige un valor de llave al azar, x , denominado pivote. Seguidamente se recorre el arreglo desde la izquierda hasta encontrar una llave mayor que el pivote, y desde la derecha hasta encontrar una menor. Entonces se intercambian. El proceso se repite hasta que los índices de incremento y decremento se crucen. En esta situación, el arreglo está dividido en dos partes, la izquierda con todos los elementos menores que el pivote y la derecha con todos los mayores. A cada una de estas partes se le denomina partición. Se repite el método para cada una de las particiones y así sucesivamente hasta que no haya particiones a ordenar.

La Figura 2.14 muestra un ejemplo de ordenación por partición para un arreglo de nueve elementos. En todas las particiones se ha escogido como pivote el elemento situado en la posición central de la partición (división entera entre 2). En la primera partición se obtiene una partición con los 3 primeros elementos (3, 5 y 8) y otra con el resto (9, 17, 23, 14, 13 y 22). Como se ha comentado antes, todos los de la primera partición son menores o iguales que el pivote y todos los de la segunda son mayores. A continuación se aplica el mismo método a cada una de estas particiones: en la primera el pivote será el 5 y en la segunda el 23. Esto da lugar a nuevas particiones hasta que al final todas las particiones quedan de tamaño 1 con lo que el

arreglo está ya ordenado. En la Figura 2.14 se indica el pivote con un pequeño triángulo invertido y con un círculo los elementos que se deben intercambiar para obtener las sucesivas particiones. En la obtención de la primera partición se precisan intercambiar dos pares de valores que quedan emparejados mediante las líneas que aparecen debajo del arreglo. En cada nivel, se muestra en negro la partición que se está considerando (particionando) y en atenuado la parte del arreglo que no interviene en ese nivel. La Figura 2.13 muestra con detalle la obtención de la primera partición.

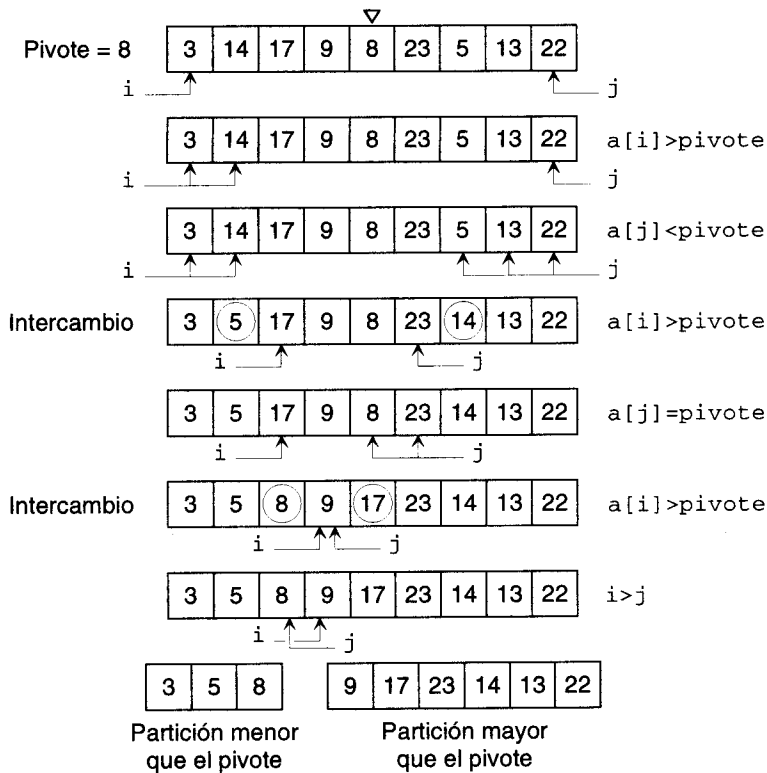


Figura 2.13 Ejemplo que muestra la obtención de la primera partición para el ejemplo de la Figura 2.14.

En esta Figura (2.13) se comienza inicializando los índices i y j a la primera y última posición del arreglo (elementos 3 y 22). A continuación, se comienza a incrementar el índice del inicio (i) hasta que se encuentre un elemento que no sea menor que el pivote, lo que se consigue al llegar al segundo elemento (14). Una vez localizado un elemento mayor o igual, se irá decrementando el índice de final (j) hasta encontrar un elemento que sea menor (o igual) que el pivote (el 5). Se intercambian estos dos elementos y se avanzan ambos índices (se incrementa i y se decrementa j). Ahora se sigue con el mismo proceso: se comienza a incrementar i hasta localizar un elemento no menor que el pivote (17) y después se decrementa j hasta localizar

zar uno no mayor (el 8), se intercambian y se avanzan los dos índices. Se continúa el proceso realizando todos los intercambios necesarios hasta que ambos índices se crucen. En este momento, tenemos en la parte inferior del arreglo los elementos menores o iguales al pivote y en la otra los mayores. La primera parte viene dada por los elementos desde la primera posición hasta el índice de final (en este caso $j=3$) y la segunda por el índice de inicio ($i=4$) hasta el final.

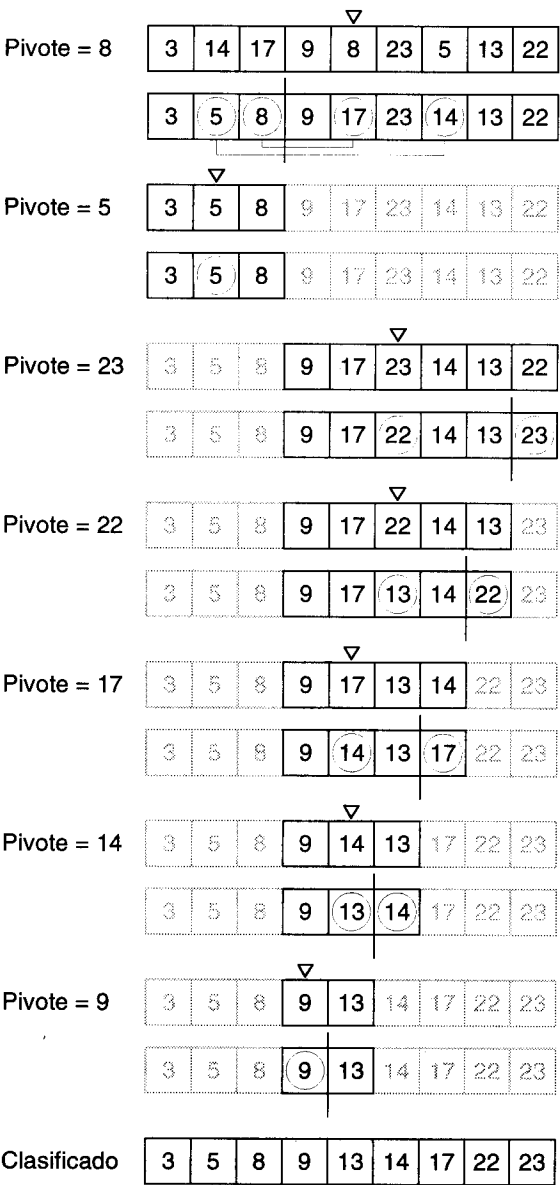


Figura 2.14 Ejemplo de clasificación por partición (quicksort).

Programa 2.11 Clasificación por partición (quicksort).

```

PROCEDURE particion;
  PROCEDURE ordena (L, R : Tipo_indice);
    VAR
      i,j      : Tipo_indice;
      piv,aux  : Tipo_datos;
    BEGIN
      i:=L; j:=R;
      piv:=a[(L+R)DIV 2];      (* pivote *)
      REPEAT
        WHILE a[i] < piv DO i := i+1 END;
        WHILE piv < a[j] DO j := j-1 END;
        IF i <= j THEN
          aux:=a[i]; a[i]:=a[j]; a[j]:=aux;  (*intercambio*)
          i:=i+1;
          j:=j-1
        END
      UNTIL i>j;
      IF L<j THEN ordena(L,j) END;
      IF i<R THEN ordena(i,R) END;
    END ordena;
  BEGIN
    ordena (1,n);
  END particion;

```

El Programa 2.11 muestra la implementación correspondiente en Modula2. Obsérvese que la implementación es recursiva, tal y como sugiere el algoritmo.

La elección del pivote es un aspecto fundamental. Nótese que se ha tomado el elemento central ($a[(L+R) \text{ DIV } 2]$). Esta elección pretende evitar que el método sea ineficaz cuando el arreglo esté preordenado. En efecto, en estas situaciones elegir el primer elemento haría que el coste del algoritmo fuese de orden n^2 ya que en cada pase una de las particiones tendría un tamaño 1, y la otra todos los demás elementos. Y así para todas las sucesivas recursiones. Obsérvese que la posibilidad de esta situación no se da necesariamente para todo el arreglo, es decir, el arreglo podría ser de 100.000 elementos y entre ellos cabe la posibilidad de que 5.000 (5 por ciento) estuviesen ordenados y la clasificación de este 5 por ciento sería muy costosa. Sin embargo la elección del elemento central puede presentar el mismo problema; de hecho sucede cuando es el menor o el mayor de los elementos de la partición, aunque bien es cierto que es muy improbable que así suceda para todas las sucesivas recursiones. Evidentemente el mejor pivote sería la mediana de los elementos del arreglo, ya que divide el arreglo en dos particiones de igual tamaño ± 1 . Sin embargo, su cálculo previo a la clasificación tiene un coste excesivo para utilizarlo. En principio, para calcular la mediana se clasificaría el arreglo y posteriormente se elegiría el de la mitad. Evidentemente este método es absurdo puesto que el pro-

blema a resolver es el de clasificar. Soluciones intermedias como calcular la media de un número reducido de elementos pueden ser útiles en este sentido. Por ejemplo, se puede utilizar como pivote la media del primero, el último y el central.

Análisis de la clasificación por partición

El método de clasificación por partición es de naturaleza recursiva, por lo que su análisis hará uso de fórmulas de recurrencia. Para el análisis se supone un pivote de valor aleatorio. El coste vendrá dado por dos llamadas recursivas más el tiempo que transcurre en la partición. Así, la fórmula de la clasificación por partición será:

$$T(n) = T(i) + T(n - i - 1) + cn \quad T(1) = T(0) = 1$$

donde i es el número de elementos en la partición de los menores.

En el peor caso el pivote es un extremo (el menor o el mayor elemento) en todas las particiones. Para el análisis es indiferente qué extremo es en cada llamada, por lo que por sencillez se considerará que en todas es el menor. Entonces $i=0$ y por tanto:

$$T(n) = T(n - 1) + cn, \quad n > 1$$

(se desprecia $T(0) = 1$ al no ser significativo).

Aplicando esta ecuación:

$$T(n - 1) = T(n - 2) + c(n - 1)$$

$$T(n - 2) = T(n - 3) + c(n - 2)$$

$$T(2) = T(1) + c(2) \text{ (se desprecia } T(0) = 1 \text{ al no ser significativo)}$$

y sustituyendo se obtiene:

$$T(n) = T(1) + c \sum_{i=2}^n i = O(n^2)$$

Por tanto, en el peor de los casos el coste es de orden n^2 .

En el mejor de los casos el pivote queda en el centro. Para simplificar se supone que las dos particiones son exactamente la mitad de la original, lo que no es significativo en un análisis asintótico. En este caso:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

Dividiendo por n :

$$\frac{T(n)}{n} = \frac{T(n/2)}{n/2} + c$$

Aplicando esta ecuación:

$$\frac{T(n/2)}{n/2} = \frac{T(n/4)}{n/4} + c$$

$$\frac{T(n/4)}{n/4} = \frac{T(n/8)}{n/8} + c$$

$$\frac{T(2)}{2} = \frac{T(1)}{1} + c$$

Sustituyendo las $\log n$ ecuaciones:

$$\frac{T(n)}{n} = \frac{T(1)}{1} + \sum_{i=1}^{\log n} c = \frac{T(1)}{1} + c \log n$$

Por tanto:

$$T(n) = cn \log n + n = O(n \log n)$$

En conclusión, en el mejor caso la clasificación por partición tiene un coste de orden $n \log n$.

En el caso promedio se supone que todos los tamaños de la partición con elementos menores al pivote son igualmente probables. Por tanto, su probabilidad es $1/n$. Obsérvese que esta hipótesis no la satisface cualquier elección del pivote. Por ejemplo, no es aplicable si se escoge sistemáticamente la media.

Con esta hipótesis el promedio de $T(i)$, y por tanto de $T(n-i-1)$ es

$$\frac{1}{n} \sum_{j=0}^{n-1} T(j)$$

Aplicando la ecuación $T(n) = T(i) + T(n-i-1) + cn$, en promedio será:

$$T(n) = \frac{2}{n} \left[\sum_{j=0}^{n-1} T(j) \right] + cn$$

y multiplicando por n :

$$nT(n) = 2 \left[\sum_{j=0}^{n-1} T(j) \right] + cn^2$$

Aplicando la ecuación para $n-1$:

$$(n-1)T(n-1) = 2 \left[\sum_{j=0}^{n-2} T(j) \right] + c(n-1)^2$$

Restando:

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2cn - c$$

Despreciando el término constante y reagrupando se obtiene:

$$nT(n) = (n+1)T(n-1) + 2cn$$

que es una expresión de $T(n)$ en términos de $T(n-1)$.

Dividiendo por $n(n+1)$ y aplicándola:

$$\frac{T(n)}{(n+1)} = \frac{T(n-1)}{n} + \frac{2c}{n+1}$$

$$\frac{T(n-1)}{n} = \frac{T(n-2)}{n-1} + \frac{2c}{n}$$

$$\frac{T(n-2)}{n-1} = \frac{T(n-3)}{n-2} + \frac{2c}{n-1}$$

$$\frac{T(2)}{3} = \frac{T(1)}{2} + \frac{2c}{3}$$

Sumando finalmente se obtiene:

$$\frac{T(n)}{n+1} = \frac{T(1)}{2} + 2c \sum_{i=3}^{n+1} \frac{1}{i}$$

y haciendo uso de los números armónicos la suma es aproximadamente:

$\ln(n+1) + \gamma - \frac{3}{2}$, siendo γ la constante de Euler. Así:

$$\frac{T(n)}{n+1} = O(\log n)$$

y por tanto:

$$T(n) = O(n \log n)$$

En conclusión en el caso promedio el coste de la clasificación por partición es de orden $n \log n$.

Cálculo del k -ésimo mayor elemento

El algoritmo de clasificación por partición permite, además de clasificar un arreglo, calcular de forma eficaz el k -ésimo elemento mayor del mismo. Obsérvese que calcular la mediana consiste en calcular el k -ésimo elemento mayor, con $k = n/2$. Por generalidad se resolverá el problema para k .

En primer lugar, se aplica el proceso de partición de la clasificación rápida a todo el arreglo, con pivote x . Los índices resultantes son tales que:

1. $a_h < x$ para toda $h < i$.
2. $a_h > x$ para toda $h > i$.
3. $i > j$.

Entonces pueden presentarse los tres casos ilustrados en la Figura 2.15. En el primero, el pivote es demasiado pequeño, y el proceso de partición debe repetirse para $a_i \dots a_R$. En el segundo, es demasiado grande y debe repetirse para $a_L \dots a_j$. En el tercer caso el arreglo está dividido en dos particiones iguales, y por tanto se ha encontrado el k -ésimo elemento mayor.

El Programa 2.12 muestra el código en Modula2 para calcular el k -ésimo elemento mayor.

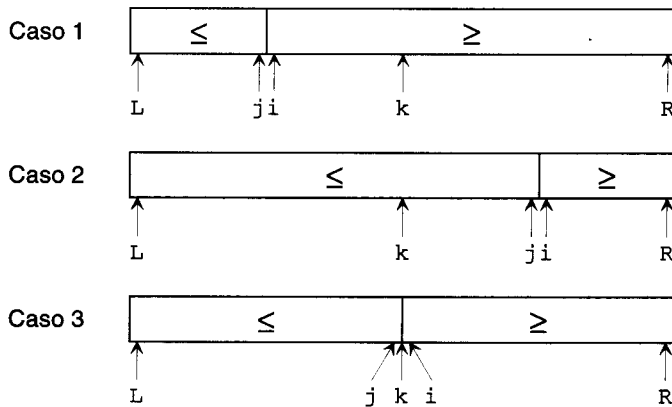


Figura 2.15 Posibles casos en el cálculo del k -ésimo mayor.

Si se supone que en promedio cada división divide a la mitad el tamaño de la partición en la que está el elemento k -ésimo buscado, el número de comparaciones necesarias será:

$$n + \frac{n}{2} + \frac{n}{4} + \dots + 1 = 2n$$

Obsérvese que clasificando primero y eligiendo el valor después el orden es, en promedio, $n \log n$. Así este método es excelente para calcular el k -ésimo elemento mayor ya que es de orden lineal, si bien en el peor caso es de orden n^2 ya que, como en la clasificación por partición, el tamaño de una partición es 1 y todos los demás elementos se disponen en la otra partición, teniendo como coste una suma aritmética.

Programa 2.12 Cálculo del k -ésimo elemento mayor.

```

VAR L,R,i,j: INTEGER;w: Tipo_datos;
BEGIN
  L:=1;R:=n;
  WHILE L<R DO
    x:=a[k];i:=L;j:=R;
    REPEAT
      WHILE a[i]<x DO i:=i+1 END;
      WHILE x<a[j] DO j:=j-1 END;
      IF i<=j THEN
        w:=a[i];a[i]:=a[j];a[j]:=w;i:=i+1;j:=j-1
      END;
    UNTIL i>j;
    IF j<k THEN L:=i END;
    IF k<i THEN R:=j END;
  END;
  x:=a[k];
END;
```

2.3.4 COMPARACIÓN DE LOS MÉTODOS AVANZADOS DE CLASIFICACIÓN

El análisis de la clasificación rápida revela el magnífico comportamiento de este método en los casos mejor y promedio. Sin embargo, no es la solución definitiva ni relega los otros métodos de clasificación. En efecto, obsérvese que en el peor caso es deficiente por lo que cuando las llaves son muy parecidas en valor sería aconsejable otro método, como por ejemplo la clasificación por montón.

La siguiente consideración final debe tenerse muy en cuenta. Aunque, como se ha demostrado, los métodos avanzados presentan unos costes computacionales claramente inferiores a los directos, no son los más eficaces para todo n . Por un lado, los métodos avanzados requieren más operaciones auxiliares que los directos, y por

otro, debe recordarse que los análisis realizados son asintóticos. Por ello, para n pequeño son preferibles los métodos directos ya que en este entorno la diferencia entre los órdenes n^2 y $n \log n$ no es muy significativa y las operaciones auxiliares de los avanzados hacen que estos métodos sean menos eficaces (Figura 2.16). Por tanto, para n pequeños son recomendables los métodos directos, mientras que para n grandes lo son los avanzados.

En el caso de la clasificación por partición debe tenerse en cuenta la anterior consideración. Así, en principio, se realizará el proceso de partición, pero sólo hasta que el tamaño de las particiones es lo suficientemente pequeño para aplicar un método directo eficaz, como la selección directa.

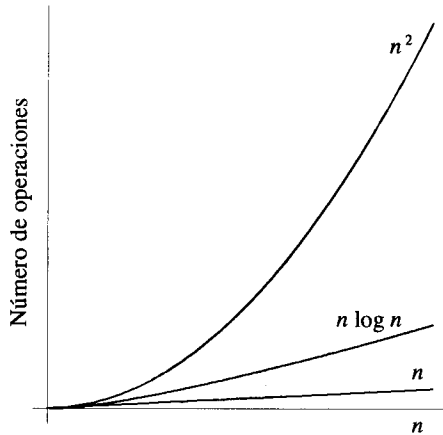


Figura 2.16 Órdenes n , n^2 y $n \log n$.

2.4 CONCLUSIONES

En este capítulo se han presentado los métodos más conocidos para la clasificación sobre arreglos. En todos ellos se presenta la idea en que se basan, seguidamente un ejemplo ilustrativo y finalmente su análisis. En general, los métodos de clasificación de inserción, selección e intercambio directo, y sus mejoras, se aplican cuando el número de datos a ordenar no es muy elevado.

Por otra parte, los métodos de clasificación avanzados, tales como la inserción por incremento decreciente (Shell), la clasificación por montículo de Floyd o la clasificación por partición, son sofisticaciones de las ideas en las que se basan los directos, y aunque su análisis muestra que mejoran los costes de los directos sólo se aplican cuando el número de datos a clasificar es elevado. Esto es debido a que en caso contrario las operaciones auxiliares, que no son evaluadas en un análisis asintótico, afectan sensiblemente al coste total de la implementación haciendo que los directos sean más eficaces.

En cualquier caso, cuando sea necesario decidir qué método de clasificación sobre arreglos se va a aplicar en una situación práctica es imprescindible recordar el análisis realizado para ellos y contrastarlo con la información que se tenga de los datos. Por ejemplo, si la cantidad de datos es elevada y se sabe que el número de datos con llaves iguales es significativo, la clasificación rápida sería desaconsejable ya que estaríamos en una situación cercana al peor de los casos, y en ella ya se ha mostrado que el coste de este método es alto. En conclusión, siempre es conveniente reflexionar brevemente sobre la idoneidad de un método atendiendo a sus costos computacionales.

CAPÍTULO 3

Clasificación en memoria secundaria

3.1 INTRODUCCIÓN

La clasificación sobre arreglos analizada en el capítulo anterior se realiza sobre memoria principal. La cantidad de datos a almacenar en ella puede ser muy grande, y aumenta día a día. Sin embargo, no siempre puede realizarse la clasificación en memoria. Evidentemente si la cantidad de datos es tan grande que sólo es almacenable en memorias secundarias, tales como cintas magnéticas o discos duros, no pueden aplicarse los métodos de clasificación sobre arreglos.

Las memorias secundarias se caracterizan en general porque el acceso es, desde el punto de vista de la programación a alto nivel, secuencial. Es decir, en un instante dado sólo se tiene acceso al primer elemento. Por ello el TDA que se considerará será *la secuencia*, implementada mediante una cinta. El hecho de utilizar las cintas como dispositivos fuente y destino de las secuencias tiene un carácter conceptual y se debe a que es el prototipo clásico de dispositivo secuencial. En un disco duro la secuencia se implementará lógicamente sobre un archivo (o fichero) secuencial. Recuérdese que en los lenguajes de alto nivel estas implementaciones incorporan un puntero de lectura y uno de escritura. El de lectura apunta al primer elemento del fichero cuando éste se abre para lectura. Seguidamente, cada vez que se lee un elemento, el puntero de lectura queda apuntando al siguiente, que es el próximo a leer. Como puede observarse no es posible leer el k -ésimo elemento sin haber leído los $k-1$ anteriores.

El hecho de que el acceso sea secuencial no permite aplicar los métodos de clasificación sobre arreglos. Recuérdese que éstos tenían la característica, muy ventajosa, de poder acceder a cualquier elemento con el mismo coste. Ahora esto no es posible. Por ejemplo, la clasificación Shell compara los elementos en las posiciones i e $i-h_k$, la clasificación por montón compara los elementos en la posición i con los de las posiciones $2i$ y $2i+1$, y la clasificación por partición requiere sucesivas comparaciones del pivote. El coste de estas comparaciones con una estructura de acceso secuencial es inaceptable.

Ciertamente los discos duros tienen a bajo nivel un direccionamiento con acceso directo, esto es, un método de acceso doble: en principio el acceso a los distintos sectores es aleatorio y posteriormente dentro de estos el acceso es secuencial [Dormido, 2000]. Este hecho podrá ser aprovechado por algunas estrategias, como las tablas de transformación Hash, pero para clasificar archivos secuenciales no tiene utilidad alguna. Además el periférico presenta limitaciones añadidas, hay que tener en cuenta los tiempos necesarios para posicionar el disco y mover la cabeza, con lo que las comparaciones de las clasificaciones internas son muy costosas.

En este capítulo se presentan los métodos básicos para la clasificación externa o en memoria secundaria. En primer lugar, se tratarán los métodos para clasificar archivos secuenciales. Los más representativos son:

- Mezcla directa.
- Mezcla natural.
- Mezcla balanceada múltiple.
- Clasificación polifásica.

En este contexto el término mezcla debe entenderse no como un sinónimo de clasificación, sino como una estrategia específica en la que se obtiene una secuencia de salida a partir de dos o más secuencias de entrada. La siguiente definición precisa este término.

Definición 3.1: Se entiende por *mezcla* la tarea de combinar varias secuencias en una sola mediante la selección repetida de los componentes accesibles en cada momento.

La mezcla es una operación auxiliar utilizada como estrategia para desarrollar la tarea de clasificación, y es previa a ésta.

Relacionados con la tarea de mezcla se utilizan los dos términos siguientes:

Definición 3.2: Se entiende por *fase* cada operación que trata un conjunto completo de datos.

Definición 3.3: Se entiende por *pase* (o *etapa*) al proceso más corto que por repetición constituye el proceso de clasificación.

Tras analizar las estrategias basadas en la mezcla, se introducirán dos métodos de clasificación de especial interés para las tareas de ordenación:

- Archivos indexados.
- Tablas de transformación Hashing.

3.2 CLASIFICACIÓN EXTERNA BASADA EN MEZCLA

3.2.1 MEZCLA DIRECTA

La mezcla directa se apoya en la idea más evidente para clasificar secuencias basada en la mezcla. En primer lugar, se divide la cinta a ordenar, c_1 , denominada *cinta fuente*, en dos mitades, c_2 y c_3 , denominadas *cintas destino* de forma que c_2 contiene los elementos de las posiciones impares de c_1 y c_3 los de las posiciones pares. Seguidamente se mezclan las cintas destino combinando el elemento accesible de cada una de las dos cintas y de esta forma la nueva cinta estará compuesta por pares de elementos ordenados. Entonces, la cinta resultante se considera fuente, c_1 , y se repite el proceso anterior con lo que la mezcla producirá una cinta con cuádruplos ordenados. Este proceso se repite hasta que toda la cinta esté ordenada. En cada pase, las subsecuencias ordenadas multiplican su tamaño por dos. Así, el algoritmo puede resumirse de la siguiente forma:

- Tomar como fuente la secuencia original c_1 .
- Dividir la fuente en dos mitades, en las cintas destino c_2 y c_3 .
- Mezclar c_2 y c_3 combinando cada elemento accesible en pares ordenados, en c_1 .
- Repetir el proceso: se obtiene una cinta con cuádruplos ordenados.
- Repetir el proceso hasta que toda la cinta esté ordenada.

La Figura 3.1 ilustra este método. Para crear las dos cintas auxiliares, se recorre la cinta fuente y sus elementos se van enviando alternativamente a las cintas c_2 y c_3 . De esta forma, las dos cintas destino contendrán el mismo número de elementos, salvo que la cinta original tuviese un número impar de elementos en cuyo caso la cinta c_2 tendrá un elemento más. Los elementos de las posiciones impares (marcados con fondo sombreado en la Figura 3.1) se copian en c_2 y los de las posiciones pares (con fondo blanco) se copian en c_3 .

Tras la primera división sólo se tiene acceso al primer elemento en cada cinta destino. Así se comparan el primero de c_2 , con llave 8 y el primero de c_3 con llave 14, se ordenan y se escriben en la *cinta temporal* c . Seguidamente ya se tiene acceso a lo que eran los segundos elementos de c_2 y c_3 , 5 y 9, se ordenan y se escriben en c , y así sucesivamente. El resultado es que c tiene pares de elementos ordenados.

Una vez terminado el proceso, c pasa a ser c_1 y se divide por segunda vez. En la división 2 se crean dos nuevas cintas, enviando a cada una de ellas pares de elementos, ya ordenados en la etapa anterior, en lugar de elementos individuales. De esta forma en la cinta c_2 se copian los elementos 8 y 14 (fondo sombreado) y en la cinta c_3 se copian el 5 y el 9 (fondo blanco) y así sucesivamente tal y como se ilustra en la Figura 3.1. A continuación se pasa a un nuevo proceso de mezcla (mezcla 2). En esta mezcla se compara el 8 y el 5 y se escoge el 5 para ser enviado a c . Después se comparan el 8 y el 9 y se envía a c el 8. A continuación se comparan el 9 y el 14 y se

escriben en c primero el 9 y luego el 14. Obsérvese que el 14 se ha copiado sin ninguna comparación con el siguiente elemento de c_3 (el 13) ya que al igual que en la mezcla 1 se comparaban elementos individuales, ahora, en la mezcla 2 se combinan pares de elementos que en este caso son el 8 y el 14 de c_2 y el 5 y el 9 de c_3 mientras que el 13 pertenece al siguiente par de elementos que deberán ser comparados.

Una vez combinados estos dos pares de elementos se pasa a combinar los siguientes dos pares: el 3 y el 23 de c_2 y el 13 y el 17 de c_3 . Este proceso continua hasta agotar alguna de las dos cintas auxiliares. Cuando sucede esto, se copian de forma directa el resto de elementos de la otra secuencia, que ya están ordenados, lo que se conoce como *copiado de cabos*.

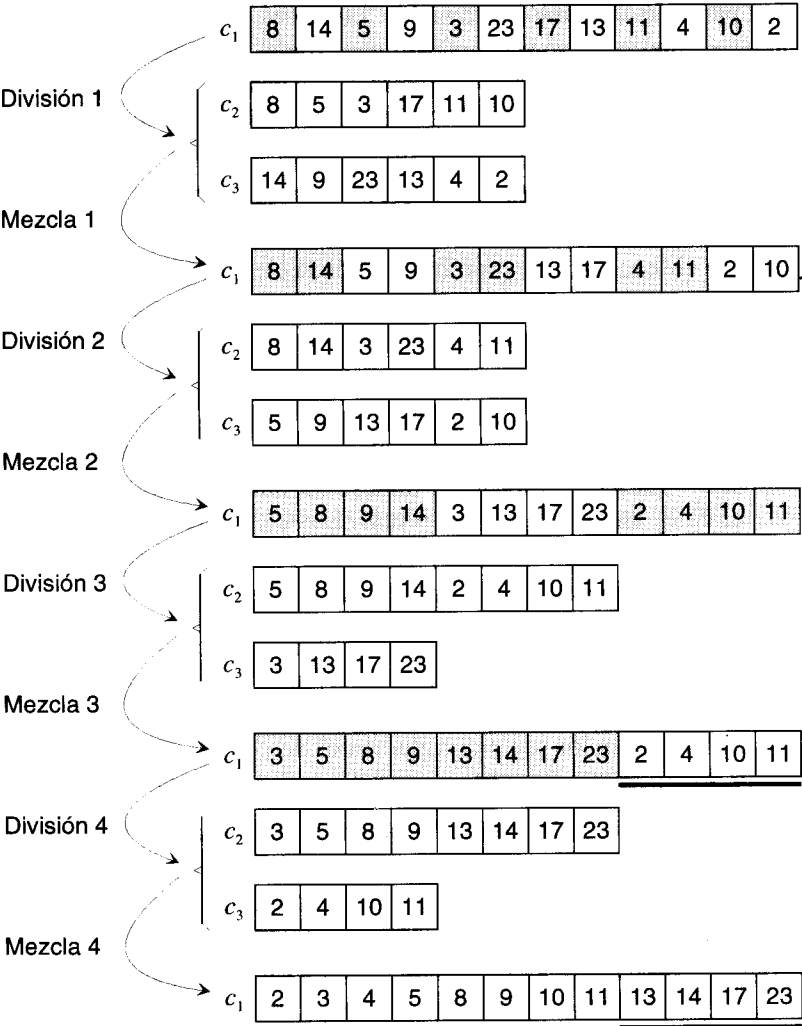


Figura 3.1 Ejemplo de mezcla directa. La barra bajo algunos elementos indica el copiado de cabos.

Obsérvese que el hecho de que se vayan ordenando en potencias de dos se aprovecha para no tener que realizar todas las comparaciones. Así por ejemplo, en la cuarta mezcla el detalle del proceso es el siguiente:

Se comparan (c_2/c_3)	Se escribe en c
3 / 2	2
3 / 4	3
5 / 4	4
5 / 10	5
8 / 10	8
9 / 10	9
13 / 10	10
13 / 11	11 y c. de cabos: (13, 14, 17 y 23)

En la estrategia de mezcla directa expuesta, cada pase o etapa consta de dos fases, una de división y otra de mezcla. Por ello se denomina *mezcla de dos fases*. También es conocida como *mezcla de tres cintas* ya que es el número de cintas que utiliza para realizar la clasificación. Obsérvese que el ejemplo ilustrado en la Figura 3.1 requiere cuatro pases o etapas para completar la clasificación.

El Programa 3.1 muestra la implementación en Módulo2 de la fase de división, y el Programa 3.2 el código de la mezcla directa de dos fases. Finalmente, el Programa 3.3 implementa el algoritmo de clasificación por mezcla directa.

Programa 3.1 Fase de división: Procedimiento Separa.

```

PROCEDURE Separa(VAR c,a,b: File;p: CARDINAL; VAR longc: CARDINAL);
  (*Divide c en a y b en grupos de p elementos y en long devuelve la
  longitud de c*)
VAR
  i,j: CARDINAL;
BEGIN
  Reset(c);
  Create(a, 'nuevoa.dat');
  Create(b, 'nuevob.dat');
  i:=0;
  j:=0;
  longc:=0;
  ReadWord(c,w);
  WHILE ~c.eof DO
    WHILE (~c.eof) & (i<p) DO
      WriteWord(a,w);
      ReadWord(c,w);
      i:=i+1;
    
```

```

        longc:=longc+1;
    END;
    i:=0;
    WHILE (~c.eof)& (j<p) DO
        WriteWord(b,w);
        ReadWord(c,w);
        j:=j+1;
        longc:=longc+1;
    END;
    j:=0;
END;
END Separa;

```

Programa 3.2 Procedimiento mezcla para el algoritmo de mezcla directa.

```

PROCEDURE mezcla(VAR a,b,c: File;p,longc:CARDINAL);
    (*Mezcla a y b en c*)
VAR
    q, (*número de elementos que quedan por mezclar de la primera subsec*)
    r, (*número de elementos que quedan por mezclar de la segunda*)
    m: CARDINAL; (*número de elementos por mezclar*)
    wa,wb:INTEGER;
BEGIN
    Reset(a);
    Reset(b);
    Reset(c);
    m:=longc;
    REPEAT
        IF m>=p THEN q:=p ELSE q:=m END;
        m:=m-q;
        IF m>=p THEN r:=p ELSE r:=m END;
        m:=m-r;
        (* Proposicion de mezcla *)
        ReadWord(a,wa);ReadWord(b,wb);
        WHILE (q#0)&(r#0) DO
            IF wa<wb THEN
                WriteWord(c,wa); q:=q-1;
                IF q#0 THEN
                    ReadWord(a,wa);
                END
            ELSE
                WriteWord(c,wb); r:=r-1;
                IF r#0 THEN
                    ReadWord(b,wb);
                END
            END
        END
    END

```

```

        END
    END;
    (* Copia de cabos *)
    WHILE r>0 DO
        WriteWord(c,wb); r:=r-1;
        IF r#0 THEN
            ReadWord(b,wb);
        END
    END;
    WHILE q>0 DO
        WriteWord(c,wa); q:=q-1;
        IF q#0 THEN
            ReadWord(a,wa);
        END
    END;
    UNTIL m=0;
END mezcla;

```

Programa 3.3 Programa principal que implementa el algoritmo de clasificación por mezcla directa.

```

MODULE Directa;
    FROM InOut IMPORT WriteInt,WriteString,WriteLn,ReadInt;
    FROM FileSystem IMPORT File,Close,Reset,WriteWord,ReadWord,Create;
BEGIN
    p:=1;
    longc:=0;    (*contador del numero de elementos de c*)
    REPEAT
        Separa(c,a,b,p,longc);
        mezcla(a,b,c,p,longc);
        WriteString('c=');
        ListSec(c);
        p:=2*p;
    UNTIL p>=longc;
    WriteString('Fin de ordenación');
    WriteLn;
    ListSec(c);
    Close(c);
    Close(a);
    Close(b);
END Directa.

```

Debe observarse que la fase de división no aporta nada a la clasificación ya que no se combinan los elementos de ninguna manera, y sin embargo tienen un coste significativo ya que realizan aproximadamente la mitad de las operaciones de copia de elementos. Por tanto, sería interesante reducir esta fase.

Las fases de división pueden eliminarse completamente combinando la fase de división con la de mezcla. En vez de mezclar en una sola cinta, *c*, que posteriormente será dividida, puede irse separando en dos, de manera que en el siguiente pase ya estarán divididas. Este método se conoce como *mezcla de una fase*, o *mezcla directa balanceada*. La Figura 3.2 muestra un ejemplo de esta clasificación.

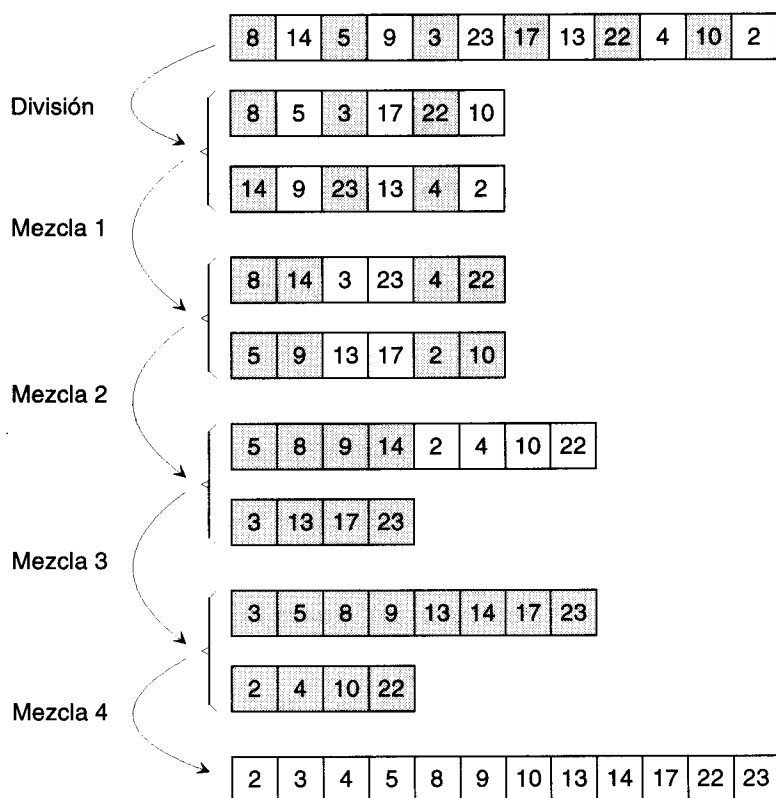


Figura 3.2 Mezcla de una fase.

Como puede observarse una vez que se realiza la división, la mezcla de los conjuntos ordenados impares (con fondo sombreado en la Figura 3.2) se escriben en una cinta y la de los pares (con fondo blanco) en otra distinta.

Evidentemente el coste en tiempo de la mezcla de una fase es menor que la de dos fases ya que sólo realiza la mitad de las operaciones de copia de elementos. El hecho de que necesite una cuarta cinta puede ser un inconveniente, si no se dispone de ella. Obsérvese que aquí se requieren siempre cuatro cintas simultáneamente, ya que el trabajo se realiza leyendo de dos y escribiendo en otras dos. En el método anterior (mezcla directa) se leía de una y se escribía en dos o se leía de dos y se escribía en una. En general, en clasificaciones sobre discos duros por ejemplo, esto

no es ningún inconveniente ya que de hecho el tamaño de estas dos cintas fuente no es significativamente superior al de la cinta fuente de la mezcla de tres cintas, de hecho se puede comprobar que la cantidad de información a almacenar es la misma en ambos casos.

Análisis de la mezcla directa

Antes de nada deben considerarse cuáles son las operaciones más frecuentes que caracterizan el algoritmo. Como puede observarse, éstas serán los movimientos que se realizan en los procesos de copiado de elementos entre cintas, y las comparaciones. Sin embargo, en los métodos de clasificación sobre memoria secundaria el análisis detallado de las comparaciones no es significativo debido a que éstas se realizan en memoria principal, mientras que los movimientos se realizan en memoria secundaria, con lo que el coste de un movimiento es de varios órdenes de magnitud superior al de las comparaciones.

En la mezcla directa se producen $\lceil \log n \rceil$ pases, ya que en cada uno se multiplica por dos el tamaño de las partes ordenadas. Además, en cada pase se copian todos los n elementos una vez. Por tanto, el número de movimientos es:

$$M = n \log n$$

El número de comparaciones es inferior a $n \log n$ ya que en las operaciones de copiado de cabos no se realiza ninguna.

Obsérvese que este método de clasificación tiene un orden similar a los avanzados para clasificar arreglos. Sin embargo, su utilización exige duplicar el tamaño del arreglo, ya que se necesitará almacenar las secuencias fuente y destino, duplicando la información. Debido a este hecho y por economía de almacenamiento, este algoritmo no se suele utilizar para clasificación sobre arreglos.

3.2.2 MEZCLA NATURAL

La mezcla natural aprovecha el hecho de que entre los elementos de la secuencia original, algunos elementos consecutivos ya se encontrarán ordenados entre sí. En efecto, obsérvese que en la clasificación externa la cantidad de llaves a ordenar es grande y entonces la probabilidad de que partes consecutivas de ellas estén ordenadas es elevada. Dadas dos subsecuencias ordenadas de longitud l_1 y l_2 , ambas se mezclan inmediatamente en una de longitud $l_1 + l_2$.

Obsérvese que la mezcla directa va creando subsecuencias ordenadas de tamaño conocido, $(2, 4, \dots, 2^k, \dots)$. En el k -ésimo pase la longitud de todas las subsecuencias ordenadas en la mezcla directa es menor o igual que 2^k , independientemente de las posibles ordenaciones parciales que había inicialmente y que podrían ser significativamente mayores. La mezcla natural aprovecha al máximo las posibles ordenaciones parciales que pudieran estar presentes en los datos originales, consiguiendo un mayor rendimiento.

La siguiente definición precisa el concepto de subsecuencia ordenada.

Definición 3.4: Se denomina *subsecuencia ordenada* al máximo conjunto de elementos consecutivos $a_i \dots a_j$ de la secuencia a ordenar, que satisfacen:

$$(a_{i-1} > a_i) \quad \text{AND} \quad (a_k \leq a_{k+1}, \forall k \ni i \leq k \leq j) \quad \text{AND} \quad (a_j > a_{j+1})$$

La mezcla natural se basa en la combinación de subsecuencias ordenadas, de manera que al mezclar dos secuencias con n subsecuencias ordenadas se obtiene una secuencia con n subsecuencias ordenadas como máximo, ya que se pueden producir subsecuencias ordenadas más largas como se comentará más adelante y se ilustra en el ejemplo de la Figura 3.5. Así el número de subsecuencias ordenadas pendientes de mezcla se divide al menos a la mitad en cada pase. En la última etapa quedará una sola subsecuencia que contiene todos los elementos de la secuencia original completa y por tanto, esta última está ordenada.

En primer lugar, las subsecuencias ordenadas de la cinta a clasificar, c , denominada cinta fuente, se distribuyen en dos cintas destino auxiliares, a y b . Seguidamente se mezcla una subsecuencia ordenada de cada cinta auxiliar. Por reiteración de este proceso se obtiene la secuencia ordenada. Así, el algoritmo puede resumirse de la siguiente forma:

- Fuente c .
- Distribuir las subsecuencias ordenadas de la fuente en las cintas destino a y b .
- Mezclar a y b en c , combinando subsecuencias ordenadas de cada cinta auxiliar.

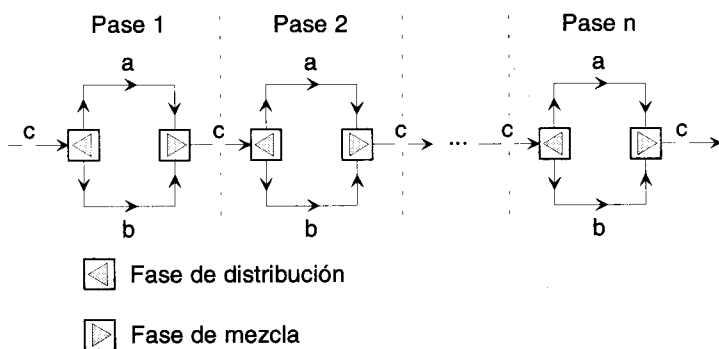


Figura 3.3 Fases y pases de la mezcla natural.

Cada pase en la mezcla natural consta de dos fases, una de distribución y otra de mezcla. La Figura 3.3 ilustra estas fases, donde a y b son las cintas auxiliares destino y c es la cinta fuente.

La Figura 3.4 muestra un ejemplo de mezcla natural. Inicialmente consta de siete subsecuencias ordenadas. En el primer pase se distribuyen alternativamente las

subsecuencias ordenadas entre las dos cintas auxiliares. Las subsecuencias impares (con fondo sombreado en la figura) se escriben en la cinta *a* y las pares (con fondo blanco) en la cinta *b*. En la fase de distribución se copian los elementos de la cinta fuente *c* a una de las cintas hasta que se detecta el final de una subsecuencia ordenada, y entonces se cambia el destino de la copia. Una vez hecha esta división se mezclan la cintas *a* y *b* en una tercera cinta *c* que tendrá la mitad de secuencias ordenadas que la cinta original pero con subsecuencias ordenadas más largas.

Una vez mezclado el primer par de secuencias ordenadas (una de *a* y otra de *b*) se continúa con el siguiente par de la misma forma hasta que se alcance el final del fichero. Repitiendo el proceso de forma iterativa al final se consigue una sola subsecuencia ordenada que incluye todos los elementos, por lo que el trabajo de clasificación ha concluido.

Análogamente en el proceso de mezcla se combinan elementos hasta detectar el fin de una subsecuencia ordenada y entonces se copian los cabos de la subsecuencia aún no copiada completamente.

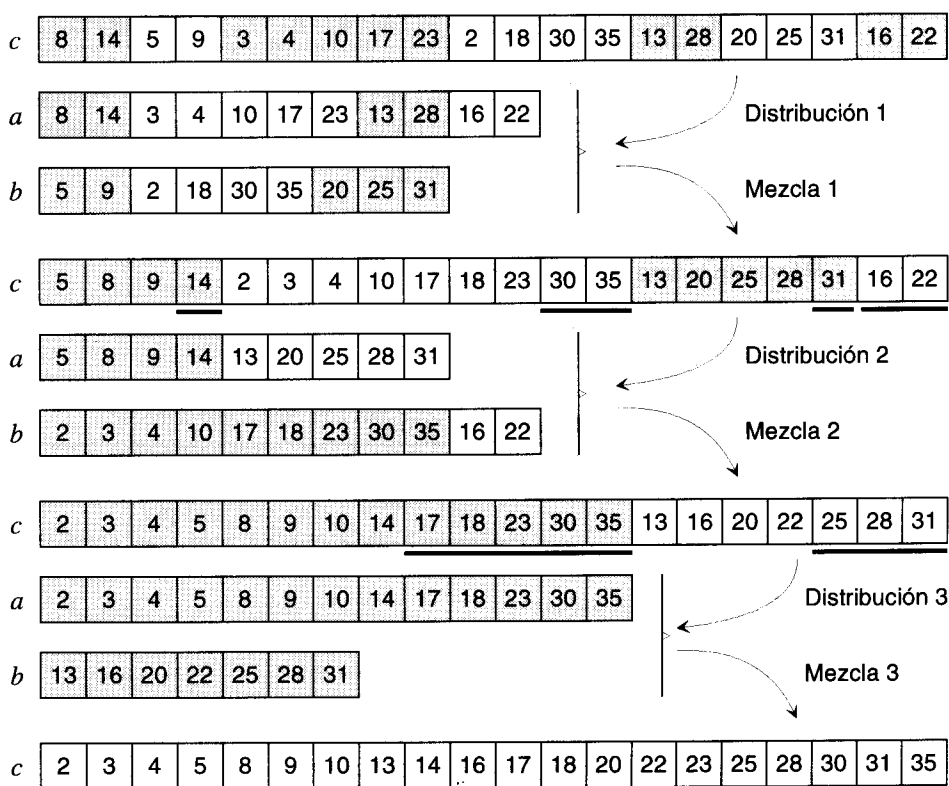


Figura 3.4 Mezcla natural. Se marcan con distinto fondo las subsecuencias ordenadas adyacentes. La barra bajo algunos elementos indica los que se han escrito durante el copiado de cabos, es decir, cuando una subsecuencia ha terminado.

En principio, el algoritmo de la mezcla natural parece indicar que la forma de implementarlo debe basarse en contar las subsecuencias ordenadas que se van distribuyendo de manera que en la fase de mezcla la secuencia combinada tendrá la mitad de subsecuencias ordenadas, o la mitad más uno si el número de subsecuencias ordenadas es impar. En este último caso bastaría copiar la subsecuencia ordenada que no se empareja para la mezcla. Sin embargo, este razonamiento no es correcto debido a que en la fase de distribución pueden realizarse nuevas mezclas. Para analizar este hecho supóngase que en la secuencia destino *a* se distribuyen las subsecuencias ordenadas impares de la secuencia fuente *c*. En el caso en que el primer elemento de la tercera subsecuencia ordenada de *c* sea mayor que el último elemento de su primera subsecuencia ordenada entonces, cuando después de distribuir las en *a* y *b*, ambas subsecuencias (primera y tercera) se combinan en una sola subsecuencia ordenada en *a*, en vez de en las dos que se esperaba. En general se produce mezcla en la fase de distribución cuando el primer elemento de la subsecuencia ordenada $(k+2)$ -ésima es mayor que el último de la k -ésima.

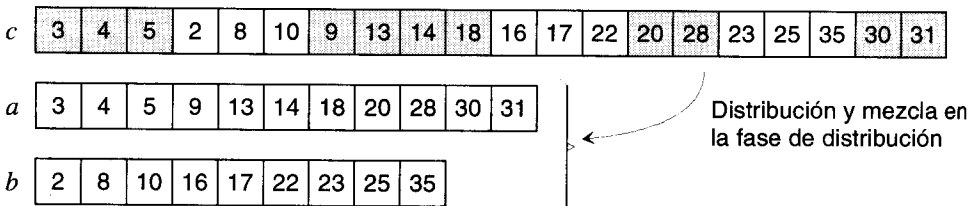


Figura 3.5 Mezcla en la fase de distribución.

Programa 3.4 Fase de distribución de la mezcla natural.

```

PROCEDURE distribuir (VAR a,b,c: File);
VAR
  wc1: INTEGER;          (*elemento de c que se copia en a o b*)
BEGIN
  Create(a, 'nuevov.dat');
  Create(b, 'nuevob.dat');
  Reset(c);
  ReadWord(c, wc1);
  REPEAT copiar(c, a, wc1);
    IF ~c.eof THEN copiar(c, b, wc1); END;
  UNTIL c.eof;
END distribuir;

```

La Figura 3.5 ilustra este comportamiento. Se observa como, inicialmente, la cinta fuente dispone de siete subsecuencias ordenadas, pero en la fase de distribución ambas cintas destino tienen una sola subsecuencia ordenada. En este ejemplo las subsecuencias 1ª, 3ª, 5ª y 7ª se combinan en una única subsecuencia ordenada en *a* al hacer la distribución y lo mismo sucede con las subsecuencias 2ª, 4ª y 6ª en *b*.

Las dos fases del proceso de mezcla natural se implementan en Modula2 en los Programas 3.4 y 3.5.

Programa 3.5 Fase de mezcla para el algoritmo de mezcla natural.

```

PROCEDURE mezclar(VAR L: INTEGER; VAR a,b,c: File);
  (*mezcla a,b en c*)
VAR
  wa1,wa2,wb1,wb2: INTEGER;
  eor: BOOLEAN;
BEGIN
  Reset(a);
  Reset(b);
  Reset(c);
  L:=0;
  ReadWord(a,wa1); ReadWord(b,wb1);
  REPEAT
    LOOP
      IF wa1<wb1 THEN
        copiado(a,c,eor,wa1,wa2);
        wa1:=wa2;
        IF eor THEN
          copiar(b,c,wb1); EXIT;END;
        ELSE
          copiado(b,c,eor,wb1,wb2);
          wb1:=wb2;
          IF eor THEN
            copiar(a,c,wa1); EXIT; END;
          END;
        END;
      L:=L+1
    UNTIL a.eof OR b.eof;
    (* Copias de colas finales: evita el truncamiento *)
    WHILE ~a.eof DO copiar(a,c,wa1); L:=L+1 END;
    WHILE ~b.eof DO copiar(b,c,wb1); L:=L+1 END;
  END mezclar;

```

En cuanto a los detalles de la implementación obsérvese que la fase de distribución debe ir copiando alternativamente subsecuencias máximas ordenadas de la

secuencia de entrada (c) a las dos secuencias de salida (a y b) hasta que se alcance el final del fichero de entrada (c.eof). Para ello, se utiliza el procedimiento copiar (PROCEDURE copiar (VAR x,y:File;VAR wx1: INTEGER);), que copia en y una subsecuencia ordenada de x y en wx1 devuelve el último valor leído de x. Para copiar una subsecuencia ordenada se van copiando elementos de la secuencia de entrada x en la secuencia destino y hasta que se alcanza el final de la subsecuencia ordenada de x (eor:=TRUE). Para ello se hace uso del procedimiento copiado, (PROCEDURE copiado (VAR x,y,:File;VAR eor: BOOLEAN; wx1: INTEGER; VAR wx2: INTEGER);). Este procedimiento copia wx1 (valor de la secuencia de entrada x) en y; además devuelve en wx2 el valor siguiente de x y determina si se ha alcanzado o no el final de la subsecuencia ordenada (eor:=(x.eof) OR (wx2<wx1);).

La devolución del valor wx1 en el procedimiento copiar viene motivada por la naturaleza de los ficheros. La determinación de las subsecuencias máximas ordenadas exige la comparación de elementos consecutivos. Sin embargo, en las secuencias sólo se puede acceder a un elemento, y el último valor que se lee del fichero x, pero que no se escribe en y (porque únicamente sirve para determinar cuando se alcanza el final de una subsecuencia ordenada), será el primer valor que se deba escribir en el siguiente paso de la distribución.

Estos dos procedimientos, copiar y copiado, se podrían reducir a uno solo en la fase de mezcla, pues cualquier llamada a copiado se realiza dentro de copiar. Sin embargo, esta simplificación no se lleva a cabo porque en la fase de mezcla se van a utilizar independientemente.

Una vez realizada la fase de distribución de c a las secuencias destino a y b, la fase de mezcla, haciendo uso de los procedimientos copiar y copiado, va comparando y seleccionando elementos de a y b hasta que una de las secuencias se agote (a.eof OR b.eof). Después de esto, la secuencia que todavía contiene elementos se copia (sin realizar comparaciones). Este procedimiento de mezcla devuelve una variable L que indica el número de subsecuencias mezcladas.

Finalmente el Programa 3.6 muestra la implementación de la mezcla natural en Modula2. En el programa principal se realizan alternativamente las fases de distribución y de mezcla hasta que el número de subsecuencias mezcladas (L) sea 1, lo que significará el final del proceso de clasificación. Se ha hecho uso en el programa principal del procedimiento ListSeq que genera un listado de la secuencia especificada.

Programa 3.6 Mezcla natural.

```
MODULE MezclaNatural;
  FROM InOut IMPORT WriteInt,WriteString,WriteLn,ReadInt;
  FROM FileSystem IMPORT File,Close,Reset,WriteWord,ReadWord,Create;
  VAR L: INTEGER; (* número de subsecuencias ordenadas mezcladas *)
      ch: CHAR;
      w:INTEGER;
      a,b,c:File;
```

```

f: prueba;      (*para la prueba*)
longc,i,
p: CARDINAL;    (*longitud de las subsecuencias por combinar*)
  (*****)
PROCEDURE ListSec(VAR f: File);
VAR w:INTEGER;
BEGIN
  Reset(f);
  ReadWord(f,w);
  WHILE ~f.eof DO
    WriteInt(w,3);
    ReadWord(f,w);
  END;
  WriteLn
END ListSec;
  (*****)
PROCEDURE copiado(VAR x,y:File; VAR eor:BOOLEAN;wx1:INTEGER;VAR wx2:INTEGER);
  (* Copia el valor de x en y. En wx1 recibe el valor a escribir en y. En wx2
  devuelve el valor siguiente de x. Si eor es TRUE entonces se ha llegado al
  final de una subsecuencia ordenada de x*)
BEGIN
  WriteWord(y,wx1);
  ReadWord(x,wx2);
  eor:=(x.eof) OR (wx2<wx1);
END copiado;
PROCEDURE copiar (VAR x,y: File;VAR wx1: INTEGER);
  (*Copia en y una subsecuencia ordenada de x. En wx1 devuelve el último
  valor leído de x*)
VAR
  wx2:INTEGER;
  eor:BOOLEAN;  (*TRUE: Detectado final de subsecuencia ordenada*)
BEGIN  (*de x hacia y*)
  REPEAT copiado(x,y,eor,wx1,wx2);
    wx1:=wx2;
  UNTIL eor;
END copiar;
  (*****)
PROCEDURE distribuir (VAR a,b,c: File);
VAR
  wc1,          (*elemento de c que se copia en a o b*)
  wc2:INTEGER;  (*elemento siguiente a wc1*)
BEGIN
  Create(a,'nuevoa.dat');
  Create(b,'nuevob.dat');
  Reset(c);
  ReadWord(c,wc1);

```

```

    REPEAT copiar(c,a,wcl);
      IF ~c.eof THEN copiar(c,b,wcl);END;
    UNTIL c.eof;
  END distribuir;
  (*****)
  PROCEDURE mezclar(VAR L: INTEGER; VAR a,b,c: File);
    (*mezcla a,b en c*)
  VAR
    wa1,wa2,wb1,wb2: INTEGER;
    eor: BOOLEAN;
  BEGIN
    Reset(a);
    Reset(b);
    Reset(c);
    L:=0;
    ReadWord(a,wa1); ReadWord(b,wb1);
    REPEAT
      LOOP
        IF wa1<wb1 THEN
          copiado(a,c,eor,wa1,wa2);
          wa1:=wa2;
          IF eor THEN
            copiar(b,c,wb1); EXIT;END;
        ELSE
          copiado(b,c,eor,wb1,wb2);
          wb1:=wb2;
          IF eor THEN
            copiar(a,c,wa1); EXIT; END;
        END;
      END;
      L:=L+1
    UNTIL a.eof OR b.eof;
    (* Copias de colas finales: evita el truncamiento *)
    WHILE ~a.eof DO copiar(a,c,wa1); L:=L+1 END;
    WHILE ~b.eof DO copiar(b,c,wb1); L:=L+1 END;
  END mezclar;
  BEGIN
    (*Listado de la secuencia inicial*)
    Reset(c);
    WriteString('c= ');
    ListSec(c);
    REPEAT
      distribuir(a,b,c);
      mezclar(L,a,b,c);
    UNTIL L=1;
  
```

```
WriteLn;  
WriteString('L=1: Fin de la clasificación.');
```

WriteLn;
(*Listado de la secuencia de salida*)
WriteString('c= ');
ListSec(c); Close(a); Close(b); Close(c);
END MezclaNatural.

Análisis de la mezcla natural

El análisis de la mezcla natural es sencillo y similar a los desarrollados anteriormente. En el peor caso el número de movimientos es de orden $n \log n$, e inferior en el caso promedio. El número de comparaciones es mucho mayor ya que se requieren para seleccionar elementos y para determinar el final de una subsecuencia ordenada; sin embargo, como ya se ha comentado, en la clasificación en memoria secundaria, el coste de una comparación es muy inferior al de un movimiento por lo que este incremento no resulta significativo.

3.2.3 MEZCLA BALANCEADA MÚLTIPLE

Como ocurría en la mezcla directa la operación de copia es la más costosa y por tanto, una disminución del número de copias reducirá significativamente el coste. Como puede observarse, el algoritmo de mezcla natural copia tanto en la fase de distribución como en la de mezcla. En esta última es difícil disminuir las copias ya que al fin y al cabo todos los elementos deberán ser copiados para conseguir su clasificación. Sin embargo las copias pueden disminuirse si se consigue reducir el número de pases. Para ello la fase de distribución puede realizarse entre más de dos cintas, de manera que en la mezcla se combinan simultáneamente varias (más de dos) subsecuencias ordenadas, con lo que el número de pases será menor. Además se utilizará una mezcla en una sola fase, tal y como se hizo en la mezcla directa. Esto consiste en eliminar la fase de distribución mediante la copia de las subsecuencias ordenadas, durante el proceso de mezcla, en cintas destino auxiliares que harán de fuente en el siguiente pase, y así las cintas fuente y destino se alternan consecutivamente. Recuérdese que desde el punto de vista de la economía de almacenamiento esto no supone inconveniente alguno. Cuando el número de cintas fuente y destino es el mismo, este método de clasificación se conoce como mezcla balanceada.

La Figura 3.6 muestra un ejemplo de mezcla balanceada múltiple con seis cintas, de las cuales tres son fuente y las otras tres destino. Una vez agotadas las subsecuencias ordenadas en las fuentes, éstas pasan a ser destino y las destino serán las cintas fuente. Obsérvese que las subsecuencias ordenadas se mezclan de tres en tres.

El primer paso consiste en distribuir las subsecuencias ordenadas presentes en la cinta original en tres cintas. A continuación se van realizando sucesivas mezclas con tres cintas fuente y tres cintas destino. Esta acción se realiza combinando en la primera cinta destino (c_4 en este caso) las tres subsecuencias iniciales de c_1 , c_2 y c_3 .

Es decir, se combinan en c_4 las secuencias formadas por los elementos 8 y 14 de c_1 , 5, 9, 13 y 28 de c_2 y 3, 4, 10, 17 y 23 de c_3 . Obsérvese que en c_2 sólo hay una subsecuencia ya que las dos iniciales (elementos 5 y 9 y elementos 13 y 28) se han combinado en una única subsecuencia ordenada.

Como se ha realizado la combinación de las subsecuencias que ocupan la primera posición en las tres cintas destino, se pasa a combinar las que ocupan la segunda posición en las tres cintas fuente (2, 18, 30 y 35 de c_1 y 20, 25 y 31 de c_3 ya que c_2 no tiene más subsecuencias) que se combinan en la siguiente cinta destino (c_5 en este ejemplo). Las subsecuencias en la tercera posición se combinarán en la tercera cinta destino y así sucesivamente hasta agotar todas las cintas destino. Cuando ya se han utilizado todas las cintas destino, se continúa añadiendo elementos a la primera. En el caso general, si hay n cintas destino, la combinación de las subsecuencias de la posición i irán a la cinta $(i - 1) \bmod n + 1$.

El proceso continúa hasta que todas las cintas fuente han sido recorridas completamente. En este momento las cintas destino pasan a ser cintas fuente y las cintas fuente pasan a ser cintas destino, repitiéndose el proceso hasta que se obtenga una única subsecuencia ordenada como se muestra en la última mezcla de la Figura 3.6.

Evidentemente en cada cinta no tiene por qué haber el mismo número de subsecuencias ordenadas y esto debe ser tenido en cuenta a la hora de hacer la implementación. El Programa 3.7 muestra el código en Modula2 para la mezcla balanceada múltiple.

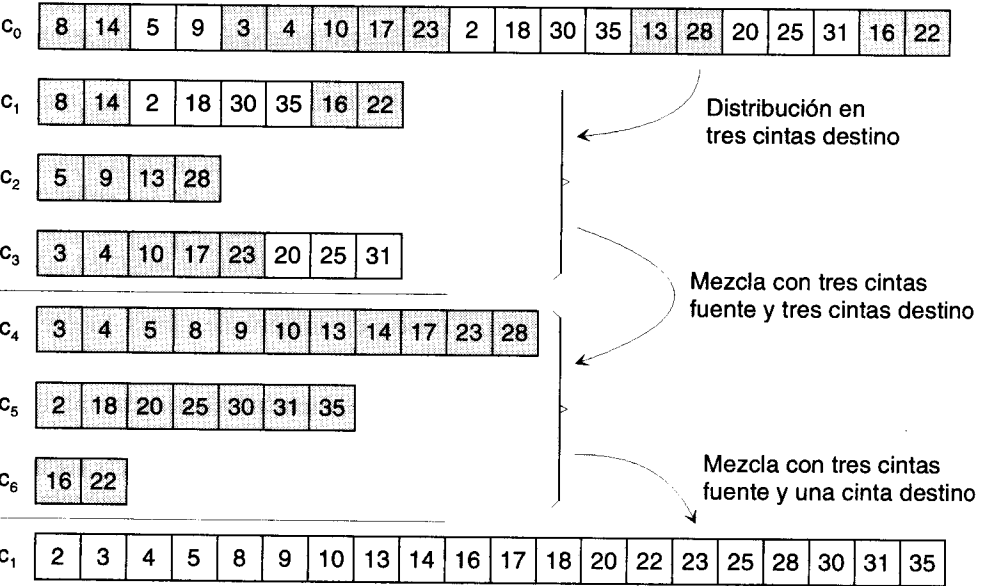


Figura 3.6 Mezcla balanceada con seis cintas. Se han marcado con distinto fondo las distintas subsecuencias ordenadas.

Programa 3.7 Mezcla balanceada.

```

MODULE MezclaBalanceada;
  FROM InOut IMPORT WriteInt,WriteString,WriteLn,ReadString;
  FROM FileSystem IMPORT File,Close,Reset,WriteWord,ReadWord,Create;
  FROM Strings IMPORT Assign;
CONST
  long=20;      (*para la prueba, no se utiliza en el algoritmo*)
  N=6;          (*número de cintas*)
  Nh=N DIV 2;
TYPE
  secno=[1..N];      (*el índice es el n° de secuencia*)
  TipoCadena=ARRAY [1..10]OF CHAR;
VAR i,j,mx,tx: secno;
  L,              (*número de corridas distribuidas*)
  k1,k2: CARDINAL;
  p,q,l,wfini1,wfini2,min,x,wx1,wx2:INTEGER;
  fin1,          (* secuencia inicial de elementos *)
  a,b,c,d : File; (* 4 secuencias en las que reparto *)
  f : ARRAY secno OF File;(*Número de cintas en el proceso de clasificación*)
  nombre: ARRAY secno OF TipoCadena; (*Nombre de cada secuencia*)
  entrada,nombrex: TipoCadena;
  t,             (*mapa de índice de cintas*)
  ta: ARRAY secno OF secno;
  f1: prueba;    (*para la prueba*)
  eor,encontrado:BOOLEAN;
  wx:ARRAY secno OF INTEGER; (*vector de elementos leídos de secuencias*)
  fin:ARRAY secno OF INTEGER; (*si fin[i]=1 se ha alcanzado eor de la
                                secuencia f[i]*)
  (*****);
PROCEDURE ListSec(VAR f: File);
VAR w:INTEGER;
BEGIN
  Reset(f);
  ReadWord(f,w);
  WHILE ~f.eof DO
    WriteInt(w,3);
    ReadWord(f,w);
  END;
  WriteLn
END ListSec;
  (*****);
PROCEDURE copiado(VAR x,y:File; VAR eor:BOOLEAN;wx1:INTEGER;VAR wx2:INTEGER);
(*Copia el valor de x en y;lee el primero de x y determina si es fin de corrida *)

```

```

BEGIN
  WriteWord(y,wx1);
  ReadWord(x,wx2);
  eor:=(x.eof) OR (wx2<wx1);
END copiado;
BEGIN
  (*Creación de todas las secuencias*)
  WriteString('Introducir nombres de los ficheros:');
  WriteLn;
  FOR i:=1 TO N DO
    WriteString('Fichero'); WriteInt(i,2);
    WriteLn;
    ReadString(entrada);
    WriteLn;
    Assign(entrada,nombre[i]);
    Create(f[i],entrada);
  END;
  (*****)
  (*Listado de la secuencia inicial*)
  Reset(fini);
  WriteString('fini= ');
  ListSec(fini);
  (*distribución de corridas iniciales at[1]...t[Nh] *)
  FOR i:=1 TO Nh DO Reset(f[i]) END;
  j:=Nh; L:=0;Reset(fini);
  (*****)
  ReadWord(fini,wfini1);
  REPEAT
    IF j<Nh THEN j:=j+1 ELSE j:=1 END;
    (*copiar una corrida de fini a la secuencia j*)
    REPEAT
      copiado(fini,f[j],eor,wfini1,wfini2);
      IF ~fini.eof THEN
        wfini1:=wfini2;
      END;
    UNTIL eor;
    L:=L+1
  UNTIL fini.eof;
  WriteString('Distribución');
  WriteLn;
  FOR i:=1 TO Nh DO
    WriteString(nombre[i]);
    ListSec(f[i])
  END;
  (*****)
  FOR i:=1 TO Nh DO

```

```

    fin[i]:=0;
END;
FOR i:=1 TO N DO t[i]:=i END;
REPEAT (* mezcla de t[1]...t[Nh] a t[Nh+1]... t[N] *)
(*restablecer secuencias de entrada*)
    IF L<Nh THEN k1:=L ELSE k1:=Nh END;
    FOR i:=1 TO k1 DO
        Reset(f[t[i]]);
    END;
    L:=0;
    j:=Nh+1; (* j=índice de la secuencia de salida *)
    k2:=k1;
    i:=1;
    WHILE i<=k2 DO
        ReadWord(f[t[i]],wx[t[i]]);
        i:=i+1;
    END;
    WriteLn;
    REPEAT (*mezclar una corrida de las entradas a t[j]*)
        L:=L+1;k2:=k1;
        i:=1;
        encontrado:=FALSE;
        REPEAT (*seleccionar la llave mínima*)
            (* búsqueda lineal del menor*)
            IF (~f[t[1]].eof) AND (fin[t[1]]=0) THEN
                i:=1;
            ELSE
                (*búsqueda de la primera secuencia que no ha alcanzado eor ni eof*)
                i:=2;
                WHILE (i<=Nh) AND (NOT encontrado) DO
                    IF ~f[t[i]].eof THEN
                        IF fin[t[i]]=1 THEN (*final de corrida*)
                            i:=i+1
                        ELSE
                            encontrado:=TRUE;
                        END
                    ELSE
                        i:=i+1;
                    END
                END;
            END;
            mx:=i;
            min:=wx[t[i]];
            WHILE i<Nh DO
                i:=i+1;
                IF (~f[t[i]].eof) AND (fin[t[i]]=0) THEN

```

```

        x:=wx[t[i]];
        IF x<min THEN min:=x; mx:=i END;
    END
END;
(* sea t[mx] el n° de secuencia en que ocurre el mínimo*)
(* copiado(f[t[mx]],f[t[j]],eor,min,wx2);*)
copiado(f[t[mx]],f[t[j]],eor,min,wx2);
wx[t[mx]]:=wx2; (*almacena el elemento leído en copiado*)
IF eor THEN
    fin[t[mx]]:=1
END;
IF f[t[mx]].eof THEN      (*eliminar cinta t[mx]*)
    Create(f[t[mx]],nombre[mx]);
    (* decrementar k1 y k2 y reasignar fin*)
    k1:=k1-1;k2:=k2-1;
ELSIF eor THEN  (*terminar corrida en f[t[mx]]*)
    k2:=k2-1;
END;
UNTIL k2=0;  (*agotar todas las entradas*)
IF j<N THEN j:=j+1 ELSE j:=Nh+1 END;
FOR i:=1 TO N DO  (*Poner a cero el indicador de fin de corrida*)
    fin[t[i]]:=0
END;
UNTIL k1=0;
FOR i:=1 TO N DO
    WriteString(nombre[t[i]]);
    ListSec(f[i]);
END;
(*cambiar secuencias*)
FOR i:=1 TO Nh DO
    tx:=t[i];t[i]:=t[i+Nh];t[i+Nh]:=tx;
    nombrex:=nombre[t[i]];Assign(nombre[t[i]],nombre[t[i+Nh]]);
    Assign(nombre[t[i+Nh]],nombrex);
END;
UNTIL L=1; (* la secuencia clasificada es t[1] *)
    WriteString('Secuencia ordenada:');
    ListSec(f[t[1]]);
END MezclaBalanceada.

```

Consideraciones sobre la implementación

Aunque este programa haga uso del procedimiento copiado descrito en la mezcla natural, porque igual que allí necesitamos determinar subsecuencias máximas ordenadas, aquí se introducen cambios significativos respecto al anterior. Estas diferencias vienen originadas por varias circunstancias:

1. Utilización de mayor número de cintas: esto conlleva tener que definir un arreglo de registros f de N elementos ($\text{secno}=[1..N]$), siendo N el número de cintas utilizadas.
2. Determinar cuando se ha alcanzado el final de la fase de mezcla implica algo más que el estudio de si se ha agotado una única secuencia de entrada; se tendrá que almacenar la información de aquellas secuencias no agotadas.
3. Además, al realizar la clasificación por mezcla balanceada en una sola fase se tendrá que restablecer de forma adecuada cuáles son las secuencias fuente y destino tras la finalización de un pase. Para abordar este problema de cambio de cintas, se introduce un indexado de cintas, t , de manera que en lugar de direccionar directamente una cinta por su índice i , se hace mediante el indexado, t . Así, en lugar de f_i , se escribirá f_{t_i} . De esta forma si inicialmente $t_i = i$ para todos los i , entonces un cambio consiste en intercambiar t_i con $(t_{N_h} + 1)$ para todos los $i = 1 \dots N_h$, siendo $N_h = N/2$. Se consideran entonces $f_{t_1} \dots f_{t_{N_h}}$ como secuencias de entrada y $f_{t_{N_h+1}} \dots f_{t_N}$ como secuencias de salida (se llamará cinta j a f_{t_j}).

Una vez que se han creado todas las cintas que se van a utilizar en el proceso de clasificación, ($\text{Create}(f[i], \text{entrada})$), la distribución inicial de subsecuencias ordenadas a partir de la secuencia inicial de elementos (fini), es análoga a la del algoritmo de mezcla natural. Se van copiando en $t[1] \dots t[N_h]$ alternativamente subsecuencias ordenadas (haciendo uso del procedimiento copiado), determinando el final de la subsecuencia cuando se encuentra el primer elemento de la subsecuencia siguiente o cuando se llega al final del fichero fini . Nótese que en este caso no se ha llamado al procedimiento copiar, pues se han incluido sus líneas en el código de la distribución.

El problema de restablecer las secuencias de entrada, al que se hacía alusión en el punto 3 se comenta a continuación. En primer lugar, hay que notar que el número de entradas activas puede ser menor que $N/2$. Puede ser, por ejemplo, que al inicio de la última clasificación haya menos de $N/2$ subsecuencias. Se va a utilizar la variable $k1$ para denotar el número real de entradas utilizadas. Cada vez que se realiza la mezcla de una subsecuencia a partir de las entradas a t_j , $k1$ se decrementa. Cuando $k1=0$, significará el final de un pase, al haberse agotado todas las secuencias de entrada.

Queda por detallar el proceso de mezclar una subsecuencia a partir de las entradas a t_j . En el programa se utiliza el arreglo wx para almacenar el elemento de cada una de las secuencias de entrada que debe ser tratado en cada paso, es decir el último elemento leído de cada una de las cintas de entrada. Esta tarea significa seleccionar la llave menor entre las fuentes disponibles y su colocación en la secuencia de salida actual. Es en este proceso en el que se presenta el problema mencionado en el punto 2). El final de una subsecuencia se alcanza:

- a) Porque la llave siguiente es menor que la actual, o bien
- b) porque se alcanza el final de la fuente.

Si se llega a la situación b), $k_1=0$ y por tanto, la fuente se elimina. Si la situación es la a), se necesita una nueva variable, k_2 , para indicar el número de fuentes disponibles para la siguiente elección de elemento: k_2 inicialmente es igual a k_1 y se decrementa cada vez que se termina una subsecuencia debido a la condición a).

La introducción de k_2 no es suficiente. Además de conocer el número de subsecuencias, se necesita saber cuáles se emplean todavía. Para ello, se utiliza un arreglo (fin: ARRAY secno OF INTEGER) con componentes que indiquen la disponibilidad de las secuencias. Si $\text{fin}[i]=1$ significará que se ha alcanzado el final de la secuencia ordenada en la secuencia $f[i]$. Con todo esto, la eliminación de una cinta, que hay que realizar si se alcanza la situación b), supondrá un decremento de k_1 , además de un decremento de k_2 .

En cuanto a la selección de la llave menor que se realiza en el proceso de mezcla se ha utilizado una búsqueda lineal entre las secuencias disponibles en cada pase: $(\sim f[t[i]].\text{eof}) \text{ AND } (\text{fin}[t[i]]=0)$.

Análisis de la mezcla balanceada múltiple

Supóngase que se utilizan N cintas destino en la fase de distribución. Mezclar m subsecuencias ordenadas que están distribuidas uniformemente en N cintas origina, en una primera fase de mezcla, una subsecuencia ordenada de m/N subsecuencias ordenadas. En la segunda fase de mezcla se tendrán m/N^2 , y en la k -ésima habrá m/N^k . El número total de pases necesarios para clasificar n subsecuencias ordenadas con N cintas será en el peor de los casos $k = \lceil \log_N n \rceil$. Como en cada pase se necesitan n copias, el número total de copias vendrá dado por:

$$M = n \lceil \log_N n \rceil.$$

3.2.4 CLASIFICACIÓN POLIFÁSICA

Este método de clasificación, propuesto por R.L. Gilstad [Gilstad, 1960], mejora el rendimiento de la mezcla balanceada. Recuérdese que la mezcla balanceada es una mezcla de una sola fase, eliminando las copias que no aportan mezcla mediante la unión de las fases de distribución y mezcla. Para ello utiliza N cintas de las cuales $N/2$ son fuente y las otras $N/2$ destino, intercambiando sucesivamente su papel. La propuesta de la clasificación polifásica consiste en utilizar las cintas de modo más complejo, de manera que las cintas fuente y destino no son establecidas *a priori*, sino como consecuencia de la mezcla realizada. Así, por ejemplo, si se tienen tres cintas y en un proceso de mezcla dos hacen de fuente y una tercera de destino, al finalizar las combinaciones, hará de cinta destino aquella que se haya agotado, la cual sólo podrá ser identificada tras la mezcla.

La Figura 3.7. muestra un ejemplo de clasificación polifásica con tres cintas. Se indica el número de subsecuencias ordenadas que contiene cada cinta en cada momento. Así inicialmente la cinta c_1 contiene 13 subsecuencias ordenadas, la c_2 8 y la c_3 se encuentra vacía. En el primer proceso de mezcla se combina una subsecuencia ordenada de c_1 con una de c_2 y se copia la subsecuencia ordenada resultante en la cinta vacía, c_3 , y así sucesivamente hasta que una de las cintas fuente se agote. Entonces se identifica la cinta agotada, en este caso c_2 , que será la cinta destino en el proceso de mezcla. Por tanto, en esta primera etapa se han mezclado 8 subsecuencias ordenadas de c_1 con las 8 de c_2 , copiándose las 8 resultantes en c_3 y quedando 5 en c_1 . En la segunda mezcla se combinan en c_2 las 5 subsecuencias que quedan en c_1 con las 5 primeras de c_3 resultantes de la combinación anterior, quedando 3 subsecuencias ordenadas en c_3 , y siendo c_1 la próxima cinta destino pues ha quedado vacía. En la tercera mezcla se combinan 3 primeras subsecuencias ordenadas de c_2 con las 3 de c_3 en c_1 , quedando 2 en c_2 y siendo c_3 la siguiente cinta destino, y así sucesivamente hasta la última mezcla en la que se combinan en c_1 una subsecuencia ordenada de c_2 y otra de c_3 , quedando una sola subsecuencia ordenada en c_1 , con lo que el proceso de clasificación ha finalizado.

	c_1	c_2	c_3
Distribución inicial	13	8	0
Mezcla 1	5	0	8
Mezcla 2	0	5	3
Mezcla 3	3	2	0
Mezcla 4	1	0	2
Mezcla 5	0	1	1
Mezcla 6	1	0	0

Figura 3.7 Clasificación polifásica de tres cintas.

	c_1	c_2	c_3
Distribución inicial	14	8	0
Mezcla 1	6	0	8
Mezcla 2	0	6	2
Mezcla 3	2	4	0
Mezcla 4	0	2	2
Mezcla 5	2	0	0

Figura 3.8 Clasificación polifásica de tres cintas no satisfactoria.

Como puede observarse la clasificación polifásica aprovecha al máximo las cintas ya que con N cintas realiza mezclas de $N-1$ subsecuencias ordenadas, mientras que la balanceada lo hacía de $N/2$ en $N/2$. Sin embargo, no resulta satisfactoria en todos los casos. La Figura 3.8 muestra un ejemplo de ello. En la última mezcla, quedando dos cintas agotadas, la cinta c_1 tiene todavía dos subsecuencias ordenadas, con lo que no se ha clasificado la secuencia.

Por tanto, y teniendo en cuenta que este método aprovecha al máximo las cintas, es imprescindible realizar un análisis detallado del mismo. Para analizar cualitativamente cuándo la clasificación es satisfactoria se construye la Figura 3.9. Lo que se pretende es que al final haya una sola subsecuencia ordenada en una cinta y las demás estén agotadas. Sean c_1 y c_2 las cintas que en cada pase tienen subsecuencias ordenadas. Así en un nivel 0 la cinta c_1 tendrá la secuencia ordenada y la c_2 estará vacía. Esto sucederá cuando en el pase anterior, nivel 1, cada una de ellas tenga una subsecuencia ordenada. Para ello en el nivel anterior, la distribución de secuencias podría ser 2 y 1, por lo tanto en el nivel 3 la distribución podría ser 3 y 2, y así sucesivamente se pueden construir situaciones en las que la distribución en cada nivel permite la clasificación satisfactoria.

En la Figura 3.9 sólo se incluyen las cintas no vacías en cada nivel. Hay que tener en cuenta que en una clasificación polifásica de tres cintas, en cada nivel siempre hay una vacía por lo que se elimina para el análisis. Con $c_1(n)$ y $c_2(n)$ se indican las cintas que hacen la función de c_1 y c_2 en cada nivel n .

Nivel	$c_1(n)$	$c_2(n)$	$c_1(n)+c_2(n)$
0	1	0	1
1	1	1	2
2	2	1	3
3	3	2	5
4	5	3	8
5	8	5	13
6	13	8	21
7	21	13	34

Figura 3.9 Construcción de la clasificación polifásica satisfactoria de tres cintas.

La Figura 3.9 muestra que la suma en cada nivel debe ser un número de Fibonacci. Sin embargo, esto no es condición suficiente como muestra la Figura 3.10. En ella se dispone de 34 subsecuencias ordenadas repartidas inicialmente de manera que hay 22 en una cinta y 12 en otra. Desarrollando el proceso de clasificación polifásica se observa que no se clasifica adecuadamente. Por tanto, el hecho de que el número de subsecuencias ordenadas sea un número de Fibonacci no garantiza la clasificación.

	c_1	c_2	c_3
Distribución inicial	22	12	0
Mezcla 1	10	0	12
Mezcla 2	0	10	2
Mezcla 3	2	8	0
Mezcla 4	0	6	2
Mezcla 5	2	4	0
Mezcla 6	0	2	2
Mezcla 7	2	0	0

Figura 3.10 Clasificación polifásica de tres cintas no satisfactoria.

Analíticamente, el proceso de clasificación satisfactoria mostrado en la Figura 3.9 puede expresarse de la forma que se describe a continuación.

Para $n=0$

$$c_1(0) = 1 \quad y$$

$$c_2(0) = 0$$

y para cada nivel, $n > 0$ se observa que:

$$c_2(n+1) = c_1(n)$$

$$c_1(n+1) = c_1(n) + c_2(n)$$

Así:

$$c_1(n+1) = c_1(n) + c_1(n-1)$$

$$c_2(n+1) = c_1(n).$$

Por tanto, $c_1(n)$ son números de Fibonacci, en los que cada uno se obtiene sumando los dos predecesores, y además $c_2(n)$ es el predecesor de $c_1(n)$. En conclusión, la clasificación polifásica con tres cintas resultará satisfactoria si la distribución inicial de subsecuencias ordenadas entre las cintas fuente es tal que son números consecutivos de Fibonacci.

Debe tenerse en cuenta que esta condición es suficiente pero no necesaria, es decir, existen otras distribuciones iniciales para las que se clasifica satisfactoriamente. La Figura 3.11 muestra un ejemplo ilustrativo de una distribución alternativa satisfactoria.

	c_1	c_2	c_3
Distribución inicial	11	7	0
Mezcla 1	7	0	4
Mezcla 2	3	4	0
Mezcla 3	0	1	3
Mezcla 4	1	0	2
Mezcla 5	0	1	1
Mezcla 6	1	0	0

Figura 3.11 Ejemplo de distribución alternativa para la clasificación polifásica de tres cintas.

Así por ejemplo, con 21 subsecuencias ordenadas en total, distribuyendo 20 en una cinta y 1 en otra, tras 20 pases se clasifica trivialmente. Sin embargo, no debe olvidarse que se trata de obtener la clasificación en el menor número de pases. En este sentido, la discusión sobre la distribución inicial óptima de subsecuencias ordenadas entre las cintas fuente, queda fuera del alcance de este texto.

Cuando el número de cintas es mayor que tres, el proceso de clasificación es el mismo, si bien el análisis y las conclusiones difieren ligeramente. La Figura 3.12 muestra una clasificación polifásica satisfactoria con seis cintas y la Figura 3.13 ilustra la construcción de la clasificación polifásica de seis cintas.

Para la obtención de esta última figura, hay que tener en cuenta que en el nivel más bajo (nivel 0) sólo puede haber una única subsecuencia. En el nivel 1 deberá haber por tanto una subsecuencia en cada cinta. En el nivel 2 deberá haber en cada cinta un número de subsecuencias tal que al combinarlas queden todas las cintas con una única subsecuencia, tal y como se ha visto para el nivel 1. Esta situación sólo se puede alcanzar si todas las cintas tienen dos subsecuencias menos una que sólo puede tener una subsecuencia ordenada. Esta cinta puede ser cualquiera, pero en la figura se ha supuesto que es la cinta c_5 . Continuando con el mismo argumento se obtiene el número de subsecuencias de cada cinta para cada uno de los niveles.

	c_1	c_2	c_3	c_4	c_5	c_6
Distribución inicial	16	15	14	12	8	0
Mezcla 1	8	7	6	4	0	8
Mezcla 2	4	3	2	0	4	4
Mezcla 3	2	1	0	2	2	2
Mezcla 4	1	0	1	1	1	1
Mezcla 5	0	1	0	0	0	0

Figura 3.12 Ejemplo de clasificación polifásica con seis cintas

Nivel	$c_1(n)$	$c_2(n)$	$c_3(n)$	$c_4(n)$	$c_5(n)$	$\sum_{i=1}^5 c_i(n)$
0	1	0	0	0	0	1
1	1	1	1	1	1	5
2	2	2	2	2	1	9
3	4	4	4	3	2	17
4	8	8	7	6	4	33
5	16	15	14	12	8	65

Figura 3.13 Construcción de la clasificación polifásica de seis cintas satisfactoria.

Analíticamente:

$$c_5(n+1) = c_1(n)$$

$$c_4(n+1) = c_1(n) + c_5(n) = c_1(n) + c_1(n-1) \quad (3.1)$$

$$c_3(n+1) = c_1(n) + c_4(n) = c_1(n) + c_1(n-1) + c_1(n-2),$$

$$c_2(n+1) = c_1(n) + c_3(n) = c_1(n) + c_1(n-1) + c_1(n-2) + c_1(n-3)$$

$$c_1(n+1) = c_1(n) + c_2(n) = c_1(n) + c_1(n-1) + c_1(n-2) + c_1(n-3) + c_1(n-4)$$

con $c_4 = 1$,

y $c_i = 0$ para $i < 4$,

que son los números de Fibonacci de orden 4.

Generalizando este resultado para una clasificación polifásica con N cintas, tenemos que para que la clasificación sea satisfactoria (o perfecta) el número de subsecuencias ordenadas iniciales tiene que ser suma de cualquier $N-1$, $N-2$, ..., 1 sumas de números de Fibonacci de orden $N-2$. La Figura 3.14 muestra estos números.

Obsérvese la eficacia del método. Por ejemplo, para clasificar 28697 subsecuencias ordenadas sólo hacen falta 14 pases con una clasificación polifásica de seis cintas. Esto da idea del gran potencial de la clasificación polifásica.

Cuando no se satisfacen las condiciones de distribución perfecta, tanto en el caso de tres cintas como en el de más de tres, la solución consiste en simular subsecuencias ordenadas vacías hasta satisfacer la condición. A tales subsecuencias se les denomina subsecuencias ordenadas *ficticias*.

	Número de cintas (N)					
	3	4	5	6	7	8
Números de Fibonacci de orden N-2	3	5	7	9	11	13
	5	9	13	17	21	25
	8	17	25	33	41	49
	13	31	49	65	81	97
	21	57	94	129	161	193
	34	105	181	253	321	385
	55	193	349	497	636	769
	89	355	673	977	1261	1531
	144	653	1297	1921	2501	3049
	233	1201	2500	3777	4961	6073
	377	2209	4819	7425	9841	12097
	610	4063	9289	14597	19521	24097
	987	7473	17905	28697	38721	48001

Figura 3.14 Número de subsecuencias ordenadas iniciales.

Consideraciones sobre la implementación

Detallemos el problema de la distribución inicial de las subsecuencias ordenadas reales y las ficticias en las $N-1$ cintas. Para ello veamos el significado de la mezcla de subsecuencias reales y ficticias. Si se selecciona una ficticia de una secuencia i , significa que la secuencia i se ignora durante esta combinación, dando como resultado una mezcla proveniente de menos de $N-1$ fuentes. Si la mezcla se produce entre $N-1$ subsecuencias ficticias, provenientes de todas las $N-1$ fuentes, en la secuencia de salida se registrará una subsecuencia ficticia. Por tanto, interesa que las subsecuencias ficticias se distribuyan en las $N-1$ secuencias con la mayor uniformidad posible, para que así las mezclas se produzcan a partir del mayor número posible de secuencias reales.

Del análisis desarrollado anteriormente queda claro que los números de Fibonacci de orden $N-2$ especifican los números deseados de subsecuencias en cada fuente en el proceso de distribución de un número desconocido de subsecuencias en $N-1$ cintas. Suponiendo por ejemplo, $N=6$ y consultando la Figura 3.13, comenzamos por distribuir las subsecuencias como lo indica la línea correspondiente al nivel $n=1$ (1,1,1,1,1). Si se dispone de más subsecuencias ordenadas, pasamos a la siguiente línea (2,2,2,2,1); y si todavía hay más a la siguiente y así sucesivamente. Cada uno de estos niveles se designan en el programa con la letra n . Luego en la distribución inicial de subsecuencias es necesario realizar los siguientes pasos:

1. Objetivo de la distribución: números de Fibonacci de orden $N-2$, nivel 1.
2. Distribuir conforme al objetivo establecido.
3. Si se alcanza el objetivo, se calcula el siguiente nivel de los números de Fibonacci (a partir de las expresiones 3.1); la diferencia entre ellos y los del paso ante-

rior constituye el nuevo objetivo de la distribución. Volver al paso 2. Si el objetivo no puede alcanzarse, por haberse agotado la fuente, finalizar el proceso de distribución.

Queda por detallar en que consiste el paso 2. Es en este paso en el que van a intervenir las subsecuencias ficticias. La forma de calcular el número de subsecuencias ficticias será restando las que debería haber en una cinta en un nivel dado (Ecuación 3.1) menos las reales que se han ido introduciendo. Así pues, se va a utilizar un contador s_i para $i = 1, \dots, N - 1$ e inicializarlo para cada nivel al valor que debe tener para cada cinta (número de subsecuencias ordenadas que hay que poner en la cinta i en ese paso). Posteriormente al introducir las subsecuencias ordenadas reales estos contadores se irán decrementando, de manera que si se agota la fuente, s_i indicará el número de subsecuencias ficticias en la cinta i .

Este algoritmo de distribución hace uso de un procedimiento llamado seleccionar, que se activa cada vez que se ha copiado una subsecuencia ordenada y se escoge una nueva fuente para la siguiente. La variable global j va a designar el índice de la secuencia destino actual. Los arreglos c y s van a contener los números de subsecuencias ordenadas ideales y ficticias para cada secuencia i respectivamente. Con esto, el procedimiento seleccionar sirve para calcular el siguiente nivel de la Figura 3.13, esto es, los valores $c_1(n) \dots c_{N-1}(n)$ cada vez que aumente el nivel. En este procedimiento también se calcula el siguiente objetivo, es decir el cálculo de las diferencias $s_i = c_i(n) - c_i(n - 1)$. La última asignación del procedimiento seleccionar ($s[j] := s[j] - 1$) sustituye al cambio de una subsecuencia ficticia en la cinta j por una subsecuencia ordenada real.

Una formulación inicial de la distribución, suponiendo que se dispone de un procedimiento copiar que copia una subsecuencia de la fuente f_0 a la fuente f_j (y suponiendo siempre que la fuente contiene al menos una subsecuencia ordenada) es:

```
REPEAT
  seleccionar;
  copiar
UNTIL f0.eof;
```

Sin embargo, aquí hay que tener en cuenta lo que se explicó en el algoritmo de mezcla natural relativo a que la simple distribución puede ocasionar un menor número de subsecuencias ordenadas de salida que de entrada. El hecho de que dos subsecuencias lleguen al mismo destino y constituyan una sola subsecuencia ordenada hace que los números de subsecuencias ordenadas, supuestos inicialmente, sean incorrectos. La corrección del algoritmo de mezcla natural no depende del número de etapas, por lo que este efecto se podía 'ignorar'. Sin embargo, la clasificación polifásica precisa del control del número exacto de subsecuencias ordenadas en cada cinta. Para tener esto en cuenta se necesita conservar las llaves del último elemento de la subsecuencia ordenada anterior en cada cinta. Para ello se utiliza un

arreglo *wesc* que almacena en cada una de sus componentes el último elemento escrito en cada una de las cintas y que por tanto se tiene que actualizar después de realizar cada llamada a copiar. De esta forma *wesc* obtendrá valores sólo después de copiar la primera subsecuencia ordenada. Según ésto, en la fase de distribución se distribuirá primero una subsecuencia ordenada en cada una de las $N-1$ cintas destino, sin examinar *wesc*[*j*]. A continuación se distribuyen el resto de las subsecuencias ordenadas, tal y como se observa en el código del Programa 3.8.

Finalizada la distribución queda por comentar cómo se efectúa el proceso de mezcla. Su estructura es semejante a la de la mezcla balanceada. Hay un bucle externo que mezcla las subsecuencias ordenadas hasta que quedan agotadas las fuentes; un ciclo interno que mezcla una sola subsecuencia ordenada proveniente de cada fuente y un ciclo más interno que selecciona la llave inicial y transmite el elemento en cuestión al archivo en blanco. Las principales diferencias son las siguientes:

1. En vez de $N/2$ sólo hay una secuencia de salida en cada pase.
2. En vez de cambiar las $N/2$ secuencias de entrada y las $N/2$ secuencias de salida al terminar cada pase, las secuencias se rotan. Esto se consigue, como ya se detalló, utilizando un indexado de cintas t .
3. El número de cintas de entrada varía entre una subsecuencia ordenada y la siguiente, y depende de los contadores de las subsecuencias ficticias. Si $s_i > 0$ para toda i , entonces se combinan $N-1$ subsecuencias ficticias en una subsecuencia ficticia con sólo incrementar el contador s_n de la secuencia de salida. Cuando $s_i = 0$ se mezcla una subsecuencia a partir de todas las fuentes y se disminuye en todas las otras secuencias, lo cual indica que se extrajo una subsecuencia ficticia. Denotamos con k el número de secuencias de salida que participan en una combinación.
4. Es imposible determinar la terminación de una fase mediante la condición final de la secuencia $(N-1)$ -ésima, ya que pueden requerirse más combinaciones que incluyan subsecuencias ficticias provenientes de la fuente. El número teóricamente necesario de subsecuencias depende de los coeficientes c_j . Estos, se calcularon en la fase de distribución y ahora se pueden recalcular en el sentido inverso.

La operación de la mezcla es muy parecida a la de la mezcla balanceada. Se ha introducido en este caso un cambio respecto a aquél y es el utilizar un nuevo mapa de índice de cintas t_c para determinar la disponibilidad de una cinta en lugar del arreglo *fin* que se utilizaba en la mezcla N -tuple. Este mapa se emplea en lugar de t tal que $t_{c1} \dots t_{ck}$ sean los índices de las cintas disponibles. Todos estos comentarios pueden estudiarse con detenimiento en el Programa 3.8 que muestra el algoritmo completo de la clasificación polifásica.

Programa 3.8 Clasificación polifásica.

```

MODULE Polifasica;
  FROM InOut IMPORT WriteInt,WriteString,WriteLn,ReadString;
  FROM FileSystem IMPORT File,Close,Reset,WriteWord,ReadWord,Create;
  FROM Strings IMPORT Assign;
CONST N=6;
  long=18;
TYPE secno=[1..N];
  TipoCadena=ARRAY[1..10] OF CHAR;
  prueba=ARRAY[0..17] OF INTEGER;
VAR i,j,mx,tx,tn: secno;
  l,k,sn,z,nivel: CARDINAL;
  min,x: INTEGER;
  c,s : ARRAY secno OF CARDINAL;
  t,tc: ARRAY secno OF secno;
  f0: File;
  wx,wesc: ARRAY secno OF INTEGER;
  wx1,wx2,waux: INTEGER;
  nombre: ARRAY secno OF TipoCadena;
  entrada: ARRAY[1..15] OF CHAR;
  f: ARRAY secno OF File;
  eor: BOOLEAN;
(*****)
PROCEDURE ListSec(VAR f: File);
VAR
  w: INTEGER;
BEGIN
  Reset(f);
  ReadWord(f,w);
  WHILE ~f.eof DO
    WriteInt(w, 4);
    ReadWord(f,w);
  END;
  WriteLn
END ListSec;
(*****)
PROCEDURE copiado(VAR x,y:File; VAR eor:BOOLEAN;wx1:INTEGER;VAR wx2:INTEGER);
  (* Copia el valor de x en y. En wx1 recibe el valor a escribir en y. En wx2
  devuelve el valor siguiente de x. Si eor es TRUE entonces se ha llegado al
  final de una subsecuencia ordenada de x*)
BEGIN
  WriteWord(y,wx1);
  ReadWord(x,wx2);
  eor:=(x.eof) OR (wx2<wx1);
END copiado;

```

```

PROCEDURE copiar (VAR x,y: File;VAR wx1: INTEGER;VAR wesc: INTEGER);
  (*Copia en y una subsecuencia ordenada de x. En wx1 devuelve el último
  valor leído de x, y en wesc el último escrito *)
VAR
  wx2:INTEGER;
  eor:BOOLEAN;  (*TRUE: Detectado final de subsecuencia ordenada*)
BEGIN  (*de x hacia y*)
  REPEAT
    copiado(x,y,eor,wx1,wx2);
    wesc:=wx1;
    wx1:=wx2;
  UNTIL eor;
END copiar;
  (*****)
PROCEDURE seleccionar;
VAR
  i,z: CARDINAL;
BEGIN
  IF s[j]<s[j+1] THEN j:=j+1
  ELSE
    IF s[j]=0 THEN
      nivel:=nivel+1; z:=c[1];
      FOR i:=1 TO N-1 DO
        s[i]:=z+c[i+1]-c[i];c[i]:=z+c[i+1]
      END
    END;
    j:=1
  END;
  s[j]:=s[j]-1
END seleccionar;
  (*****)
BEGIN
  (*Listado de la secuencia inicial*)
  Reset(f0);
  WriteString('f0= ');
  ListSec(f0);
  WriteString('Introducir nombres de los ficheros:');
  WriteLn;
  FOR i:=1 TO N DO
    WriteString('Fichero'); WriteInt(i,2);
    WriteLn;
    ReadString(entrada);
    WriteLn;
    Assign(entrada,nombre[i]);
    Create(f[i],entrada);
  END;

```

```

FOR i:=1 TO N DO Reset(f[i]) END;
(*distribución de corridas iniciales *)
FOR i:=1 TO N-1 DO c[i]:=1;s[i]:=1;Reset(f[i]) END;
nivel:=1;j:=1; c[N]:=0;s[N]:=0;Reset(f0);
ReadWord(f0,wx1);
wx[1]:=wx1;          (*wx[i]: último elemento leído en f[i]*)
REPEAT
    seleccionar;
    copiar(f0,f[j],wx1,waux);
    wesc[j]:=waux;      (*wesc[i]: último elemento escrito en f[i]*)
    wx[j+1]:=wx1;
UNTIL f0.eof OR (j=N-1);
IF NOT f0.eof THEN
    wx1:=wx[N];  (*elemento siguiente a tratar de f0*)
END;
WHILE NOT f0.eof DO
    seleccionar;      (*primero=último elemento escrito sobre f[j]*)
    IF wesc[j]<=wx1 THEN
        copiar(f0,f[j],wx1,waux);
        wesc[j]:=waux;
        IF f0.eof THEN s[j]:=s[j]+1
        ELSE
            copiar(f0,f[j],wx1,waux);wesc[j]:=waux END
        ELSE
            copiar(f0,f[j],wx1,waux);wesc[j]:=waux;END
    END;
WriteString('Distribucion');
WriteLn;
FOR i:=1 TO N DO
    WriteString(nombre[i]);
    ListSec(f[i]);
END;
FOR i:=1 TO N-1 DO t[i]:=i; Reset(f[i]) END;
t[N]:= N;
i:=1;
WHILE i<=N-1 DO
    IF ~f[i].eof THEN
        ReadWord(f[t[i]],wx[t[i]])
    END;
    i:=i+1;
END;
wx[N]:=0;
REPEAT  (*mezclar de t[1] ...t[N-1] a t[N]*)
    z:=c[N-1];s[N]:=0; Reset(f[t[N]]);
    REPEAT k:=0;  (*mezclar una corrida*)
        FOR i:=1 TO N-1 DO

```

```

        IF s[i]>0 THEN s[i]:=s[i]-1
        ELSE k:=k+1; tc[k]:=t[i];
        END
    END;
    IF k=0 THEN s[N]:=s[N]+1
    ELSE (*meclar una corrida real de t[1] ... t[k] a t[N]*)
        REPEAT (*seleccion de llave mínima*)
            i:=1;
            mx:=1;min:=wx[tc[1]];
            WHILE i<k DO
                i:=i+1;x:=wx[tc[i]];
                IF x<min THEN min:=x; mx:=i END
            END;
            (*sea ta[mx] el n° de secuencia en que ocurre el mínimo*)
            copiado(f[tc[mx]],f[t[N]],eor,min,wx2);
            wx[t[N]]:=min; (*almacena último elemento escrito en sec.
                           de salida*)
            IF ~f[tc[mx]].eof THEN
                wx[tc[mx]]:=wx2; (*almacena elemento leído en copiado*)
            END;
            IF eor THEN (*omitir esta fuente*)
                tc[mx]:=tc[k];k:=k-1
            END;
        UNTIL k=0;
    END;
    z:=z-1;
    UNTIL z=0;
    Reset(f[t[N]]); (*girar secuencias*)
    ReadWord(f[t[N]],wx[t[N]]);
    tn:=t[N]; sn:=s[N];z:=c[N-1];
    FOR i:=N TO 2 BY -1 DO
        t[i]:=t[i-1];s[i]:=s[i-1];c[i]:=c[i-1]-z;
        Assign(nombre[t[i]],nombre[t[i-1]]);
    END;
    t[1]:=tn;s[1]:=sn;c[1]:=z;
    Create(f[t[N]],nombre[t[N]]);
    nivel:=nivel-1;
    UNTIL nivel=0;
    WriteString('Secuencia ordenada');
    WriteLn;
    ListSec(f[t[1]]);
    FOR i:=1 TO N DO Close(f[i]) END
END Polifasica.

```

3.3 ARCHIVOS INDEXADOS

Los métodos de clasificación presentados en la sección anterior son secuenciales. Sin embargo, las implementaciones de secuencias en algunos dispositivos de memoria secundaria, tales como los discos duros, permiten además un acceso cuasi-arbitrario. Al ser dispositivos de acceso directo, si se conoce una dirección física dada, se puede acceder a ella mediante un acceso aleatorio al sector seguido de un acceso secuencial, denominado ‘offset’, dentro de él. Este tipo de acceso y almacenamiento en memoria secundaria se conoce desde el punto de vista de los lenguajes de alto nivel como archivos o ficheros aleatorios.

Por otro lado, la información almacenada no suele consistir únicamente en la llave. En general, constará de un registro con múltiples campos, siendo uno de ellos la llave que se utiliza para la clasificación.

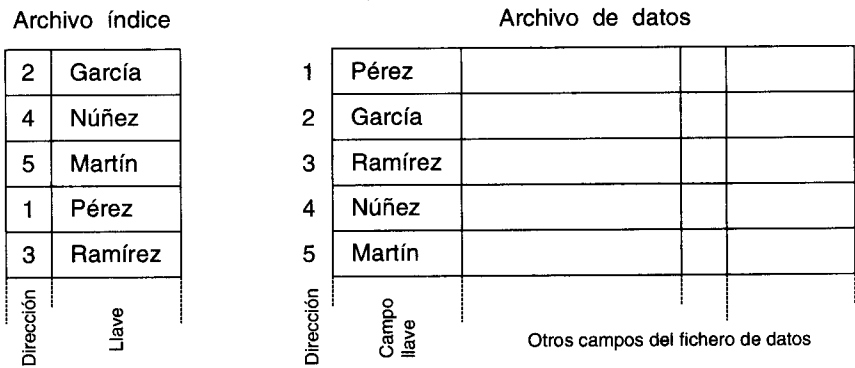


Figura 3.15 Archivos indexados con índice denso.

El método de clasificación mediante archivos indexados se basa en considerar, asociada a cada llave, la dirección física del registro que caracteriza. Así, se crea un segundo archivo, denominado archivo de índices, en el que se almacenan pares (llave, dirección). La Figura 3.15 ilustra este método. El archivo de datos contiene información relativa a una persona en varios campos (Nombre, Primer Apellido, Segundo Apellido, Dirección, Teléfono, ...), y se utiliza como criterio de clasificación la ordenación alfabética según el Primer Apellido. Así este campo será el campo llave. Para crear el archivo de índices se asocia el valor de este campo llave con la dirección del registro (Peréz → 4, García → 1, ...). Posteriormente se realiza la clasificación del archivo índice, cuyo resultado se muestra en la Figura 3.15. Cuando el archivo índice contiene la dirección del registro de la llave, se conoce como *indexado de índice denso*.

La clasificación no se realiza sobre el archivo de datos sino que es el archivo de índices el que se ordena. Dado que el tamaño de éste es muy inferior al del archivo

de datos suele ser posible realizarlo sobre memoria principal. Para recuperar un registro con una llave dada se busca en el archivo de índices la llave deseada, y entonces se accede a la dirección física que tiene asociada. Las búsquedas suelen también realizarse en memoria primaria, tras cargar el archivo de índices en un arreglo. La búsqueda podrá ser secuencial aunque lo más habitual es realizar una búsqueda dicotómica. La eliminación tampoco se realiza en un primer paso sobre el archivo de datos. Para eliminar un registro se indica con una marca en el archivo de índices de manera que será ignorado por las siguientes búsquedas y ordenaciones. Este hecho se observa claramente en cualquier programa de bases de datos (clipper, dbase,...) en los que la eliminación es rápida pero no disminuye el tamaño del archivo de datos. Sólo cuando se utiliza una orden específica se realiza la 'eliminación física', proceso que sí reduce el tamaño del archivo de datos pero que es mucho más lento. Esto se debe a que la forma habitual de eliminar registros es copiando en otro archivo todo el original menos los eliminados y posteriormente dándole el nombre original. Evidentemente este proceso es costoso y sólo conviene hacerlo cuando el archivo ha sufrido un número elevado de eliminaciones. Si se desea ordenar el archivo de datos según distintos criterios, se creará un archivo índice distinto para cada uno. De esta forma, si en el ejemplo anterior deseamos ordenar también por DNI tendremos un segundo archivo de índices en el que el campo llave es el DNI y el de dirección el que corresponde a ese registro.

La idea básica de los archivos indexados suele refinarse agrupando las llaves en bloques. En este caso, al archivo de índices se le denomina de *índice disperso* y está formado por grupos de pares (x, b) donde b es la dirección física del bloque en el cual el primer registro tiene la llave de valor x . Para utilizar esta estrategia inicialmente se clasifica el archivo de índices. Para recuperar una llave x , se busca el par (x, b) en el archivo índice, localizando la mayor llave menor o igual que x , y posteriormente se busca en el bloque. De esta forma la búsqueda se realiza por bloques lo que evita comparar con todos y cada uno de los elementos. La Figura 3.16 muestra esta estrategia con tres registros por bloque.

En la distribución de los registros en bloques una vez ordenado el archivo de índices no tiene que ser máxima, es decir, suele preferirse dejar espacio para registros adicionales que puedan insertarse posteriormente. De esta forma se evitan sobrecargas posteriores en los bloques. Cuando se requiera insertar un nuevo registro se busca mediante el archivo de índices cuál es el bloque que le corresponde, si no es el primero únicamente se inserta en su lugar, mientras que si es el primero es necesario actualizar el archivo de índices. Si el registro nuevo no cabe en el bloque que le corresponde, b_i , la estrategia más sencilla es buscar el siguiente bloque, b_{i+1} , mediante el archivo de índices. Si en b_{i+1} cabe un registro más entonces el último registro de b_i se pasa a b_{i+1} , se coloca el registro nuevo en b_i y se actualiza el archivo de índice. Si no cabe, o si b_{i+1} es el último bloque, entonces se obtiene un bloque nuevo del sistema de archivos, se inserta en él el registro nuevo y se coloca después de b_i , actualizando el archivo de índices.

Las técnicas basadas en archivos indexados dan soporte a gran número de aplicaciones en bases de datos, y es en este ámbito en el que se realiza un estudio detallado de las mismas.

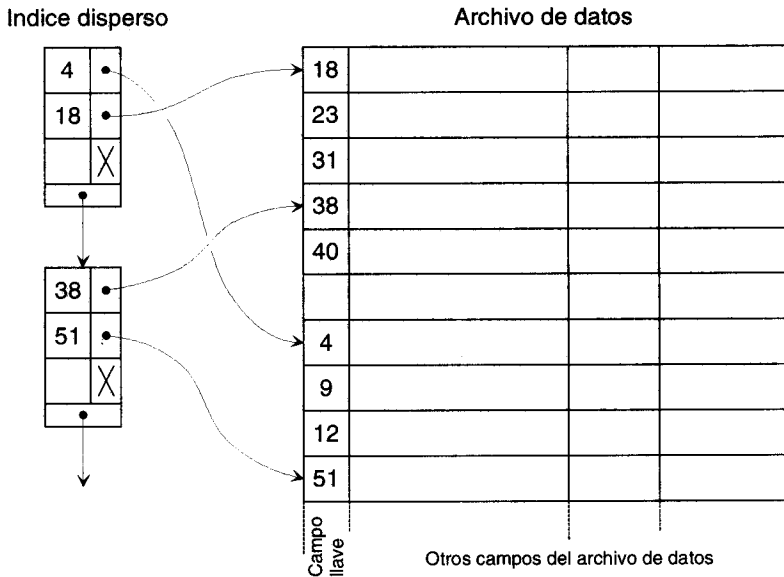


Figura 3.16 Archivo de datos e índice disperso.

3.4 TABLAS DE DISPERSIÓN (HASHING)

Las tablas hashing constituyen un tipo de datos abstracto de especial importancia en la manipulación y almacenamiento de información en memoria secundaria.

La idea básica consiste en transformar las llaves en direcciones de memoria mediante una función de transformación. Estas direcciones pueden hacer referencia a memoria primaria, en cuyo caso se utilizaría un arreglo, pero en general las tablas de dispersión adquieren su máximo potencial cuando se implementan en memoria secundaria.

Para comprender la idea básica se analizará el siguiente ejemplo. Supóngase que se requiere desarrollar una aplicación para registrar los participantes en un congreso. Se esperan alrededor de 3000 participantes y se requerirá almacenar los datos de cada uno, como nombre, apellidos, DNI o pasaporte, etc. Se utilizará como campo llave el primer apellido. Suponiendo que los apellidos pueden tener hasta 16 caracteres habrá 26^{16} llaves posibles (suponiendo un alfabeto de 26 caracteres).

Evidentemente es absurdo pretender reservar memoria para todas ellas, obviamente no van a presentarse 26^{16} personas, entre otras cosas, porque la Tierra está superpoblada, pero no tanto. Si se sabe *a priori* el número aproximado de asistentes al congreso, como por ejemplo 5000 bastará con reservar memoria suficiente para este orden de magnitud. El problema radica en cómo asignar una llave de todas las posibles a una dirección de memoria. Este cálculo lo resuelve la función de transformación.

En conclusión las tablas de dispersión se aplican cuando el conjunto de llaves posibles es mucho mayor que el de llaves reales a almacenar. Así la función de transformación es una aplicación suprayectiva, de muchos contra uno. Es seguro, por tanto, que para una función de transformación existen varias llaves distintas de todo conjunto de llaves que se aplican en la misma dirección de memoria. A los registros con estas llaves se les denomina *sinónimos*. Cuando aparecen dos sinónimos se dice que se ha producido una colisión, ya que ambos deberían ocupar la misma posición de memoria.

Una vez comprendidas las ideas básicas a continuación se precisan los términos expuestos.

Definición 3.5: Dado un conjunto de llaves posibles, X , y un conjunto de direcciones de memoria, D , una función de transformación, $H(x)$, es una aplicación suprayectiva del conjunto de llaves posible en el conjunto de direcciones de memoria:

$$H: X \rightarrow D$$

Definición 3.6: El TDA *tabla de dispersión* es un tipo de datos homogéneo, denominadas celdas, de tamaño fijo, T_{tabla} , compuesto por un número fijo de componentes a las que se accede mediante una dirección de memoria resultante de una función de transformación. Sobre este TDA se definen los operadores *Insertar*, *Buscar* y *Eliminar*.

Las celdas de las tablas de dispersión suelen estar constituidas por un único registro con la información asociada a una llave. Sin embargo, es posible definir las celdas como bloques con varios registros (como se hacía en los archivos indexados).

En las tablas de dispersión sólo se definen los operadores *Insertar*, *Buscar* y *Eliminar*, por razones de eficiencia. Estos tres operadores tienen un coste constante en promedio. Sin embargo, las tablas de dispersión son muy ineficaces para realizar otras funciones típicas, especialmente aquellas que requieren información sobre algún tipo de ordenación en los datos, como encontrar el máximo, imprimir la tabla ordenadamente, etc.

Definición 3.7: Se dice que dos llaves distintas x_1 y x_2 son sinónimas para una función de transformación $H(x)$ si $H(x_1) = H(x_2)$.

Definición 3.8: Se dice que se ha producido *desbordamiento* cuando una nueva llave se aplica a una dirección de memoria completamente ocupada, y se dice que se ha producido una *colisión* cuando dos llaves distintas se aplican sobre la misma

celda. En el caso habitual en el que una celda contiene un único registro el desbordamiento y la colisión se producen simultáneamente.

Definición 3.9: Se denomina *densidad de llaves* al cociente entre el número de llaves en uso, m , y el número total de llaves posibles, n_x . Se denomina *factor de carga* (o densidad de carga), α , al cociente entre el número de llaves en uso y el número total de registros almacenables en la tabla de dispersión. Así:

$$\alpha = \frac{m}{s \cdot b}$$

donde s es el número de registros por bloque y b el número de bloques que hay en la tabla de dispersión.

La Figura 3.17 muestra un ejemplo de tabla de dispersión que consta de mil bloques con dos registros por bloque. El conjunto de llaves posibles es el conjunto de números del DNI (00000000...99999999) y la función de dispersión se define tal que los tres primeros dígitos del DNI asignan la dirección de memoria (0...999). Suponiendo que el número de llaves en uso es de 500 entonces el factor de carga es:

$$\alpha = (500/2000) = 0.25$$

000	00077641	00034567
001	00131178	00139179
549	54913110	54939189
999	99911011	99931419

Figura 3.17 Tabla de dispersión.

Obsérvese que las llaves 54913110 y 54939189 son sinónimos y por tanto, ocupan el mismo bloque. Supóngase que en esta situación se requiere almacenar la llave 54900000. Al intentar la inserción en su bloque se encuentra que la primera celda está ocupada produciéndose una colisión. Como cada bloque tiene dos celdas se buscaría en la siguiente y al estar ocupada, además de la colisión, se produce desbordamiento. Será por tanto, muy importante analizar estrategias para encontrar una ubicación para la llave cuando se produce desbordamiento.

3.4.1 FUNCIONES HASH

La función hash debe ser fácil de calcular, es decir, la dirección de memoria debe ser calculada con sencillez y rapidez a partir de la llave. Pero además es importante que se produzcan el menor número posible de colisiones. En general, el número de colisiones será menor cuando la función de transformación utilice toda la tabla para ubicar la información con la misma probabilidad, es decir, la función de transformación es tal que la ocupación de los bloques es equiprobable. A estas funciones de transformación se les denomina *uniformes*. Obsérvese que la función de transformación de la Figura 3.17 satisface esta propiedad. Una consecuencia obvia de ello es que el número de llaves del conjunto de llaves posible que son sinónimas es similar para todos los bloques. De esta forma lo que se está haciendo es dividir el conjunto de llaves posibles en grupos de llaves aproximadamente iguales y asignar a cada grupo una dirección de bloque. A este hecho hace referencia el término *hashing* (picadillo, trocear en partes iguales).

La función de transformación más conocida es el resto de la división entera (mod):

$$f(x) = X \bmod M.$$

Con esta función se divide la llave en uso X por M y el resto será la dirección. Por tanto, el espacio de direcciones de los bloques será $[0, M - 1]$ y la tabla tendrá como mínimo M bloques. La elección de M es crucial, y debe ser considerada detenidamente en función de las características del conjunto de llaves posibles. Un ejemplo evidente es que si las llaves posibles son números acabados en cero, la elección $M = 10$ es desastrosa, ya que el resto es siempre cero, existen numerosas colisiones y no se reparten uniformemente por la tabla (no se utilizan nunca las direcciones de 1 a 9). En el caso en que las llaves sean cadenas de caracteres la elección de M no es en absoluto sencilla. Esto se debe a que éstas no son equiprobables, pertenecerán a un lenguaje y en ninguno las terminaciones de las palabras son equiprobables. Así por ejemplo, las terminaciones *ado*, *ión* o *ente* son más probables (en español) que *kzt*, *ryx* o *pqo*. Es decir, en todo lenguaje existen sufijos que aparecen frecuentemente mientras otros no lo hacen nunca. Supóngase que los caracteres se almacenan en un byte, y que las cadenas de caracteres son de 8 bits (Figura 3.18).

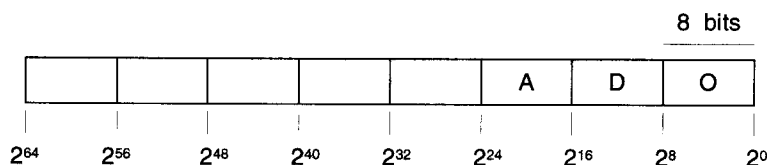


Figura 3.18 Cadena de ocho caracteres de ocho bits cada uno.

Si por ejemplo $M = 2^k$, $k \leq 24$, todas las cadenas que tengan los tres últimos caracteres iguales se asignarán a la misma dirección, ya que dividir entre dos es desplazarse dos bits hacia la izquierda. Por ello, si M se escoge como potencia de 2 las direcciones asociadas a sufijos que no se usan no se ocuparían nunca mientras que los sufijos usuales colisionarían a menudo. Por tanto, produciría un gran número de colisiones y no repartiría uniformemente las llaves por la tabla. Este hecho se debe a que las potencias de M tienen divisores. Por ello suele recomendarse que M sea un número primo, de manera que sus únicos divisores son M y 1. Realizando un análisis riguroso Knuth [Knuth, 1998] demuestra que si M divide a $r^{k \cdot a}$, donde a y k son números pequeños y r el número de bits utilizados para representar el carácter, entonces la función $f(x) = X \bmod M$ tiende a ser la superposición de los caracteres de X . Con ello se evita la dependencia de un conjunto de caracteres de la cadena, teniendo un efecto semejante a promediar sobre todos los caracteres.

a)

68	75	82	71	63	93	102	75
----	----	----	----	----	----	-----	----

	75		75		75		
	102		201		102		
	93		93		93		
	63		36		252		
+	71	+	71	+	71		
	82		28		74		
	75		75		75		
	68		86		34		
	Dirección = 629		665		776		

b) c) d)

e)

102 => 01100110		01100110 => 102
63 => 00111111	→	11111100 => 252
82 => 01010010		01001010 => 74
68 => 01000100		00100010 => 34

Figura 3.19 a) Cadena de ocho caracteres representada por los números de orden dentro de la secuencia de cotejo correspondiente. b) Plegado por desplazamiento. c) Plegado en las fronteras en base decimal. d) Plegado en las fronteras en base binaria. e) Detalle del plegado en base binaria.

Otra función hashing clásica es la conocida como *plegado* (folding). La dirección se obtiene dividiendo la llave en partes iguales y sumando todas ellas. La suma de las partes puede realizarse de dos formas. La primera se conoce como *plegado por desplazamiento* y consiste en sumar las partes directamente. La segunda se

denomina *plegado por las fronteras*, y consiste en plegar el identificador por las fronteras de las partes y sumar los dígitos coincidentes. La Figura 3.19 muestra un ejemplo de ambas técnicas de plegado.

En el caso en que la llave sea una cadena de caracteres, los dígitos de cada carácter vendrán determinados como es habitual por su valor decimal de la secuencia de cotejo correspondiente. Como puede observarse, esta técnica también considera todos los caracteres de la cadena.

Existen otras muchas posibilidades para escoger una función hashing. Así por ejemplo se suele recurrir a operaciones lógicas como la *o exclusiva*, para aplicarla a algunas partes de la llave, representada ésta en binario. Estas operaciones suelen ser muy rápidas, incluso en algunos computadores son más rápidas que la división entera. Sin embargo, en ocasiones la distribución es muy poco uniforme. Su aplicabilidad o no, deberá analizarse en cada caso considerando las características del conjunto de llaves posibles.

3.4.2 MANEJO DE DESBORDAMIENTO O SOBRECARGA

Como ya se ha explicado anteriormente, cuando se ha de insertar una nueva llave, si la celda que le corresponde está ocupada se produce una colisión y si todo el bloque está lleno, se ha producido un desbordamiento o sobrecarga. Lo mismo sucede cuando se va a recuperar una llave. Para poder determinar si existe o no colisión, debe inicializarse la tabla de manera que se establezca sin ambigüedad cuando la tabla está vacía. Se supondrá que se puede inicializar al valor cero. Obsérvese que esto sólo es válido cuando el cero no es una llave posible. Si lo fuese habría que determinar un valor en función de las características de las llaves del caso dado.

El método de manejo de sobrecargas más evidente es la *exploración lineal*, que consiste en buscar en el bloque siguiente. Se distinguen dos acciones: inserción y búsqueda. Para la inserción se busca en el bloque siguiente una celda libre, si vuelve a producirse sobrecarga se busca en el siguiente y así sucesivamente. Para la búsqueda de una llave se busca en la dirección siguiente y así sucesivamente hasta encontrar la llave, o encontrar que la celda del bloque está vacía o la tabla está llena. En los dos últimos casos la llave no se encuentra en la tabla. Por ejemplo, suponiendo una tabla *TH* con bloques de una celda, la búsqueda con exploración lineal calculará $f(x)$ y examinará las direcciones $TH(f(x))$, $TH(f(x) + 1)$, ..., $TH(f(x) + j)$, tal que $TH(f(x) + j)$ es la llave buscada, o está vacía (la llave no está en la tabla) o no quedan más bloques por recorrer (la tabla está llena y no está la llave).

En general la exploración puede expresarse como:

$$\text{dirección} = f(x) + g(x)$$

donde $f(x)$ es la función de dispersión y $g(x)$ es la función de tratamiento de sobrecargas. Considerando la tabla circular, la dirección vendrá dada por:

$$\text{dirección} = (f(x) + g(x)) \bmod \text{TamañoTabla}$$

donde *TamañoTabla* es obviamente el tamaño de la tabla hash.

En el caso de la exploración lineal:

$$g(x) = i, \quad i = 1 \dots \text{TamañoTabla}.$$

La exploración lineal tiende a colocar las llaves poco uniformemente a lo largo de la tabla, de manera que las sobrecargas producidas irán llenando los bloques cercanos a los ocupados. Una técnica que mejora este comportamiento es la *exploración cuadrática*, definida tal que:

$$g(x) = i^2, \quad i = 1 \dots \text{TamañoTabla}.$$

Con esta función de exploración la búsqueda se realiza examinando los bloques:

$$f(x), \quad (f(x) + i^2) \bmod \text{TamañoTabla} \quad \text{y} \quad (f(x) - i^2) \bmod \text{TamañoTabla},$$

con $1 \leq i \leq (\text{TamañoTabla} - 1)/2$.

Cuando *TamañoTabla* es un número primo de la forma $4j + 3$, j entero, la exploración cuadrática puede recorrer todos los bloques de la tabla. La demostración de este hecho se basa en resultados de la teoría de números y puede encontrarse en [Radke 1970]. Los números 3, 7, 11, 19, 23, 31, 43, 59, ..., 503, 1019, ..., satisfacen esta condición para $j=0, 1, 2, 4, 5, 7, 10, 14, \dots, 125, 254, \dots$, respectivamente.

La técnica de *rehashing* generaliza los conceptos anteriores, de forma que la función de exploración será una familia de funciones de dispersión que se examinan sucesivamente en un orden dado. Así:

$$g(x) = f_i(x), \quad i = 1 \dots m$$

donde cada $f_i(x)$ es una función de dispersión.

El Programa 3.9 muestra un esquema general del algoritmo de búsqueda en tablas hash.

Programa 3.9 Esquema general del algoritmo de búsqueda en tablas hash.

Declaración de la Tabla Hash

TYPE

Tipo_tabla_hash=array[1..TamañoTabla] of Tipo_llave;

PROCEDURE Busqueda_exploracion_lineal(x:Tipo_llave; TH:Tipo_tabla_hash;
var bloque:Tipo_dirección);

(* Búsqueda con exploración lineal en una tabla hash con una celda por bloque
Se supone que el identificador global Blanco indica que la celda está vacía y
las funciones Encontrado(), No_encontrado() y Tabla_llena() indican que el
elemento se ha encontrado, no está el elemento en la tabla y que la tabla está

```

llena respectivamente.Se supone la tabla circular*)
VAR
    dir, dir_inicial: Tipo_dirección;
    enc, llena: boolean;
    i: integer;
BEGIN
    dir_inicial:=f(x);
    dir=dir_inicial;
    enc=false;
    llena=false;
    i=0;
    REPEAT
        IF (TH(dir)=x) THEN
            Encontrado();
            bloque=dir;
            encontrado=true;
        END
        ELSIF TH(dir) = Blanco then No_encontrado();
        ELSE (* colisión *)
            i=i+1;
            dir=(f(x)+g(x) ) mod TamañoTabla
            IF dir=dir_inicial THEN
                Tabla_llena();
                llena=true;
            END;
        END;
    UNTIL enc OR llena;
END Busqueda_exploracion_lineal;

```

Tanto la exploración lineal como la cuadrática o el rehashing requiere comparaciones sucesivas de la llave con los elementos de la tabla hash. Muchas de ellas pueden evitarse utilizando otras estrategias de tratamiento de colisiones. La más elemental consiste en mantener una lista asociada a cada bloque de la tabla hash, de manera que en ella se almacenen todos los sinónimos que no se pueden insertar en el bloque correspondiente. La implementación más adecuada de esta lista es mediante una lista enlazada ya que el número de sobrecargas es desconocido *a priori* (si fuese conocido podría dimensionarse una tabla hash para acomodar todas las llaves). Esta estrategia, denominada *encadenamiento*, requiere disponer en cada bloque del espacio necesario para almacenar un enlace (por ejemplo, un puntero).

De las consideraciones realizadas anteriormente puede deducirse que el comportamiento de las tablas hash depende únicamente de la estrategia de tratamiento de los desbordamientos y que es independiente de la función de dispersión, si ésta es uniforme. Esto es cierto siempre y cuando las llaves en uso se tomen aleatoriamente del conjunto de llaves posibles. Esto no suele suceder en la práctica, ya que en gene-

ral siempre hay unas llaves más probables que otras (recuérdese el ejemplo de los sufijos frecuentes y los nunca usados en un lenguaje).

La Figura 3.20 muestra algunos de los resultados experimentales presentados por Lum *et al* [Lum, 1971]. Se presentan el promedio de los accesos a bucles para tablas hash con cuatro factores de carga diferentes, y dos métodos de tratamiento de sobrecargas: *encadenamiento* y *exploración lineal*. Como puede observarse la función de dispersión elegida influye en los resultados, y además la división entera (con un número primo) es en general superior a las otras funciones hash. Por tanto, éste será el método recomendado generalmente.

Tipo de función Hash	$\alpha=0.5$		$\alpha=0.5$		$\alpha=0.5$		$\alpha=0.5$	
	E	L	E	L	E	L	E	L
División entera	1.19	4.52	1.31	7.20	1.38	22.42	1.41	25.79
Plegado	1.33	21.75	1.48	65.10	1.40	77.01	1.51	118.57
Plegado en la frontera	1.39	22.97	1.57	48.70	1.55	69.63	1.51	97.56
Teórico	1.25	1.50	1.37	2.50	1.45	5.50	1.48	10.50

Figura 3.20 Resultados experimentales [Lum, 1971]. Número promedio de los accesos a bloques para distintas funciones Hash con estrategia de encadenamiento (E) o lineal (L).

Análisis de las tablas hash

En el caso de que no se produzcan colisiones, el tiempo para introducir o recuperar una llave depende del tiempo para realizar la transformación con la función hash y el tiempo necesario para direccionar un bloque. Dentro de un bloque el número de celdas suele ser pequeño, con lo que las búsquedas pueden ser secuenciales y de coste despreciable. Por tanto, la primera conclusión importante es que en ausencia de sobrecargas el coste es independiente del número de llaves en uso, n .

Sin embargo, la inserción y la búsqueda por transformación de llaves tiene evidentemente un rendimiento pésimo en el peor caso, ya que si siempre hay colisiones y sobrecargas se tendrán que realizar todas las comparaciones.

Para analizar el caso promedio se supone que todas las llaves tienen la misma probabilidad y que la función de dispersión es uniforme. Supóngase que una llave debe ser insertada en una tabla de tamaño n , y que la tabla contiene k elementos. La probabilidad de encontrar un bloque libre en la primera comparación, p_1 , será $1 - k/n = (n - k)/n$, ya que k/n será la probabilidad de que la llave se encuentre en la tabla. La probabilidad de que se necesite una segunda comparación será la probabilidad de que se haya producido un desbordamiento en la primera, k/n , por la probabilidad de hallar libre el segundo bloque en el que se consulta,

$(n-k)/(n-1)$ (hay un bloque menos en la tabla a considerar, el del primer intento). Por tanto:

$$p_1 = (n-k)/n$$

$$p_2 = (k/n) \times (n-k)/(n-1)$$

$$p_3 = (k/n) \times (k-1)/(n-1) \times (n-k)/(n-2)$$

y en general, la probabilidad p_i de que una inserción requiera i exploraciones será:

$$p_i = (k/n) \times (k-1)/(n-1) \times (k-2)/(n-2) \times \dots \times (n-k)/(n-i+1).$$

Por tanto, el valor esperado, E , de exploraciones requeridas después de insertar la llave $(k+1)$ -ésima será:

$$E_{k+1} = \sum_{i=1}^{k+1} i \times p_i = 1 \times \frac{n-k}{n} + 2 \times \frac{k(n-k)}{n(n-1)} + \dots + (k+1) \times \frac{k(k-1)(k-2)}{n(n-1)(n-2)} \times \dots$$

$$\dots \times \frac{1}{(n-k+1)} = \frac{(n+1)}{(n-k+1)}$$

Suponiendo que n es el tamaño de la tabla y que m es el número de llaves almacenadas en ella, el número promedio de exploraciones necesarias para recuperar una llave aleatoria de la tabla, E , vendrá dado por:

$$E = \frac{\sum_{k=1}^m E_k}{m} = (n+1) \frac{\sum_{k=1}^m \frac{1}{(n-k+2)}}{m} = (n+1) \frac{(H_{n+1} - H_{n-m+1})}{m}$$

Aproximando los números armónicos por $H_n = \ln n + \gamma$, y sustituyendo $\alpha = \frac{m}{n}$, tenemos:

$$E \approx \frac{(\ln(n+1) - \ln(n-m+1))}{\alpha} = -\frac{\ln\left(\frac{n-m+1}{n+1}\right)}{\alpha} \approx -\frac{\ln(1-\alpha)}{\alpha}$$

El cálculo anterior será válido siempre que la función de dispersión sea uniforme, pero además, la estrategia de sobrecargas debe ser tal que los razonamientos probabilísticos realizados sean válidos. Esto sucede, por ejemplo, con la exploración cuadrática, o con la estrategia de rehashing.

Finalmente, se presenta el análisis para la estrategia de encadenamiento. En este caso, cuando una llave x se busca en la dirección $i = f(x)$, y la lista de la direc-

ción i tiene k nodos, (lo que no incluye al enlace que apunta al primer nodo de la lista) entonces si x no está en la tabla se necesitan k comparaciones. Si x está en la lista a una distancia j ($1 \leq j \leq k$) entonces se necesitan j comparaciones. Cuando la llave i -ésima, x_i , se inserta en la tabla, el número esperado de llaves sobre cualquier lista es $(i-1)/n$. Suponiendo que las llaves nuevas se insertan al final de la lista, entonces el número esperado de comparaciones necesarias para recuperar x_i será la suma de las $(i-1)/n$ realizadas en promedio sobre la lista, más la de la llave x_i , es decir, $((i-1)/n + 1)$. Por tanto, el número promedio de comparaciones necesarias para recuperar una llave será:

$$E = \frac{1}{m} \sum_{i=1}^m \left(1 + \frac{i-1}{n} \right) = 1 + \frac{m-1}{2n} \approx 1 + \frac{\alpha}{2}.$$

3.5

CONCLUSIONES

En este capítulo se han presentado los métodos básicos de clasificación sobre memoria secundaria. En primer lugar, se trataron los métodos más representativos para clasificar estructuras que se caracterizan en general porque el acceso es, desde el punto de vista de la programación a alto nivel, secuencial. Todos ellos se fundamentan en la operación de mezcla: mezcla directa, mezcla natural, mezcla balanceada múltiple y clasificación polifásica. El primero de ellos explota el proceso más intuitivo. La mezcla natural mejora el proceso de mezcla mediante la utilización del concepto de subsecuencia ordenada y la mezcla balanceada es una sofisticación de esta última utilizando una estrategia con más recursos dispuestos adecuadamente. Pero sin duda la clasificación polifásica se muestra como un potente método cuando el número de datos a clasificar es muy elevado.

Los métodos de clasificación externa basados en la mezcla presentados en este capítulo apenas utilizan memoria primaria, salvo para el código y las variables auxiliares. Sin embargo, hoy en día cualquier computador dispone de una apreciable cantidad de memoria primaria, y ésta podría ser aprovechada para realizar una clasificación externa más eficaz. En este contexto el lector interesado puede consultar el texto del Profesor N. Wirth [Wirth, 1987], en el que se muestra un método de excelente rendimiento basado en una combinación de la clasificación polifásica y la clasificación por montón.

En la segunda parte del capítulo se muestran brevemente los conceptos básicos de otras estrategias de clasificación, no basadas en la operación de mezcla, como los archivos indexados y las tablas hash. Un estudio en profundidad de ellas puede encontrarse en [Enbody, 1988].

Las tablas hash clásicas tienen como inconveniente fundamental su carácter estático. Por ello, cuando el número de llaves a almacenar llena la tabla, es necesario un proceso de redimensionamiento de la tabla, cuyo coste es elevado. Para superar estos inconvenientes se introducen las tablas hash dinámicas, de aplicación inmediata en lenguajes de sistemas de desarrollo de bases de datos, como por ejemplo SQL. Un estudio detallado de estas posibilidades puede encontrarse en [Weiss, 1997].

CAPÍTULO 4

Tipos de datos abstractos dinámicos lineales

4.1 INTRODUCCIÓN

En el Capítulo 1 se presentaron los TDA básicos, bien conocidos de programación estructurada. Recuérdese que un TDA es una colección de componentes cuya organización se caracteriza por las funciones de acceso que se usan para almacenar los elementos individuales. Así por ejemplo, los arreglos unidimensionales se definen como una colección estructurada de elementos del mismo tipo, identificada con un mismo nombre, y tal que se accede a cada componente mediante un índice que indica su posición dentro de la colección. Del mismo modo, el registro es un tipo de datos estructurado compuesto por un número fijo de componentes no necesariamente del mismo tipo (denominadas campos del registro), y tal que a cada componente del registro se accede mediante un selector de campo. En Modula2, este selector está formado por el nombre del registro seguido de un punto y del nombre del identificador de campo.

Estas estructuras de datos tienen dos características reseñables. En primer lugar, son estructuras incorporadas en los lenguajes de alto nivel, es decir, estos lenguajes disponen de palabras clave para su declaración y expresiones sintácticas determinadas para el acceso a sus elementos. En segundo lugar, son TDA estáticos. Recuérdese que un TDA es estático cuando el número de elementos que lo componen es fijo y es dinámico si es variable. Para los estáticos la memoria se reserva en tiempo de compilación. Por ejemplo, los arreglos unidimensionales son TDA estáticos por su propia definición ya que en ellos el número de elementos es fijo y viene

determinado por el número de valores diferentes que puede tomar su índice. Análogamente, los registros son TDA estáticos puesto que el número de campos es fijo. Debe observarse que la naturaleza estática o dinámica de un TDA se debe a su propia definición y es independiente del uso que se haga de ella.

Otros TDA presentados en el Capítulo 1 tenían carácter dinámico. En ellos la memoria se reserva a medida que se va necesitando a lo largo de la ejecución del programa. La asignación dinámica de memoria permite construir otros tipos de datos abstractos generalizando el concepto de nodo. Así por ejemplo, en algunas aplicaciones suele ser un inconveniente el hecho de que las listas enlazadas sólo se puedan recorrer en un sentido. Si se requiere recorrer las listas en ambos sentidos puede definirse un TDA dinámico basado en un tipo similar al nodo de la lista enlazada pero que incorpore dos punteros, tal que uno apunte al predecesor y otro al sucesor. O puede ser necesario mantener, además del puntero externo a la cabeza de la lista, otro puntero al final de la misma. En conclusión, es posible definir los TDA en función de las necesidades de la aplicación.

En este capítulo y en los dos siguientes se presentan los TDA dinámicos fundamentales para la organización de datos. En este capítulo se presentarán los TDA dinámicos lineales, es decir, aquellos que mantienen una organización secuencial con un predecesor y un sucesor. En primer lugar se introducen algunos TDA similares a las listas enlazadas, como ejemplo de construcción de TDA dinámicos. Seguidamente se presentan los dos TDA dinámicos lineales más característicos: las pilas y las colas.

4.2 EJEMPLOS DE TDA DINÁMICOS LINEALES

Como ya se ha explicado anteriormente, el TDA dinámico más elemental es la lista enlazada, en la que los enlaces se realizan en un único sentido, pero generalizando el campo enlace del tipo nodo visto en el primer capítulo (Figura 1.8), puede definirse cualquier TDA dinámico con diversos enlaces. Así por ejemplo, cuando se necesita recorrer la estructura en ambos sentidos puede definirse el tipo nodo de manera que el campo enlace tenga dos punteros tal que uno apunta al sucesor y otro al predecesor de la lista. La Figura 4.1 muestra un nodo de este tipo.



Figura 4.1 Representación esquemática de un nodo con enlace al nodo siguiente y al anterior.

De este modo puede definirse el TDA *lista doblemente enlazada* de la siguiente forma:

Definición 4.1: El TDA *lista doblemente enlazada* es una colección de nodos ordenada según su posición, tal que cada uno de ellos es accedido mediante el puntero

anterior del campo enlace del nodo siguiente y por el puntero siguiente del campo enlace del nodo anterior.

La Figura 4.2 ilustra el concepto de lista doblemente enlazada.

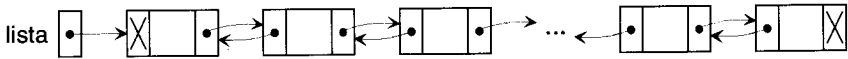


Figura 4.2 Lista doblemente enlazada.

Sobre este TDA pueden definirse funciones análogas a las realizadas sobre listas enlazadas, como insertar, suprimir, etc. El Programa 4.1 muestra la declaración de tipo en Modula2 para las listas doblemente enlazadas y la implementación de la función de inserción por el principio. Los Programas 4.2, 4.3 y 4.4 muestran respectivamente tres funciones típicas: Insertar en orden, Suprimir un elemento dado e Imprimir en sentido inverso; el procedimiento para la impresión en sentido directo es idéntico al de las listas enlazadas. Las Figuras 4.3 y 4.4 ilustran los pasos necesarios para la realización de estas dos primeras funciones básicas.

Programa 4.1 Declaración de la estructura de la lista doblemente enlazada e inserción por el principio.

```

TYPE
  tipo_datos= INTEGER;
  tipo_doble= POINTER TO nododoble;
  nododoble= RECORD
    datos: tipo_datos;
    sig,ant: tipo_doble
  END;
  tlista=tipo_doble;
VAR
  lista : tlista;

PROCEDURE ins_ini(VAR lista:tlista; nuevo:tipo_datos);
VAR
  aux: tipo_doble;
BEGIN
  ALLOCATE(aux, SIZE(nododoble));
  aux^.datos:=nuevo;
  aux^.sig:=lista;
  aux^.ant:=NIL;
  IF lista#NIL THEN lista^.ant:=aux; END;
  lista:=aux;
END ins_ini;

```

Programa 4.2 Inserción en orden en una lista doblemente enlazada.

```
PROCEDURE ins_orden(VAR lista:tlista;nuevo: tipo_datos);
VAR
    aux,ant,tempo: tipo_doble;
BEGIN
    IF lista=NIL THEN ins_ini(lista,nuevo)
    ELSE
1      ALLOCATE(tempo,SIZE(nododoble));
2      tempo^.datos:=nuevo;
3      aux:=lista; ant:=NIL;
4      WHILE (aux#NIL) AND (nuevo>aux^.datos)DO
          ant:=aux;
          aux:=aux^.sig;
        END;
5      tempo^.sig:=aux;
6      tempo^.ant:=ant;
7      IF ant=NIL THEN lista:=tempo ELSE ant^.sig:=tempo END;
8      IF aux#NIL THEN aux^.ant:=tempo; END;
    END;
END ins_orden;
```

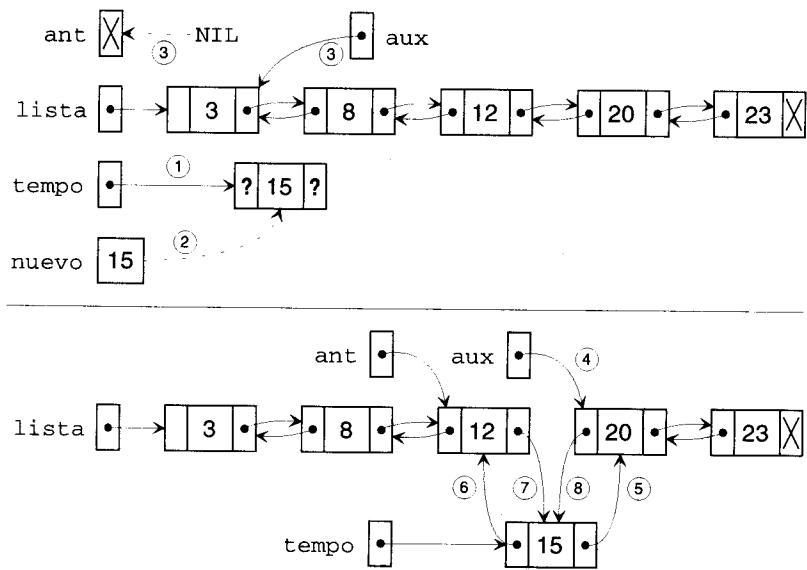


Figura 4.3 Inserción en orden en una lista doblemente enlazada. Los números muestran las acciones correspondientes a las sentencias del Programa 4.2. Estas figuras muestran el caso general de inserción por el centro pero las sentencias condicionales 7 y 8 del mencionado programa garantizan la inserción al principio y al final de la lista.

Programa 4.3 Eliminación de la primera aparición de un elemento dado en una lista doblemente enlazada.

```

PROCEDURE sup_nodo(VAR lista:tlista; dato:tipo_datos);
VAR
  aux: tipo_doble;
BEGIN
1   aux:=lista;
2   WHILE (aux#NIL) & (aux^.datos#dato) DO
      aux:=aux^.sig;
    END;
    IF aux#NIL THEN (* Si existe el nodo *)
3     IF aux^.sig#NIL THEN (* Si tiene siguiente *)
        aux^.sig^.ant:=aux^.ant;
      END;
4     IF aux^.ant#NIL THEN (* Si tiene anterior *)
        aux^.ant^.sig:=aux^.sig
      ELSE (* Si es el primero *)
        lista:=aux^.sig
      END;
5     DEALLOCATE(aux,SIZE(nododoble));
    END;
END sup_nodo;

```

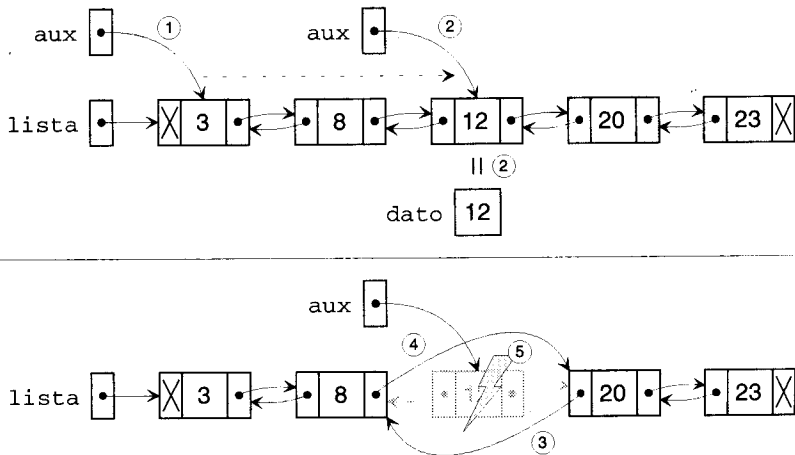


Figura 4.4 Eliminación de un nodo en una lista enlazada. Los números hacen referencia a las correspondientes sentencias del Programa 4.3. De nuevo se muestra únicamente el caso general, pero las sentencias condicionales 3 y 4 aseguran la eliminación del primer y último nodo, aunque sea el único (primero y último simultáneamente).

Programa 4.4 Impresión en sentido inverso de una lista doblemente enlazada.

```

PROCEDURE Imprimir(lista: tlista);
(*Imprime lista en orden inverso*)
VAR
    aux: tipo_doble;
BEGIN
    aux:=lista;
    IF lista#NIL THEN
        WHILE aux^.sig#NIL DO
            aux:=aux^.sig
        END;
        WriteInt(aux^.datos,4); (* último *)
        WHILE aux^.ant#NIL DO
            ~ WriteInt(aux^.ant^.datos,4);
            aux:=aux^.ant
        END;
    END;
    WriteLn;
END Imprimir;

```

En principio, podía pensarse que las listas doblemente enlazadas son mejores que las listas enlazadas habituales ya que permiten mayor flexibilidad en su manejo y menor coste. Sin embargo, esto sólo es aparentemente. En primer lugar, es evidente que requieren más memoria, un puntero más por nodo, y como el número de nodos será elevado, éste no es un hecho despreciable. Además, el coste de las funciones sobre ellas será mayor, ya que se requerirá el doble de actualizaciones de valores de los enlaces. Por tanto, estos dos incrementos en el coste, tanto de almacenamiento como computacional, tendrán que estar justificados en función de la aplicación requerida. Supóngase, por ejemplo, que la aplicación requiere una lista y que sobre ella se suele necesitar acceder a un elemento y a su predecesor, además de las funciones típicas de una lista enlazada. En este caso no se justificaría la utilización de la lista doblemente enlazada ya que se podría implementar una función de búsqueda en la que se mantengan dos punteros, Actual y Anterior por ejemplo, con lo que el problema estaría resuelto.

Una solución similar podría realizarse si se requiriese encontrar un elemento y su quinto predecesor. Entonces, ¿cuándo se justificaría la utilización de una lista doblemente enlazada? Evidentemente cuando la necesidad de recorrer la estructura hacia los predecesores sea frecuente. Por ejemplo, si en función de un valor en un nodo se necesita volver a examinar sus predecesores.

Como ejemplo de aplicación supóngase que por un dispositivo serie se reciben cadenas de caracteres de dos periféricos tal que el primero las envía en orden y el segundo en orden inverso. El inicio de cadena se identifica mediante el carácter '1' si es del primer periférico y '2' si es del segundo y el final de cadena se produce cuando

se encuentra la marca que indica el comienzo de otra. Si por ejemplo, el periférico 1 envía las cadenas 'HOLA', 'BIEN' y 'ADIÓS', y el periférico 2 envía las cadenas 'BUENA' y 'SUERTE', la secuencia de caracteres que llega será:

1	H	O	L	A	1	B	I	E	N	2	A	N	E	U	B	1	A	D	I	O	S	2	E	T	R	E	U	S
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Para imprimir en pantalla los mensajes de ambos periféricos habría que recorrer la lista e ir imprimiendo los caracteres en el sentido de los sucesores siempre que el identificador de cadena encontrado fuese '1', mientras que si se encuentra un '2' habría que recorrer la lista hasta encontrar el fin de la cadena (un nuevo identificador '1' o '2') y recorrerla hacia los predecesores, imprimiendo los caracteres, hasta encontrar el principio de la cadena (identificador '2'). Esto podría resolverse también insertando las cadenas del segundo periférico en una pila. Ya se verá más adelante que este tipo de estructura devuelve la información almacenada en ella en orden inverso a como se introdujo.

Otro ejemplo para listas doblemente enlazadas podría ser una lista enlazada con fichas de libros. Supongamos que éstos están ordenados por temas. Una vez localizado un libro determinado puede ser interesante recorrer las fichas adyacentes por delante y por detrás de ésta para acceder a libros de temas similares.

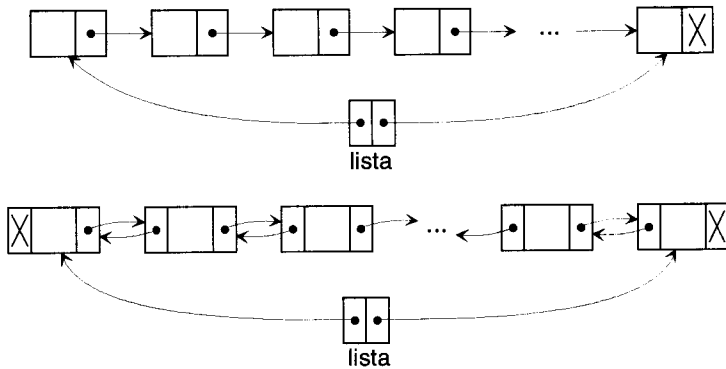


Figura 4.5

Lista enlazada y lista doblemente enlazada con puntero al final de la lista.

Al igual que se han definido las listas doblemente enlazadas puede establecerse cualquier otra que sea necesaria. Por ejemplo, en numerosas aplicaciones suele bastar una lista enlazada pero además se necesita tener un acceso rápido al último elemento. Con las listas enlazadas habituales sería necesario recorrer toda la lista para llegar al mismo. Entonces puede definirse una *lista enlazada con puntero al final de la lista* sin más que añadir un puntero externo que mantenga la dirección del último nodo. La Figura 4.5 muestra este TDA. Evidentemente las funciones realizadas sobre este TDA deben actualizar el puntero final de la lista. En esta figura se muestra también una lista doblemente enlazada con puntero al final, situación muy habi-

tual debido a que de esta forma se obtiene una estructura completamente simétrica desde ambos extremos y se aprovecha mejor la circunstancia del doble enlace de cada nodo.

Hay aplicaciones en las que resulta interesante la utilización de listas enlazadas 'con memoria'. En este tipo de listas se añade un puntero externo adicional, denominado índice, que está apuntando siempre al último elemento accedido. Esto permite detener el recorrido sobre la lista para realizar cualquier otra tarea y continuar después en el nodo donde se paró, sin necesidad de comenzar el recorrido desde el principio. Obviamente, también podemos definir listas doblemente enlazadas con memoria, con lo que después de detener el recorrido, podríamos continuar el mismo hacia delante o hacia detrás del nodo en el que se detuvo éste. En este tipo de listas, podemos definir nuevas operaciones como son:

- Avanzar y Retroceder: hacen que el puntero índice pase a apuntar al elemento siguiente o al anterior al que está apuntando en ese momento.
- Ir al Inicio o al Final. Colocar el puntero índice apuntando al primer o último elemento.
- Consultar el elemento apuntado por el índice.
- Comprobar si el índice está en el primer o el último elemento que podría utilizarse para detener un recorrido externo de la lista.

Este tipo de listas con memoria, permiten recorridos externos de la lista, de forma que éstos lo va realizando el programa que la utiliza. Esto puede ser útil por ejemplo para aplicar un determinado procesamiento a todos los elementos de la lista. Para hacer esto con una lista enlazada convencional, tenemos dos opciones: la primera sería crear una operación que lo realizase y que debería formar parte de la definición del tipo de datos, con lo que debería cambiarse esta definición si cambiase el tipo de procesamiento a realizar. La segunda opción sería ir solicitando los elementos de forma sucesiva, pero esto obligaría a que para cada elemento habría que recorrer la lista nuevamente desde el principio. Las listas enlazadas con memoria eliminan estas dos dificultades de forma que se podría cambiar el procesamiento sin modificar la definición de la lista y para acceder a cada elemento sólo habría que asignar un puntero, siempre claro está, que el acceso sea secuencial. Este tipo de tratamiento puede resumirse en el siguiente esquema:

- Poner el índice al principio (final) de la lista.
- Mientras el índice sea distinto de NIL:
 - Consultar la lista para obtener el elemento apuntado por el índice.
 - Procesar el dato.
 - Avanzar (retroceder) a la posición siguiente (anterior).

Lo que realiza este esquema también se podría hacer si una lista enlazada convencional dispusiera de una función de recorrido que admitiese como parámetro el procedimiento de procesamiento, funcionalidad que permiten la mayor parte de lenguajes de alto nivel. Pero en este caso el recorrido completo sería incondicional y no podría alterarse (parar o retroceder) en función del contenido de los elementos.

La Figura 4.6 muestra una lista doblemente enlazada con memoria. Obviamente, también se puede diseñar una lista simplemente enlazada con memoria, pero dada la funcionalidad de las listas con memoria es habitual hacerlo con listas doblemente enlazadas y punteros al inicio y al final.

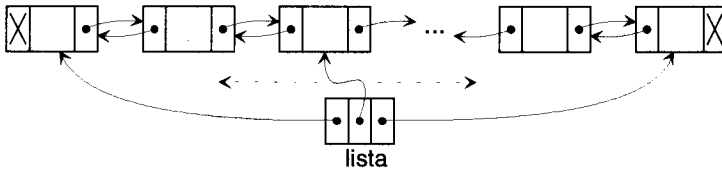


Figura 4.6 Esquema de una lista doblemente enlazada con memoria.

Una variación habitual en algunas aplicaciones de las listas doblemente enlazadas son las conocidas como *listas circulares doblemente enlazadas*. En ellas, el último nodo de la lista tiene como sucesor al primero, y el primero tiene como predecesor al último. La Figura 4.7 muestra este TDA, incluyendo el caso especial en el que la lista contiene un único nodo y es a la vez predecesor y sucesor de sí mismo. Obsérvese que en estas listas ningún nodo tiene un enlace con el valor NIL. Esto tiene como consecuencia que los recorridos que se realicen sobre la estructura deberán hacerse mediante un contador u observando el contenido de los distintos nodos.

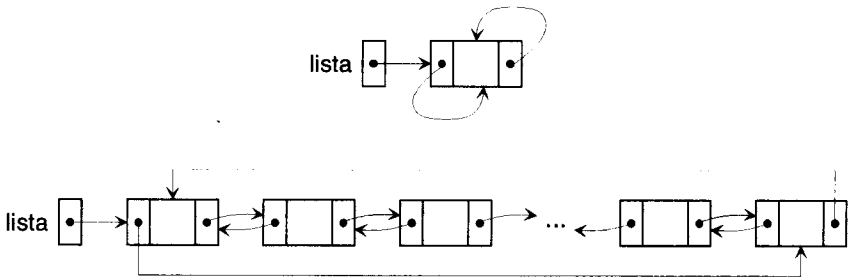


Figura 4.7 Lista circular doblemente enlazada, mostrando el caso general con varios elementos y el caso particular de elemento único. La situación de lista vacía es la habitual: lista=NIL.

Un tipo de estructura especialmente interesante son las listas multireferenciadas, ya que explotan especialmente las características de la asignación dinámica de memoria. En estas listas, el nodo que vertebra la estructura no contiene la información directamente sino un puntero a la misma, tal y como se muestra en la Figura 4.8. La información se almacena en una variable referenciada a la que se añade un contador que lleva la cuenta de cuántos punteros (listas) están apuntando en cada momento a la mencionada información.

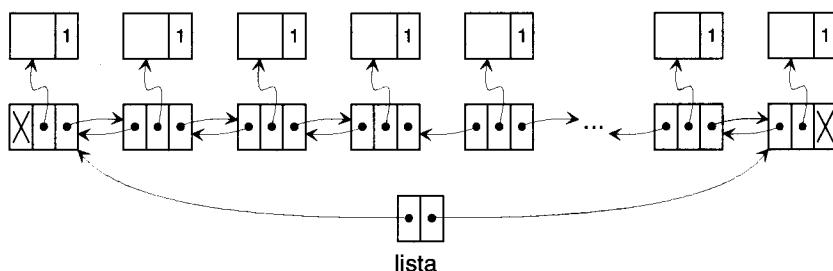


Figura 4.8 Lista doblemente enlazada donde cada nodo almacena un puntero a la información, en lugar de la información en sí. El registro referenciado incluye un contador para tener bajo control el número de punteros que en cada momento apuntan a la información que contiene.

El interés de este tipo de estructura está en aquellas aplicaciones en las que una misma información puede pertenecer simultáneamente a más de una lista. Podemos imaginar una aplicación en la que en una lista enlazada de este tipo tenemos almacenadas las fichas correspondientes a una colección de libros. Si hacemos una búsqueda, por ejemplo todos los libros de un determinado autor, podría ser interesante crear una nueva lista que contuviese sólo los libros encontrados. De esta forma podríamos operar con ella como una lista completamente independiente, sin necesidad de acceder y recorrer la lista completa, que será normalmente mucho más larga. Si ahora deseamos todos los libros que incluyan en su título la palabra ‘algoritmo’, crearíamos otra lista con los libros que cumplen esta segunda condición y así sucesivamente para distintas búsquedas. Una vez hecho esto podríamos realizar búsquedas combinadas, sin necesidad de recorrer la lista completa atendiendo sólo a las nuevas listas obtenidas que, en el caso general, tendrán una longitud sensiblemente inferior que la lista original.

Sin embargo, con este tipo de técnica estaríamos ‘devorando’ memoria de forma considerable si empleamos un tipo de lista convencional, ya que habría que duplicar la información en las distintas listas. De esta forma, si la ficha de un libro pertenece en un momento dado a siete listas, se multiplica por siete el espacio de almacenamiento necesario para esa ficha. Otro problema que se plantea es el de la inconsistencia de la información; si se modifica la ficha de un libro, habría que asegurar que la modificación se realiza en todas las listas.

Con una lista multireferenciada como la que se muestra en la Figura 4.8, se reduce considerablemente el problema del almacenamiento y se elimina el de la consistencia, ya que cuando creamos una nueva lista, no se duplica la información, que sigue siendo única, sino que basta con incrementar el contador de referencias de la misma. Este contador es necesario, porque al eliminar un elemento de una de las listas, la memoria sólo deberá ser liberada y devuelta al espacio libre, si es la única lista que contiene ese elemento. Es decir, al eliminar un elemento de la lista, lo primero que habrá que hacer será reducir su contador en una unidad y sólo si después de esto el contador vale cero se producirá una liberación de memoria. La Figura 4.9

muestra un ejemplo en el que una misma información está siendo compartida por varias listas. En esta figura se puede apreciar como el contador mencionado anteriormente lleva el control del número de referencias que tiene la información referenciada. Aunque en la Figura 4.9 se muestran listas doblemente enlazadas con punteros a los dos extremos, no tiene por qué ser así y puede adaptarse a cualquier tipo de listas.

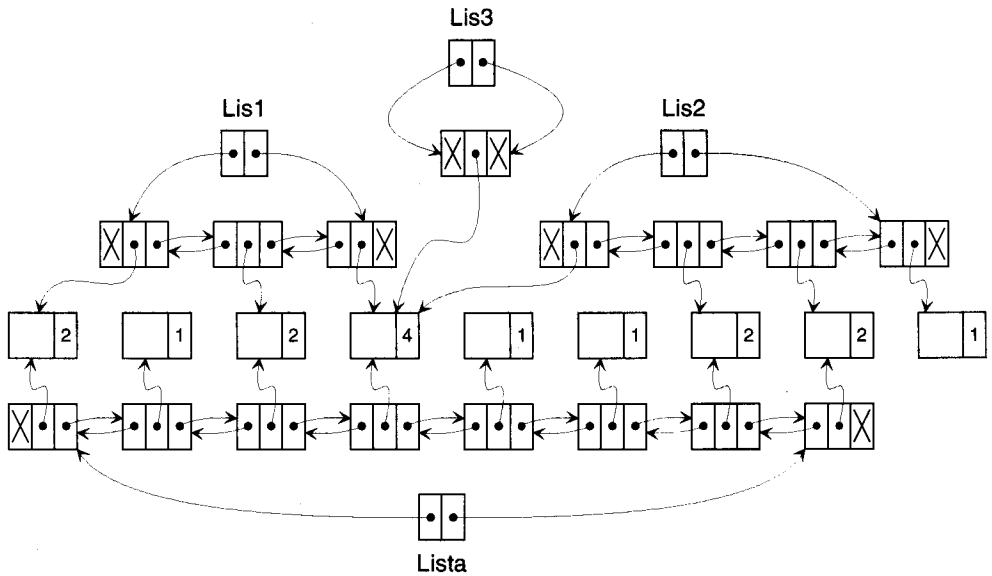


Figura 4.9 Ejemplo de lista multireferenciada con varias listas compartiendo una misma información.

Un sencillo cálculo nos permite valorar la ventaja en cuanto al espacio de almacenamiento de este tipo de listas. En el ejemplo mencionado supongamos que todos los campos necesarios para almacenar la información de cada libro necesitan 1000 bytes, cada puntero requiere 4 bytes y los enteros ocupan dos. Con una lista doblemente enlazada normal, cada nodo requeriría 1008 bytes (un campo de información y dos punteros de enlace). Con una lista multireferenciada requeriría, sin embargo, 1014 bytes (1000 de la información, 4 del puntero de referencia, 8 de los punteros de enlace y 2 del contador). El incremento de memoria necesario es $<1\%$ (6 bytes) que es un valor pequeño. Por otra parte, cada vez que un nodo que ya existe deba añadirse a una nueva lista, el espacio necesario será de sólo 12 bytes (1 puntero a la información y dos de enlace), mientras que con una lista convencional se requerirían otros 1008 bytes además del tiempo necesario para reservar la memoria y copiar la información. El interés de este tipo de lista será tanto mayor cuanto más grande sea el tamaño de las fichas y/o cuanto mayor sea el número de listas que deben compartir información. La Figura 4.9 no muestra claramente la ventaja de este tipo de listas porque los tamaños no están dibujados de forma proporcional, pero téngase en cuenta que, con los valores mencionados, que no son nada desproporcionados, un

sólo nodo convencional ocupa el equivalente a más de 100 nodos de una lista de este tipo. Si en lugar de considerar listas doblemente enlazadas, fuesen listas con un solo enlace, el ahorro sería aún mayor.

Las listas que incluyen más de un puntero en cada nodo, no siempre se enlazan simultáneamente con el anterior y el posterior según un criterio de ordenación único. Imaginemos que una misma información la queremos mantener ordenada según dos criterios distintos. En el ejemplo de las fichas de libros podría ser deseable tenerlas ordenadas por autores y por años de edición. Esto puede resolverse fácilmente con dos listas independientes, pero la información estaría duplicada. Otra alternativa es el empleo de listas multireferenciadas, ya que evitan la duplicación de información. Sin embargo, si tenemos nodos con dos punteros, podemos emplear uno para enlazar según el orden de autores y el otro para enlazar según el año de edición. La Figura 4.10 muestra una lista de este estilo.

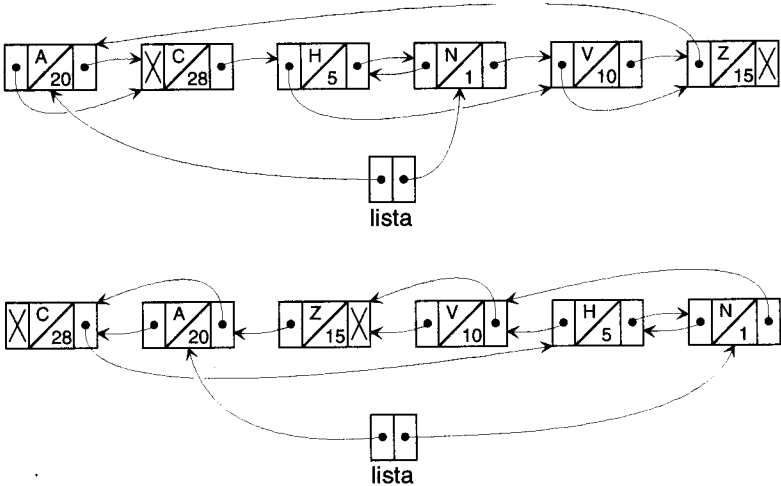


Figura 4.10 Lista con dos punteros que enlazan los elementos según dos criterios distintos. Las dos gráficas muestran la misma lista enlazada pero ordenando la representación gráfica de sus nodos según cada uno de los criterios.

Esta estructura puede generalizarse a cualquier número de punteros, tantos como criterios de ordenación distintos queramos mantener. En el caso extremo de que quisiéramos ordenar por todos los posibles campos de la información almacenada, habría un puntero por cada uno de estos campos.

Debe tenerse en cuenta que aunque sean listas doblemente enlazadas, son bastante distintas a las mostradas en los ejemplos anteriores, ya que ahora no se puede acceder desde un nodo a su predecesor, sino a dos sucesores. Cada nodo de esta lista indica quién es el nodo que le sigue según cada uno de los criterios pero no mantiene ninguna información sobre quién le precede, para ello sería necesario otro par de punteros adicional. Este tipo de listas deben considerarse como listas absoluta-

mente independientes, y cada vez que se quiera hacer una manipulación con un nodo (inserción, búsqueda, eliminación, etc.), habría que tenerlo en cuenta en las dos listas de forma absolutamente independiente, lo que hace el tratamiento bastante lento y complicado.

Como puede observarse, hay aplicaciones en las que las listas doblemente enlazadas son sensiblemente mejores que las listas enlazadas, pero no son tan numerosas como en un principio podría pensarse, además del exceso de complejidad y carga computacional que conllevan; de ahí la amplia utilización de las listas enlazadas simples.

En conclusión, será la aplicación la que determine la necesidad de definir un TDA dinámico u otro, pero al hacerlo deben considerarse detenidamente las ventajas e inconvenientes que pueden aportar. Se deja como ejercicio la realización de las operaciones básicas sobre los distintos tipos de listas enlazadas comentadas en este apartado.

4.3

PILAS

Una pila se suele definir simplemente como una estructura de datos en la que el último elemento en entrar es el primero en salir. A estas estructuras se les denomina LIFO (*Last In, First Out*).

Evidentemente es una estructura en la que los elementos están ordenados y se añaden y suprimen únicamente por un único extremo, conocido como *tope o cabeza de la pila*. Una estructura de tipo pila responde fielmente al concepto cotidiano de pila al que estamos muy acostumbrados en la vida real. Un ejemplo típico es una pila de cajas. Sólo se puede coger la caja de arriba de la pila, y sólo ahí se puede dejar una caja. Obsérvese que esta disposición de las cajas es una pila porque se actúa así sobre ella. Efectivamente, podría pensarse en suprimir una caja que no es la primera o se podría ser lo suficientemente habilidoso como para introducir una entre dos ya colocadas. En este caso la disposición de las cajas no es una estructura pila, será otra cosa (una lista enlazada, por ejemplo).

Como puede observarse por otro lado, la estructura pila es dinámica ya que el número de elementos que la componen es variable. Por ello una pila puede encontrarse definida en la literatura, usualmente en la dedicada a principios básicos de programación estructurada, como una lista enlazada en la que el último elemento en entrar es el primero en salir.

Sin embargo, deben realizarse algunas consideraciones acerca de esta sencilla definición. La definición inicial de la pila sólo resalta la característica más sobresaliente del TDA pila, su acceso LIFO, y definirla como una lista enlazada sólo indica una manera de implementarla. Es decir, ninguna de ellas es una definición rigurosa de la pila como TDA. Esto no sólo tiene importancia formal o académica. Las definiciones de los TDA precisan los términos y las operaciones que pueden realizarse sobre los datos evitando ambigüedades. En el ejemplo de las pilas de cajas, ¿podría

mirarse el contenido de la última caja? Si es una lista enlazada por supuesto que sí, y la definición de acceso LIFO no se infringiría por 'mirar' la última caja. Pero si se tiene en cuenta que para mirar el contenido de la caja habría que sacarla entonces sí se violaría la condición LIFO. Para no hacerlo habría que pensar en cajas transparentes, por ejemplo. Podría hacerse así un conjunto de elucubraciones, todas ellas correctas o incorrectas según el punto de vista. Recuérdese que la abstracción y el encapsulamiento de datos tienen como objetivo establecer con rigor y sencillez las estructuras, y por ello, estas definiciones iniciales no pueden considerarse en absoluto correctas en el contexto de los TDA.

Atendiendo a las consideraciones anteriores el TDA pila se define de la siguiente manera:

Definición 4.2: Una pila es un TDA dinámico homogéneo al que sólo se tiene acceso por la cabeza o cima de la pila; dicho acceso es de tipo LIFO ('Last In - First Out': último en entrar - primero en salir), los operadores básicos asociados son Meter y Sacar elementos de la pila, y los operadores auxiliares asociados son:

- **Inicia_pila:** crea una pila o limpia una existente inicializándola a una pila vacía, que es aquella que no contiene elementos. Permite partir de un estado previamente establecido.
- **Pila_Vacia:** operación utilizada para consultar si la pila está vacía.
- **Pila_Llena:** operación utilizada para consultar si la pila está llena.
- **Consultar_Pila:** operación utilizada para consultar el contenido de la cima de la pila sin sacar el elemento de la misma.

La operación **Pila_Vacia** será necesaria cuando se vaya a sacar un elemento de la pila ya que no se pueden sacar elementos de una pila vacía, y **Pila_Llena** cuando se vaya a meter un elemento en la pila ya que no se pueden introducir elementos en una estructura llena.

La operación **Consultar_Pila** se utiliza para consultar el último elemento introducido en la pila sin sacarlo de la misma. Obsérvese que esta operación no infringe la característica de la pila que indica que *sólo* se tiene acceso por la cabeza o cima de la pila. La Figura 4.11 muestra la inserción y la eliminación de los elementos de una pila.

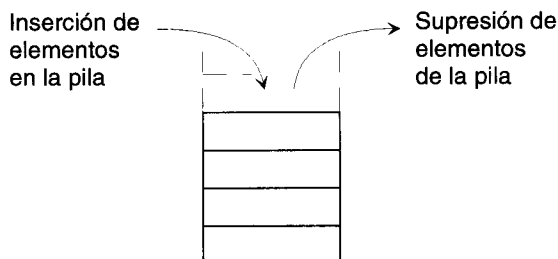


Figura 4.11 Inserción y eliminación de los elementos de una pila.

La Definición 4.2 establece el concepto de pila en sentido estricto, y así deberán considerarse todos los TDA. Con ella no existe ambigüedad alguna ya que todas las funciones a realizar deberán implementarse en función de sus operaciones básicas y auxiliares.

Es importante resaltar el hecho de que el carácter dinámico de la pila es conceptual, como en todos los TDA, y se debe a que el tamaño de la pila es variable, dependiendo de la cantidad de elementos almacenados en la misma. Este hecho es independiente de la implementación que se realice de la pila, en base a estructuras estáticas o dinámicas. Tanto con unas como con otras, la pila es dinámica. Seguidamente se presentan las implementaciones estática, mediante arreglos, y dinámica, mediante listas enlazadas, de la pila.

4.3.1 IMPLEMENTACIÓN DE PILAS MEDIANTE ARREGLOS

La implementación del TDA consta de dos partes. Por un lado, hay que definir la estructura sobre la que se almacenarán los datos, y por otro, la realización de los operadores básicos y asociados.

Dado que la pila es un tipo de datos homogéneo, es decir, sus componentes son del mismo tipo, es posible realizar su implementación mediante arreglos. Para ello, la pila vendrá representada por un registro con dos campos, uno el arreglo en el que se almacenarán los elementos de la pila y otro, denominado cima, que almacenará la posición del tope de la pila, es decir, la posición del último elemento introducido en la pila. Indexando el arreglo mediante enteros, el campo cima será de tipo entero. La declaración de una pila en Modula2 es la siguiente:

```
TYPE Tipo_Indice = [1..MAXPILA];
    Tipo_pila = RECORD
        datos : ARRAY Tipo_indice OF Tipo_datos;
        cima : INTEGER;
    END;
VAR pila : Tipo_pila;
```

donde MAXPILA es el tamaño máximo de la pila.

La realización de los operadores básicos y auxiliares se describe a continuación.

La operación *Inicia_pila* deberá establecer que la pila no contiene elementos, en función de un valor dado por convenio. Evidentemente la forma más natural de inicializar la pila es considerar que el valor de la cima es cero, indicando que la pila contiene cero elementos. Así, se toma como *criterio* que *cima indica la posición del último elemento introducido*. Cuando cima es cero indica que no ha habido último elemento introducido, con lo que la pila está vacía. Esta operación se implementa en Modula2 mediante el procedimiento del Programa 4.5.

Programa 4.5 Procedimiento para la inicialización de una pila implementada mediante arreglos.

```

PROCEDURE Inicia_pila (VAR pila: Tipo_pila);
BEGIN
    pila.cima := 0;
END Inicia_pila;

```

Para implementar los operadores básicos es necesario analizar su efecto sobre la pila. La Figura 4.12.a muestra una pila vacía. Obsérvese que para ‘vaciar’ la pila no es necesario realizar ninguna acción sobre los elementos del arreglo. Basta con considerar los posibles contenidos de `pila.datos` como ‘basura’.

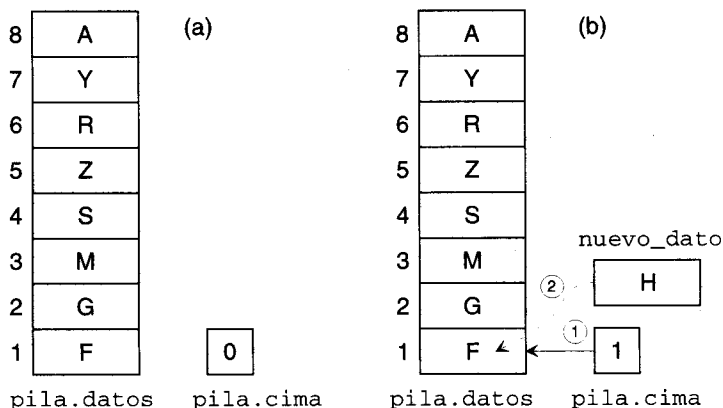


Figura 4.12 Pila vacía (a). Introducción de un nuevo elemento en la pila (b).

Dado que la cima indica por criterio la posición del último elemento introducido, para introducir el nuevo elemento habrá que incrementar primero la cima y seguidamente introducir el elemento, tal y como se ilustra en la Figura 4.12.b. Así el procedimiento en Modula2 que implementa la operación `Meter` viene dado por el Programa 4.6.

Programa 4.6 Implementación de la operación `Meter` de la pila implementada mediante arreglos.

```

PROCEDURE Meter_pila (VAR pila: Tipo_pila; nuevo_dato : Tipo_datos);
BEGIN
    pila.cima := pila.cima+1;
    pila.datos[pila.cima] := nuevo_dato;
END Meter_pila;

```

La operación Sacar actuará en orden inverso, de manera que primero se toma el elemento al que apunta cima y posteriormente se decrementa el valor de la cima. El Programa 4.7 muestra este procedimiento.

Programa 4.7 Implementación de la operación Sacar de la pila implementada mediante arreglos.

```
PROCEDURE Sacar_pila (VAR pila: Tipo_pila; VAR dato : Tipo_datos);
BEGIN
    dato := pila.datos[pila.cima];
    pila.cima := pila.cima-1;
END Sacar_pila;
```

Debe observarse el carácter dinámico de la pila. La cantidad de elementos que pertenecen a la pila es variable. Así, en la situación de la Figura 4.12.a la pila no tiene elementos mientras que en la de la Figura 4.12.b tiene un elemento. En esta situación los elementos del arreglo de índice 2, 3, etc., no forman parte de la pila. Ésta está formada únicamente por el elemento de índice 1. No debe confundirse la pila con el arreglo utilizado para implementarla.

Finalmente, las operaciones auxiliares de consulta vendrán dadas por las funciones mostradas en el Programa 4.8.

Programa 4.8 Operaciones auxiliares de consulta de la pila implementada mediante arreglos.

```
PROCEDURE Pila_vacia (pila:Tipo_pila) : BOOLEAN;
BEGIN
    RETURN pila.cima = 0;
END Pila_vacia;

PROCEDURE Pila_llena (pila:Tipo_pila) : BOOLEAN;
BEGIN
    RETURN pila.cima = MAXPILA
END Pila_llena;

PROCEDURE Consultar_pila (pila:Tipo_pila; VAR dato : Tipo_datos);
BEGIN
    dato := pila.datos[pila.cima]
END Consultar_pila;
```

Es importante observar que esta implementación no es la única válida. La implementación de la pila sobre arreglos anteriormente realizada ha tomado como criterio que la cima apunta a la posición del último elemento introducido. Sin embargo, se podría haber decidido que la *cima* apunte a la posición del último ele-

mento sacado; o lo que es lo mismo: apuntará a la posición donde deberá añadirse el próximo elemento. Con este criterio, la pila estará vacía cuando el valor del campo cima sea 1, para Meter un nuevo elemento habrá que introducir primero el elemento y seguidamente incrementar la cima y la operación Sacar actuará en orden inverso, de manera que primero se decrementa el valor de la cima y a continuación, se toma el elemento al que apunta cima. Las operaciones auxiliares de consulta cambiarán con el nuevo criterio de manera que la pila estará vacía si cima vale 1, y estará llena si vale MAXPILA+1. Finalmente la operación Consultar_Pila tomará el dato de la posición cima-1. Ambas implementaciones son igualmente válidas. Una vez realizada la implementación, el uso de la pila es independiente de la misma, es decir, para realizar cualquier función con la pila se utilizan los procedimientos y funciones de los operadores asociados y no se vuelve a atender a los detalles de implementación.

4.3.2 IMPLEMENTACIÓN DE PILAS MEDIANTE LISTAS ENLAZADAS

La implementación de pilas mediante estructuras dinámicas es inmediata partiendo de una lista enlazada. La declaración de la estructura en Modula2 será:

```
TYPE Tipo_pila = POINTER TO Nodopila;
   Nodopila = RECORD
       datos : Tipo_datos;
       enlace : Tipo_pila;
   END;
VAR pila : Tipo_pila;
```

y se accede a la pila mediante el puntero externo que apunta a la lista enlazada. La cima de la pila es obviamente la cabeza de la lista enlazada.

La operación Meter se implementa de la misma forma que se hacía en la inserción por la cabeza de la lista enlazada, y análogamente la operación Sacar es equivalente a sacar por la cabeza de la lista enlazada. Los procedimientos que implementan estas operaciones básicas se muestran en el Programa 4.9.

Programa 4.9 Implementación de las operaciones básicas de la pila basadas en una lista enlazada.

```
PROCEDURE Meter_pila (VAR pila: Tipo_pila; nuevo_dato : Tipo_datos);
VAR
   Nuevo_nodo : Tipo_pila;
BEGIN
   ALLOCATE(Nuevo_nodo, SIZE(Nodopila));
   Nuevo_nodo^.datos := nuevo_dato;
   Nuevo_nodo^.enlace := pila;
   pila := Nuevo_nodo;
END Meter_pila;
```

```

PROCEDURE Sacar_pila (VAR pila: Tipo_pila; VAR dato : Tipo_datos);
VAR
    Aux : Tipo_pila;
BEGIN
    Aux := pila;
    dato := pila^.datos;
    pila := pila^.enlace;
    DEALLOCATE(Aux, SIZE(Nodopila));
END Sacar_pila;

```

Las operaciones auxiliares *Iniciar_Pila* y *Pila_Vacia* se implementan también directamente a partir de los conceptos elementales de listas enlazadas. Así, para inicializar la pila bastará que el puntero externo sea NIL y para comprobar si está vacía habrá que consultar si tiene este valor.

La operación *Pila_llena* no puede obviarse aunque la implementación sea dinámica. En el caso de la implementación estática el hecho de que la pila estuviese llena podía resolverse en memoria principal dotando al programa de procedimientos alternativos para manejar esta situación, utilizando otra pila por ejemplo. Sin embargo, debe observarse que con implementación dinámica el hecho de que la pila esté llena significa que no existe memoria libre suficiente para crear un nuevo nodo. Ésta es una situación crítica en la que sólo se puede intentar liberar memoria principal, pasando datos a memoria secundaria, e informar que no queda memoria libre. El hecho de que no quede memoria libre se detecta en *Modula2* mediante la función *Available* cuya declaración es:

```

PROCEDURE Available(size: CARDINAL): BOOLEAN;

```

En el Programa 4.10 se muestra la realización de estas funciones auxiliares de pilas implementadas como una lista enlazada.

Programa 4.10 Implementación de las operaciones auxiliares de la pila empleando una lista enlazada.

```

PROCEDURE Inicia_pila (VAR pila: Tipo_pila);
BEGIN
    pila := NIL;
END Inicia_pila;

PROCEDURE Pila_vacia (pila: Tipo_pila) : BOOLEAN;
BEGIN
    RETURN pila = NIL;
END Pila_vacia;

```

```

PROCEDURE Pila_llena (pila:Tipo_pila) : BOOLEAN;
BEGIN
    RETURN ~Available(SIZE(Nodopila));
END Pila_llena;

```

```

PROCEDURE Consultar_pila (VAR pila: Tipo_pila; VAR dato : Tipo_datos);
BEGIN
    dato := pila^.datos;
END Consultar_pila;

```

4.4 COLAS

Una cola se suele definir simplemente como una estructura de datos en la que el primer elemento en entrar es el primero en salir. A estas estructuras se les denomina FIFO (*First In, First Out*). Como ocurría con las pilas, es una estructura en la que los elementos están ordenados, se añaden por un extremo, denominado *final de cola*, pero en este caso se suprimen por el otro, conocido como *principio de cola*.

Un ejemplo típico es la cola de un cine. El primero que llega a la cola es el primero que compra la entrada y sale de la cola. Podría pensarse en introducirse por el medio, pero colarse no está permitido. Además, debe observarse que tampoco está permitido, por la definición de cola, abandonarla si no se está al principio. Por tanto, si se desiste de comprar la entrada y se sale en una posición intermedia la estructura ya no es una cola, puesto que no satisface su definición. Al igual que con las pilas, las colas responden fielmente al concepto cotidiano de cola y todo el mundo conoce sus reglas y las respeta. Si en una cola, no respetamos sus reglas, con toda seguridad seremos fuertemente increpados y obligados a someternos a las reglas establecidas. Nuevamente puede observarse que la estructura cola es dinámica, ya que el número de elementos que la componen es variable.

Aplicaciones típicas de colas, aunque hay muchas más, son todas aquellas en las que debemos sincronizar la velocidad de llegada de datos con la velocidad de procesamiento. Todos hemos empleado alguna vez el término cola de impresión. Es un ejemplo típico en el que el sistema operativo de una máquina forma una cola con los trabajos a imprimir y son atendidos por riguroso orden de llegada. En este caso, podemos suponer que la velocidad de impresión es uniforme pero en determinados momentos pueden llegar los trabajos con mayor frecuencia. Es exactamente la misma situación que en el cine, la velocidad de servir entradas puede considerarse constante, pero la afluencia de espectadores es variable y durante algún tiempo superior a la primera.

Como sucedía con las pilas, las colas pueden encontrarse definidas en la literatura orientada a la formación en principios básicos de programación estructurada,

como una lista enlazada con acceso FIFO. Las consideraciones realizadas para las pilas en relación a este hecho son igualmente aplicables en este caso.

Al igual que en el caso de las pilas, esta definición inicial de la cola sólo resalta la característica más sobresaliente del TDA cola, su acceso FIFO, y definirla como una lista enlazada sólo hace referencia a una manera de implementarla.

En sentido estricto, que es el único correcto desde el punto de vista de la abstracción y el encapsulamiento, el TDA cola se define de la siguiente manera:

Definición 4.3: Una cola es un TDA dinámico homogéneo con acceso de tipo FIFO (primero en entrar - primero en salir), al que sólo se tiene acceso al principio de la cola para sacar elementos, y al final de la misma para meterlos. Los operadores básicos asociados son Meter y Sacar elementos de la cola, y los operadores auxiliares asociados son:

- **Inicia_cola:** crea una cola o limpia una existente inicializándola a una cola vacía, que es aquella que no contiene elementos. Permite partir de un estado previamente establecido.
- **Cola_Vacia:** operación utilizada para consultar si la cola está vacía.
- **Cola_Llena:** operación utilizada para consultar si la cola está llena.
- **Consultar_Cola:** operación utilizada para consultar el contenido del principio de la cola sin sacar el elemento de la misma.

La operación Cola_Vacia será necesaria cuando se vaya a sacar un elemento de la cola ya que no se pueden sacar elementos de una cola vacía, y Cola_Llena cuando se vaya a meter un elemento en la cola, ya que si está llena no se puede introducir más elementos en ella.

La operación Consultar_Cola se utiliza para consultar el último elemento introducido en la cola sin sacarlo de la misma. Esta operación, como ocurría en el TDA pila, no infringe la característica de la cola que indica que *sólo* se tiene acceso por el principio o final de ella. Análogamente podría definirse una operación que consultase el inicio de la cola.

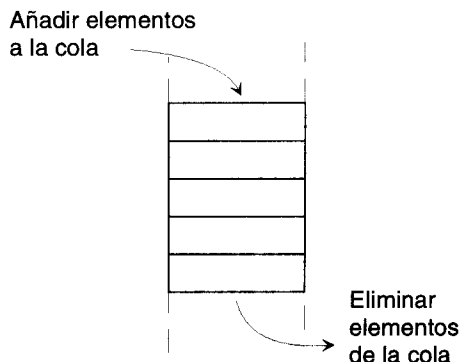


Figura 4.13 Inserción y la eliminación de los elementos de una cola.

La Figura 4.13 muestra la inserción y la eliminación de los elementos de una cola.

Con esta definición del TDA cola, no existe ambigüedad alguna ya que todas las funciones a realizar deberán implementarse en función de sus operaciones básicas y auxiliares.

Como ocurría con las pilas, el carácter dinámico de las colas es conceptual y se debe a que el tamaño de la cola depende de la cantidad de elementos almacenados en la misma, y por tanto, es variable independientemente de su implementación. Seguidamente se presentan las implementaciones mediante arreglos y mediante listas enlazadas.

4.4.1 IMPLEMENTACIÓN DE COLAS MEDIANTE ARREGLOS

La cola es un TDA homogéneo, por lo que es factible su implementación mediante arreglos. Seguidamente se exploran diversas implementaciones con el fin de analizar la más adecuada.

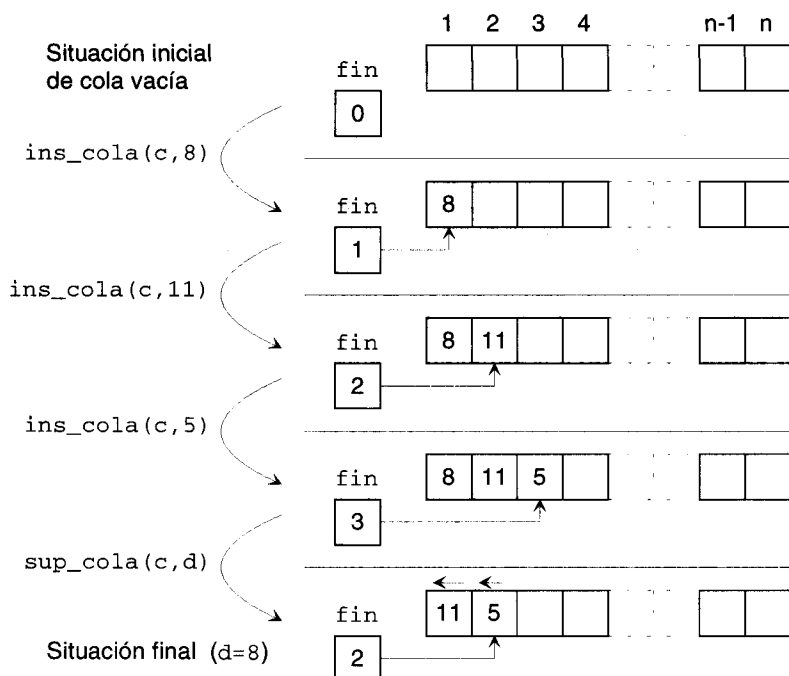


Figura 4.14 Cola con un único índice (fin = índice de final de cola).

En una primera implementación con arreglos se podría considerar la utilización de un único índice para almacenar el final, presuponiendo que el primer elemento de la cola está siempre en la primera posición del arreglo. Sin embargo, esta estructura es inadecuada por ineficaz ya que al sacar un elemento por el principio de la cola

(posición 1 del arreglo) deberemos seguidamente mover todos los elementos del arreglo una posición hacia el principio, tal y como se ilustra en la Figura 4.14. Este elevado coste computacional hace que la estructura no sea admisible, aunque 'funcione'. La situación en la cola del cine es básicamente la misma. Sin embargo, podemos pensar que se trata de un sistema en el que cada elemento dispone de su propio procesador para realizar el desplazamiento. Supóngase que alguien tuviese que encargarse de hacer avanzar un paso a los espectadores de la cola, con toda seguridad requeriría mucho más esfuerzo. Si, como sucede con los elementos de la cola, los espectadores no dispusieran de movilidad propia, sería mucho más eficiente que fuese la persona que sirve las entradas la que avanza sobre la cola de forma que bastaría mover un único elemento.

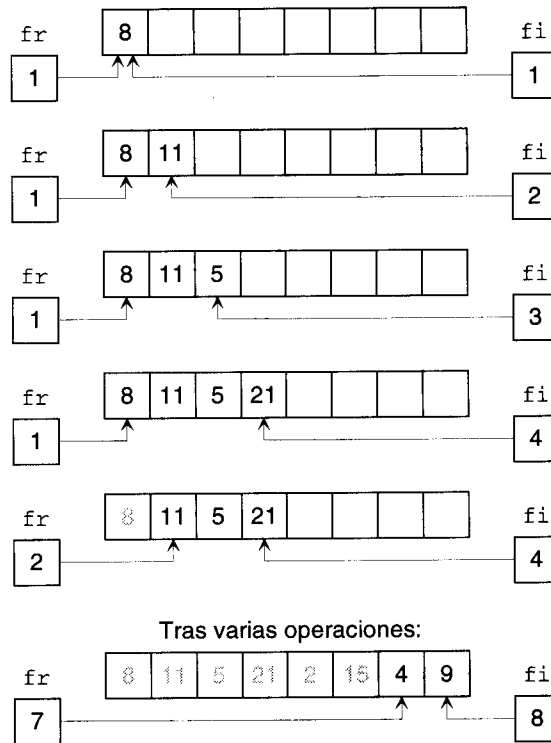


Figura 4.15 Implementación de la cola con dos índices. Obsérvese la situación final: hay muchos elementos libres pero, según el criterio de índices dado, no se puede seguir insertando.

Por tanto, es conveniente utilizar dos índices, uno que indique el final (*fi*) y otro el frente (*fr*) de la cola. Puede pensarse en adoptar como *criterio* que el final, *fi*, indique el último elemento introducido, aumentando cada vez que se introduce un nuevo elemento en la cola, y que frente, *fr*, apunte al primer elemento a sacar, aumentando al eliminar un elemento. Con esta implementación, las posiciones del arreglo en las que se han introducido elementos y posteriormente se han sacado que-

dan inutilizadas, debido al hecho de que los índices no se decrementan nunca. Este comportamiento que se ilustra en la Figura 4.15 hace que se alcance el final del arreglo y no se puedan seguir añadiendo elementos aunque haya numerosas posiciones libres al comienzo del mismo. Si se piensa en solucionar este problema decrementando los índices para reutilizar esas posiciones, esto implica mover todos los elementos de la cola, como sucedía en el caso de la estructura con un único índice y, por los mismos motivos que antes, esta opción debe desestimarse.

Para aprovechar las posiciones del arreglo que han sido utilizadas y están disponibles, se vuelve a comenzar por el principio una vez que hayamos llegado al final. De esta manera estamos considerando el arreglo como si fuera circular, de tal forma que el elemento que sigue al último es el primero. Lo que se consigue con esto es sustituir la estructura lineal del arreglo por una de tipo circular, tal y como se muestra en la Figura 4.16.

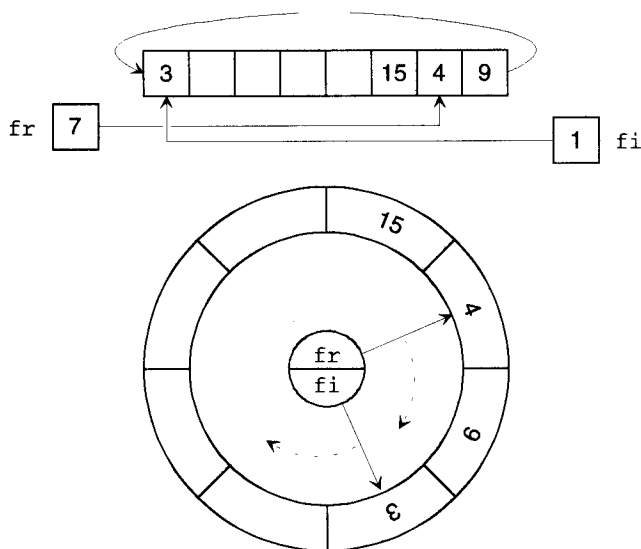


Figura 4.16 Estructura circular de la cola.

Antes de aceptar esta estructura debe analizarse el comportamiento de la misma. Las operaciones Meter y Sacar elementos no presentan ningún inconveniente ni en cuanto a economía de almacenamiento (como ocurría en la opción con dos índices y arreglo lineal) ni en relación con el coste computacional (como ocurría en la opción con un único índice).

Sin embargo, debe tenerse en cuenta que con la estructura circular la cola puede quedar vacía o llena en posiciones intermedias del arreglo. Analicemos detenidamente cómo detectar si la cola está vacía o llena.

Recuérdese que el *criterio* adoptado para los apuntadores es:

- Final (fi) apunta al último elemento introducido, y aumenta cada vez que se introduce un nuevo elemento en la cola.
- Frente (fr) apunta al primer elemento a sacar, y aumenta al eliminar un elemento de la cola.

Supóngase que la cola tiene un único elemento y va a quedar vacía. En esta situación ambos índices tienen el mismo valor ($fr=fi$) ya que el último elemento introducido y el primero a sacar son el mismo. Cuando el elemento es eliminado de la cola, el índice frente (fr) se incrementa. Esta situación se ilustra en la Figura 4.17. Por tanto, la condición de cola vacía será tal que el frente esté apuntando al siguiente elemento al que apunta final. Este siguiente elemento ofrece dos alternativas según sea el último del arreglo o sea otro cualquiera. Si es un elemento cualquiera distinto del último, el siguiente se obtiene incrementando en 1, si por el contrario es el último elemento su siguiente será el primero:

$$\text{Condición de cola vacía: } fr = \text{sig}(fi) = \begin{cases} \text{Si } fi = \text{MAXCOLA:} & fr = 1 \\ \text{Si } fi \neq \text{MAXCOLA:} & fr = fi + 1 \end{cases}$$

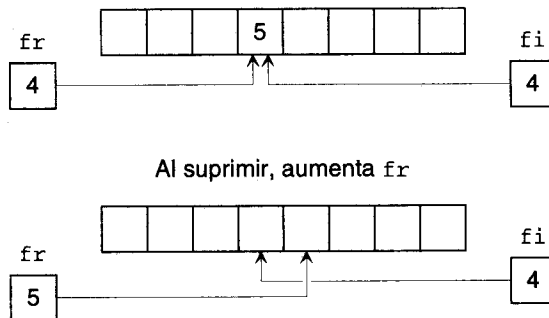


Figura 4.17 Condición de cola vacía: $fr=5$, $fi=4$, $fr=\text{sig}(fi)$: (1 o $fi+1$).

Por otro lado, supóngase ahora que a la cola le falta un elemento para estar llena, y tras una nueva inserción va a quedar llena (Figura 4.18). En esta situación entre ambos índices hay una posición libre y entonces $fr=fi+2$, ya que el final apunta al último elemento introducido, $fi+1$ será la posición que aún está libre y $fi+2$ será la posición del primer elemento de la cola, y por tanto el frente de la misma. Cuando un nuevo elemento es introducido en la cola, el índice final (fi) se incrementa, quedando la cola llena. Por tanto, la condición de cola llena será:

$$\text{Condición de cola llena: } fr = \text{sig}(fi) = \begin{cases} \text{Si } fi = \text{MAXCOLA:} & fr = 1 \\ \text{Si } fi \neq \text{MAXCOLA:} & fr = fi + 1 \end{cases}$$

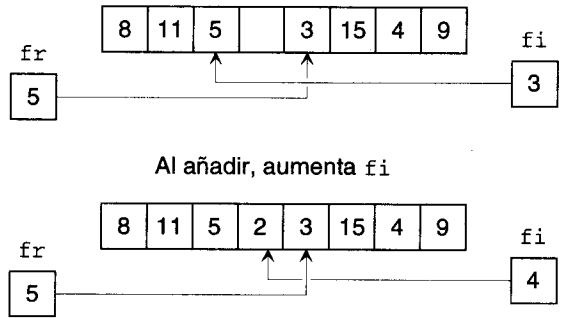


Figura 4.18 Condición de cola llena, $fi=4$ y $fr=5$, $fr=\text{sig}(fi)$: (1 o $fi+1$).

En conclusión, con el criterio adoptado para los apuntadores, tal que final (fi) apunta al último elemento introducido, y aumenta cada vez que se introduce un nuevo elemento en la cola, y frente (fr) apunta al primer elemento a sacar, y aumenta al eliminar un elemento de la cola, *no es posible distinguir si la cola está llena o vacía*.

Una primera solución que puede considerarse es mantener un contador que indique los elementos que hay en la cola. Esta solución lleva a la necesidad de incrementar este contador con cada nueva inserción y decrementarlo con cada eliminación. Siempre será mejor una solución que evite realizar estas operaciones adicionales. Para conseguirlo será necesario cambiar el criterio de asignación de índices, de manera que se distingan claramente las situaciones de cola llena y vacía. Así, la condición de cola vacía puede establecerse de forma que en esta situación el frente y el final de la cola coincidan, y la de cola llena tal que en esta situación el frente sea el final más uno. En resumen:

Condición de cola vacía: $fr=fi$

Condición de cola llena:
$$fr=\text{sig}(fi) = \begin{cases} \text{Si } fi=\text{MAXCOLA}: & fr=1 \\ \text{Si } fi\neq\text{MAXCOLA}: & fr=fi+1 \end{cases}$$

Estas condiciones se satisfacen para el siguiente *criterio*, que define los apuntadores frente y final.

Criterio 1

- El frente, fr, apunta al elemento anterior al primero de la cola.
- El final, fi, apunta al último elemento introducido en la cola.

Obsérvese que frente apunta a un elemento que nunca se rellena, con lo que cuando la cola está llena se está dejando una posición libre en el arreglo.

La Figura 4.19 muestra la situación en la que la cola queda vacía y la Figura 4.20 muestra la cola llena.

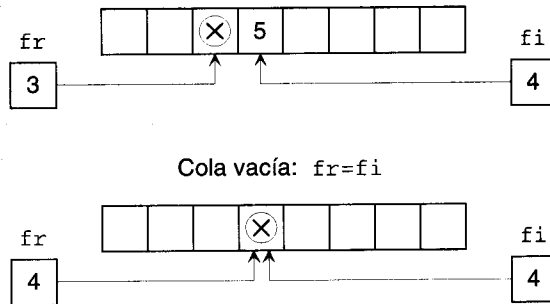


Figura 4.19 Cola vacía con $fr=4$ y $fi=4$ ($fr=fi$).

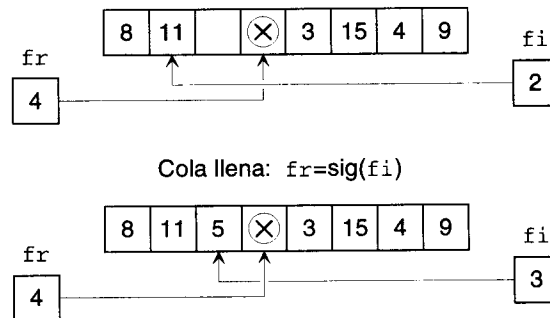


Figura 4.20 Cola llena con $fr=4$ y $fi=3$ $fr=sig(fi)$.

Con el *Criterio 1* y atendiendo a las consideraciones anteriores ya se está en disposición de implementar el TDA cola mediante arreglos. En primer lugar, la estructura consistirá en un registro con el arreglo en el campo de datos, y los dos apuntadores frente (fr) y final (fi). Si el arreglo se indexa mediante enteros los dos apuntadores serán de tipo entero. La declaración de la estructura en Modula2 es la siguiente:

```

CONST MAXCOLA=MAXCOLAN+1;
TYPE
  TipoIndice = [1..MAXCOLA];
  TipoCola = RECORD
    datos : ARRAY TipoIndice OF Tipo_datos;
    fr, fi : TipoIndice;
  END;
VAR cola: TipoCola;
```

En estas declaraciones `MAXCOLAN` es el máximo número de elementos que pueden almacenarse en la cola.

Obsérvese que $\text{MAXCOLA} = \text{MAXCOLAN} + 1$ es el tamaño necesario del arreglo para implementar la cola, que viene determinado por el número de elementos a almacenar más el elemento que siempre quedará libre. El apuntador `fi` es el índice que indica el final de la cola, es decir, la posición del arreglo en la que se introdujo el último elemento, y `fr` indica el frente de la cola, es decir, la posición anterior al elemento a sacar de la cola.

Los operadores básicos se implementan de forma inmediata atendiendo al criterio utilizado y al carácter circular de la cola. Así, habrá que considerar dos situaciones, una cuando las operaciones se realizan en los extremos del arreglo sobre el que se implementa la cola, y otra cuando se realizan en posiciones intermedias del mismo. De este modo, para implementar el operador `Meter_cola`, se manipulará el índice de final (`fi`). Como `fi` apunta al último elemento introducido en la cola, para introducir el nuevo elemento habrá que incrementar primero el final, `fi`, y seguidamente introducir el nuevo dato en la posición que esté siendo apuntada por este índice. En el incremento del índice `fi` se deben considerar dos casos: si éste se encuentra apuntando al final del arreglo (`cola.fi = MAXCOLA`), el nuevo elemento habrá que introducirlo en la primera posición del mismo (`cola.fi:=1`), y en cualquier otro caso bastará con incrementar el índice `fi` en 1 (`cola.fi := cola.fi+1`). Esta distinción es necesaria para convertir en circular la estructura puramente lineal del arreglo.

El razonamiento es análogo para implementar el operador `Sacar`. Teniendo en cuenta que por el *Criterio 1* el índice frente (`fr`) apunta al elemento anterior al primero de la cola, para sacar un elemento habrá que incrementar primero el índice `fr` y posteriormente asignarlo. De nuevo, si el índice se encuentra en el final del arreglo (`cola.fr = MAXCOLA`) el primer elemento de la cola estará en la primera posición del arreglo (`cola.fr:=1`), y si está en las posiciones intermedias del arreglo basta con incrementar el índice `fr` en 1 (`cola.fr := cola.fr+1`).

El Programa 4.11 muestra la implementación en Modula2 de los operadores básicos de la cola implementada estáticamente.

Programa 4.11 Implementación de los operadores básicos de la cola implementada estáticamente.

```

PROCEDURE Meter_cola (VAR cola : Tipo_cola; nuevo_dato : Tipo_datos);
BEGIN
  IF cola.fi = MAXCOLA THEN
    cola.fi := 1
  ELSE cola.fi := cola.fi+1
  END;
  cola.datos[cola.fi] := nuevo_dato
END Meter_cola;

```

```

PROCEDURE Sacar_cola (VAR cola: Tipo_cola; VAR dato : Tipo_datos);
BEGIN
    IF cola.fr = MAXCOLA THEN
        cola.fr := 1
    ELSE cola.fr := cola.fr + 1
    END;
    dato := cola.datos[cola.fr]
END Sacar_cola;

```

El operador auxiliar *Iniciar* puede implementarse de muy diversas formas. Al inicializar la cola, ésta estará vacía y por tanto, debe satisfacerse que $cola.fr = cola.fi$. Dado que la estructura del arreglo es circular, cualquier elemento es igualmente válido para imponer esta condición, de ahí las múltiples implementaciones posibles de este operador.

Si se adopta el criterio de que partiendo de una cola vacía, el primer elemento se introducirá en la primera posición del arreglo, entonces con la cola vacía, *fi* deberá apuntar a *MAXCOLA* para que al incrementarse para introducir el nuevo elemento, apunte a la primera posición del arreglo. Y como la condición de cola vacía exige $fr = fi$, entonces la cola se inicializará con:

$cola.fr = cola.fi = MAXCOLA.$

El operador de consulta *Cola_Vacia* se implementa sencillamente comprobando si $fr = fi$, y *Cola_Llena* comprobará la condición $cola.fr = cola.fi + 1$ en las posiciones intermedias del arreglo, y en el extremo del arreglo, cuando $cola.fi = MAXCOLA$, la cola estará llena si $cola.fr = 1$.

Los operadores de consulta al principio y al final de la cola se implementan de forma análoga. Las implementaciones estáticas en Modula2 de los operadores auxiliares de la cola se muestran en el Programa 4.12.

Programa 4.12 Implementación de los operadores auxiliares de la cola implementada estáticamente.

```

PROCEDURE Inicia_cola (VAR cola: Tipo_cola);
BEGIN
    cola.fr := MAXCOLA;
    cola.fi := MAXCOLA
END Inicia_cola;

PROCEDURE Cola_vacia (cola: Tipo_cola) : BOOLEAN;
BEGIN
    RETURN cola.fi = cola.fr;
END Cola_vacia;

```

```

PROCEDURE Cola_llena (cola:Tipo_cola) : BOOLEAN;
BEGIN
  IF cola.fi = MAXCOLA THEN RETURN cola.fr =1
  ELSE RETURN cola.fr = cola.fi + 1;
  END
END Cola_llena;

```

```

PROCEDURE Consultar_final_cola (cola : Tipo_cola; VAR dato : Tipo_datos);
BEGIN
  dato := cola.datos[cola.fi];
END Consultar_final_cola ;

```

```

PROCEDURE Consultar_frente_cola (cola: Tipo_cola; VAR dato : Tipo_datos);
BEGIN
  IF cola.fr = MAXCOLA THEN
    dato := cola.datos[1]
  ELSE dato := cola.datos[cola.fr + 1]
  END;
END Consultar_frente_cola;

```

La implementación desarrollada, que se basa en el *Criterio 1* no es la única válida. En efecto, las condiciones de distinguibilidad de cola vacía y cola llena también se satisfacen con el siguiente criterio de definición de los apuntadores frente y final:

Criterio 2

- El frente, *fr*, apunta al primer elemento de la cola.
- El final, *fi*, apunta al elemento siguiente al último introducido en la cola.

La Figuras 4.21 y 4.22 ilustran este criterio.

Como puede observarse los índices del *Criterio 2* son los del *Criterio 1* incrementados una posición. A continuación recordamos el primer criterio con el fin de observar este hecho:

Criterio 1

- El frente, *fr*, apunta al elemento anterior al primero de la cola.
- El final, *fi*, apunta al último elemento introducido en la cola.

La implementación de los operadores básicos y auxiliares es diferente según el criterio que se decida emplear, aunque ambos satisfagan la misma condición de distinguibilidad de cola llena y cola vacía. Sin embargo, una vez desarrollada la implementación de los operadores, las tareas a desarrollar con la cola se realizan utilizándolos sin atender a sus detalles de implementación.

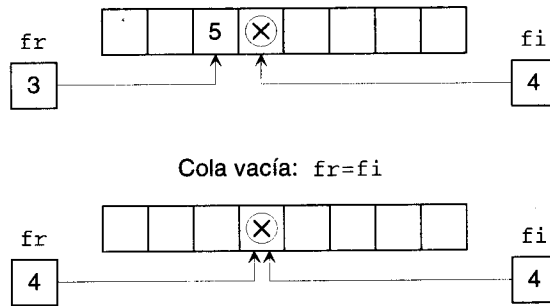


Figura 4.21 Cola vacía con $fr=4$ y $fi=4$ ($fr=fi$).

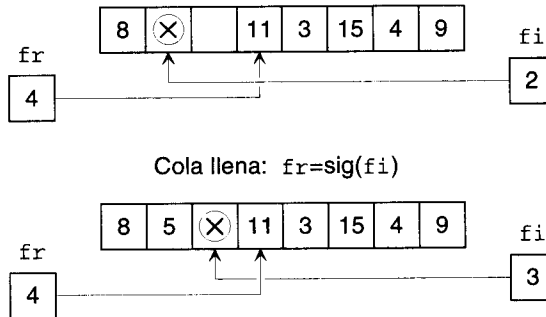


Figura 4.22 Cola llena con $fr=4$ y $fi=3$ ($fr=fi+1$).

Antes de pasar a comentar la implementación de las colas con asignación de memoria dinámica basadas en listas enlazadas, veamos una forma alternativa y muy habitual de recircular los índices de frente y final de una cola. Para ello se hace uso del operador resto de una división entera y se define el arreglo que contiene los datos entre 0 y $MAXCOLA-1$. En este caso el siguiente elemento se calcula con $sig:=(actual+1) \text{ MOD } MAXCOLA$. Esto es válido para todos los elementos incluido el último y no es necesario considerar este último elemento de distinta forma al resto. Supongamos como ejemplo que $MAXCOLA=10$ y que el índice de final vale 7. Al aplicar la expresión anterior obtendríamos $(7+1) \text{ MOD } 10=8$, que es equivalente a $7+1$. Supongamos ahora que está en la última posición del arreglo ($fi=9$). Si aplicamos nuevamente la misma expresión tendremos: $(9+1) \text{ MOD } 10=0$, con lo que el índice de final pasa a apuntar a la primera posición tal y como se requiere para recircular los índices a lo largo del arreglo.

El Programa 4.13 muestra las declaraciones necesarias y la implementación de las operaciones fundamentales que deben ser reescritas para emplear esta alternativa de recirculación de índices.

Programa 4.13 Implementación de una cola circular empleando el operador módulo. Sólo se muestran aquellas operaciones que se ven alteradas por esta nueva estrategia de recirculación.

```

TYPE
  TipoIndice = 0..MAXCOLA;
  Tipo_cola = RECORD datos : ARRAY[0..MAXCOLA-1] OF Tipo_datos;
                  fr,fi : TipoIndice;
                  END;

PROCEDURE Meter_Cola(VAR cola: Tipo_cola; nuevo : Tipo_datos);
BEGIN
  cola.fi:=(cola.fi+1) MOD MAXCOLA;
  cola.datos[cola.fi]:=nuevo;
END;

PROCEDURE Sacar_Cola(VAR cola : Tipo_cola; VAR dato : Tipo_datos);
BEGIN
  cola.fr:=(cola.fr+1) MOD MAXCOLA;
  dato:=cola.datos[cola.fr];
END;

PROCEDURE Cola_Llena(VAR cola : Tipo_cola) : BOOLEAN;
BEGIN
  RETURN cola.fr=(cola.fi+1) MOD MAXCOLA;
END;

```

4.4.2 IMPLEMENTACIÓN DE COLAS MEDIANTE LISTAS ENLAZADAS

En principio, puede pensarse en utilizar una lista enlazada para implementar una cola, empleando operaciones análogas a insertar por la cabeza y suprimir por el final (el frente de la cola sería el final de la lista y el final de la cola sería el principio de la lista), o bien insertar por el final y suprimir por la cabeza (el frente de la cola sería el principio de la lista y el final de la cola sería el final de la lista). En cualquier caso, con una lista enlazada, la inserción y la supresión por el final requiere un bucle hasta alcanzar este extremo. Esto significa que siempre una operación básica (Sacar o Meter) tiene un número de comparaciones y asignaciones de punteros igual a la cantidad de elementos que tenga la cola.

La solución es sencilla, y consiste en modificar la lista enlazada dotándola de un puntero externo más, que apunte al final de la misma. Esta estructura ya fue presentada en la Figura 4.5.

La declaración de la cola dinámica en Modula2 será, por tanto:

```

TYPE Ptr_nodo = POINTER TO Nodocola;
Nodocola= RECORD
    datos : Tipo_datos;
    enlace : Ptr_nodo;
END;
TipoCola = RECORD
    frente,final : Ptr_nodo;
END;

```

Ahora se nos plantea la cuestión de ver qué función asignamos a cada uno de los extremos de la lista. Es decir, decidir si añadimos por el principio de la lista y eliminamos por el final o lo hacemos a la inversa. En principio, parecen situaciones simétricas, pero no lo son, ya que los punteros apuntan desde el inicio hacia el final pero no a la inversa.

Consideremos que el puntero externo *frente* apunta al principio de la lista y el puntero externo *final* apunta al último elemento. Debe observarse que, con esta estructura de lista enlazada, no se puede suprimir por el final. Para poder hacerlo, se debería asignar el valor NIL al penúltimo elemento, pero su dirección no es conocida a partir del último nodo o del puntero externo *final*. Para conseguirlo, habría que emplear un bucle que recorriese toda la lista desde el principio, estando en la misma situación que una lista con un único puntero, con su considerable costo computacional. Otra alternativa sería una lista doblemente enlazada, pero sólo se necesita un enlace doble y de esta forma tendríamos todos los nodos doblemente enlazados con el consiguiente gasto de memoria y de actualización de punteros. Sin embargo, si añadimos por el final de la lista y eliminamos por el principio de la misma, no se plantea este problema. En conclusión, la operación *Meter* debe realizarse al final de la lista, y la operación *Sacar* al principio, lo que por supuesto no es ningún inconveniente.

Como es habitual, es necesario establecer un criterio para determinar de forma única la situación en que la cola está vacía. Estableciendo como criterio que la cola está vacía cuando el puntero *final* sea NIL, la inicialización de la cola se realiza simplemente asignando NIL a *cola.final*.

La operación de *Meter* se realizará considerando el puntero *final*. En primer lugar se crea un nuevo nodo, a continuación se introduce el nuevo dato en su campo *datos*, y se establece NIL en su enlace, indicando que será el último elemento. Estos tres primeros pasos se muestran en las Figuras 4.23 y 4.24. Seguidamente se distinguen dos situaciones: el caso general en que la cola ya contiene elementos y el caso especial en el que esté inicialmente vacía.

Si la cola ya contiene elementos (Figura 4.23) se enlaza el último nodo de la lista con el nuevo nodo (*cola.final^.enlace:=Nuevo_nodo*). Sin embargo, si la cola está vacía (Figura 4.24) no puede hacerse esto. En este caso es el puntero externo *frente* el que debe apuntar a este nuevo nodo.

Obsérvese que, en principio, el puntero frente sólo se modifica al eliminar y el puntero final sólo al añadir. Sin embargo, como frente apunta al primer elemento que saldrá de la cola, si el que se inserta es el primero (y por tanto único), habrá que hacer que frente apunte a él.

Por último, se asigna la dirección del nuevo nodo al puntero externo final independientemente del estado inicial de la cola. La Figura 4.23 ilustra la inserción en una cola de este tipo que ya contiene elementos y en la Figura 4.24 se muestra la misma acción pero sobre una cola inicialmente vacía. En estas figuras, FR y FI representan los campos frente y final del registro que forma la cola.

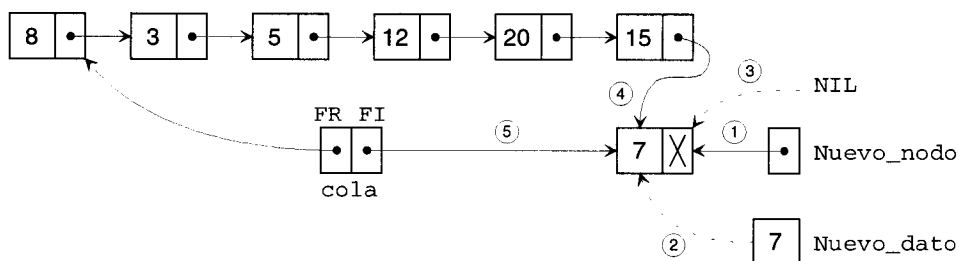


Figura 4.23 Inserción en una cola no vacía implementada con asignación dinámica de memoria.

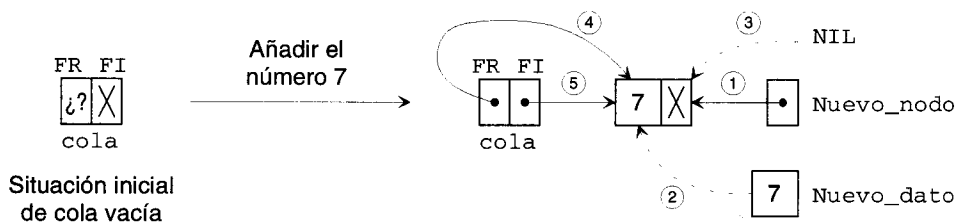


Figura 4.24 Inserción en una cola inicialmente vacía con asignación dinámica de memoria.

La operación Sacar toma los datos de la variable referenciada a la que apunta cola.frente y asigna el puntero externo frente al siguiente nodo (cola.frente:=cola.frente^.enlace;). Nuevamente hay que distinguir la situación de elemento único (al sacarlo la cola quedará vacía) de la situación en que la lista tiene más de un elemento. Por tanto, es necesario comprobar si tras la eliminación la cola ha quedado vacía, en cuyo caso debe asignarse NIL al puntero externo final, indicativo de este hecho para posteriores operaciones. Como puede verse la situación es prácticamente simétrica a la que se tenía al insertar: sólo se actúa sobre el puntero final si el elemento eliminado es el último, en caso contrario basta con manipular adecuadamente el puntero frente. Finalmente, se libera el nodo con la ayuda de un puntero auxiliar (aux) que se habrá asignado al último elemento como primer paso de la eli-

minación. La Figura 4.25 muestra la eliminación en una cola con más de un elemento y la Figura 4.26 la eliminación en una cola con un único elemento. El Programa 4.14 muestra la implementación en Modula2 de estos operadores básicos.

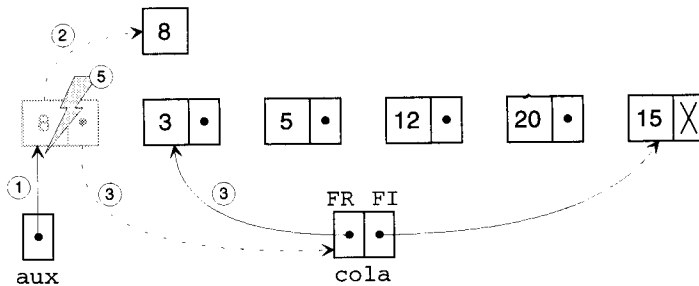


Figura 4.25 Eliminación de un elemento no único en una cola implementada dinámicamente.

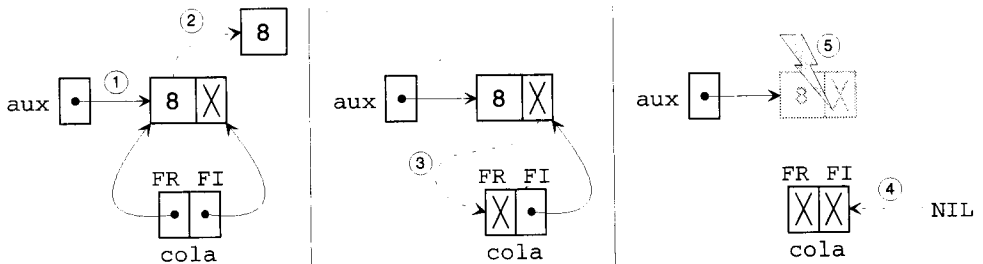


Figura 4.26 Eliminación del último elemento en una cola implementada dinámicamente. En este caso, el paso 3 no es necesario pero se mantiene por ser común con el caso general.

Programa 4.14 Implementación en Modula2 de los operadores básicos Meter y Sacar para la cola con implementación dinámica.

```

PROCEDURE MeterCola (VAR cola : TipoCola; nuevo_dato : TipoDatos);
VAR
  Nuevo_nodo : Ptr_nodo;
BEGIN
1   ALLOCATE(Nuevo_nodo, SIZE(NodoCola));
2   Nuevo_nodo^.datos := nuevo_dato;
3   Nuevo_nodo^.enlace := NIL;
4   IF cola.final=NIL THEN cola.frente := Nuevo_nodo
   ELSE cola.final^.enlace := Nuevo_nodo END;
5   cola.final := Nuevo_nodo
END MeterCola;

```

```

PROCEDURE SacarCola (VAR cola: TipoCola; VAR dato : Tipo_datos);
VAR
    aux : Ptr_nodo;
BEGIN
1   aux := cola.frente;
2   dato := cola.frente^.datos;
3   cola.frente := cola.frente^.enlace;
4   IF cola.frente = NIL THEN cola.final := NIL END;
5   DEALLOCATE(aux,SIZE(NodoCola));
END SacarCola;

```

Los operadores auxiliares son sencillos de implementar, y se muestran en el Programa 4.15. Iniciar ya fue comentado anteriormente. La consulta sobre si la cola está vacía se lleva a cabo siguiendo el criterio adoptado mediante la comparación del puntero externo final con NIL, y las consideraciones realizadas sobre la operación Pila_llena en la implementación dinámica de las pilas son igualmente aplicables en este caso. Las operaciones de consulta en los extremos de la cola son evidentes.

Programa 4.15 Implementación dinámica en Modula2 de los operadores auxiliares para la cola.

```

PROCEDURE IniciaCola (VAR cola: TipoCola);
BEGIN
    cola.final := NIL;
END IniciaCola;

PROCEDURE Cola_vacia (cola:TipoCola) : BOOLEAN;
BEGIN
    RETURN cola.final = NIL;
END Cola_vacia;

PROCEDURE Cola_llena (cola:TipoCola) : BOOLEAN;
BEGIN
    RETURN ~Available(SIZE(Nodo));
END Cola_llena;

PROCEDURE Consultar_frenteCola (cola: TipoCola; VAR dato : Tipo_datos);
BEGIN
    dato := cola.frente^.datos;
END Consultar_frenteCola;

```

```
PROCEDURE Consultar_finalCola (cola: TipoCola; VAR dato : TipoDatos);  
BEGIN  
    dato := cola.final^.datos;  
END Consultar_frenteCola;
```

Obsérvese que en esta implementación se ha tomado como criterio de cola vacía que el puntero final sea NIL. Análogamente podría haberse establecido como criterio que la cola estuviese vacía cuando el puntero frente apunte a NIL. Esto obliga a una implementación diferente de los operadores asociados, pero de nuevo, una vez desarrollados, las tareas que hacen uso de la cola no atienden a los detalles de implementación y se realizan haciendo uso exclusivamente de los operadores asociados.

Como se ha dicho anteriormente, la estructura con dos punteros externos permite un menor coste computacional de las operaciones. Además debe observarse que el gasto en memoria es mínimo ya que es sólo un puntero más (y recordar que su valor es simplemente una dirección).

Es conveniente considerar la posibilidad de implementar una cola haciendo uso de una lista doblemente enlazada con dos punteros externos (frente y final). Evidentemente ahora ya no se tiene el problema del costo del bucle para realizar una de las operaciones, como pasaba en la lista enlazada simple, ni tampoco la dificultad para realizarla insertando al principio y suprimiendo al final, tal y como sucedía en la lista enlazada con dos punteros externos al frente y al final. Pero también es verdad que no presenta ninguna ventaja ni desventaja en relación al coste computacional respecto a la implementación mediante una lista enlazada con los dos punteros frente y final. Por tanto, ¿puede decirse que ambas implementaciones son igualmente buenas? La respuesta es claramente no. La implementación mediante una lista doblemente enlazada es inadecuada siguiendo el criterio de economía de almacenamiento. Obsérvese que para cada nodo se ha declarado un puntero más y por tanto, con cada elemento almacenado se está reservando memoria que no se necesita para implementar la cola. Además, al tener dos enlaces por nodo, la actualización de punteros requerirá el doble de operaciones. Por tanto, sería una solución incorrecta, aunque ‘funcione’.

En conclusión, la implementación desarrollada mediante una lista enlazada con dos punteros externos, frente y final, es la más adecuada.

4.5 EJEMPLOS DE APLICACIÓN

4.5.1 EVALUACIÓN DE LA PARENTIZACIÓN DE EXPRESIONES

Las expresiones aritméticas, como es bien sabido de matemáticas básicas, se representan habitualmente de manera que los operadores se encuentran entre los operandos. Así, la suma de los operandos A y B se expresa como $A+B$. Además, se

utilizan unas reglas de precedencia para los operadores y el uso de paréntesis para romper dichas reglas cuando se requiera. Este tipo de notación es conocida como notación ‘infija’.

En la mayoría de los lenguajes de alto nivel se utiliza esta notación para representar expresiones aritméticas. Una tarea inicial en todos ellos será analizar que la expresión está bien parentizada. Supóngase por ejemplo que las cadenas a evaluar están formadas por caracteres entre los que se encuentran los símbolos de parentización. Éstos son paréntesis, corchetes y llaves, abiertos y cerrados (‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’). Entonces la expresión está bien parentizada si los símbolos abierto y cerrado de un mismo tipo se emparejan como indican las reglas de las expresiones aritméticas. La Figura 4.27 muestra algunos ejemplos de cadenas bien y mal parentizadas.

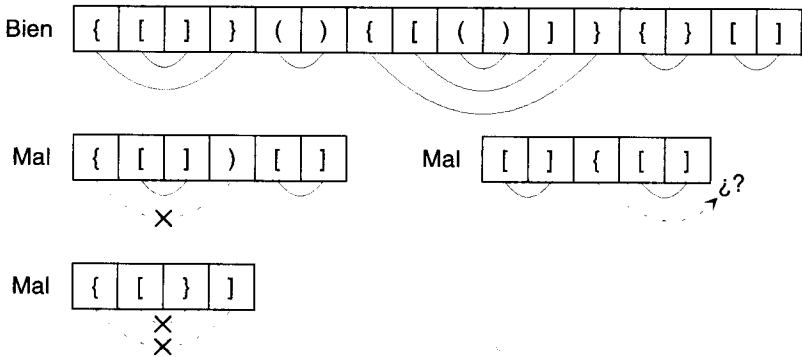


Figura 4.27 Ejemplo de varias cadenas bien y mal parentizadas.

Consideremos dos situaciones ligeramente distintas:

- a) Supóngase que las cadenas se leen desde consola con buffer.
- b) Supóngase que las cadenas se leen desde un fichero.

Independientemente del dispositivo periférico del que vengan los caracteres, el tratamiento de la cadena para su evaluación es el mismo. Cuando se leen los caracteres de la cadena se dan dos situaciones. Si el símbolo es abierto se almacenará a la espera de un símbolo cerrado. Si el símbolo es cerrado entonces se analiza si el *último* símbolo abierto almacenado es del mismo tipo. Si lo es, ambos parentizan correctamente y no será necesario analizarlos de nuevo, pudiendo continuar con la evaluación del resto de la cadena. Si no lo es, entonces la cadena está mal parentizada. Con lo dicho el TDA utilizado para el almacenamiento debe ser tal que el último en entrar sea el primero en salir, es decir, una pila. El Programa 4.16 muestra la implementación en Modula2 de la solución. Dicha solución es la misma, desde el punto de vista abstracto, independientemente del periférico del que venga la cadena. Sin embargo, la implementación de la pila deberá ser distinta. Cuando las cadenas se leen de consola —situación a)— la pila deberá ser estática ya que es más rápida y el número máximo de caracteres es conocido (el tamaño máximo del buffer de con-

sola). Sin embargo, si vienen de un fichero —situación b)— es posible que la cadena tenga un tamaño grande y desconocido *a priori*, por lo que será conveniente una implementación dinámica. Estos razonamientos deben entenderse desde el punto de vista académico, ya que difícilmente encontraremos expresiones aritméticas que requieran ser almacenadas en ficheros por su elevada longitud.

Programa 4.16 Evaluación de la parentización de una expresión.

```

MODULE parentizar;
(*Comprueba si una expresion esta bien parentizada*)
FROM InOut IMPORT WriteString,Read,ReadInt, WriteLn,Write,WriteInt;
FROM pilarray IMPORT Tipo_pila, Inicia_pila,
Meter_pila,Sacar_pila,Pila_vacia;
CONST
    long = 30;
TYPE
    Tipo_conj = SET OF CHAR;

VAR
    abierto      : Tipo_pila;
    ch1,ch2      : CHAR;
    error        : BOOLEAN;
    abre,cierra  : Tipo_conj;

BEGIN
    abre := Tipo_conj{'(','[','{'};
    cierra := Tipo_conj{')','}','}'};
    Inicia_pila(abierto);
    error := FALSE;
    WriteString('Introduzca cadena con un punto para terminar:');
    WriteLn;
    Read(ch1);
    WHILE (ch1#'.') & (NOT error) DO
        Write(ch1);
        (*Tratamiento de cada caracter leído*)
        IF ch1 IN abre THEN
            Meter_pila(abierto,ch1);
        ELSIF ch1 IN cierra THEN
            error := Pila_vacia(abierto);
            IF NOT error THEN
                Sacar_pila (abierto,ch2);
                error := error AND (((ch1=')') & (ch2='(')) OR
                    (((ch1=']') & (ch2='[')) OR
                    (((ch1='}') & (ch2='{'})))
            END
        END
        Read(ch1);
    END;
END;
```

```

    Read(ch1);
END;
IF NOT Pila_vacia(abierto)OR error THEN
    WriteString('Mal parentizado')
ELSE
    WriteString('Bien parentizado')
END;
END parentizar.

```

4.5.2 NOTACIÓN DE EXPRESIONES PREFIJA Y POSTFIJA

La notación de expresiones constituye un ejemplo clásico de aplicación de TDA dinámicos. El problema de la notación infija es la necesidad de utilizar los paréntesis para evitar ambigüedades. Esto, además de requerir espacio de almacenamiento para ellos, necesita un proceso de evaluación de la parentización. Las expresiones en notación prefija o postfija no necesitan paréntesis. Su aplicación es característica en las calculadoras, donde la memoria es un bien escaso.

En notación prefija, el operador precede a los operandos. Así, la suma de los operandos A y B se expresa como $+AB$. La expresión infija $(A+B) \times C$ se expresa como $\times +ABC$, mientras que $A+B \times C$ es $+A \times BC$.

En notación postfija, el operador sucede a los operandos. Así, la suma de los operandos A y B se expresa como $AB+$. La expresión infija $(A+B) \times C$ se expresa como $AB+C \times$, mientras que $A+B \times C$ es $ABC \times +$.

Obsérvese que ninguna de las dos notaciones, al contrario que la notación infija, requiere paréntesis, y que una no es la mera inversión de la otra.

Para convertir una expresión en notación infija a prefija o postfija, el TDA a utilizar será la pila, ya que se trata de nuevo de almacenar los elementos (operandos en prefija y operadores en postfija) y recuperar siempre el último almacenado y que no haya sido ya recuperado. Por ejemplo $A+B \times C$ se convertiría en prefija almacenando A , localizado el operador, éste se coloca en primer lugar en el TDA destino, se recupera el último operando almacenado (A) para colocarlo a continuación en el destino, seguidamente se almacena B , se encuentra el siguiente operador ' $\times$ ' que se escribe en el destino, se recupera el último operando almacenado (B), se escribe en el destino, se lee C , se almacena y como no hay más operandos ni operadores se recupera y se escribe en el destino, resultando finalmente: $+A \times BC$. Para convertir una expresión en notación infija a postfija el proceso es análogo salvo que son los operadores los que se almacenan en la pila.

El Programa 4.17 muestra el código en Modula2 para la conversión de expresiones de notación infija a prefija y postfija.

Programa 4.17 Código en Modula2 para la conversión de expresiones de notación infija a prefija y postfija.

```

MODULE Notacion;
FROM InOut IMPORT WriteString, Read, ReadInt, WriteLn, Write, WriteInt;
FROM pilacad IMPORT Tipo_pila, Tcadena, Inicia_pila, Meter_pila,
Sacar_pila, Pila_vacia, Consultar_pila, Imprimir_pila;
CONST
  maxlong = 30;
TYPE
  (* Tcadena = RECORD
      long : INTEGER;
      a : ARRAY[1..maxlong] OF CHAR
    END; *)

  Tipo_conj = SET OF CHAR;

VAR
  infija, post, pref : Tcadena;
  i, j, num : INTEGER;
  error : BOOLEAN;

PROCEDURE Concatenar(VAR Cad1 : Tcadena; Cad2 : Tcadena; VAR error : BOOLEAN);
(* Añade los caracteres de Cad2 al final de Cad1;
   error indica si la longitud de Cad1 excede el máximo long*)
VAR
  ultima, i : INTEGER;
BEGIN
  error := (Cad1.long + Cad2.long) > maxlong;
  IF NOT error THEN
    ultima := Cad1.long;
    FOR i := 1 TO Cad2.long DO
      ultima := ultima + 1;
      Cad1.a[ultima] := Cad2.a[i]
    END;
    Cad1.long := ultima;
  END;
  (* Si error Cad 1 se devuelve sin cambiar *)
END Concatenar;

PROCEDURE prefija (infija: Tcadena; VAR pref: Tcadena; VAR error: BOOLEAN);
VAR
  operando, operador : Tipo_pila;
  letra, simbolo : Tipo_conj;
  i : INTEGER;
  aux, ch_op1, ch_op2, ch_operador : Tcadena;
BEGIN

```

```

letra := Tipo_conj{'a'..'z'}; (* Representa operandos *)
simbolo := Tipo_conj{'+', '-', '*', '/'}; (* Operadores *)
(*Paso a prefija*)
Inicia_pila(operando);
Inicia_pila(operador);
i:=1;
WHILE (infija.a[i]#'.')&(infija.long <= maxlong)&(NOT error) DO
  (*Tratamiento de cada caracter leído*)
  IF infija.a[i] IN letra THEN
    (* Transformar el operando a Tcadena antes de meterlo en la pila *)
    aux.long := 1;
    aux.a[1] := infija.a[i];
    Meter_pila(operando,aux);
  ELSIF infija.a[i] IN simbolo THEN
    aux.long := 1;
    aux.a[1] := infija.a[i];
    Meter_pila(operador,aux);
  ELSIF infija.a[i] = '(' THEN
    error := infija.long>maxlong;
  ELSE (*caso de que sea un ) *)
    Sacar_pila(operando,ch_op1);
    Sacar_pila(operando,ch_op2);
    Sacar_pila(operador,ch_operador);
    Concatenar(ch_operador,ch_op2,error);
    Concatenar(ch_operador,ch_op1,error);
    aux.long := ch_operador.long;
    FOR j := 1 TO ch_operador.long DO
      aux.a[j] := ch_operador.a[j];
    END;
    Meter_pila(operando,aux);
  END;
  i := i+1;
  infija.long := infija.long+1;
  Read(infija.a[i]);
  Write(infija.a[i]);
END;

(* Si se considera expresion totalmente parentizada este IF sobra *)
IF operando.cima>1 THEN
  (* Quedaran 2 elementos en operando y 1 en operador *)
  Sacar_pila(operando,ch_op1);
  Sacar_pila(operando,ch_op2);
  Sacar_pila(operador,ch_operador);
  Concatenar(ch_operador,ch_op2,error);
  Concatenar(ch_operador,ch_op1,error);
  aux.long := ch_operador.long;

```

```

        FOR j := 1 TO ch_operador.long DO
            aux.a[j] := ch_operador.a[j];
        END;
        Meter_pila(operando,aux);
    END;
    Consultar_pila(operando,pref);
END prefija;

PROCEDURE postfija(infija : Tcadena; VAR post : Tcadena; error : BOOLEAN);
VAR
    operador : Tipo_pila;
    letra,simbolo : Tipo_conj;
    i : INTEGER;
    aux,oper : Tcadena;

BEGIN
    letra := Tipo_conj{'a'..'z'}; (*representa operandos*)
    simbolo := Tipo_conj{'+', '-', '*', '/'}; (*operadores*)
    Inicia_pila(operador);
    post.long:=0;
    i:=1;
    WHILE (infija.a[i]#'.') & (infija.long<=maxlong) & (NOT error) DO
        (* Tratamiento de cada caracter leído *)
        IF infija.a[i] IN letra THEN
            post.long := post.long+1;
            post.a[post.long] := infija.a[i];
        ELSIF infija.a[i] IN simbolo THEN
            aux.long := 1;
            aux.a[1] := infija.a[i];
            Meter_pila(operador,aux);
        ELSIF infija.a[i]='(' THEN
            error := infija.long>maxlong;
        ELSE (*caso de que sea un ) *)
            Sacar_pila(operador,oper);
            Concatenar(post,oper,error);
        END;
        i := i+1;
        infija.long := infija.long+1;
        Read(infija.a[i]);
        Write(infija.a[i]);
    END;

    WHILE NOT Pila_vacia(operador) DO
        Sacar_pila(operador,aux);
        Concatenar(post,aux,error);
    END;

```

```

END postfija;

BEGIN
  WriteString('Elija opcion: ');
  WriteString(' 0. Terminar');
  WriteString(' 1. Paso de infija a prefija');
  WriteString(' 2. Paso de infija a postfija');
  ReadInt(num);

  WHILE num<>0 DO
    i:=1;
    WriteString('Introduzca expresion infija con un punto para terminar:');
    WriteLn;
    Read(infija.a[i]);
    Write(infija.a[i]);
    infija.long := 1;
    error := FALSE;

    CASE num OF
      1: prefija(infija,pref,error); (* Paso de infija a prefija *)
        j := 1;
        WHILE (j<=pref.long) DO
          Write(pref.a[j]);
          j := j+1;
        END;
      2: postfija(infija,post,error); (* Paso de infija a postfija *)
        FOR j := 1 TO post.long DO
          Write(post.a[j]);
        END;
    END;
    WriteLn;
    WriteString('Elija opcion: ');
    WriteString(' 0. Terminar');
    WriteString(' 1. Paso de infija a prefija');
    WriteString(' 2. Paso de infija a postfija');
    ReadInt(num);
  END;
END Notacion.

```

En cuanto a la implementación, obsérvese que para el paso de infija a prefija se utilizan dos pilas, una de operadores y otra de operandos, siendo necesaria en el paso de infija a postfija el uso de una sola. Por otro lado, las pilas de operandos y de operadores utilizadas contienen elementos de tipo cadena, esto se ha de tener en cuenta a la hora de meter los datos en las pilas, pues han de ser del tipo de datos

cadena. Este tipo de datos es el que aparece en el programa entre comentarios y se importa desde el modulo `PilaCad`.

4.5.3 BÚSQUEDA DE INFINITIVOS EN UN TEXTO

Vamos a tratar aquí un ejemplo sencillo de un tipo de problema más general como es el tratamiento de una secuencia de caracteres en función de su terminación. Supongamos que deseamos imprimir en pantalla todas las palabras del texto de un fichero que representen verbos en infinitivo.

Es obvio que no podemos decidir si la palabra es un infinitivo hasta alcanzar el final de la misma, y una vez alcanzado y comprobado que efectivamente se trata de un infinitivo, necesitamos todos sus caracteres para imprimirlos, y además deberán ser impresos en el mismo orden en que llegaron. Para realizar esto, resulta ideal la utilización de una cola. Cada vez que comience una palabra, se inicializa la cola y vamos introduciendo en ella los sucesivos caracteres. Una vez que se ha llegado al final y se ha comprobado si realmente cumple la condición buscada (que termine en -ar, -er o -ir) se imprime la palabra extrayendo los caracteres de la cola. Si la condición no se cumple, la cola deberá ser vaciada para comenzar de nuevo mientras queden más caracteres en el fichero.

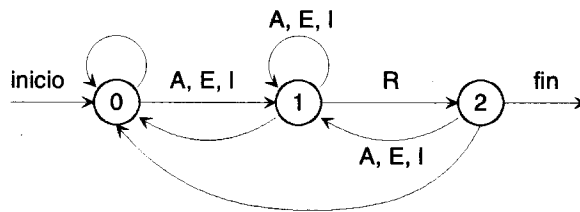


Figura 4.28 Esquema de la máquina de estados para detectar palabras acabadas en -ar, -er o -ir.

Para decidir si la palabra es un infinitivo se ha diseñado una máquina de estados que detecta la cadena de caracteres buscada. El esquema de la misma se representa en la Figura 4.28. El estado inicial se ha etiquetado como 0, es el estado en el que comienza, y permanece en él mientras no llegue un carácter 'A', 'E' o 'I'. Si llega uno de estos caracteres pasa al estado 1. Si en este estado llega una 'R' se pasa al estado 2 y si llega cualquier otro se retorna al estado 0. Si estando en el estado 2 llega un carácter 'A', 'E' o 'I' pasa al estado 1, si llega un carácter separador o de fin de palabra ('.', ',', ';', ':', '\n', etc.) se sale del bucle y si llega cualquier otro se retorna nuevamente al estado 0. Si después de terminar el bucle, la máquina está en el estado 2, es que se ha encontrado una palabra en infinitivo y se escribe por consola, extrayendo los elementos de la cola. Como es de suponer, aunque en la Figura 4.28 y en la descripción precedente de la transición de estados sólo se han incluido las letras mayúsculas, habrá que tener en cuenta también sus correspon-

```
        Sacar_cola(cola,dato);
        Write(dato);
    END;
    contador := contador+1;
    WriteLn;
    (* Si no es infinitivo se vacia la cola *)
    ELSE Inicia_cola(cola);END;
END;
Close(f);
WriteString('Se han encontrado ');
WriteInt(contador,1);
WriteString(' infinitivos');
WriteLn;
END infinitivo.
```

4.6 CONSIDERACIONES GENERALES

Antes de finalizar es conveniente realizar algunas consideraciones generales con el fin de ver con una perspectiva de conjunto los conceptos presentados en este tema y relacionarlos con otros ya conocidos.

1. A lo largo del tema se han ido analizando diversos TDA y diferentes algoritmos. En ocasiones, se han encontrado soluciones que no son adecuadas aunque 'funcionen'. Debemos insistir en este punto. No todo lo que funciona es la solución correcta. Recordemos el ejemplo de la implementación de la cola mediante una estructura dinámica. Una lista doblemente enlazada no es adecuada por su innecesario coste de almacenamiento y procesamiento (pero funciona), y una lista lineal con un único puntero externo tampoco es adecuada por su coste computacional (aunque también funciona).

La conclusión es clara: la lista enlazada con dos punteros externos (frente y final) es la solución adecuada. La idoneidad de una solución vendrá dada, como siempre, por los criterios de economía de almacenamiento y coste computacional.

2. En el Capítulo 1 ya se definió un TDA dinámico como aquel en el que el número de elementos que lo componen es variable. En este capítulo se han definido las pilas y las colas y se ha comentado su naturaleza dinámica. Posteriormente se ha estudiado su implementación estática y dinámica. Por otro lado, una definición alternativa (y también correcta) de una estructura dinámica es la siguiente: aquella estructura que puede aumentarse o reducirse en tiempo de ejecución. ¿Es esto contradictorio con el hecho de que una pila, por ejemplo, implementada mediante arreglos sea dinámica? La respuesta es claramente no. Los tipos de datos abstractos son estáticos o dinámicos dependiendo exclusivamente de su definición, nunca de su implementación. Así, en la implementación estática no debe confundirse la pila con el arreglo. El arreglo no crece ni

disminuye en tiempo de ejecución, pero la pila sí. Las componentes del arreglo que no han sido ocupadas introduciendo datos en la pila no forman parte de ésta. Tan dinámica es una pila implementada mediante arreglos como una implementada con una lista enlazada.

3. Las implementaciones estáticas tienen la ventaja de que son más rápidas. El manejo de arreglos utilizando el índice es más rápido que la realización del conjunto de manipulaciones de punteros que son necesarias en las estructuras basadas en variables dinámicas.
4. La clave fundamental para decidir entre las implementaciones estáticas y dinámicas de tipos de datos abstractos dinámicos radica en el hecho de si se conoce o no *a priori* un número máximo de datos que deberán ser almacenados. Si este número máximo es conocido, la implementación estática no es sólo posible sino que es más adecuada atendiendo a las consideraciones expuestas en el punto 3. Como ejemplo ilustrativo considérese el ejemplo de la evaluación de la parentización de una expresión en notación infija.

Al solucionar este ejercicio de programación se llegó a la conclusión de que el TDA adecuado es la pila. La siguiente cuestión a resolver era si debe ser estática o dinámica. En el primer caso (a) la especificación decía que las cadenas se leían de consola. Si las consolas tienen un buffer que acepta un número limitado de caracteres, éste puede ser conocido *a priori*. Por tanto, es posible una implementación estática y atendiendo a las consideraciones realizadas en el punto 3 es más adecuada. Sin embargo, en el segundo caso (b) las cadenas debían leerse desde un fichero. Entonces no podemos conocer *a priori* un número máximo de caracteres que tienen que almacenarse. Por tanto, debería utilizarse una implementación dinámica.

5. También debe tenerse en cuenta que la utilización de un TDA dinámico para resolver un problema en el que el número de elementos a almacenar es variable no es siempre adecuada. Supongamos, por ejemplo, que se necesita almacenar un apellido en una estructura de datos, y que las operaciones a realizar con él descartan la utilización de una pila o una cola. Tendremos que considerar un arreglo o una lista enlazada. Como los apellidos tienen diferente cantidad de caracteres podría pensarse que su almacenamiento en una lista enlazada es más adecuado que en un arreglo. Sin embargo, esto no es cierto. Es verdad que el número de elementos es variable, pero por otro lado sabemos que el número de caracteres es limitado (¿cuántos apellidos tienen más de quince caracteres?, ¿o más de veinticinco?). Si se utilizase una lista enlazada para cada apellido se ahorraría la memoria de las posiciones en las que no hay caracteres. Pero también debe tenerse en cuenta que se gastaría memoria para almacenar los punteros de los nodos (y un carácter ocupa menos que una dirección) y además su manejo es más lento como se indicó en 3. Por tanto, la solución con un arreglo es la adecuada.
6. Por último, es muy importante realizar una consideración acerca de la implementación de funciones para Vaciar un estructura dinámica. Supóngase que se necesita Vaciar una pila o una cola que contiene elementos. Como Vaciar no es

un operador básico hay que implementarla en función de los operadores básicos de la misma. En sentido estricto hay que Sacar todos los elementos hasta que la pila o la cola esté vacía. Si se piensa en las implementaciones con arreglos Vaciar puede implementarse mediante una simple llamada a Iniciar, ya que los demás antiguos elementos del arreglo serán 'basura' cuando se haya inicializado. Sin embargo, hacer esto con una implementación dinámica tiene resultados catastróficos puesto que la lista enlazada sobre la que se implementó la pila o la cola no ha sido liberada, ocupa memoria y no estará ya accesible para liberarla. Éste es un error muy grave. Entonces, ¿hay que considerar la implementación para realizar las funciones sobre los TDA? Es decir, por un lado tendremos que desarrollarlas en función de los operadores básicos y auxiliares definidos sobre ellas, pero por otro lado Vaciar sacando todos los elementos con una implementación estática tiene un coste computacional innecesario. La solución en sentido estricto es realizar la función sin atender a los detalles de implementación, aunque el coste computacional sea innecesario. Sin embargo, observado el hecho, una solución elegante es definir un nuevo TDA en el que se incorpore la operación Vaciar entre las básicas, llamándola por ejemplo *Pila con vaciado*, o *Cola con vaciado*. No debe nunca olvidarse que los TDA se definen de forma estricta pero siempre está permitido definir cualquier TDA que sea necesario. Lo importante es evitar ambigüedades.

4.7

CONCLUSIONES

En conclusión, no existen reglas ni para decidir de forma absoluta entre los TDA dinámicos o estáticos ni tampoco para concluir que una implementación dinámica es siempre mejor que una estática para las estructuras dinámicas. Sin embargo, sí se dispone de criterios claros para adoptar la solución adecuada al resolver un problema concreto. Estos criterios son la economía de almacenamiento y el coste computacional. Como sucedía en los algoritmos de clasificación se deben conocer todas las posibilidades, en este caso los TDA y sus posibles implementaciones, y al resolver unas especificaciones dadas elegir razonadamente las más adecuadas.

En este capítulo se han presentado los TDA dinámicos lineales fundamentales para la organización de datos, que mantienen una organización secuencial con un predecesor y un sucesor. En primer lugar se han introducido, como ejemplos de construcción de TDA dinámicos, algunos similares a las listas enlazadas. Seguidamente se han estudiado detalladamente los dos TDA dinámicos lineales más característicos: las pilas y las colas, desarrollando completamente sus implementaciones, tanto estáticas como dinámicas, y se han explorado las diversas posibilidades hasta llegar a una implementación determinada satisfactoria. Como es habitual, en ocasiones se han encontrado soluciones que no son adecuadas aunque 'funcionen'.

Debe observarse que se ha insistido en el hecho, bien conocido y detallado desde el comienzo del texto, de que el carácter estático o dinámico del TDA es independiente de su implementación. Las implementaciones estáticas tienen la ventaja

de que son más rápidas, mientras que las dinámicas permiten un mejor aprovechamiento de la memoria. La clave fundamental para decidir entre las implementaciones estáticas y dinámicas de tipos de datos abstractos dinámicos radica en el hecho de si se conoce o no *a priori* un número máximo de datos que deberán ser almacenados. Sin embargo, esto no significa que la utilización de un TDA dinámico para resolver un problema en el que el número de elementos a almacenar es variable no es siempre adecuada.

En conclusión, no existen reglas ni para decidir para todos los casos que un TDA es más adecuado ni para considerar únicamente una implementación determinada. El estudio de los conceptos desarrollados en este capítulo permite disponer de criterios claros para adoptar la solución adecuada al resolver un problema concreto.

CAPÍTULO 5

Tipos de datos abstractos dinámicos no lineales: árboles

5.1 INTRODUCCIÓN

En los Capítulos 1 y 4 se han descrito los tipos de datos dinámicos elementales, que se caracterizan por su definición abstracta en función de las operaciones que se pueden realizar sobre ellos independientemente de la implementación que se utilice. Entre ellos, las listas enlazadas permitían almacenar organizadamente datos del mismo tipo y acceder a cualquiera de ellos. Cuando la cantidad de información a almacenar y a tratar es elevada, el tiempo de acceso de las listas enlazadas es elevado ya que son TDA lineales y, por tanto, el tratamiento de la información es ineficaz.

En este capítulo se presenta un *Tipo de Datos Abstracto* más adecuado para el tratamiento de grandes cantidades de información: los *árboles*. Las aplicaciones de esta estructura son diversas. Así por ejemplo, se utilizan para el almacenamiento y búsqueda de información ya que pueden establecerse estructuras en las que el tiempo medio de las operaciones de búsqueda es del orden de $\log n$. Otros ámbitos de aplicación característicos son la implementación del sistema de archivos en los sistemas operativos, manejo de ficheros en la realización de bases de datos, etc.

En primer lugar, se introducen las definiciones básicas y la terminología y posteriormente se estudian con detalle varios tipos fundamentales de árboles como son los árboles perfectamente balanceados, los árboles de búsqueda y los árboles AVL que son una solución de compromiso entre los dos tipos anteriores.

5.2

CONCEPTOS Y DEFINICIONES

Los árboles se basan en el concepto de *nodo*. En el capítulo 1 ya se presentaron diversos TDA que hacían uso de este concepto. Recuérdese que fue introducido como un tipo sobre el que se basan las listas enlazadas y, en principio, constaba de un campo de datos y un campo de enlace. Para la definición de las listas enlazadas el campo enlace estaba constituido por un único puntero que se utilizaba para apuntar al nodo sucesor. Sin embargo para otros TDA dinámicos, como las listas doblemente enlazadas, los nodos tenían dos punteros, uno para apuntar al nodo sucesor y otro para apuntar al nodo predecesor. En general, los nodos pueden definirse de la siguiente forma:

Definición 5.1: Se denomina *nodo* a cualquier tipo cuyos elementos son registros formados por un campo *Datos* y un número dado de *apuntadores o enlaces*.

Los elementos de este tipo se ilustran en la Figura 5.1. Por economía de lenguaje una vez definido un TDA con tipo nodo, en el que se especifican tanto el número de punteros del campo enlace como su uso, los elementos de este TDA se denominan simplemente nodos. Así se suele denominar nodo a los utilizados tanto en una lista enlazada como en una doblemente enlazada, o a cualquiera que se ajuste a la Definición 5.1.

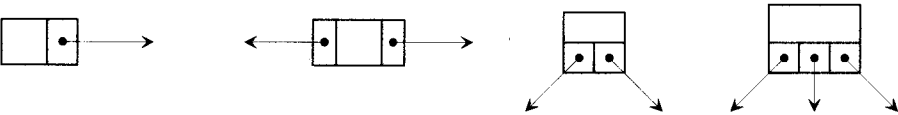


Figura 5.1 Concepto de nodo. Se muestran, por este orden, los nodos correspondientes a una lista enlazada, una lista doblemente enlazada, un árbol binario y un árbol ternario.

Haciendo uso del tipo nodo, los árboles se definen formalmente de la siguiente manera:

Definición 5.2: El TDA *árbol* de grado n está formado por nodos con n apuntadores, cada uno de ellos es apuntado por un único nodo (salvo uno) y apunta a un árbol de grado n (denominado subárbol). Se dice que el árbol está vacío si no contiene ningún nodo.

Debe observarse que atendiendo a esta definición los árboles son tipos de datos abstractos claramente dinámicos. Como es habitual no se atiende a la implementación del tipo.

Debe prestarse especial atención al hecho de que la definición del TDA árbol se realiza en términos de recursión. Ésta es una característica sobresaliente de los árboles. Son un ejemplo característico y paradigmático en el que la recursión se utiliza de forma elegante para definir un Tipo de Datos Abstracto.

Existen numerosas maneras equivalentes de representar los árboles. En la Figura 5.2 se muestran dos de ellas. La representación gráfica es sin lugar a dudas la más utilizada y la que mejor pone de manifiesto las relaciones entre los nodos. La representación de paréntesis anidados ofrece por su parte la ventaja de ser más compacta.

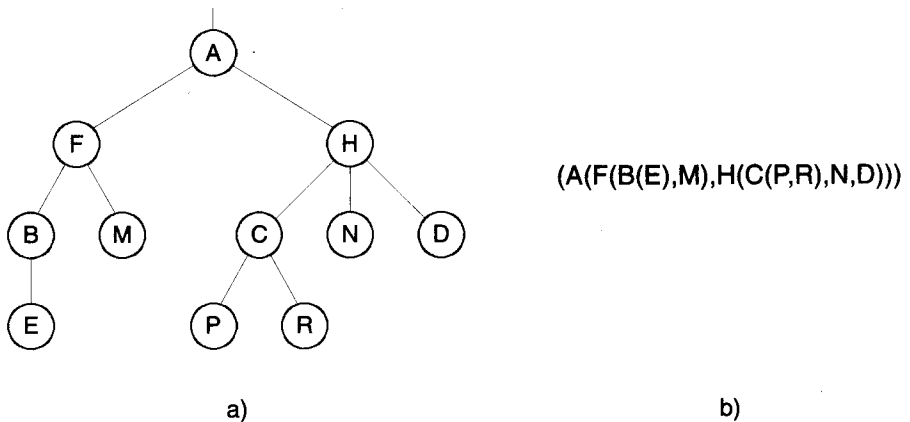


Figura 5.2 Representación de una estructura de árbol. a) Gráfica. b) Paréntesis anidados.

Teniendo en cuenta la Definición 5.2, las listas enlazadas pueden considerarse un caso particular del TDA árbol, en las que cada nodo tiene como máximo un subárbol. Por este motivo las listas enlazadas suelen denominarse árboles degenerados.

En el estudio de los árboles se hace uso de la siguiente terminología, que se ilustra en la Figura 5.3.

Definición 5.3: Se dice que un nodo y es *descendiente directo* (o hijo) de un nodo x si y es apuntado por x , y es *ascendiente directo* (padre) si y apunta a x . La *raíz* del árbol se define como el nodo que no tiene ascendiente. Si un elemento no tiene descendientes se le denomina *hoja* o *nodo terminal*. Un elemento que no es terminal es un nodo *interior*. El enlace entre dos nodos se conoce como *arista* o *rama*.

Definición 5.4: Se denomina *grado* del nodo al número de descendientes directos del nodo. El grado del árbol, como se establece en la Definición 5.2, es el mayor de los grados de los nodos.

Según el grado, los árboles se denominan *binarios* (como máximo dos hijos), *ternarios* (como máximo tres hijos), y así sucesivamente.

Los árboles binarios son de especial interés y serán tratados detalladamente en este capítulo. Cada nodo tiene dos descendientes, denominados *subárbol izquierdo* y *subárbol derecho*. La definición de tipo, en Modula2, para un árbol binario es la siguiente:

```

TYPE Ptr_Nodo = POINTER TO Nodo;
TYPE Nodo = RECORD
  Dato : Tipo_datos;
  izq, dch : Ptr_Nodo;
END

```

en la que el árbol vacío se referencia por NIL.

El costo necesario para acceder a un elemento almacenado en un nodo de un árbol depende de varios factores. Uno de ellos es su cercanía o lejanía a la raíz del mismo. Así, por ejemplo, si en el árbol de la Figura 5.2.a se accede sólo por el subárbol izquierdo del nodo raíz, siempre costará más acceder al nodo de valor E que al nodo de valor F. Las siguientes definiciones son fundamentales para realizar los razonamientos en este sentido.

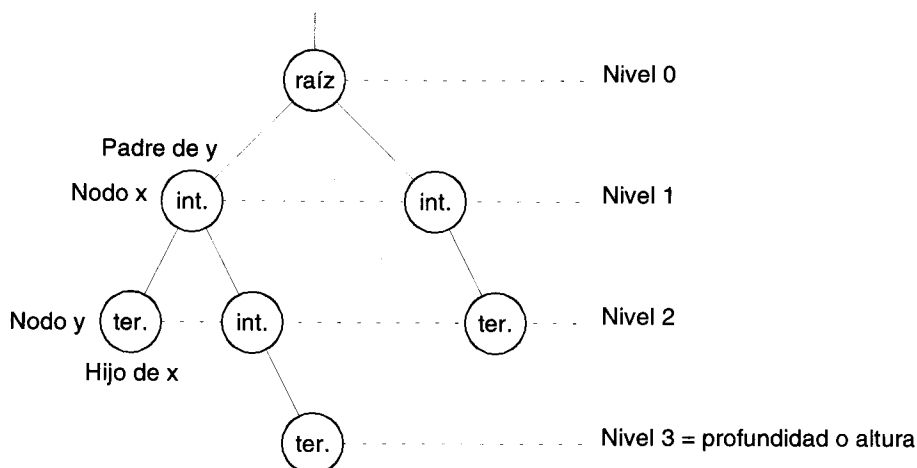


Figura 5.3 Terminología de árboles. 'int.' indica un nodo interior y 'ter.' un nodo terminal u hoja.

Definición 5.5: Se denomina *nivel* de un nodo al número de descendientes que deben recorrerse desde la raíz hasta el nodo. El nivel de la raíz es cero. El nivel máximo en el que se encuentran los elementos de un árbol se denomina *profundidad* o *altura*.

Definición 5.6: La *longitud de trayectoria interna* de un nodo, es el número de aristas o ramas que se recorren desde la raíz al nodo.

La *longitud de trayectoria* de un árbol o *longitud de trayectoria interna*, L_p , es la suma de todas las longitudes de trayectoria de sus nodos.

La *longitud de trayectoria media* es:

$$L_I^m = \frac{\sum_{i=1}^n n_i \cdot i}{n} \quad (5.1)$$

donde n_i es el número de nodos en el nivel i .

En principio, un nodo de nivel i tiene una longitud de trayectoria i . Sin embargo, es posible definir árboles en los que esto no sucede, como se detallará al estudiar los *árboles B*.

La longitud de trayectoria interna del árbol binario de la Figura 5.3 es $2 \times 1 + 3 \times 2 + 1 \times 3 = 11$ y la del de la Figura 5.2 es 21.

Como puede observarse la longitud de trayectoria interna de un nodo contabiliza el recorrido necesario para llegar hasta él. Los siguientes términos están relacionados con el recorrido necesario al dirigirse a un nodo que no está en el árbol.

Definición 5.7: Dado un árbol, su *árbol extendido* es el árbol ampliado con nodos especiales tal que todos los subárboles son completos, de manera que todos los apuntadores sin nodos descendientes apuntan a un nodo especial. Los nodos especiales no tienen descendientes.

La Figura 5.4 muestra el árbol extendido asociado al árbol de la Figura 5.3.

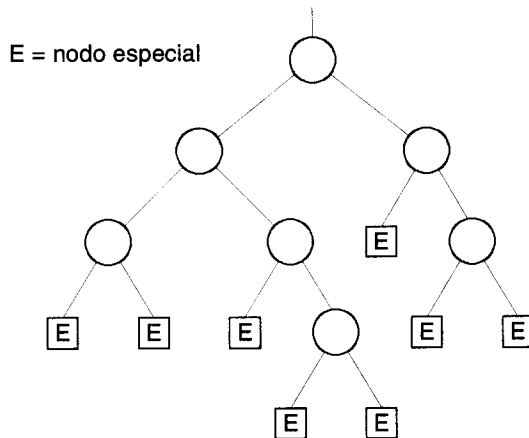


Figura 5.4 Árbol binario extendido asociado al árbol de la Figura 5.3.

Definición 5.8: Se define la *longitud de trayectoria externa* de un árbol, L_E , como la suma de las longitudes de trayectoria de todos los nodos especiales.

La *longitud de trayectoria externa media* es:

$$L_E^m = \frac{\sum_{i=1}^m m_i \cdot i}{m} \quad (5.2)$$

donde m_i es el número de nodos especiales en el nivel i .

La longitud de trayectoria externa del árbol de la Figura 5.3 es:

$$0 \times 1 + 1 \times 2 + 5 \times 3 + 2 \times 4 = 25$$

Las siguientes propiedades son de especial interés para el cálculo de los costes computacionales en los árboles.

Propiedad 5.1. Sean g y n el grado y el número de nodos del árbol respectivamente. Entonces el número de nodos especiales, m , que debe añadirse para calcular la longitud de trayectoria externa viene dado por:

$$m = (g - 1)n + 1 \quad (5.3)$$

Demostración:

La demostración de esta propiedad se basa en el cálculo del número de aristas del árbol extendido. Dado que cada nodo será apuntado por un enlace, el número de aristas del árbol extendido en función del número de nodos es $m + n$.

Por otro lado, el número de aristas puede también expresarse en función del grado del árbol. Cada nodo es apuntado por una y sólo una arista, y cada uno apunta a g nodos, salvo los nodos especiales que no apuntan a ninguno. Por tanto, las aristas del árbol extendido serán todas las que existen suponiendo que todos los punteros de los nodos apuntan a un nodo, y el número de aristas del árbol extendido será $g \times n + 1$, donde el '+1' se debe a la arista que apunta a la raíz y que no proviene de ningún nodo. Entonces se satisface que $m + n = g \times n + 1$, de donde se obtiene $m = (g - 1)n + 1$.

Esta propiedad indica que el número de nodos especiales que debe añadirse para calcular la longitud de trayectoria externa sólo depende del número de nodos originales y del grado del árbol.

Propiedad 5.2. Dado un árbol de grado g y altura h el número máximo de nodos N_g viene dado por:

$$N_g = \sum_{i=0}^{h-1} g^i \quad (5.4)$$

Demostración:

Dado un árbol de grado g y altura h , el número máximo de nodos se obtendrá cuando todos los punteros, excepto los pertenecientes a los nodos del nivel h , refieren a un nodo. Así, en el nivel cero hay un nodo, en el uno los g descendientes de la raíz, en el nivel dos cada uno de los g nodos del nivel uno habrá producido g descendientes, es decir, g^2 descendientes, para el tercer nivel g^3 , etc. Por tanto, en total se tiene la suma :

$$N_g = \sum_{i=0}^{h-1} g^i$$

Por ejemplo, para los árboles binarios el número máximo de nodos es:

$$N_2 = \sum_{i=0}^{h-1} 2^i = 2^h - 1$$

Propiedad 5.3. En los árboles binarios, la longitud de trayectoria de un árbol o longitud de trayectoria interna, L_I , y la longitud de trayectoria externa, L_E , están relacionadas por la siguiente expresión:

$$L_E = 2n + L_I$$

Demostración:

La longitud de trayectoria interna, L_I , es la suma de todas las longitudes de trayectoria de sus nodos y la longitud de trayectoria externa, L_E , es la suma de las longitudes de trayectoria de todos los nodos especiales. En los árboles binarios un nodo de nivel i tiene una longitud de trayectoria i , y por tanto:

$$L_I = \sum_{i=1}^n n_i \cdot i \quad \text{y} \quad L_E = \sum_{i=1}^m m_i \cdot i$$

donde n_i es el número de nodos en el nivel i y m_i es el número de nodos especiales en el nivel i .

Supóngase que se elimina una hoja o nodo terminal, es decir un nodo tal que sus dos descendientes son nodos especiales, en el nivel k . El cambio producido en L_I es $-k \times 1$. Como es una hoja, L_E disminuye en $2(k+1)$ ya que el número de nodos especiales es dos y estaban en el nivel $k+1$. Pero además L_E aumenta en k ya que el nodo eliminado se convierte en terminal en el nivel k . De este modo, el cambio neto en L_E es $k - 2(k+1) = -k - 2$. Si se elimina otro nodo terminal, en un nivel l , entonces:

$$L_E = L_E(\text{anterior}) - k - 2 - l - 2 = L_E(\text{anterior}) - k - l - 2 \times 2 \text{ y}$$

$$L_I = L_I(\text{anterior}) - k - l.$$

Por tanto, para n niveles se observa que, por inducción: $L_E = L_I + 2n$.

5.2.1 ÁRBOLES PERFECTAMENTE BALANCEADOS

Como ya se indicó anteriormente si sólo se utiliza uno de los subárboles, el árbol degenera en una lista enlazada. Es deseable que el árbol esté '*equilibrado*', es decir, que se utilicen de forma similar los árboles izquierdo y derecho de modo que su longitud de trayectoria media sea la menor posible. De otro modo, el posterior acceso a sus elementos es ineficaz, al tener que recorrer más nodos. Los árboles *perfectamente balanceados* son un ejemplo ilustrativo de esta característica. Se definen de la siguiente forma:

Definición 5.9: Un árbol *perfectamente balanceado* es aquel en el que para cada nodo el número de nodos en sus subárboles derecho e izquierdo difieren como máximo en uno.

Estos árboles se caracterizan por tener altura mínima. Para conseguir este objetivo será necesario asignar el número máximo de nodos en todos los niveles salvo en el mayor. La Figura 5.5 muestra un ejemplo de árbol perfectamente balanceado.

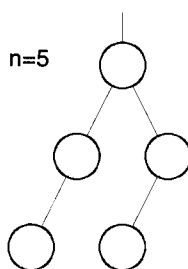


Figura 5.5 Árbol binario perfectamente balanceado con cinco nodos.

Supóngase que se conoce el número n de elementos a almacenar, uno en cada nodo, y quiere construirse un árbol binario perfectamente balanceado. Para satisfacer la definición es suficiente que para cada nodo los elementos a almacenar se distribuyan de igual forma entre su subárbol derecho e izquierdo. Dado que esta regla de igual distribución debe satisfacerse *en cada nodo*, la estrategia es evidentemente recursiva. El procedimiento del Programa 5.1 construye un árbol binario perfectamente balanceado. El algoritmo es sencillo y se describe a continuación:

Para cada nodo se construye un subárbol izquierdo perfectamente balanceado de $n_{izq} = n \text{ DIV } 2$ nodos y otro derecho de $n_{dch} = n - n_{izq} - 1$ nodos. Debe observarse que cuando se dispone de un elemento a introducir, se crea un nodo y se introduce su valor en él, y seguidamente se llama al procedimiento recursivamente para crear

el subárbol izquierdo. No se producen los enlaces hasta que finaliza la recursión. Entonces, en el último nivel de recursión se devuelve el puntero del `Nuevo_nodo` y es asignado al puntero `izq` del nivel anterior. Realizada esta asignación se ejecuta recursivamente el procedimiento para crear el subárbol derecho del nodo. Finalizado este procedimiento, que puede constar de nuevo de varias llamadas recursivas, el `Nuevo_nodo` del nivel inferior se asigna al puntero `dch` del nivel actual. Seguidamente se devuelve el `Nuevo_nodo` del nivel actual a la llamada en el nivel anterior.

Programa 5.1 Procedimiento para construir un árbol binario perfectamente balanceado, conocido previamente el número de nodos a insertar [Wirth, 1987].

```

PROCEDURE Arbol_perf_bal(n:INTEGER):Ptr_Nodo;
VAR
  Nuevo_nodo: Ptr_Nodo;
  n_izq, n_dch, valor: INTEGER;
BEGIN
  IF n=0 THEN Nuevo_nodo := NIL
  ELSE n_izq := n DIV 2;
      n_dch := n-n_izq-1;
      ReadInt(valor);
      ALLOCATE(Nuevo_nodo,SIZE(Nodo));
      WITH Nuevo_nodo^ DO
        Dato := valor;
        izq := Arbol_perf_bal(n_izq);
        dch := Arbol_perf_bal(n_dch);
      END;
  END;
  RETURN Nuevo_nodo
END Arbol_perf_bal;

```

La Figura 5.6 muestra el árbol perfectamente balanceado de 17 nodos con los siguientes datos:

13, 22, 18, 15, 9, 2, 17, 32, 16, 4, 14, 12, 69, 23, 99, 42, 10.

Evidentemente el Programa 5.1 tiene valor meramente académico ya que requiere conocer previamente el número de nodos a introducir y esto no tiene sentido para los TDA dinámicos. El principal interés del Programa 5.1 está en la forma en que se asignan los punteros. Es conveniente que el lector lo analice detenidamente como ejercicio de programación. El Programa 5.2 muestra una implementación en Modula2 para construir un árbol perfectamente balanceado sin necesidad de conocer previamente el número de nodos, y la Figura 5.7 muestra el árbol resultante para la misma secuencia de la Figura 5.6. Supóngase que se toma como criterio insertar primero por la izquierda. Los primeros pasos son sencillos: se inserta el 13 como raíz, el 22 como hijo izquierdo y el 18 como hijo derecho. Seguidamente, el 15 se inserta como hijo izquierdo de 22. El 9 deberá insertarse necesariamente como

hijo del 18. Si se insertase como hijo del 22 el nodo con valor 13 tendría el subárbol izquierdo con tres nodos y el derecho con uno, con lo que no cumpliría la definición de árbol perfectamente balanceado. Y será el hijo izquierdo por el criterio inicial de insertar primero por la izquierda. El 2 y el 17 se insertarán como hijo derecho del 22 y del 18 respectivamente. Seguidamente el 32 será hijo izquierdo del 15 y el 16 del 9, ya que si se colocase en el subárbol izquierdo del raíz, de nuevo se infringiría la definición de árbol perfectamente balanceado. Aplicando los mismos razonamientos se insertan los demás valores sucesivamente.

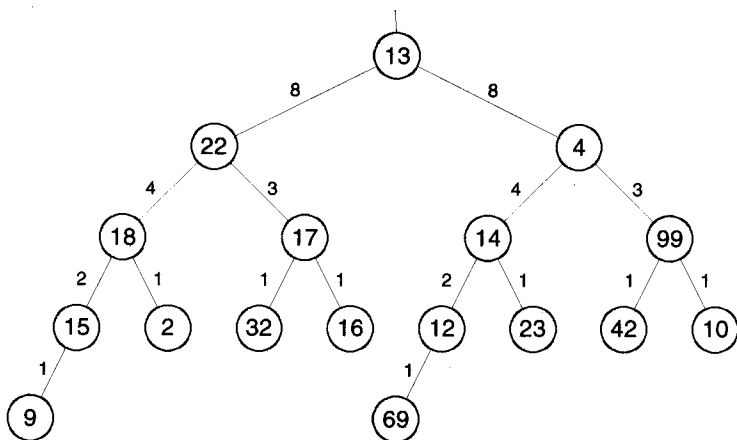


Figura 5.6 Construcción de un árbol binario perfectamente balanceado conociendo previamente el número de nodos. Los números junto a cada rama indican el número de nodos de esa rama.

Programa 5.2 Procedimiento para construir un árbol binario perfectamente balanceado sin conocer previamente el número de nodos a insertar [Wirth, 1987].

```

PROCEDURE Perfec_balanceado(f:File; VAR k:Ptr_Nodo);
  (* f: fichero con los datos a almacenar en el árbol *)
  VAR w: INTEGER;
  PROCEDURE Insertar_Arbol_perf(w: INTEGER;VAR k:Ptr_Nodo);
    (*w:elemento a introducir en el árbol;k:raíz del árbol*)
    VAR
      numdch,numizq : INTEGER;
  PROCEDURE NumNodos(a: Ptr_Nodo):INTEGER;
    (*Devuelve el número de nodos del arbol apuntado por a*)
  BEGIN
    IF a=NIL THEN
      RETURN 0
    ELSE
      RETURN 1+NumNodos(a^.izq)+NumNodos(a^.dch);
  
```

```

        END
    END NumNodos;
BEGIN
    (*lectura hasta que termine la entrada de datos*)
    IF k#NIL THEN
        numdch:=NumNodos(k^.dch);
        numizq:=NumNodos(k^.izq);
    END;
    IF k=NIL THEN      (*insertar*)
        ALLOCATE(k,SIZE(Nodo));
        k^.Dato:=w;
        k^.dch:=NIL;
        k^.izq:=NIL;
    ELSIF numdch=numizq THEN
        Insertar_Arbol_perf(w,k^.izq)
    ELSIF numizq>numdch THEN
        Insertar_Arbol_perf(w,k^.dch)
    END;
    END Insertar_Arbol_perf;
BEGIN
    Reset(f);
    WriteLn;
    ReadWord(f,w);
    WHILE ~f.eof DO
        Insertar_Arbol_perf(w,k);
        ReadWord(f,w);
    END;
    Close(f);
END Perfec_balanceado;

```

Como puede observarse en este ejemplo, la utilización de procedimientos recursivos resulta muy adecuada para la construcción y manejo de árboles. Esto no puede resultar sorprendente teniendo en cuenta que la definición del TDA árbol se realiza en términos de recursión.

Los árboles de las Figuras 5.6 y 5.7 son de altura mínima y almacenan 17 elementos, pero los datos han sido almacenados meramente según su orden de aparición. Rara vez es útil almacenar los datos de este modo. Generalmente los datos se van a utilizar para realizar determinadas tareas con ellos. Estas tareas establecerán cómo ha de tratarse la información almacenada. Lógicamente, tanto los TDA como los algoritmos deben realizarse con el fin de que el tratamiento de la información se haga de forma eficaz. Así por ejemplo, los árboles de las Figuras 5.6 y 5.7 son de altura mínima, y en este sentido son eficaces. Pero supongamos que los enteros almacenados representan el número del DNI de los alumnos y en el árbol se encuentran los que se han presentado a examen. Para determinar si un alumno se ha presen-

tado a examen es necesario, con este árbol, consultar todos los nodos, es decir, recorrer el árbol por completo. Esto es claramente ineficaz. El hecho de que sea de altura mínima no es aprovechado en absoluto. Por tanto, será necesario, además, establecer un método de acceso a los elementos del árbol para que la tarea a desarrollar se realice eficazmente. Este problema se aborda en las siguientes secciones.

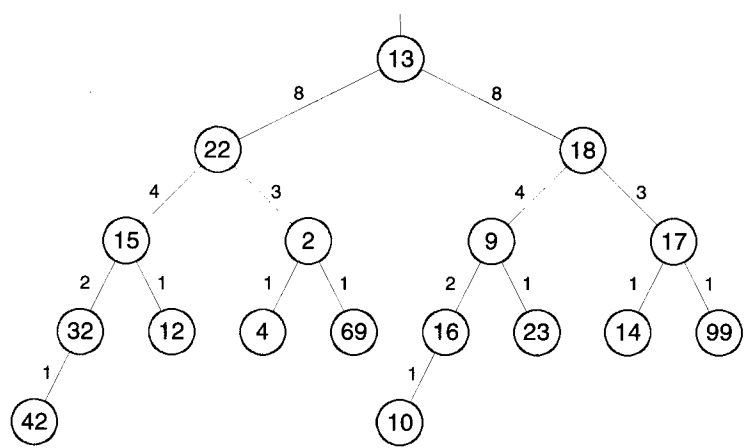


Figura 5.7 Construcción de un árbol binario perfectamente balanceado sin conocer previamente el número de nodos. Nuevamente, los números junto a las ramas indican el número de nodos de cada una y muestran la característica de árbol perfectamente balanceado.

5.3

ÁRBOLES BINARIOS ORDENADOS SEGÚN EL RECORRIDO

Es obvio que todo árbol tendrá que ser recorrido mediante un criterio dado. Por ejemplo, podría recorrerse el árbol tal que para cada nodo primero se visita el nodo, seguidamente el subárbol izquierdo y por último el derecho. O bien, primero el subárbol izquierdo, luego el derecho y por último el nodo. Este método de organización de los árboles se denomina *árboles ordenados según el recorrido*. La siguiente definición precisa formalmente este concepto.

Definición 5.10: Un *árbol binario ordenado según el recorrido* es aquél que para cada nodo se visita el nodo, su subárbol izquierdo y su subárbol derecho en un orden establecido.

Atendiendo a la naturaleza de los árboles binarios surgen tres ordenamientos de forma inmediata, que se expresan de forma recursiva en función del orden en que se visitan el nodo N y sus descendientes izquierdo I y derecho D .

1. *Preorden*: visitar el nodo antes que los subárboles.
2. *En orden*: visitar el nodo después de un subárbol y antes que el otro.
3. *Postorden*: visitar el nodo después de los subárboles.

Como puede observarse estos órdenes se establecen en función del orden de visita del nodo, y para cada uno de ellos pueden darse dos posibilidades dependiendo del orden en que se visitan los subárboles entre sí. Así, los órdenes N, I, D y N, D, I son preorden, I, N, D y D, N, I en orden y por último I, D, N y D, I, N son postorden.

Los tres métodos de recorrido se realizan de forma inmediata mediante procedimientos recursivos. El Programa 5.3 muestra estos procedimientos en los que el procedimiento implícito P representa la acción a realizar en cada nodo y el parámetro A simboliza el árbol en el que se realizará.

Programa 5.3 Ejemplo de procedimientos generales de los tres métodos de recorrido.

```
PROCEDURE Preorden(A:Ptr_nodo);
BEGIN
  IF A # NIL THEN
    P(A); Preorden(A^.izq); Preorden(A^.dch)
  END
END Preorden;

PROCEDURE Enorden(A:Ptr_nodo);
BEGIN
  IF A # NIL THEN
    Enorden(A^.izq); P(A); Enorden(A^.dch)
  END
END Enorden;

PROCEDURE Postorden(A:Ptr_nodo);
BEGIN
  IF A # NIL THEN
    Postorden(A^.izq); Postorden(A^.dch); P(A)
  END
END Postorden;
```

5.3.1 ÁRBOLES DE EXPRESIÓN

Esta forma de organizar los árboles tiene aplicación inmediata para la representación de expresiones. Como se indicó en el Capítulo 4 una tarea muy común es la de realizar un conjunto de acciones o procedimientos P con datos dados. Entonces se introdujeron las notaciones infija, prefija y postfija, y se analizó su tratamiento haciendo uso de TDA dinámicos lineales, especialmente mediante pilas. Debe recordarse que la notación infija es la que habitualmente se utiliza en matemáticas.

Las reglas de precedencia, en este tipo de notación, se establecen con la ayuda de paréntesis. Las notaciones prefija y postfija permiten escribir una expresión sin

usar paréntesis para establecer las precedencias. En la notación prefija el operador precede a los operandos y en la postfija los sucede. Atendiendo a sus características se observa que si la expresión se considera como un único proceso secuencial, entonces los nodos individuales deberán visitarse en un orden específico de acuerdo con la secuencia de acciones a realizar. De esta manera, mediante árboles binarios ordenados según el recorrido, se pueden tratar expresiones diádicas. Para ello los nodos contienen operadores y estos actúan sobre los operandos, que se almacenan en los hijos del nodo. En este caso particular estos árboles se denominan *árboles de expresión*.

Así por ejemplo la expresión aritmética con notación infija:

$$((A + ((B / C) - D)) \times (E \times (F + G)))$$

se expresa en la notación prefija como:

$$\times + A - / B C D \times E + F G$$

y en la postfija es:

$$A B C / D - + E F G \times \times$$

tal y como se introdujo en el capítulo anterior.

La Figura 5.8 muestra el árbol de expresión correspondiente. Como puede observarse, las notaciones prefija, infija y postfija corresponden a los recorridos en preorden, orden y postorden. Con una estructura de este tipo resulta trivial la conversión entre un tipo de notación y otro, pues basta con crear el árbol de expresión correspondiente y después se obtendrá una u otra notación en función del recorrido que realicemos sobre él.

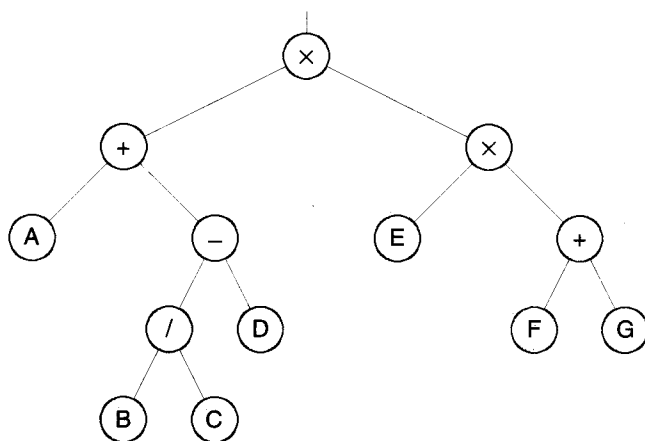


Figura 5.8 Árbol de expresión. Cada nodo interno almacena un operador y los nodos terminales almacenan los correspondientes operandos.

5.4 ÁRBOLES DE BÚSQUEDA BINARIOS

5.4.1 CONCEPTO Y DEFINICIONES

Como se ha detallado en los capítulos anteriores, una de las tareas más frecuentes es la de la clasificación de la información. Por este motivo cabe esperar que la definición y el estudio de TDA árboles adecuados para esta función sean de especial interés.

Evidentemente la organización de los árboles según recorrido sólo es útil en los casos en que se trate de realizar secuencialmente un conjunto de acciones o procedimientos P con datos dados. Sin embargo, no son de utilidad para almacenar información y recuperarla posteriormente según un campo llave. Supóngase que se desea localizar el nodo con valor 42 en el árbol binario de la Figura 5.9.

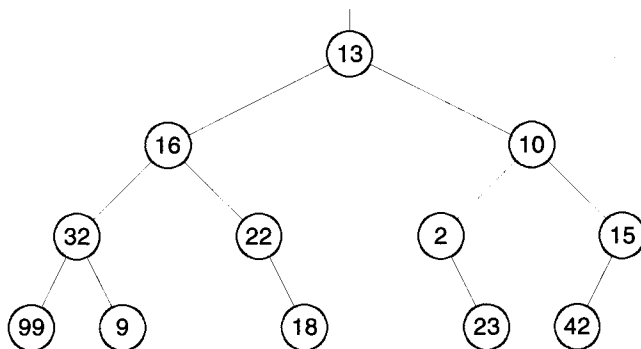


Figura 5.9 Árbol binario.

En este caso, si se recorre en preorden (N, I, D) entonces hay que recorrer todos los nodos del árbol antes de llegar a encontrar el nodo de valor 42. En los árboles ordenados según el recorrido se está tratando el árbol secuencialmente, del mismo modo que en las listas enlazadas. En conclusión, ni la simple definición del TDA árbol ni su organización según recorrido establecen mejora alguna sobre los TDA dinámicos lineales, incluyendo los casos de árboles no degenerados.

Por tanto, es clara la necesidad de establecer algún *criterio de organización* de los datos en el TDA, es decir, *ordenar* los datos de alguna manera. Evidentemente esta organización depende del modo en que vayan a utilizarse los datos. En esta sección se muestran los métodos de organización fundamentales de los árboles binarios.

Evidentemente cuando un conjunto de datos es ordenable lo será según un criterio dado sobre una característica determinada. Así por ejemplo, los datos de los alumnos de informática podrán clasificarse por orden alfabético del primer apellido, por número del DNI, etc. Debe observarse que la clasificación es posible debido al

hecho de que entre los campos llave de cualquier par de datos se puede establecer una relación lógica formada por *condiciones mutuamente excluyentes*, tal que siempre una de ellas es cierta. Por ejemplo, todo alumno tiene un DNI mayor o menor que otro. Los árboles se muestran especialmente adecuados para incorporar esta característica. Cuando las condiciones mutuamente excluyentes son dos, los árboles binarios se pueden organizar de modo que para cada nodo todas las llaves de su subárbol izquierdo satisfagan una condición, y todas las de su subárbol derecho la otra. Un árbol binario con esta organización se denomina *árbol binario de búsqueda*.

Un árbol de búsqueda puede definirse formalmente de la siguiente manera:

Definición 5.11: Un *árbol de búsqueda* es un TDA árbol en el que para cada nodo todas las llaves de cada subárbol satisfacen una y sólo una condición de un conjunto de n_c condiciones mutuamente excluyentes.

Evidentemente, si $n_c = 2$ los árboles binarios son los adecuados, si $n_c = 3$ lo son los ternarios, etc. La particularización de esta definición para los árboles binarios es de especial interés.

Definición 5.12: Un *árbol binario de búsqueda*, es un árbol binario en el que dadas dos condiciones mutuamente excluyentes, para cada nodo, todas las llaves de su subárbol izquierdo satisfacen una condición y todas las de su subárbol derecho la otra.

Dado que los árboles binarios de búsqueda son los más frecuentemente utilizados, por economía de lenguaje se suelen denominar sencillamente árboles de búsqueda.

Sea por ejemplo un conjunto de datos en el que el campo llave son números enteros, y no puede haber dos campos llave iguales. Dados dos enteros distintos cualesquiera, a y b , siempre existen las dos relaciones mutuamente excluyentes $a > b$ y $a < b$, de manera que una de ellas siempre es cierta. Entonces, el árbol puede organizarse de manera que para cada nodo las llaves del subárbol izquierdo serán menores que la suya y las del subárbol derecho mayores. La Figura 5.10 muestra el árbol binario de búsqueda correspondiente a la secuencia de datos:

13, 22, 18, 15, 9, 2, 17, 32, 16, 4, 14, 12, 69, 23, 99, 42, 10.

Como puede observarse, este TDA permite realizar una búsqueda dirigida para localizar un elemento en el árbol, de modo que según la llave de cada nodo se determina si debe buscarse en el subárbol izquierdo o en el derecho. De este modo, no se tiene que realizar todo el recorrido del árbol para encontrar el dato.

Seguidamente se presentan las operaciones fundamentales sobre los árboles binarios de búsqueda.

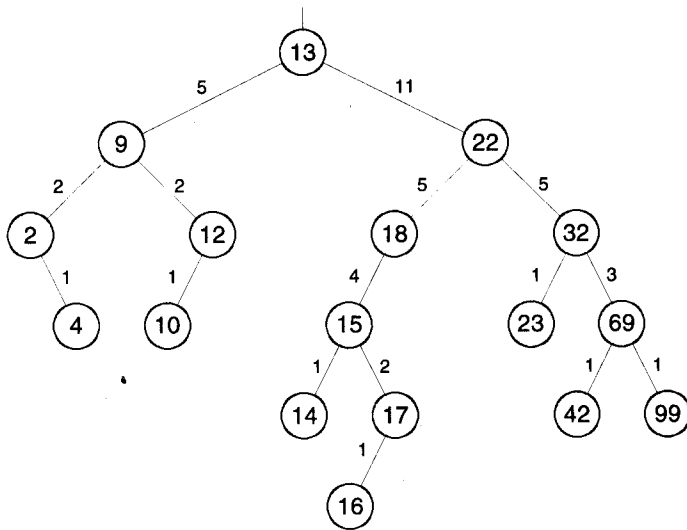


Figura 5.10 Árbol binario de búsqueda.

5.4.2 PROCEDIMIENTOS SOBRE ÁRBOLES BINARIOS DE BÚSQUEDA

Atendiendo a la definición de árbol de búsqueda se observa que ésta se realiza de forma recursiva, como corresponde a la naturaleza recursiva de los árboles. Por tanto, los procedimientos sobre árboles de búsqueda serán recursivos. Para mostrar cómo se realizan los procedimientos, se supone que ninguna de las llaves de los elementos a introducir se repite, y por tanto, las condiciones mutuamente excluyentes pueden expresarse mediante los operadores lógicos menor (<) y mayor (>).

La declaración del TDA árbol binario de búsqueda, en Modula2, es:

```

TYPE Ptr_Nodo = POINTER TO Nodo;
TYPE Nodo = RECORD
  llave : Tipo_llave;
  izq, dch : Ptr_Nodo;
END
  
```

y el árbol de búsqueda vacío se representa por NIL.

Obsérvese que en la declaración de los nodos se utiliza como campo de datos el identificador llave mientras que en otros casos se ha utilizado el identificador Datos. Con ello se pretende simplemente resaltar el contenido semántico de sus propiedades indicando con llave que es posible incorporar un criterio de clasificación. Por otro lado es obvio que en un caso dado se añadirán los campos necesarios para almacenar la información requerida en función de los datos específicos.

El primer procedimiento a considerar es la *inserción* de un elemento en el árbol de búsqueda. Para cada nodo se consulta si el dato es menor o mayor que la llave, decidiendo así si se prosigue la búsqueda por la izquierda o por la derecha respectivamente. Si el puntero que recibe el procedimiento es NIL, la búsqueda ha terminado sin haberse encontrado el elemento y por tanto, hay que insertarlo.

El Programa 5.4 muestra la implementación en Modula2 de este procedimiento. Obsérvese que el parámetro puntero se pasa por referencia, de forma que su contenido cambiará al ser asignado en el procedimiento realizándose así el enlace del nodo insertado con el árbol.

Programa 5.4 Inserción en un árbol de búsqueda.

```
PROCEDURE Insertar_busqueda(x:Tipo_llave; VAR puntero:Ptr_Nodo);
BEGIN
  IF puntero=NIL THEN (* insertar *)
    ALLOCATE(puntero,SIZE(Nodo));
    WITH puntero^ DO
      llave:=x; izq:=NIL; dch:= NIL
    END
  ELSIF x<puntero^.llave THEN Insertar_busqueda(x,puntero^.izq)
  ELSIF x>puntero^.llave THEN Insertar_busqueda(x,puntero^.dch)
  ELSE
    (*Condición de error*)
  END
END Insertar_busqueda;
```

El segundo procedimiento a considerar es la *eliminación* en el árbol de búsqueda. En este caso se dan tres situaciones diferentes:

1. No hay ningún nodo con llave igual a la que se desea eliminar.
2. El nodo a eliminar tiene como máximo un subárbol descendiente.
3. El nodo a eliminar tiene dos subárboles descendientes.

En los dos primeros casos la eliminación es directa. En el primero no hay nodo a eliminar y por tanto, no hay nada que hacer. El segundo caso contempla la posibilidad de que el nodo a eliminar no tenga ningún descendiente o tenga sólo uno. Si no tiene ningún descendiente, basta con asignar NIL al puntero que apunta al nodo a eliminar y liberar el nodo, para lo cual habrá que apuntarlo previamente con un puntero auxiliar. Un ejemplo de esta situación se muestra en la Figura 5.11.

En el caso de que tenga un descendiente, hay que tener en cuenta si es por la izquierda o por la derecha. En cualquiera de estos dos casos, el puntero que lo apunta hay que modificarlo para que apunte a su descendiente y liberar el nodo a eliminar, de nuevo utilizando un puntero auxiliar. Obsérvese que en ninguno de estos casos las asignaciones del puntero del ascendiente del nodo a eliminar pueden infringir la definición de árbol de búsqueda. La Figura 5.12 muestra un ejemplo en

el que se elimina un nodo que tiene un único descendiente por la izquierda. La única diferencia entre si el único descendiente del nodo a eliminar, es por la izquierda o por la derecha, se manifiesta en la mencionada asignación del puntero que lo apunta. Si el descendiente es por la izquierda, como se muestra en la Figura 5.12, esta asignación será: $p := q^{\wedge}.izq$, y si es por la derecha será: $p := q^{\wedge}.dch$.

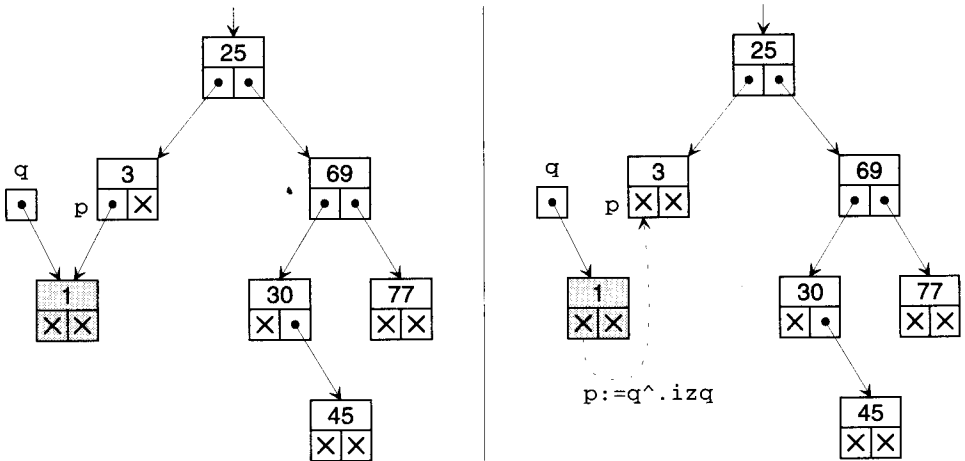


Figura 5.11 Eliminación de un nodo terminal en un árbol de búsqueda. Tras esta asignación de puntero bastaría con liberar el nodo apuntado por q con `DEALLOCATE( $q$ , SIZE(Nodo))`;

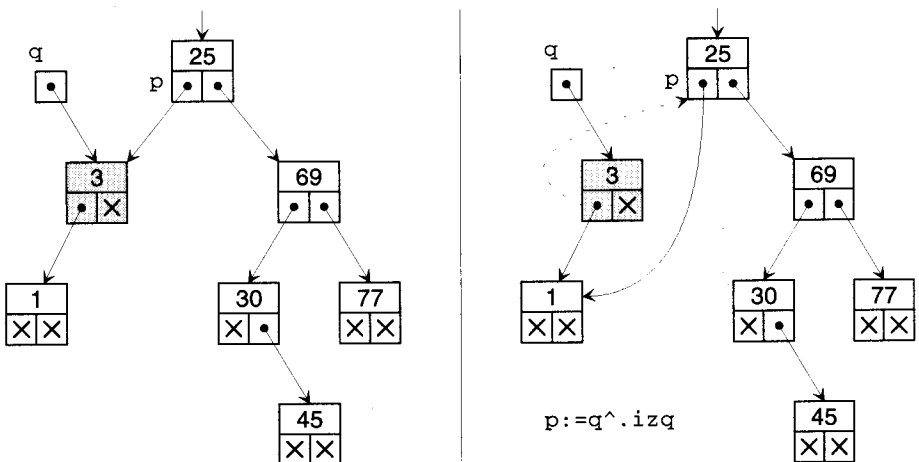


Figura 5.12 Eliminación de un nodo con un único descendiente, donde puede observarse una situación similar a la mostrada en la Figura 5.11 para un nodo terminal. Si el nodo de llave 3 hubiese tenido el descendiente por la derecha la asignación sería $p := q^{\wedge}.dch$;

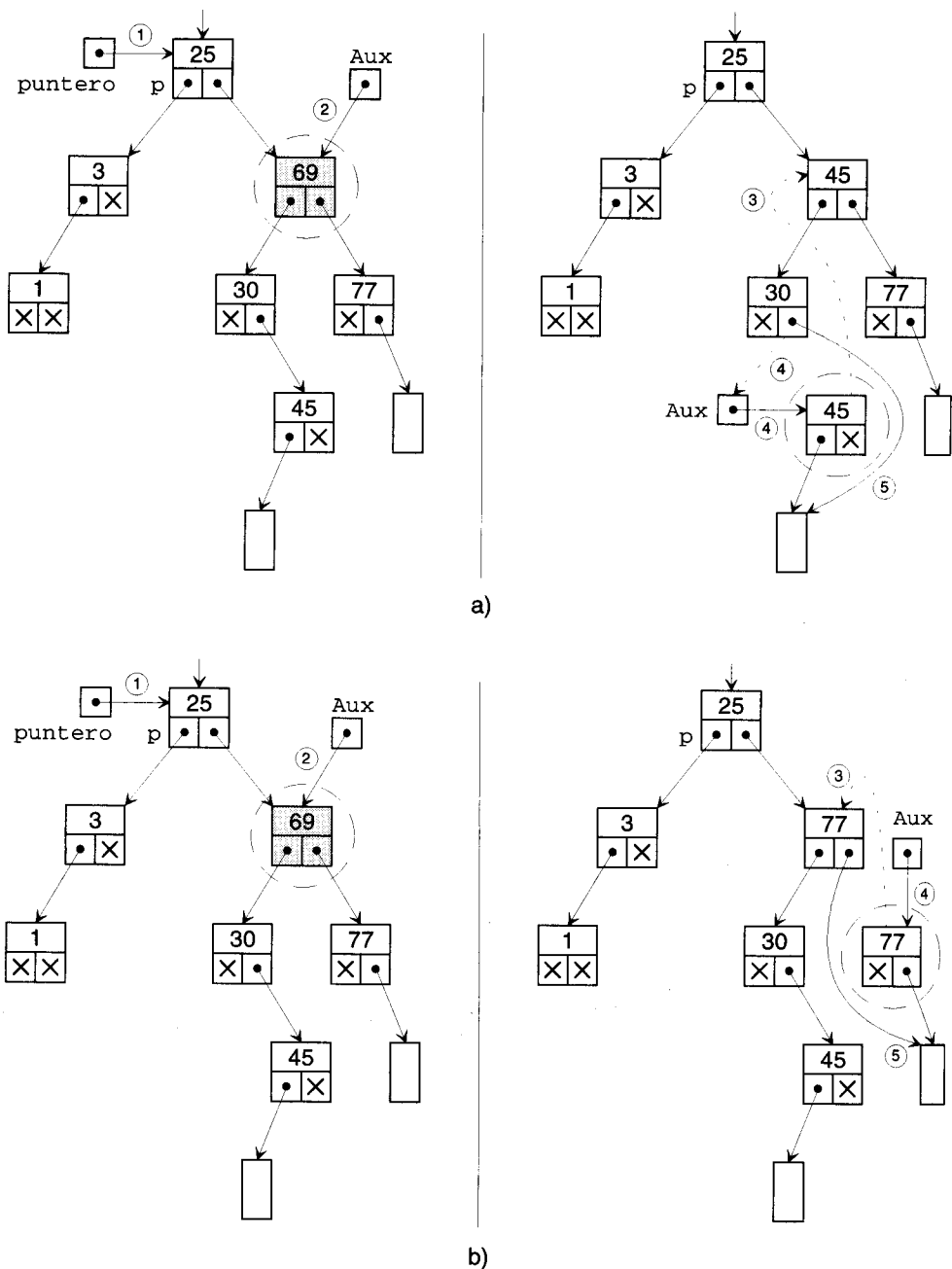


Figura 5.13 Eliminación de un nodo con dos descendientes en un árbol binario de búsqueda.
a) Búsqueda recursiva y sustitución por el mayor de los menores.
b) Búsqueda recursiva y sustitución por el menor de los mayores.

El tercer caso presenta la dificultad de que la eliminación del nodo no es directa, ya que el puntero que apunta al nodo eliminado no puede ser reasignado a los dos punteros, *izq* y *dch*, del nodo eliminado. Para mantener el árbol de búsqueda una vez eliminado el nodo hay dos posibles soluciones:

1. Sustituir el nodo eliminado por el nodo más a la derecha de su subárbol izquierdo, es decir, toma su lugar el nodo con llave mayor, de entre todos los que tienen llave menor que él.
2. Sustituir el nodo eliminado por el nodo más a la izquierda de su subárbol derecho, es decir, toma su lugar el nodo con llave menor de los que tienen llave mayor que él.

Obsérvese que estos nodos, que sustituirán al nodo eliminado, tienen como máximo un solo subárbol descendiente. La Figura 5.13 ilustra las dos posibilidades para la eliminación de un nodo con dos descendientes. En ella los rectángulos representan un subárbol con cualquier número de nodos.

El procedimiento del Programa 5.5 implementa en Modula2 la eliminación en un árbol de búsqueda con el criterio de sustitución del nodo por el mayor de los menores. Obsérvese el procedimiento local `Eliminar_dch()` que implementa esta sustitución.

Programa 5.5 Eliminación de un nodo en un árbol de búsqueda.

```

1  PROCEDURE Eliminar_búsqueda(x:Tipo_llave; VAR puntero:Ptr_Nodo);
    VAR
        Aux : Ptr_Nodo;

    PROCEDURE Eliminar_dch(VAR punt:Ptr_Nodo);
    BEGIN
        IF punt^.dch#NIL THEN Eliminar_dch(punt^.dch)
        ELSE
3         Aux^.llave:=punt^.llave;
4         Aux:=punt;
5         punt:=punt^.izq;
        END
    END Eliminar_dch;

    BEGIN
        IF puntero=NIL THEN (* Caso 1: No hay nodo a eliminar *)
        ELSIF x<puntero^.llave THEN Eliminar_búsqueda(x,puntero^.izq)
        ELSIF x>puntero^.llave THEN Eliminar_búsqueda(x,puntero^.dch)
        ELSE (* encontrado nodo a eliminar*)
2         Aux:=puntero;
            IF Aux^.dch=NIL THEN puntero:=Aux^.izq (*ningún hijo a la derecha*)
            ELSIF Aux^.izq=NIL THEN puntero:=Aux^.dch(*ningún hijo a la izquierda*)
            ELSE

```

```

      (* Dos hijos: sustitución por el elemento de más a la derecha
        del subárbol izquierdo (El mayor de los menores) *)
      Eliminar_dch(Aux^.izq)
    END;
    DEALLOCATE(Aux, SIZE(Nodo));
  END
END Eliminar_busqueda;

```

La tercera operación es evidentemente la *búsqueda*. Esta tarea se ha desarrollado implícitamente tanto en la inserción como en la eliminación. El Programa 5.6 muestra la implementación en Modula2.

Programa 5.6 Búsqueda de un nodo en un árbol de búsqueda.

```

PROCEDURE Busqueda ( x:Tipo_llave; VAR puntero:Ptr_Nodo;
                    VAR encontrado:BOOLEAN;
VAR dato:Tipo_llave);
BEGIN
  IF puntero=NIL THEN (* no encontrado *)
  ELSIF (x=puntero^.llave) AND (NOT encontrado) THEN
    encontrado:=TRUE;
    dato:=puntero^.llave;
  ELSIF (x<puntero^.llave) AND (NOT encontrado) THEN
    Busqueda(x,puntero^.izq,encontrado,dato)
  ELSIF (x>puntero^.llave) AND (NOT encontrado) THEN
    Busqueda(x,puntero^.dch,encontrado,dato)
  ELSE
    encontrado:=false; (* Condición de error *)
  END
END Busqueda;

```

Dado que la búsqueda es dirigida, es obvio que los árboles de búsqueda realizan implícitamente una tarea de clasificación. Compárese con la búsqueda en los árboles perfectamente balanceados de las Figuras 5.6 y 5.7. Es evidente que la disposición organizada de los datos en el árbol de búsqueda aumenta la eficacia. Puede observarse su semejanza con el algoritmo de clasificación por inserción analizado en el Capítulo 2, salvo en el hecho de que no es necesaria ninguna recolocación de los elementos almacenados debido a su naturaleza dinámica.

5.4.3 ANÁLISIS DE LA BÚSQUEDA Y DE LA INSERCIÓN EN ÁRBOLES BINARIOS DE BÚSQUEDA

La operación más frecuente en el algoritmo de búsqueda e inserción es la *comparación* del elemento a buscar con las llaves de los nodos. Dado que los árboles

binarios se basan en dos condiciones mutuamente excluyentes se realizará una comparación en cada nodo. Entonces, el número de comparaciones a realizar dependerá del número de nodos que debe consultarse, o lo que es lo mismo, del recorrido a realizar.

Recuérdese que en los árboles perfectamente balanceados debía recorrerse todo el árbol para determinar si el dato está o no. En los árboles de búsqueda sólo habrá que buscar en una de las ramas de cada nodo. Por tanto, si el árbol de búsqueda es perfectamente balanceado el número promedio de comparaciones es $\log n$, siendo n el número de nodos del árbol. Sin embargo en los árboles de búsqueda no se ha determinado condición alguna sobre su forma, es decir, no hay ninguna condición de equilibrio. De hecho, no es probable que su crecimiento lleve a un árbol perfectamente balanceado. Véase, por ejemplo, el árbol de búsqueda de la Figura 5.10. Es evidente que el crecimiento y la profundidad del árbol de búsqueda depende del orden en el que lleguen los elementos. Así las secuencias:

13, 22, 18, 15, 9, 2, 17, 32, 16, 4, 14, 12, 69, 23, 99, 42, 10

y 13, 9, 12, 10, 2, 4, 22, 18, 15, 14, 17, 16, 32, 23, 69, 42, 99

entre otras, forman el mismo árbol de búsqueda de la Figura 5.10. Sin embargo, la secuencia

16, 12, 32, 10, 22, 14, 69, 4, 13, 18, 42, 9, 15, 23, 99, 2, 17

entre otras, da lugar a un árbol de búsqueda perfectamente balanceado, y las secuencias ordenadas, tanto creciente como decrecientemente, producen un árbol que degenera en una lista lineal, en la que el número promedio de comparaciones es $n/2$.

En resumen, el número de comparaciones es $n/2$ en el peor de los casos y se da cuando degenera en una lista enlazada, mientras que en el mejor caso es $\log n$, que tiene lugar cuando el árbol está perfectamente balanceado.

Seguidamente se presenta el análisis del caso promedio. El objetivo es calcular el número de comparaciones promedio, C_n , para encontrar una llave en un árbol de búsqueda con n nodos. Para ello se deberá considerar la búsqueda entre los n nodos y los $n!$ árboles que se generan a partir de sus permutaciones. Se supone que todas las n llaves son igualmente probables.

En primer lugar, se analiza el número de comparaciones para cualquier árbol binario. Por definición de longitud de trayectoria, el número de comparaciones promedio cuando se encuentra la llave buscada vendrá dado por:

$$C_n = 1 + \frac{L_I}{n} \quad (5.5)$$

ya que será el número de comparaciones promedio para insertar un nodo, $\frac{L_I}{n}$, más uno (la raíz).

Cuando la llave buscada no se encuentra en el árbol, el número de comparaciones promedio, C'_n , vendrá dado por el promedio de la longitud de trayectoria externa (suma de las longitudes de trayectoria de todos los nodos especiales).

$$C'_n = \frac{L_E}{m} \quad (5.6)$$

La relación entre L_I y L_E viene dada por la expresión (Propiedad 5.3)

$$L_E = 2n + L_I \quad (5.7)$$

Teniendo en cuenta la relación $m = (g - 1)n + 1$ (Propiedad 5.1), para los árboles binarios $g = 2$ tendremos:

$$C_n = 1 + \frac{L_E - 2n}{n} = 1 + \frac{mC'_n - 2n}{n} = 1 + \frac{(n+1)C'_n - 2n}{n} = \frac{(n+1)C'_n}{n} - 1$$

es decir:

$$C_n = \left(1 + \frac{1}{n}\right)C'_n - 1 \quad (5.8)$$

Esta expresión, conocida como *fórmula de Hibbard*, se satisface para cualquier árbol binario.

La aplicación de este resultado a los árboles binarios de búsqueda es inmediata. Se supone que todas las $n!$ secuencias de las n llaves se insertan en el árbol de búsqueda con igual probabilidad, $\frac{1}{n!}$. En el árbol de búsqueda podrá insertarse el nodo 0, o el 1, etc. Entonces el número de comparaciones promedio necesarias para insertar un nodo en el árbol de búsqueda será:

$$\frac{C'_0 + C'_1 + \dots + C'_{n-1}}{n}$$

siendo $C'_0, C'_1, \dots, C'_{n-1}$ el número de comparaciones promedio cuando no se encuentran en el árbol las llaves de los nodos 0, 1, ..., $n - 1$, respectivamente.

De nuevo, el número de comparaciones promedio para buscar un nodo será el número de comparaciones promedio para insertarlo más uno. Por tanto:

$$C_n = 1 + \frac{C'_0 + C'_1 + \dots + C'_{n-1}}{n} \quad (5.9)$$

Teniendo en cuenta la fórmula de Hibbard, se obtiene la siguiente ecuación de recurrencia:

$$(n+1)C'_n = 2n + C'_0 + C'_1 + \dots + C'_{n-1}$$

Para expresar la recurrencia de la forma $C'_n = f(C'_{n-1})$, se desarrolla la expresión para $n-1$.

$$nC'_{n-1} = 2(n-1) + C'_0 + C'_1 + \dots + C'_{n-2}$$

y se resta, obteniéndose:

$$(n+1)C'_n - nC'_{n-1} = 2 + C'_{n-1}$$

Por tanto:

$$C'_n = C'_{n-1} + \frac{2}{(n+1)} \quad (5.10)$$

Esta relación puede expresarse en forma no recursiva. Desarrollándola en sus sucesivos términos, y teniendo en cuenta que $C'_0 = 0$ tenemos:

$$C'_n = 2 \left[\frac{1}{n+1} + \frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{2} + C'_0 \right] = 2 \left[\sum_{i=1}^{n+1} \frac{1}{i} - 1 \right] = 2H_{n+1} - 2$$

donde H_{n+1} son los números armónicos. Aplicando la fórmula de Hibbard:

$$C_n = \left(1 + \frac{1}{n}\right)(2H_{n+1} - 2) - 1 = 2 \left[H_{n+1} + \frac{H_{n+1}}{n} - \frac{1}{n} \right] - 3$$

Finalmente, teniendo en cuenta que $H_{n+1} = \frac{1}{n+1} + H_n$ se obtiene:

$$C_n = 2 \left[\left(\frac{1}{n+1} + H_n \right) \left(\frac{n+1}{n} \right) - \frac{1}{n} \right] - 3 = 2 \left[\frac{1}{n} + \frac{H_n(n+1)}{n} - \frac{1}{n} \right] - 3$$

y simplificando:

$$C_n = \frac{2H_n(n+1)}{n} - 3$$

es decir:

$$C_n = 2\left(1 + \frac{1}{n}\right)H_n - 3 \quad (5.11)$$

Interpretación.

La *interpretación* de este importante resultado es la siguiente. Aplicando a la Ecuación 5.11 la fórmula de Euler:

$$H_n = \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + \dots$$

donde $\gamma = 0.577\dots$ es la constante de Euler, se obtiene para n grande que:

$$C_n \approx 2 \ln n = 2 \log n / \log e = 1.386 \log n$$

Por tanto, el número de comparaciones promedio de un árbol de búsqueda, C_n , es del orden del 39 por ciento mayor que las correspondientes a un árbol perfectamente balanceado $\approx \log n$. Esta cifra es lo suficientemente baja como para que no se justifique el costo necesario para convertir el árbol de búsqueda en un árbol perfectamente balanceado en cada inserción.

5.5 ÁRBOLES DE BÚSQUEDA BALANCEADOS

5.5.1 CONCEPTO

Como se ha visto hasta ahora, la organización de los árboles de búsqueda es, en general, muy apropiada para las tareas de inserción y búsqueda. Sin embargo, el análisis del peor de los casos indica que cuando éste se produzca, el árbol degenerará en una lista lineal, perdiendo toda su eficacia. Evidentemente, es apropiado incorporar a los árboles de búsqueda alguna restricción en cuanto a su forma de crecimiento que evite las posibles construcciones poco eficaces. Por otro lado, construir un árbol de búsqueda perfectamente balanceado implica tener que reorganizar el árbol con cada inserción realizando una serie de manipulaciones de rebalanceo.

En resumen, se presenta un *compromiso* entre asegurar la eficacia en la búsqueda mediante cierto equilibrio y el costo del rebalanceo en cada inserción.

El análisis de los árboles de búsqueda muestra que la diferencia en el caso promedio con los árboles perfectamente balanceados es lo suficientemente baja como para no justificar el costo de las manipulaciones necesarias para construir un árbol

de búsqueda perfectamente balanceado. Por ello, puede relajarse el criterio de equilibrio, es decir, no es necesario asegurar el equilibrio perfecto, de modo que el costo de los rebalanceos sea menor.

La solución propuesta por Adelson-Velskii y Landis, mediante los árboles denominados *árboles AVL o balanceados*, satisface estos requerimientos. El criterio de equilibrio es el siguiente:

Definición 5.13: Se dice que un *árbol binario está balanceado* si y sólo si en cada nodo las alturas de sus dos subárboles difieren como máximo en 1.

Evidentemente, todos los árboles perfectamente balanceados son AVL, pero lo contrario no es cierto. La Figura 5.14.a muestra un árbol perfectamente balanceado (y por tanto, AVL) y la Figura 5.14.b un árbol AVL no perfectamente balanceado. En la siguiente sección se presentan las operaciones básicas sobre los árboles AVL.

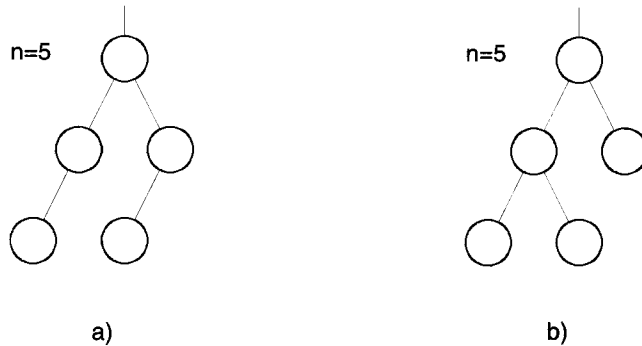


Figura 5.14 a) Árbol perfectamente balanceado. b) Árbol AVL no perfectamente balanceado.

5.5.2 INSERCIÓN EN ÁRBOLES AVL

Los árboles AVL son de búsqueda, es decir, incorporan el criterio de búsqueda mediante condiciones mutuamente excluyentes. Por tanto, la forma de insertar será en principio la misma que en los árboles de búsqueda. Pero además, incorporan un criterio de equilibrio. Por ello, una vez realizada la inserción inicial en el árbol será preciso analizar si se satisface el criterio de equilibrio y si no es así habrá que realizar las manipulaciones necesarias para que lo satisfaga. Éstas deben ser siempre sistemáticas de manera que se puedan expresar de forma algorítmica y por tanto, programable. Para ello, habrá que analizar de forma general todas las situaciones posibles y desarrollar los métodos necesarios para que se satisfagan todas las condiciones en todos los casos.

Supóngase que se dispone de un árbol AVL y que se va a insertar un elemento en un subárbol con N como nodo padre y con subárboles I y D terminales de alturas h_I y h_D respectivamente. Antes de insertar un elemento, N puede encontrarse de tres formas diferentes: $h_I = h_D$, $h_I < h_D$ o $h_I > h_D$.

Considérese que el nuevo nodo se inserta en I . Como es AVL, en cada nodo las alturas de sus dos subárboles difieren como máximo en 1 y entonces:

1. Si N tenía $h_I = h_D$ entonces el árbol seguirá siendo AVL con $h_I > h_D$.
2. Si N tenía $h_I < h_D$ entonces el árbol seguirá siendo AVL con $h_I = h_D$.
3. Si N tenía $h_I > h_D$ entonces el árbol no será AVL.

Por tanto, en los dos primeros casos la inserción es directa, pero en el tercer caso es necesario manipular el árbol de manera que siga siendo AVL y de búsqueda. A esta manipulación se le denomina *rebalanceo*.

En general, deberán analizarse dos aspectos fundamentales: en primer lugar, será preciso localizar los casos en los que es necesario rebalanceo, y en segundo lugar, habrá que desarrollar los rebalanceos adecuados.

Para detectar las posibles situaciones del árbol se define el balance del nodo N como:

$$bal(N) \equiv h_D - h_I$$

Por tanto, para realizar el rebalanceo se deberá guardar información sobre el equilibrio en cada nodo. La siguiente definición de los nodos, expresada en Modula2, tiene en cuenta explícitamente dicha información mediante el campo *bal*:

```

TYPE Ptr_Nodo = POINTER TO Nodo;
TYPE Balance = [-1..+1];
TYPE Nodo = RECORD
  llave : Tipo_llave;
  izq, dch : Ptr_Nodo;
  bal : Balance
END

```

en la que el árbol vacío se representa, como siempre, por NIL.

Inserción de un nuevo nodo por la izquierda de un subárbol

Para determinar los casos en los que es necesario el rebalanceo supóngase un árbol AVL en el que *se inserta un nuevo nodo por la izquierda* de un subárbol. Los demás casos pueden determinarse análogamente mediante sencillas consideraciones de simetría. Seguidamente se analizan los dos aspectos fundamentales para esta situación.

Análisis para determinar los casos en los que es necesario rebalanceo

Cuando se inserta un nuevo nodo por la izquierda de un subárbol de raíz N , la inserción necesita rebalanceo si tras ella $bal(N) = -2$ para algún nodo N . Se considerará como N el nodo *más profundo* tal que antes de la inserción tuviese un

balance de -1 , ya que si este subárbol es AVL también lo serán todos los ascendientes.

Sea N_l el descendiente izquierdo de N . Cuando se inserta por la izquierda de N pueden darse dos situaciones. La inserción se realizará o por el subárbol izquierdo de N_l o por el derecho.

- a) Supongamos que se insertará en el subárbol izquierdo de N_l . Entonces, si antes de la inserción $bal(N_l) = -1$, el nodo a considerar para rebalancear sería N_l y no N , ya que se debe considerar el nodo más profundo tal y como se ilustra en la Figura 5.15.a., en la que los rectángulos engloban subárboles e indican sus alturas y el cuadrado sombreado representa la inserción del nuevo nodo. Si $bal(N_l) = 0$ hay necesidad de rebalancear el nodo N (Figura 5.15.b). Finalmente si $bal(N_l) = 1$ no hay necesidad de rebalanceo (Figura 5.15.c). Por tanto, *el caso a estudiar como situación inicial para realizar rebalanceo sólo es cuando $bal(N_l) = 0$.*

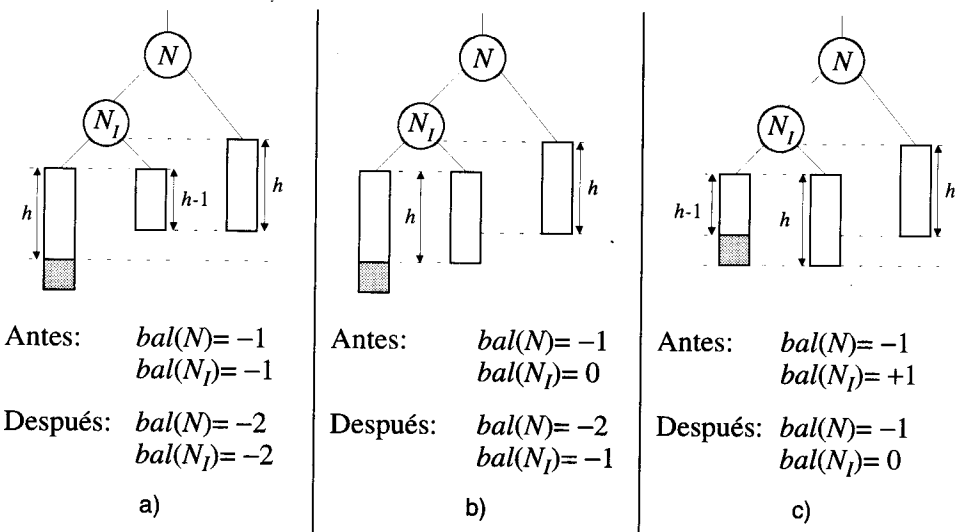


Figura 5.15 Posibles situaciones en la inserción del nuevo nodo con $bal(N) = -1$ cuando se inserta a la izquierda de N_l .

- b) Cuando el nuevo nodo se inserta en el subárbol derecho de N_l , si antes de la inserción $bal(N_l) = -1$, no hay necesidad de rebalanceo (Figura 5.16.a), si $bal(N_l) = 0$ es necesario rebalancear (Figura 5.16.b) y si $bal(N_l) = 1$ el nodo a considerar para rebalancear sería N_l y no N (Figura 5.16.c). Por tanto, *la única situación inicial a considerar es, nuevamente, cuando $bal(N_l) = 0$.*

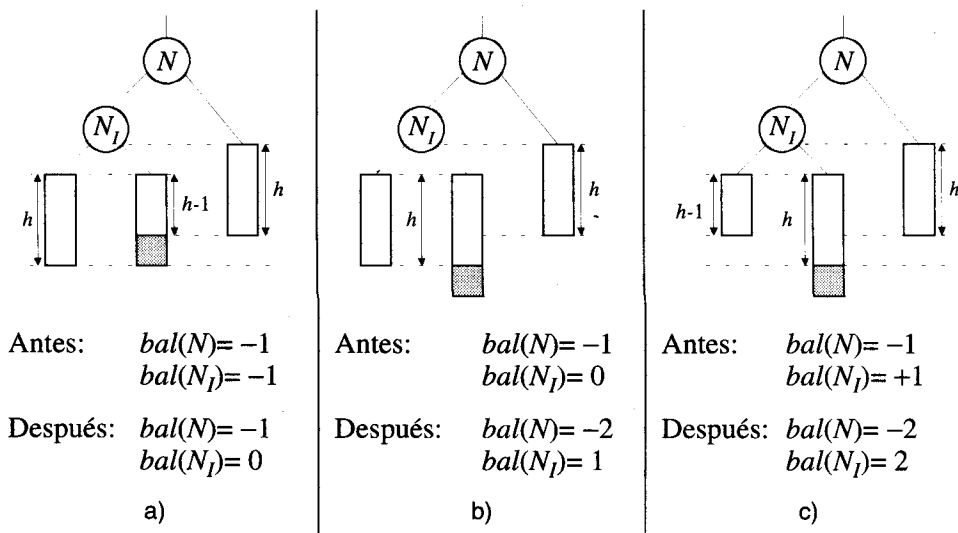


Figura 5.16 Posibles situaciones en la inserción del nuevo nodo con $bal(N) = -1$ cuando se inserta a la derecha de N_I .

En conclusión, la situación inicial en la que se debe realizar rebalanceo satisface que antes de la inserción:

$$bal(N) = -1 \quad \text{y} \quad bal(N_I) = 0$$

independientemente de si la inserción se produce por la izquierda o por la derecha del nodo N_I .

Análisis de los rebalanceos

Los rebalanceos en estos casos se denominan rebalanceos *izquierda*, L , por el hecho de que el nodo N está cargado en altura hacia la izquierda.

Como se ha concluido anteriormente, deben considerarse dos casos:

- a) El nuevo nodo se inserta en el subárbol izquierdo de N_I : *rebalanceo LL*

El rebalanceo se realizará de forma que el nodo N_I toma el lugar de N y se reasigna el subárbol derecho de N_I al subárbol izquierdo de N , manteniéndose así la característica de los árboles de búsqueda (Figura 5.17). Es una *rotación simple*.

Este rebalanceo recibe el nombre de *izquierda-izquierda*. En él, N_I está cargado en altura hacia la izquierda tras la inserción. En este texto se utilizarán las siglas inglesas LL (Left-Left) en lugar de II debido a que pueden considerarse nombres propios firmemente establecidos en la terminología informática y de las ciencias de la computación.

La Figura 5.17 muestra la secuencia de pasos necesaria para realizar un rebalanceo LL. En primer lugar se inicializan dos punteros para que apunten a los dos nodos que intervienen (p al nodo N y $p1$ al nodo N_l) lo que se muestra en el apartado a) de la figura. A continuación, se realiza la rotación de forma que N_l que es hijo por la izquierda de N pase a ser su padre por la derecha de forma que se sigan manteniendo las condiciones de árbol de búsqueda. Como primer paso para realizar esto, hay que poner el subárbol derecho de N_l como subárbol izquierdo de N con objeto de dejar libre el puntero derecho de N_l para poder asignarlo con la dirección de N , lo que se muestra en las Figuras 5.17.b y 5.17.c. Por último, se actualiza el puntero externo al árbol (p), para que apunte al nuevo nodo raíz, que a partir de ahora será N_l (Figura 5.17.d). La Figura 5.17.e muestra la situación final de la Figura anterior pero reordenada en forma de árbol. Obsérvese que antes de reasignar el puntero p , el balance del nodo N es actualizado, poniéndolo a cero.

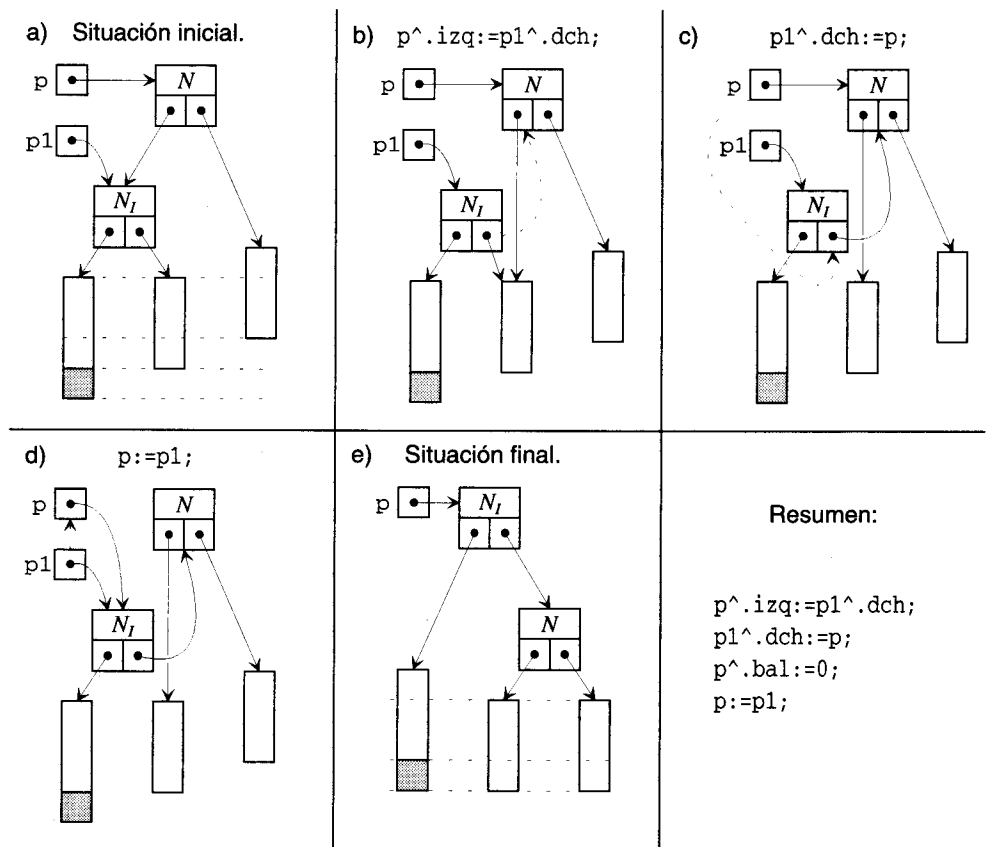


Figura 5.17 Rebalanceo LL. Tras la inserción del nuevo nodo representado con un cuadrado sombreado, (Figura a), el nodo N_l tiene un balance -1 y el nodo N tiene un balance -2. Tras el rebalanceo ambos nodos tienen un balance 0 (Figura e).

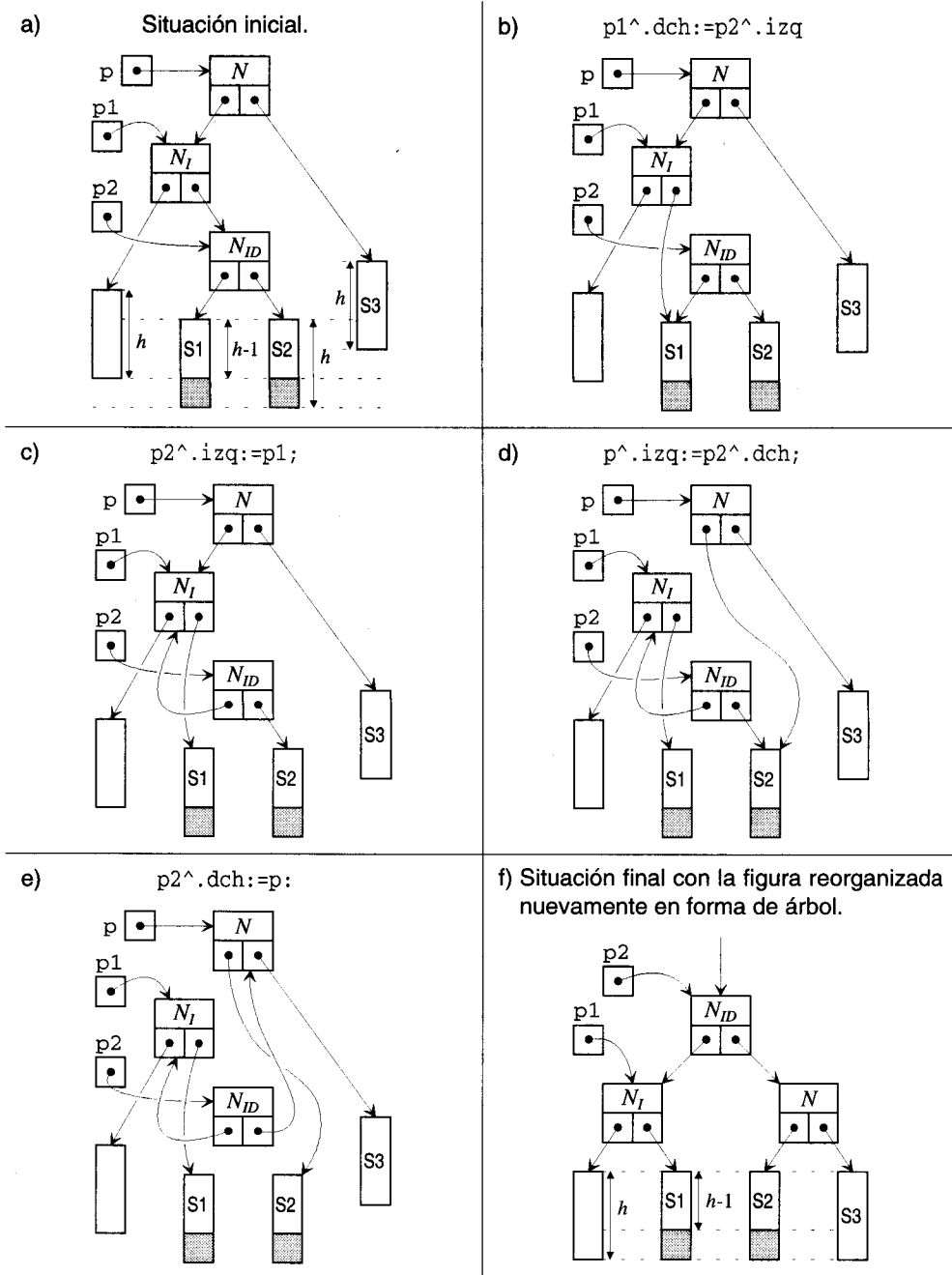


Figura 5.18 Rebalanceo LR. a) Situación inicial. b) Se pone S_1 como hijo derecho de N_I . c) N_I como hijo izquierdo de N_{ID} . d) S_2 como hijo izquierdo de N . e) N como hijo derecho de N_{ID} . f) Árbol final.

b) El nuevo nodo se inserta en el subárbol derecho de N_I : *rebalanceo LR*.

En este caso el rebalanceo no es tan simple. En principio, habrá que reorganizar el árbol tal que el descendiente por la derecha de N_I (N_{ID}), se coloque entre N y N_I , con el fin de disminuir la altura del subárbol que ha crecido (S_1 o S_2) (véase Figura 5.18). Para conseguir esto, hay que colocar el nodo N_I como hijo izquierdo de N_{ID} y N como su hijo derecho. Sin embargo, como N_{ID} tiene dos descendientes (subárboles S_1 y S_2), estas asignaciones no pueden ser directas puesto que de esta forma se perderían estos dos descendientes. Para evitarlo, hay que poner previamente a S_1 como hijo derecho de N_I y a S_2 como hijo izquierdo de N .

A la hora de hacer cualquier asignación hay que garantizar que, en todo momento, se siguen cumpliendo las condiciones de árbol de búsqueda.

Según lo comentado anteriormente, la primera acción a realizar es la de poner S_1 como hijo derecho de N_I (Figura 5.18.b). Para no perder el nodo N_{ID} se guarda su dirección previamente en un puntero auxiliar (p2). Una vez reubicado el subárbol izquierdo del nodo N_{ID} (S_1) ya podemos poner al nodo N_I como hijo izquierdo de N_{ID} (Figura 5.18.c).

A continuación, hay que realizar unos pasos simétricos a los del párrafo anterior pero con el subárbol S_2 y los nodos N y N_{ID} . El primero de estos pasos consiste en poner S_2 como hijo izquierdo de N , lo que se muestra en la Figura 5.18.d. Una vez hecho esto ya se puede realizar el último paso que consiste en poner el nodo N como hijo derecho del nodo N_{ID} (Figura 5.18.e). Estos dos últimos pasos constituyen el cuerpo central de una rotación LL, lo que se puede comprobar comparando las Figuras 5.17.b y 5.17.c de la rotación LL con las Figuras 5.18.d y 5.18.e respectivamente.

Este *rebalanceo se denomina LR* (izquierda-derecha: izquierda porque N está cargado a la izquierda y derecha porque N_I lo está a la derecha). Es una *rotación doble*.

Esto es válido tanto si se inserta a la izquierda de N_{ID} como si se hace a la derecha; en la Figura 5.18 se indica colocando el cuadrado sombreado tanto en el subárbol izquierdo como en el derecho de N_{ID} aunque obviamente en un caso particular será en uno u otro.

El único paso que resta es el de la actualización de los balances de cada nodo. Para determinar el balance de los nodos reorganizados deben considerarse las posibles situaciones en las que quedan los nodos tras la inserción y anteriores a la reorganización.

Si tras la inserción, N_{ID} tiene $bal = -1$ (suponiendo que se insertó por la izquierda de N_{ID}), el nodo N tendrá después de la reorganización $bal = 1$ puesto que queda cargado a la derecha, y el balance de N_I es cero (Figura 5.19). Si N_{ID} tiene

$bal = 1$ tras la inserción (suponiendo ahora que se insertó por la derecha de N_{ID}) entonces después de la reorganización el balance de N_I será -1 y el de N será cero (Figura 5.20). Obsérvese que si N_{ID} queda con balance cero tras la inserción entonces no es el caso en el que se necesita rebalanceo.

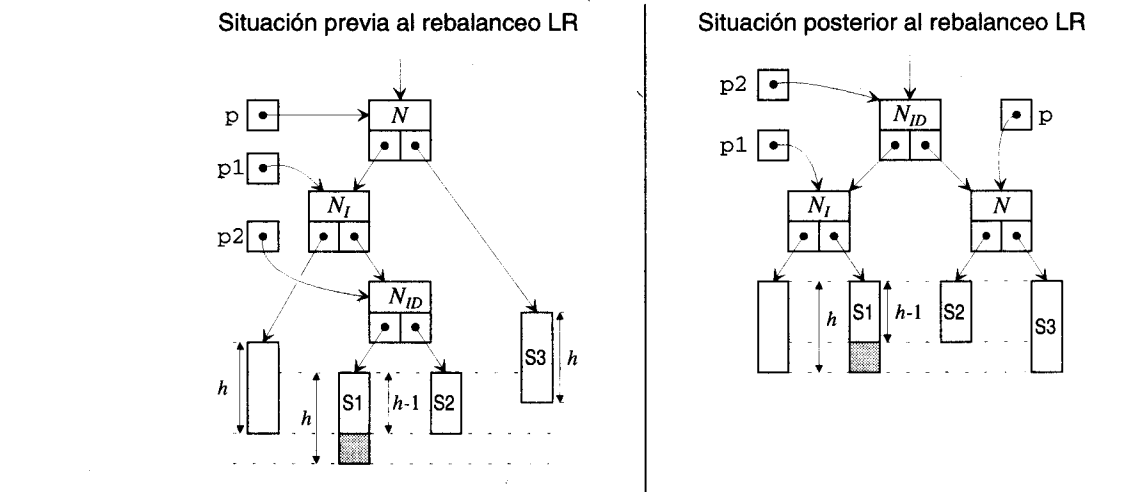


Figura 5.19 Balance tras un rebalanceo LR cuando el balance inicial del nodo N_{ID} tras la inserción es -1 .

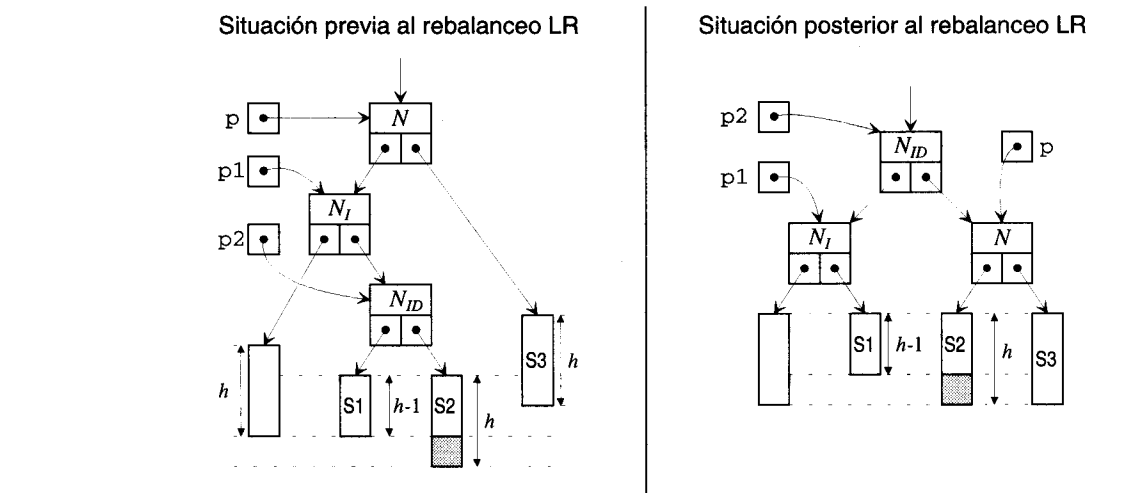


Figura 5.20 Balance tras un rebalanceo LR cuando el balance inicial del nodo N_{ID} tras la inserción es $+1$.

Por tanto, para finalizar la reorganización LR se reasignarán los balances mediante las sentencias (en Modula2):

```
IF p2^.bal=-1 THEN p^.bal:=1 ELSE p^.bal:=0 END;
IF p2^.bal=+1 THEN p1^.bal:=-1 ELSE p1^.bal:=0 END;
```

y finalmente se reasignará el puntero padre, $p:=p2$.

Inserción de un nuevo nodo por la derecha de un subárbol

Los rebalanceos, cuando el nuevo nodo se inserta por la derecha de N , se desarrollan directamente a partir de los anteriores teniendo en cuenta la simetría de ambas situaciones. Así se obtiene:

- El rebalanceo RR cuando el nuevo nodo se inserta en el subárbol derecho de N_D (descendiente derecho de N). Es una rotación simple.
- El rebalanceo RL cuando el nuevo nodo se inserta en el subárbol izquierdo de N_D . Es una rotación doble.

Algoritmo de inserción

Una vez analizados los rebalanceos, la realización de un procedimiento de inserción en árbol AVL es inmediata. En primer lugar, debe observarse que el proceso de inserción está formado por tres partes:

- Buscar siguiendo la trayectoria de búsqueda, con lo que se distinguirá la inserción por la izquierda o por la derecha.
- Insertar el nodo y determinar su balance.
- Retroceder y verificar el factor de balance en cada nodo. Realizar el rebalanceo en caso necesario.

Evidentemente, la recursión aparece de nuevo como un método adecuado para la realización de estas tareas. Obsérvese que el procedimiento es similar al de los árboles binarios de búsqueda, y las modificaciones serán las necesarias para realizar los rebalanceos.

Dada una inserción habrá que evaluar si la altura del árbol ha aumentado y transmitir esta información a los nodos ascendientes. Para ello, se hace uso de la variable booleana h que se pasa por referencia e indica si la altura ha aumentado o no. Cuando en el nivel de recursión en el que se creó un nodo, apuntado por p , se recibe información de que la altura ha aumentado pueden darse tres casos si el nodo se insertó por la izquierda y otros tres si lo hizo por la derecha.

Si se insertó por la izquierda:

- Si el balance de p era 1 ahora es cero, y no ha aumentado la altura del árbol.
- Si el balance de p era 0 ahora es -1 .
- Si el balance de p era -1 se debe realizar el rebalanceo, que podrá ser LL o LR.

Análogamente, si se insertó por la derecha:

1. Si el balance de p era -1 ahora es cero, y no ha aumentado la altura del árbol.
2. Si el balance de p era 0 ahora es 1 .
3. Si el balance de p era 1 se debe realizar el rebalanceo, que podrá ser RR o RL.

El Programa 5.7 muestra este procedimiento.

Programa 5.7 Procedimiento de inserción en un árbol AVL.

```

PROCEDURE Insertar_AVL(x:Tipo_llave;VAR p:Ptr_Nodo; VAR h:BOOLEAN);
VAR
    p1, p2 : Ptr_Nodo;
BEGIN
    IF p=NIL THEN (* insertar *)
        ALLOCATE(p,SIZE(Nodo)); h:=TRUE;
        WITH p^ DO llave:=x; izq:=NIL; dch:= NIL; bal:=0;END
    ELSIF p^.llave>x THEN Insertar_AVL(x,p^.izq,h);
    IF h THEN
        CASE p^.bal OF
            1: p^.bal:=0; h:=FALSE |
            0: p^.bal:=-1 |
            -1: (* REBALANCEO *)
                p1:=p^.izq;
                IF p1^.bal=-1 THEN      (* ROTACIÓN LL *)
                    p^.izq:=p1^.dch; p1^.dch:=p;
                    p^.bal:=0; p:=p1
                ELSE
                    (* ROTACIÓN LR *)
                    p2:=p1^.dch;
                    p1^.dch:=p2^.izq; p2^.izq:=p1;
                    p^.izq:=p2^.dch; p2^.dch:=p;
                    IF p2^.bal=-1 THEN p^.bal:=1 ELSE p^.bal:=0 END;
                    IF p2^.bal=1 THEN p1^.bal:=-1 ELSE p1^.bal:=0 END;
                    p:=p2;
                END;
                p^.bal:=0; h:=FALSE
        END
    END
    ELSIF p^.llave<x THEN Insertar_AVL(x,p^.dch,h);
    IF h THEN
        CASE p^.bal OF
            -1: p^.bal:=0; h:=FALSE |
            0: p^.bal:=1 |
            1: (* REBALANCEO *)
                p1:=p^.dch;
                IF p1^.bal=1 THEN      (* ROTACIÓN RR *)
                    p^.dch:=p1^.izq; p1^.izq:=p;

```

```

        p^.bal:=0; p:=p1
    ELSE                                     (* ROTACIÓN RL *)
        p2:=p1^.izq;
        p1^.izq:=p2^.dch; p2^.dch:=p1;
        p^.dch:=p2^.izq; p2^.izq:=p;
        IF p2^.bal=1 THEN p^.bal:=-1 ELSE p^.bal:=0 END;
        IF p2^.bal=-1 THEN p1^.bal:=1 ELSE p1^.bal:=0 END;
        p:=p2;
    END;
    p^.bal:=0; h:=FALSE
END
END
ELSE (*No se satisfacen las condiciones excluyentes*)
END
END Insertar_AVL;

```

Ejemplos

Los siguientes ejemplos, indicados por el profesor N. Wirth [Wirth, 1987] ilustran los cuatro rebalanceos. Por simplicidad, se suponen las llaves de tipo Entero.

Suponiendo el árbol inicial de la Figura 5.21.a en el que se inserta el dato 1, y atendiendo al criterio de árbol de búsqueda, la inserción se realizaría a la izquierda del 5, a la izquierda del 4 y a la izquierda del 2, quedando tal y como se indica en la Figura 5.21.a donde se muestra el nuevo nodo con fondo sombreado.

Antes de la inserción, el balance del nodo de llave 2 era 0 pues no tenía ningún descendiente y el de llave 4 era -1. Tras la inserción el balance del nodo de llave 2 pasa a ser -1 y el de llave 4 pasaría a ser -2, con lo que debe rebalancearse el nodo de llave 4 y atendiendo a la taxonomía descrita anteriormente el rebalanceo es de tipo LL.

Para realizar este rebalanceo LL, en primer lugar se asigna el subárbol derecho del nodo de llave 2 al izquierdo del nodo de llave 4 ($p^.izq:=p1^.dch$) tal y como se muestra en la Figura 5.21.b. En este caso es asignar NIL ya que este subárbol $p1^.dch$ está vacío. Seguidamente se asigna el apuntador al nodo de llave 4 al enlace derecho del nodo de llave 2, que ha sido liberado en el enlace anterior, con lo que el nodo de llave 4 pasa a ser hijo derecho del nodo de llave 2 (Figura 5.21.c.). Finalmente, se asigna el nodo de llave 2 al enlace izquierdo del nodo de valor 5 obteniéndose la situación final que se muestra en la Figura 5.21.d. La Figura 5.21.e muestra esta misma situación pero reordenando gráficamente los nodos para que presenten el aspecto de árbol habitual. Al igual que en otras figuras del texto, las flechas de trazo continuo representan punteros y las de trazo discontinuo representan trasvase de información mediante una sentencia de asignación, en la que el contenido del origen de dicho tipo de flechas es asignado al elemento al que apuntan.

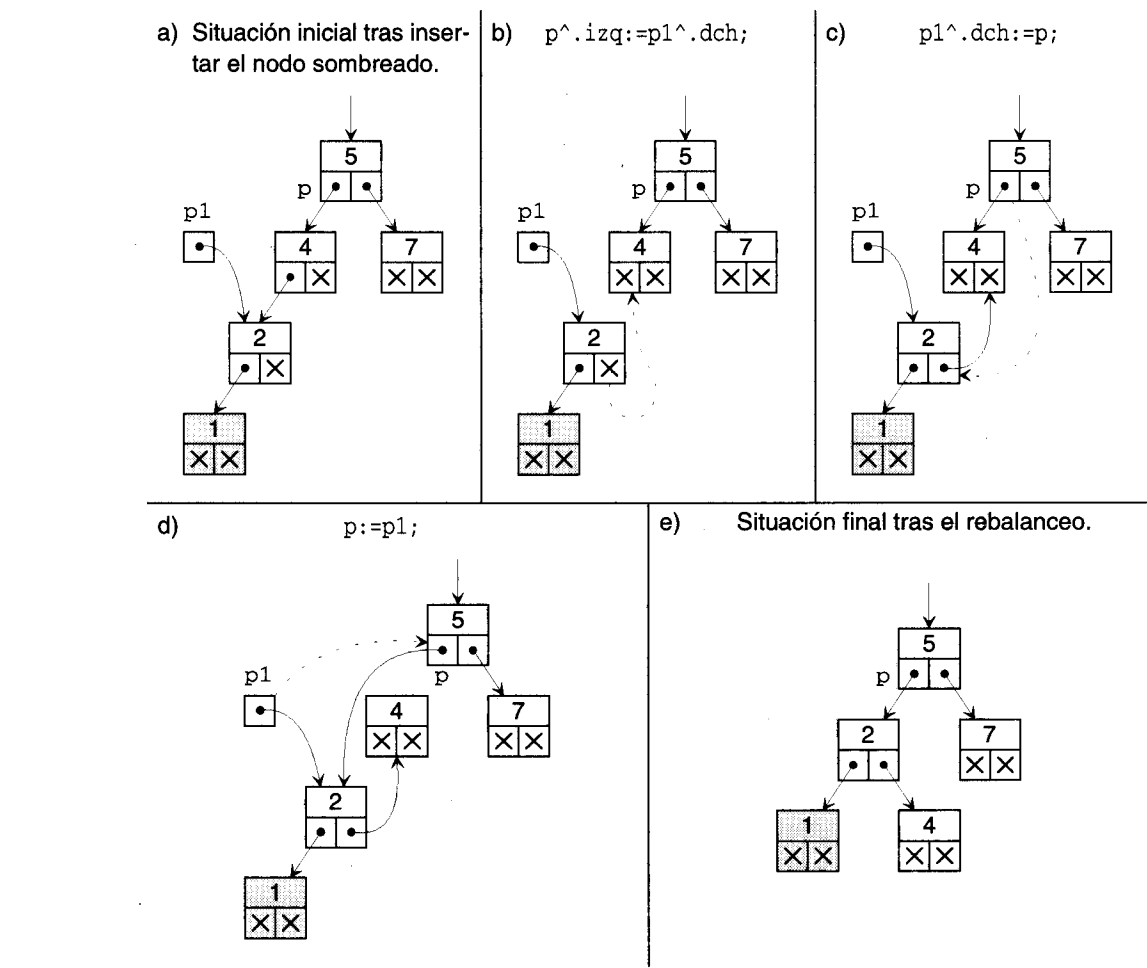


Figura 5.21 Ejemplo de rebalanceo LL.

Consideremos ahora un ejemplo en el que la inserción produce una situación que requiere un rebalanceo de tipo LR.

Esta situación se ilustra en la Figura 5.22.a en la que se muestra la posición en la que se inserta un nuevo nodo (el 3) que siguiendo el criterio de árbol de búsqueda, se insertará a la izquierda del 5, a la derecha del 2 y a la izquierda del 4, mostrándose con fondo sombreado en la mencionada figura. El balance del nodo de llave 4 era 0 antes de la inserción pues no tenía descendientes y después es -1, el del nodo de llave 2 era 0 y pasará a ser +1 y el de llave 5 era -1 y pasará a ser -2. Por tanto, el nodo a rebalancear es el de llave 5, y el rebalanceo será LR.

En primer lugar, se asigna el subárbol izquierdo del nodo de llave 4 al derecho del nodo de llave 2 con el fin de disminuir la profundidad del nodo de llave 3 (Figura 5.22.b). Seguidamente, se hace que el nodo de llave 4 apunte por la izquierda (obsérvese que este enlace acaba de ser liberado) al nodo de llave 2, manteniendo la condición de árbol de búsqueda (Figura 5.22.c). Tras esta primera rotación el árbol no está todavía rebalanceado por completo, pero se encuentra en la situación inicial de un rebalanceo LL, en la que el nodo más profundo a rebalancear es el de llave 5. A continuación, al puntero izquierdo del nodo de llave 5 se le asigna el subárbol derecho del nodo 4 (que en este caso es NIL) tal y como se ilustra en la Figura 5.22.d. Realizando esta rotación (Figura 5.22.e y 5.22.f) se obtiene el árbol AVL final.

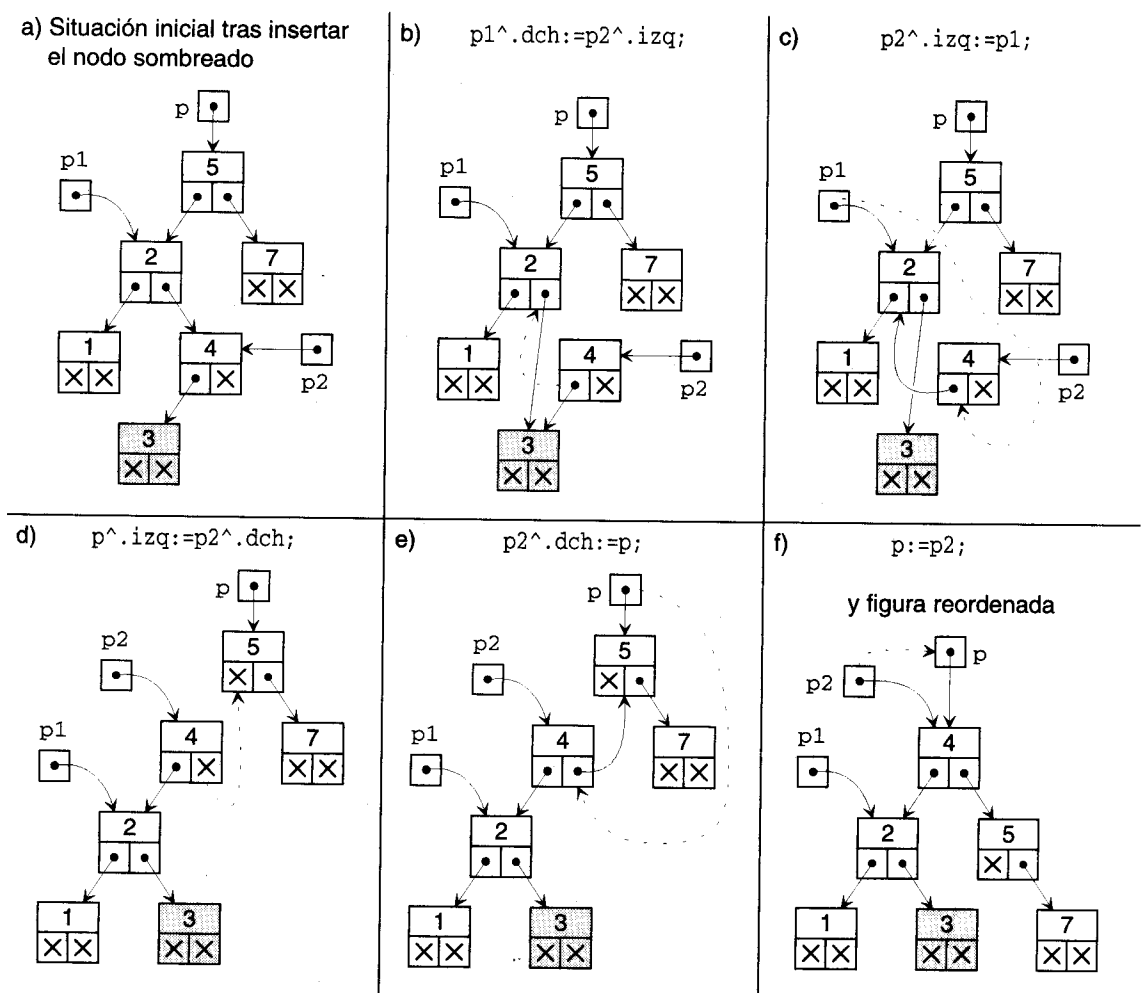


Figura 5.22 Ejemplo de rebalanceo LR. Entre las Figuras c) y d) se ha omitido el paso que se muestra en la Figura 5.23.a y que pone de manifiesto la necesidad de una rotación LL.

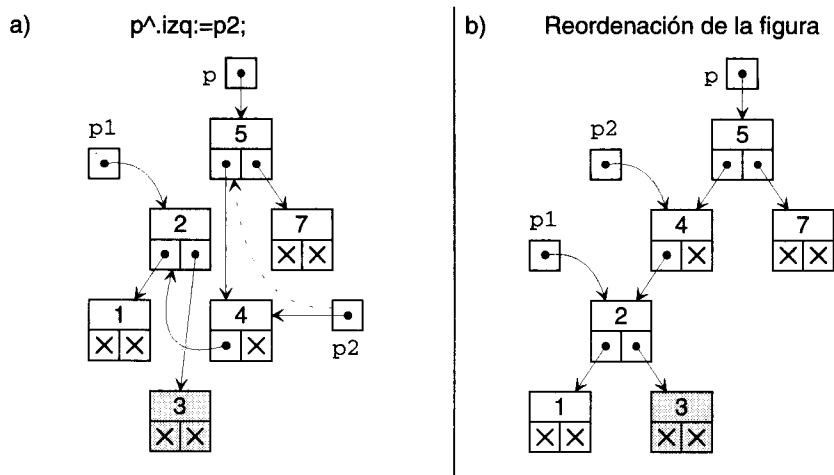


Figura 5.23 a) Paso omitido en el rebalanceo LR y que muestra la necesidad de un rebalanceo LL sobre el nodo de llave 5. b) Muestra la situación después de reorganizar la figura. Compárese el árbol resultante con el de la Figura 5.17.a que muestra la situación inicial del rebalanceo LL.

En la Figura 5.22 no se muestra evidente la situación inicial de un rebalanceo LL, ya que se ha omitido un paso que se muestra en la Figura 5.23.a. Si realizamos este paso, obtenemos el árbol de la Figura 5.23.b que comparándolo con el de la Figura 5.17.a pone de manifiesto la situación inicial de un rebalanceo LL. En este caso N es el nodo de llave 5 y N_l el de llave 4. No obstante, este paso intermedio no es necesario y se puede prescindir de él, pasando directamente de la situación de la Figura 5.22.c a la de la Figura 5.22.d que constituye el primer paso de una rotación LL. Obsérvese que si se incluyese este paso habría dos asignaciones seguidas a la misma variable puntero:

```
p^{\wedge}.izq:=p2;
p^{\wedge}.izq:=p2^{\wedge}.dch;
```

con lo que la primera resulta completamente inútil y puede eliminarse sin pérdida de información ni de generalidad.

El rebalanceo RR es análogo al LL. Supongamos la situación inicial del árbol de la Figura 5.24.a. que muestra un árbol en el que se ha insertado el nodo de llave 7. Para mantener la condición de árbol de búsqueda, se ha insertado a la derecha del nodo de llave 4 y a la derecha del de llave 5. El primer paso es poner el hijo izquierdo del nodo 5 como hijo derecho del nodo 4 (Figura 5.24.b). En este caso, este subárbol no existe pero en el caso general hay que tenerlo en cuenta para no perderlo, ya que la siguiente asignación modifica el puntero izquierdo del nodo 5 haciendo que apunte al nodo 4 con lo que este último nodo (4) se convierte en el hijo

izquierdo del anterior (5) tal y como se ilustra en la Figura 5.24.c. El último paso (Figura 5.24.d) reasigna el puntero externo del árbol para que apunte al nuevo nodo raíz (el 5). Por último, la Figura 5.24.e muestra la situación de la Figura 5.24.d pero reorganizando los nodos en forma de árbol.

Finalmente el rebalanceo RL es análogo al LR. La Figura 5.25.a muestra la situación inicial de un árbol en el que se ha insertado el nodo de llave 6, que debido al criterio de árbol de búsqueda se ha insertado a la derecha del 4, a la derecha del 5 y a la izquierda del 7. Como deben reasignarse los dos punteros del nodo 6 para que esté entre el 5 y el 7 (ascienda para convertirse en ascendente directo de ambos), primero hay que poner sus descendientes como descendientes de los nodos 5 y 7. En la Figura 5.25.b se pone el hijo derecho del 6 como hijo izquierdo del 7 y en la Figura 5.25.c se reasigna el puntero derecho del nodo 6 para que apunte al 7. La Figura 5.25.d pone el subárbol izquierdo del nodo 6 como descendiente derecho del 5 y por último, en la Figura 5.25.e se reasigna el puntero izquierdo del nodo 6 para que apunte al nodo 5 convirtiéndolo en su descendiente izquierdo. Como ya se ha comentado, estos dos últimos pasos constituyen una rotación simple RR.

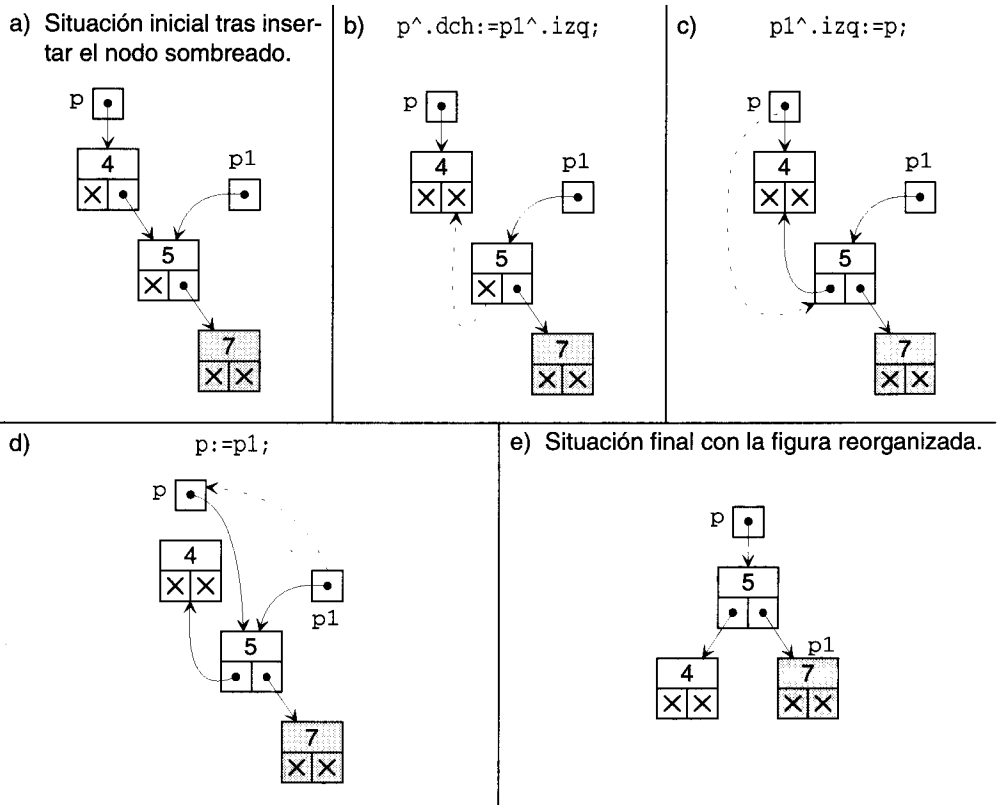


Figura 5.24 Ejemplo de rebalanceo RR.

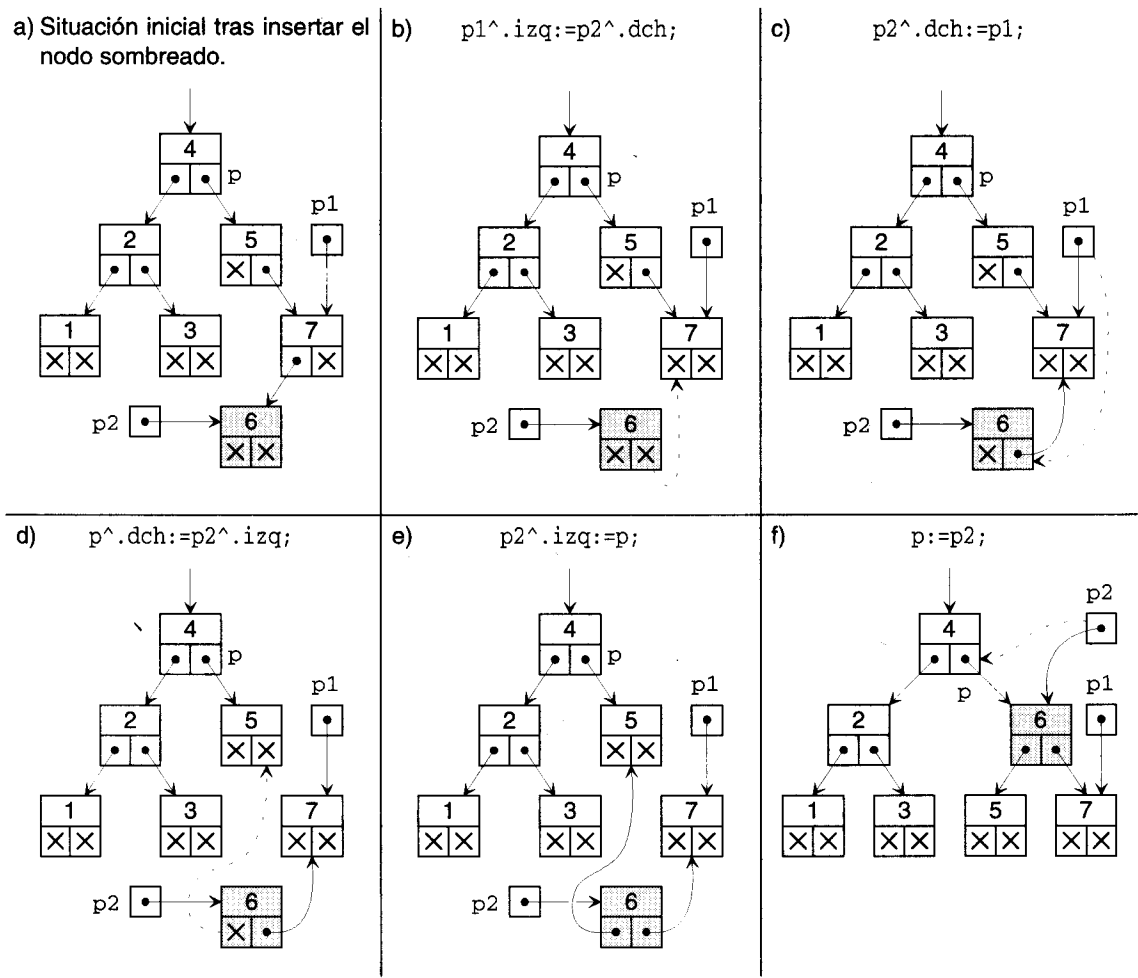


Figura 5.25 Ejemplo de rotación RL.

5.5.3 ELIMINACIÓN EN ÁRBOLES AVL

Para desarrollar el procedimiento de eliminación en árboles AVL debe tenerse en cuenta las mismas consideraciones que para la eliminación en los árboles de búsqueda más los rebalanceos necesarios cuando la altura del subárbol haya sido reducida por la eliminación.

La supresión de los nodos terminales y la de los nodos con un único descendiente es directa. Si el nodo que debe suprimirse tiene dos subárboles deberá mantenerse el árbol de búsqueda, pudiéndose realizar la eliminación, sustituyéndolo por el nodo más a la derecha de su subárbol izquierdo o por el nodo más a la izquierda de su subárbol derecho.

El Programa 5.8 muestra el procedimiento de eliminación en árboles AVL. Se hace uso de la variable booleana *h* para indicar si la altura del subárbol ha sido reducida. Si *h* es TRUE indica que se ha reducido la altura y es necesario rebalancear el árbol. Esta situación se da tras suprimir un nodo o cuando un rebalanceo produce la disminución de altura del árbol con raíz en otro nodo. Los procedimientos *Balance_R* y *Balance_L* realizan los rebalanceos por la derecha y por la izquierda respectivamente.

Cuando en un nivel de recursión el procedimiento *Eliminar_AVL* vuelve, será necesario rebalancear si *h* es devuelta del nivel inferior con el valor TRUE. Entonces si la eliminación se produjo por la izquierda (es decir, se pasó el parámetro *p^.izq*) entonces el rebalanceo será LL, y si fue por la derecha (se pasó el parámetro *p^.dch*) el rebalanceo será RR.

Para el caso en el que el nodo a eliminar tiene dos subárboles se ha tomado el criterio de sustituirlo por el nodo más a la derecha de su subárbol izquierdo (el mayor de los menores), realizado en el procedimiento *Eliminar_dch*, tal y como se hizo en la eliminación en árboles perfectamente balanceados (Programa 5.5 y Figura 5.13). Tras la sustitución del nodo a eliminar (apuntado por *q*) por el de más a la derecha de su subárbol izquierdo, la altura ha cambiado y se asigna *h:=TRUE*. Al volver por la trayectoria de búsqueda de este subárbol izquierdo será necesario un rebalanceo, que será RR pues el elemento que se ha sustituido es el de más a la derecha. Nótese que este rebalanceo en el procedimiento *Eliminar_dch* reorganiza los árboles del subárbol izquierdo del nodo a eliminar. Tras finalizar la sustitución (al terminar *Eliminar_dch*) la altura del árbol cuya raíz es el nodo sustituido puede haber variado, de forma que es necesario un rebalanceo para reorganizarlo. Este rebalanceo será LL puesto que la eliminación en este árbol se ha producido por la izquierda (el parámetro que se pasó a *Eliminar_dch* fue *q^.izq*).

Programa 5.8 Procedimiento de eliminación en un árbol AVL.

```

PROCEDURE Eliminar_AVL(x:Tipo_llave; VAR p:Ptr_Nodo; VAR h:BOOLEAN);
VAR
  q : Ptr_Nodo;
PROCEDURE Eliminar_dch(VAR pl:Ptr_Nodo; VAR h:BOOLEAN);
BEGIN
  IF pl^.dch#NIL THEN
    Eliminar_dch(pl^.dch,h);
    IF h THEN Balance_R(pl,h) END
  ELSE q^.llave:=pl^.llave; q:=pl; pl:=pl^.izq; h:=TRUE;
  END
END Eliminar_dch;
BEGIN
  IF p=NIL THEN (* La llave no está en el árbol *)
  ELSIF p^.llave>x THEN
    Eliminar_AVL(x,p^.izq,h);
    IF h THEN Balance_L(p,h) END

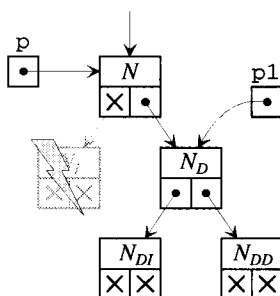
```

```

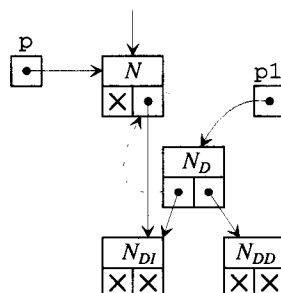
ELSIF p^.llave<x THEN
  Eliminar_AVL(x,p^.dch,h);
  IF h THEN Balance_R(p,h) END
ELSE (* Eliminación *)
  q:=p;
  IF q^.dch=NIL THEN p:=q^.izq; h:=TRUE;
  ELSIF q^.izq=NIL THEN p:=q^.dch; h:=TRUE
  ELSE
    Eliminar_dch(q^.izq,h);
    IF h THEN Balance_L(p,h) END
  END;
  DEALLOCATE(q, SIZE(Nodo));
END
END Eliminar_AVL;

```

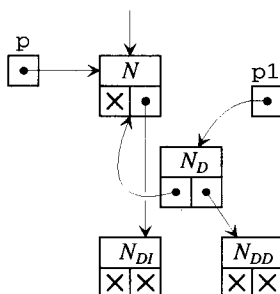
a) Situación inicial tras la eliminación de un nodo.



b) $p^.dch := p1^.izq;$



c) $p1^.izq := p;$



d) Reordenación de la figura.

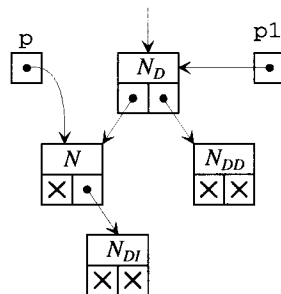


Figura 5.26 Rebalanceo tras una eliminación por la izquierda en un árbol AVL, y $bal(N_D)=0$ inicialmente.

Por último, queda por presentar los rebalanceos. Ciertamente son similares al caso de la inserción, pero no iguales. Así por ejemplo, cuando se elimina un nodo por la izquierda será necesario rebalanceo cuando el balance del nodo N sea 1, puesto que al eliminar el nodo por la izquierda el subárbol quedaría cargado en 2 por la derecha. Esta situación es la misma que se produce en la inserción cuando se inserta un elemento por la derecha. Por tanto, el rebalanceo necesario es RR. Cuando el balance de N es 0, al eliminar por la izquierda aumenta a 1 pero su altura no ha disminuido (tenía subárbol izquierdo y derecho de igual altura y ahora el izquierdo es de menor altura pero el derecho tiene la misma que antes). Por ello en este caso se asigna h a FALSE. Cuando el balance de N es -1 , pasa a cero y ha variado la altura del árbol.

Análogamente al caso de la inserción por la derecha será una rotación simple o doble dependiendo del balance del descendiente derecho de N , N_D . Si es 1 es equivalente a una inserción por la derecha de N_D , y la rotación es RR. Si es 0, de nuevo el rebalanceo es RR pero los balances son diferentes puesto que N_D tiene subárboles derecho e izquierdo de igual altura (Figura 5.26). Si es -1 entonces es análogo a una inserción por la izquierda de N_D , y la rotación es RL.

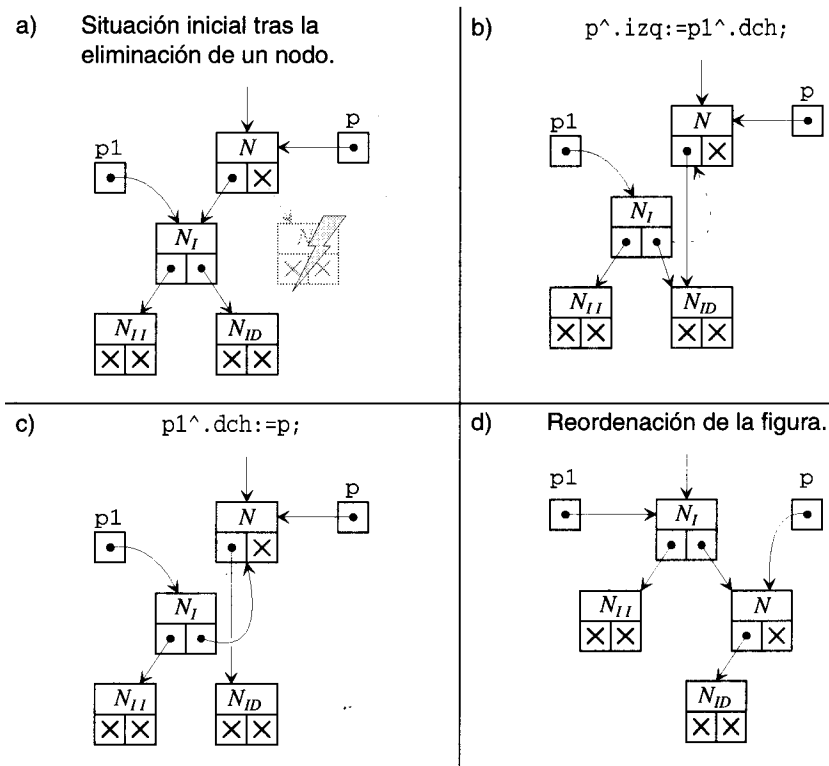


Figura 5.27 Rebalanceo tras una eliminación por la derecha en un árbol AVL, y $bal(N_i)=0$ inicialmente.

Cuando se elimina un nodo por la derecha será necesario rebalanceo si el balance del nodo N es -1 , puesto que al eliminar el nodo por la derecha el subárbol quedaría cargado en -2 , siendo análogo a la inserción por la izquierda. Por tanto, el rebalanceo es L. Si el balance de N es 0 al eliminar por la derecha pasa a -1 pero su altura no ha disminuido (tenía subárbol izquierdo y derecho de igual altura y ahora el derecho es de menor altura pero el izquierdo tiene la misma que antes). En este caso se asigna h a FALSE. Cuando el balance de N es 1 , pasa a cero y ha variado la altura del árbol.

Análogamente al caso de la inserción por la izquierda será una rotación simple o doble dependiendo del balance del descendiente izquierdo de N , N_l . Si es -1 es equivalente a una inserción por la izquierda de N_l , y la rotación es LL. Si es 0 de nuevo el rebalanceo es LL, y los balances son diferentes puesto que N_l tiene subárboles derecho e izquierdo de igual altura (Figura 5.27). Si es 1 entonces es análogo a una inserción por la derecha de N_l , y la rotación es LR.

El Programa 5.9 muestra los procedimientos de rebalanceo.

Programa 5.9 Procedimientos de rebalanceos para la eliminación en un árbol AVL.

```

PROCEDURE Balance_R(VAR p:Ptr_Nodo; VAR h: BOOLEAN);
VAR p1,p2: Ptr_Nodo; b1,b2:Balance;
BEGIN  (*h, la rama de la derecha se ha encogido*)
  CASE p^.bal OF
    1:p^.bal:=0|
    0:p^.bal:=-1;h:=FALSE|
    -1: (*rebalanceo*) p1:=p^.izq;b1:=p1^.bal;
      IF b1<=0 THEN                      (*ROTACIÓN LL*)
        p^.izq:=p1^.dch;p1^.dch:=p;
        IF b1=0 THEN p^.bal:=-1;p1^.bal:=1;h:=FALSE
        ELSE p^.bal:=0;p1^.bal:=0
        END;
        p:=p1
      ELSE                                (*ROTACIÓN LR*)
        p2:=p1^.dch;b2:=p2^.bal;
        p1^.dch:=p2^.izq;p2^.izq:=p1;
        p^.izq:=p2^.dch;p2^.dch:=p;
        IF b2=-1 THEN p^.bal:=1 ELSE p^.bal:=0 END;
        IF b2=+1 THEN p1^.bal:=-1 ELSE p1^.bal:=0 END;
        p:=p2;p2^.bal:=0
      END
  END
END
END Balance_R;
```

```

PROCEDURE Balance_L(VAR p:Ptr_Nodo; VAR h: BOOLEAN);
VAR p1,p2: Ptr_Nodo; b1,b2:Balance;
BEGIN
  CASE p^.bal OF
    -1:p^.bal:=0|
    0:p^.bal:=1;h:=FALSE|
    1: (*rebalanceo*) p1:=p^.dch;b1:=p1^.bal;
      IF b1>=0 THEN (*ROTACIÓN RR*)
        p^.dch:=p1^.izq;p1^.izq:=p;
        IF b1=0 THEN p^.bal:=1;p1^.bal:=-1;h:=FALSE
        ELSE p^.bal:=0;p1^.bal:=0
        END;
        p:=p1
      ELSE (*ROTACIÓN RL*)
        p2:=p1^.izq;b2:=p2^.bal;
        p1^.izq:=p2^.dch;p2^.dch:=p1;
        p^.dch:=p2^.izq;p2^.izq:=p;
        IF b2=+1 THEN p^.bal:=-1 ELSE p^.bal:=0 END;
        IF b2=-1 THEN p1^.bal:=1 ELSE p1^.bal:=0 END;
        p:=p2;p2^.bal:=0
      END
  END
END
END Balance_L;

```

5.5.4 ANÁLISIS DE ÁRBOLES AVL

El objetivo fundamental en el análisis de árboles AVL es conocer cuánto puede crecer como máximo, es decir, la altura en el peor de los casos. Este dato dará idea del costo de las operaciones de inserción y eliminación. Adelson-Velskii y Landis establecieron claramente cotas superiores e inferiores a la altura de los árboles AVL. Su teorema es el siguiente:

Teorema (Adelson-Velskii y Landis). La altura de un árbol balanceado con n nodos siempre es mayor o igual a $\log(n+1)$ y menor que $1.4404\log(n+2) - 0.328$, es decir:

$$\log(n+1) \leq h(n) < 1.4404\log(n+2) - 0.328$$

Demostración.

La demostración precisa de la cota superior requiere algunos resultados matemáticos de los que se hará uso sin demostración. Estos resultados se refieren al algoritmo moderno de Euclides para el cálculo del máximo común divisor de dos enteros no negativos.

La demostración de la cota inferior es inmediata. Evidentemente el número de nodos especiales de un árbol binario de altura h es como máximo 2^h . Dado que para

los árboles binarios $m = n + 1$, se verifica que $n + 1 \leq 2^h$ y por tanto, $h \geq \log(n + 1)$ en cualquier árbol binario.

Para calcular la cota superior de h se analiza cuál es el mínimo número de nodos que puede haber en un árbol balanceado de altura h . Este caso es lógicamente la peor estructura del árbol AVL. Sea A_h el árbol de altura h con el menor número de nodos. Entonces uno de los subárboles de la raíz, por ejemplo el izquierdo, tiene altura $h - 1$, y el otro $h - 1$ o $h - 2$. Como A_h tiene el mínimo número de nodos entonces la altura de su subárbol izquierdo será $h - 2$ (obsérvese que si la altura es $h - 1$ el árbol tiene más nodos que si es $h - 2$). El número de nodos con esta construcción viene dado por la siguiente relación de recurrencia:

$$N_0 = 1; \quad N_1 = 2; \quad N_h = N_{h-1} + N_{h-2} + 1$$

Obsérvese, a partir de la Tabla 5.1, que puede expresarse en función de los números de Fibonacci $F_{h+1} = F_h + F_{h-1}$, obteniéndose la expresión:

$$n = F_{h+2} - 1$$

Las características de este árbol satisfacen el Teorema de Lamé (1845). Aplicando este teorema, con el dato de que el mínimo número de nodos es $n = F_{h+2} - 1$, se obtiene la cota superior del teorema.

h	n	F_{h+2}
2	$2 + 1 + 1 = 4$	$F_4 = 5$
3	$4 + 2 + 1 = 7$	$F_5 = 8$
4	$7 + 4 + 1 = 12$	$F_6 = 13$
5	$12 + 7 + 1 = 20$	$F_7 = 21$

Tabla 5.1 Relación del número mínimo de nodos con los números de Fibonacci.

Interpretación.

Este teorema indica que la altura de un árbol AVL nunca será mayor que el 45 por ciento respecto a su árbol perfectamente balanceado. Por tanto, se ha mejorado sensiblemente el comportamiento de los árboles de búsqueda.

Desde el punto de vista práctico es de indudable interés conocer cuál es el comportamiento promedio, pero el análisis de los algoritmos sobre árboles AVL es extraordinariamente complicado. Por ello, la mayoría de los datos establecidos son resultados empíricos.

En cuanto al procedimiento de inserción, el rendimiento viene determinado fundamentalmente por dos cuestiones: cuál es la altura esperada del árbol AVL si todas las $n!$ permutaciones de las n llaves tienen igual probabilidad, y cuál es la probabilidad de que se necesite rebalanceo. Las pruebas experimentales indican que el rebalanceo es necesario cada dos inserciones, siendo igualmente probables las rotaciones simples y dobles, y que la altura esperada es $h_{esp} = \log(n + 0.25)$. Por tanto, el árbol AVL es tan satisfactorio como el árbol perfectamente balanceado, siendo además mucho más fácil de mantener.

En relación con el procedimiento de eliminación, debe destacarse el hecho de que mientras la inserción de una llave puede producir como máximo una rotación, cuando se elimina un nodo, la eliminación puede necesitar una rotación en cada nodo de la trayectoria de búsqueda (obsérvese el algoritmo y los razonamientos realizados sobre las llamadas a los rebalanceos). En principio este dato resulta alarmante. Sin embargo, los resultados empíricos muestran que sólo es necesario rebalanceo en una de cada cinco eliminaciones, con lo que en definitiva su comportamiento es comparable al de la inserción.

Finalmente debe observarse que los árboles AVL deben utilizarse sólo en el caso de que las operaciones de búsqueda sean más frecuentes que las de inserción (caso habitual, por otro lado) puesto que en otro caso las ventajas de la característica de árbol de búsqueda no es aprovechada y se está soportando el costo de los rebalanceos.

En conclusión, los árboles AVL presentan procedimientos de rebalanceo manejables, y las operaciones de búsqueda, inserción y eliminación tienen un costo del orden de $\log n$. Por tanto, los árboles AVL pueden considerarse como *la mejor solución de compromiso* entre el equilibrio y la organización ordenada del árbol.

5.6

ÁRBOLES CON VARIAS CONDICIONES MUTUAMENTE EXCLUYENTES

En las secciones anteriores se han estudiado detalladamente los árboles binarios, especialmente los de búsqueda. Antes de finalizar el capítulo es conveniente explorar la posibilidad de utilizar más de dos condiciones mutuamente excluyentes con lo que se requerirán árboles de mayor grado. La primera observación es evidente, cuanto más relaciones mutuamente excluyentes se consideren, *menor* puede ser la altura del árbol, ya que cada nivel admite más nodos. Supóngase, por ejemplo, el conjunto de datos:

13, 22, 18, 15, 9, 2, 17, 32, 16, 4, 14, 12, 69, 23, 99, 42, 10

que forman el árbol binario de búsqueda de la Figura 5.10. Dados dos enteros x e y , y suponiendo todas las llaves distintas, en los árboles binarios de búsqueda se consideraban las dos condiciones mutuamente excluyentes $x < y$ y $x > y$. Sin embargo,

podrían haberse tomado tres condiciones mutuamente excluyentes $x < y$, $y < x < 2y$, $x \geq 2y$. La Figura 5.28 muestra el árbol ternario de búsqueda con estas condiciones.

Como puede observarse, la altura del árbol es la misma que en el árbol de la Figura 5.10; sin embargo, su capacidad para una misma altura es mucho mayor. Además, debe tenerse en cuenta que con tres condiciones, en el peor caso en cada nodo deben realizarse dos comparaciones, frente a la única comparación de los árboles binarios, aumentando por tanto, el coste.

Por otro lado, habrá que establecer las condiciones de equilibrio necesarias para evitar que el árbol aumente anárquicamente, y los rebalanceos serán evidentemente más complejos y costosos que en el caso de los árboles binarios.

Teniendo en cuenta estas consideraciones y los buenos resultados de los costes de árboles de búsqueda binarios y AVL, se comprende que en la práctica para la organización de árboles en memoria primaria se utilicen habitualmente árboles binarios.

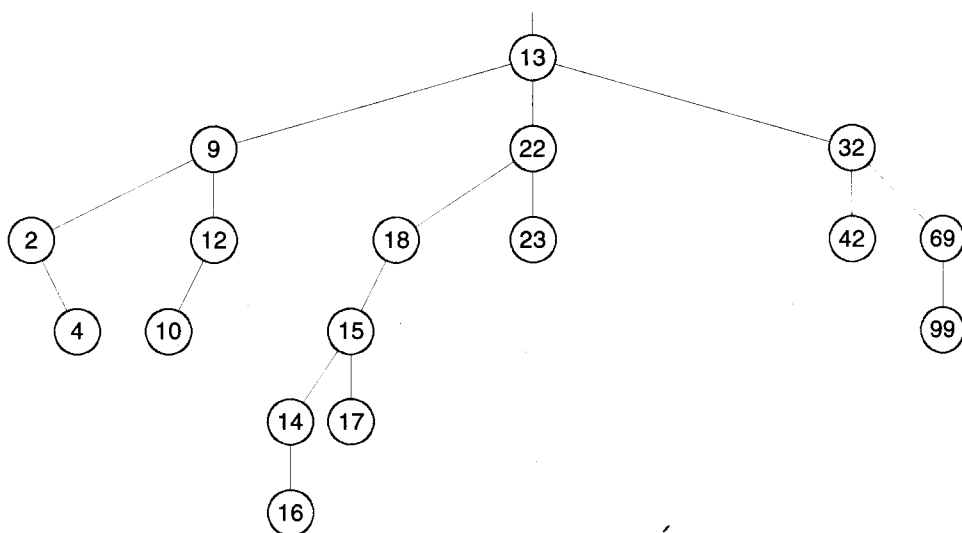


Figura 5.28 Árbol ternario de búsqueda.

5.7

CONCLUSIONES

En este capítulo se han presentado los TDA conocidos como árboles. En primer lugar, se han definido de forma general, particularizando el estudio posteriormente a los árboles binarios. Con el fin de establecer un criterio de equilibrio que evite que el TDA árbol degenera en una lista enlazada, perdiendo con ello sus posibilidades como estructura no lineal, se han definido los árboles perfectamente balanceados.

Atendiendo al estudio presentado sobre árboles binarios pueden establecerse las siguientes consideraciones. En primer lugar no debe olvidarse que los árboles son estructuras de datos dinámicas. Por tanto, su utilización es adecuada en aquellas aplicaciones en las que la cantidad de elementos a almacenar varía y puede ser elevada, al igual que en el caso de las listas enlazadas.

Entonces, ¿qué aportan los árboles sobre las listas enlazadas? Recuérdese que el tiempo de acceso de las listas enlazadas es elevado. Cuando los datos se pueden organizar, es decir, si existe un criterio de clasificación de los mismos, las estructuras de árboles presentan la posibilidad de tener en cuenta este criterio y realizar el tratamiento de la información de forma mucho más eficaz que con listas enlazadas.

En el estudio de las estructuras de árboles deben tenerse en cuenta dos factores fundamentales:

1. La longitud de trayectoria debe ser la menor posible, evitando que degeneren en una lista enlazada perdiendo su eficacia.
2. El criterio de organización de los datos.

Así, los árboles perfectamente balanceados son los que presentan el mejor comportamiento desde el punto de vista de la longitud de trayectoria. Pero debe observarse que no son árboles ordenados ya que en su definición no se ha establecido ningún criterio de clasificación. Por eso su comportamiento no es mejor que el de las listas enlazadas (piénsese, por ejemplo, en realizar una búsqueda en un árbol perfectamente balanceado).

Considerando el segundo factor, se definen los árboles ordenados, considerándose dos tipos básicos: árboles ordenados según el recorrido, y árboles de búsqueda. En los primeros la organización se da en función del orden en que se visitan los nodos y sus descendientes. Obsérvese que no se realiza en ellos ninguna consideración sobre la longitud de trayectoria. Sin embargo, este hecho no implica deterioro alguno en su eficacia, puesto que la secuenciación es el elemento a considerar en las aplicaciones de estos árboles (piénsese en el caso particular de los árboles de expresión).

Los árboles de búsqueda se definen para organizar los datos cuando en ellos se puede establecer un criterio de clasificación con condiciones mutuamente excluyentes, caso extraordinariamente común en la práctica. En este sentido satisfacen los requerimientos de organización, pero de nuevo no se realiza consideración alguna sobre la longitud de trayectoria, y para este tipo de criterio sí es importante puesto que puede degenerar en una lista enlazada y perder su eficacia. Si se establece sobre ellos el requerimiento de que sean perfectamente balanceados, el costo del mantenimiento del árbol para que tras cada inserción o eliminación siga siendo de búsqueda y perfectamente balanceado es muy elevado.

Aparece así un compromiso entre el costo de mantenimiento de la condición de equilibrio del árbol y la organización como árbol de búsqueda. Debido a este hecho se incorpora un criterio de equilibrio que garantice el comportamiento eficaz de la longitud de trayectoria en el peor de los casos, obteniéndose los árboles AVL. El

análisis realizado muestra que en el caso promedio, el árbol AVL es tan satisfactorio como el árbol perfectamente balanceado, siendo además mucho más fácil de mantener.

En conclusión, *los árboles AVL se presentan como la mejor solución de compromiso entre equilibrio y árbol de búsqueda*. Obsérvese que esta conclusión no quiere decir que los árboles AVL sean la mejor solución en todos los casos. Los rebalanceos necesarios en la inserción y eliminación de datos no se dan en un árbol de búsqueda. Por tanto, si predomina la inserción sobre la búsqueda, un árbol de búsqueda es mejor solución que un AVL. En una aplicación así puede no considerarse el criterio de equilibrio.

CAPÍTULO 6

Árboles avanzados

6.1 INTRODUCCIÓN

En el capítulo anterior se presentaron los árboles en general, pero el estudio detallado se realizó para los árboles binarios. Sólo al final del capítulo, y de forma exploratoria, se especuló sobre la posibilidad de utilizar árboles de grado mayor que dos. En aquel apartado se observaba la necesidad de establecer criterios de equilibrio, además de los de búsqueda. Sin embargo, un estudio detallado de esta posibilidad muestra la necesidad de analizar específicamente este tipo de árboles, y no como una mera extensión de los binarios. Así por ejemplo, pensar en establecer criterios de equilibrio AVL sobre árboles de grado mayor que dos implica un coste en el rebalanceo muy superior al de los árboles binarios.

En general se denominan *árboles multicamino* a aquellos árboles de grado mayor que dos. Estos árboles son muy utilizados en la construcción y mantenimiento de árboles de búsqueda con gran cantidad de nodos, y por tanto, usualmente almacenados en memoria secundaria, en los que se necesitan inserciones y supresiones. Como ya se indicó en el Capítulo 3, en los almacenamientos en memoria secundaria el costo más significativo viene dado por el tiempo de acceso a este tipo de memoria, muy superior al tiempo necesario para acceder a cualquier posición de memoria principal. Al tratarse de un dispositivo periférico, y por tanto, de acceso lento, resulta primordial minimizar en lo posible el número de accesos. Supóngase, por ejemplo, que se almacenan un millón de elementos en memoria secundaria. Con

un árbol binario el número promedio de accesos es de orden $\log 10^6 \cong 20$. Los árboles multicamino mejoran este número de accesos. Para ello se organizan de manera que un árbol se subdivide en subárboles y éstos se representan como unidades a las que *se accede simultáneamente*. Estos subárboles se denominan *páginas*, y obviamente están íntimamente relacionados con las características de direccionamiento de las memorias secundarias. Cada acceso a página requiere un único acceso a memoria secundaria. En el ejemplo anterior almacenando 100 nodos por página se necesitarán en promedio $\log_{100} 10^6 \cong 3$ accesos a memoria secundaria para buscar en un millón de elementos. Sin embargo, cuando el árbol degenera, es decir, en el peor caso, los accesos serán 10^4 . De nuevo se muestra imprescindible un criterio de crecimiento que garantice cierto equilibrio evitando las situaciones de degeneración. Ejemplos típicos de aplicación son las bases de datos [Silberschatz, 1998].

En este capítulo se presentan los resultados fundamentales para este tipo de árboles, mostrando el tipo de datos abstracto más elemental de todos los posibles: *los árboles B*. Se estudiará su organización y sus características. Las operaciones básicas asociadas se discutirán de forma conceptual, sin entrar en implementaciones detalladas. Seguidamente se analizan dos tipos de árboles, los BB y los BBS, que se obtienen a partir del concepto general de árbol B, que tendrán también aplicación en memoria primaria.

6.2

ÁRBOLES B

Los árboles B se basan en un criterio de crecimiento establecido por Bayer-McCreight, de manera que se definen de la siguiente forma:

Definición 6.1: Un árbol B de orden n es aquel árbol de búsqueda que satisface las siguientes propiedades:

1. Cada página contiene como máximo $2n$ llaves.
2. Cada página contiene como mínimo n llaves, excepto la raíz que puede contener sólo una.
3. Cada página o es una página de hoja o tiene $m+1$ descendientes, siendo m el número de llaves en esta página.
4. Todas las páginas de hoja aparecen al mismo nivel.

Atendiendo a su definición, los árboles B presentan dos características destacables. En primer lugar un árbol B con N elementos requiere en el peor caso $\log_n N$ accesos de página, y en segundo término las páginas están como mínimo llenas por la mitad, con lo que el factor de utilización de memoria será superior al 50 por ciento.

La Figura 6.1 muestra un árbol B de orden 2. Como puede observarse satisface la condición de árbol de búsqueda de manera que a la izquierda de cada llave se encuentran las llaves con valores menores que ella, y a la derecha las que tienen

valores mayores. Esto es así tanto dentro de una página como entre ellas. Además satisface las condiciones de Bayer-McCreight. Cada página contiene como máximo $2 \times 2 = 4$ llaves, y como mínimo 2 llaves, excepto la raíz que almacena una sola.

Todas las páginas de hoja están en el mismo nivel, el nivel 3, y las que no son de hoja tienen $m+1$ descendientes. Así la raíz con una sola llave ($m=1$) apunta a dos páginas ($m+1=2$ descendientes). En el segundo nivel las páginas contienen dos llaves ($m=2$), y por tanto, tienen tres descendientes ($m+1$).

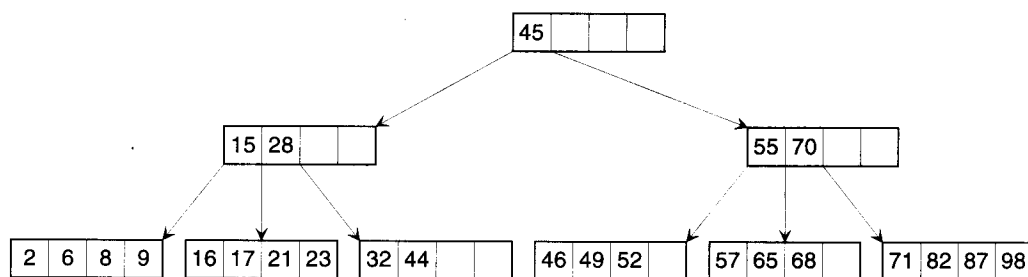


Figura 6.1 Árbol B de orden 2.

Seguidamente se presentan las tres operaciones asociadas básicas: *búsqueda*, *inserción* y *eliminación*.

6.2.1 BÚSQUEDA

La operación de búsqueda es la más habitual en los árboles de búsqueda, con la única particularidad de que en los árboles B se dispone de páginas. En general las páginas de un árbol B se representan tal y como se muestra en la Figura 6.2, con m llaves y $m+1$ apuntadores.

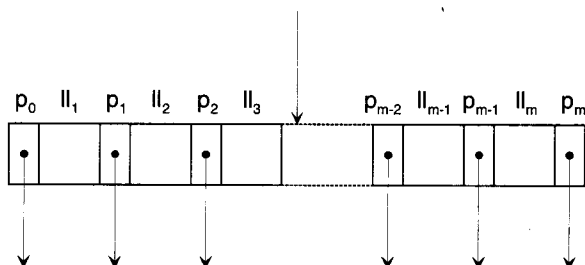


Figura 6.2 Representación de la página de un árbol B: m llaves y $m+1$ apuntadores.

La búsqueda en una página de un argumento x dado, depende del direccionamiento de la memoria en que se realice la implementación. En general en memoria secundaria suele realizarse de forma secuencial. Si se realizase sobre memoria primaria, para m grande convendría una búsqueda binaria mientras que si m es pequeño

es más adecuada la búsqueda secuencial. Si la búsqueda en una página fracasa, se pueden dar cuatro situaciones:

1. Si el argumento de búsqueda se encuentra entre dos llaves de la página, $ll_i < x < ll_{i+1}$ para $1 \leq i \leq m$, entonces la búsqueda sigue en la página apuntada por el puntero entre ellas, p_i .
2. Si el argumento de búsqueda es mayor que la última llave de la página, $ll_m < x$, entonces se sigue la búsqueda en la página apuntada por el último apuntador, p_m .
3. Si el argumento de búsqueda es menor que la primera llave de la página, $x < ll_1$, entonces se sigue la búsqueda en la página apuntada por el primer apuntador, p_0 .
4. Si en cualquier caso el apuntador a la página en la que hay que seguir la búsqueda es NIL entonces el argumento de búsqueda, x , no está en el árbol y finaliza la búsqueda.

6.2.2 INSERCIÓN

La inserción se realiza buscando el lugar donde debe insertarse de la misma manera en que se realizó la operación de búsqueda. Una vez localizada dicha posición si la página correspondiente tiene $m < 2n$ llaves se inserta en ella en la posición dada por el criterio de búsqueda, pero si la página está llena, es decir antes de la inserción tiene $m = 2n$ llaves, entonces no se puede insertar directamente, y el árbol debe reorganizarse. El rebalanceo que resuelve esta situación se ilustra en la Figura 6.3 y se describe a continuación.

Cuando la inserción en una página no es posible porque está llena, se divide en dos nuevas páginas, la llave que se encontraba en la mitad, ll_{mitad} , se sube un nivel colocándola en la página madre, las llaves que se encontraban a la izquierda se distribuyen en la página nueva apuntada por la izquierda de ll_{mitad} , y las que se encontraban a la derecha se distribuyen en la página nueva apuntada por la derecha de ll_{mitad} . A la hora de determinar la llave central hay que tener en cuenta también la nueva llave que se pretende insertar. De hecho puede ser ésta la que sea necesario subir un nivel.

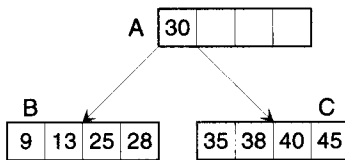
Inicialmente tenemos un árbol B de dos niveles donde las dos páginas del segundo nivel están completas tal y como se muestra en la Figura 6.3.a. Supongamos ahora que se pretende insertar la llave 42. En primer lugar, se localiza la posición de inserción en la página C, entre el 40 y el 45 (Figura 6.3.b). Como esta página está llena se divide en dos, C y D, y el elemento central (llave 40) se sube a la página de nivel inmediatamente superior, redistribuyendo las demás llaves uniformemente, 35 y 38 a la página izquierda (C) y 42 y 45 a la derecha (D) obteniéndose finalmente el árbol B rebalanceado que se muestra en la Figura 6.3.c.

Obsérvese que este método de inserción y rebalanceo conserva las propiedades de árbol B. En efecto, al subir el elemento central se conserva el criterio de árbol de

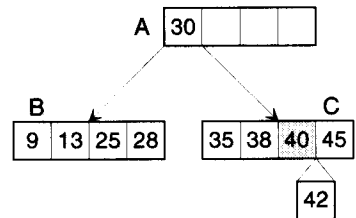
búsqueda y se posibilitan dos apuntadores, por su izquierda y por su derecha, que apuntan a las dos nuevas páginas que almacenan los demás elementos. Además, la distribución de llaves en las páginas nuevas es la mitad de $2n$ en cada una, con lo que tendrán los n elementos mínimos. Por último, obsérvese que las páginas nuevas están en el mismo nivel en el que estaba la página llena, con lo que se sigue verificando que todas las páginas de hoja estén en el mismo nivel.

La división de páginas puede propagarse hasta la raíz. Es obvio que el árbol crece de las hojas a la raíz. Esta peculiar forma de crecimiento se debe a la necesidad de satisfacer que todas las páginas hojas estén al mismo nivel.

a) Situación inicial.



b) Localización de la posición.



c) Desdoblamiento de página y ascenso de la llave central.

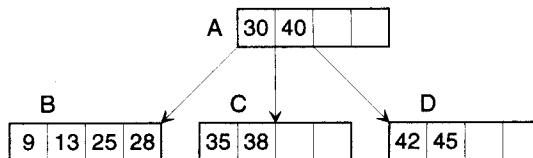


Figura 6.3 Inserción en una página llena de un árbol B de orden 2.

Ejemplo de inserción en un árbol B

Supóngase que en un árbol B de orden dos, inicialmente vacío, se introducen consecutivamente las siguientes llaves:

30, 60, 45, 8, 22, 35, 4, 28, 52, 33, 13, 39, 41, 43, 24, 25, 15.

Al insertar el 30 se colocará como raíz (Figura 6.4.a). Esta página puede contener un único elemento con lo que se satisface la definición de árbol B de orden 2. Seguidamente entraría el 60 en la misma página a la derecha del 30, el 45 entre el 30 y el 60 y el 8 a la izquierda del 30. En esta situación la página estaría llena (Figura 6.4.b). Cuando se introduce el 22 se localiza su posición entre el 8 y el 30, pero la página ya está llena por lo que ésta se divide en dos, se sube el elemento central (30) y se distribuyen el 8 y el 22 en la página izquierda (B) y en la derecha (C) el 45 y el 60 (Figura 6.4.c).

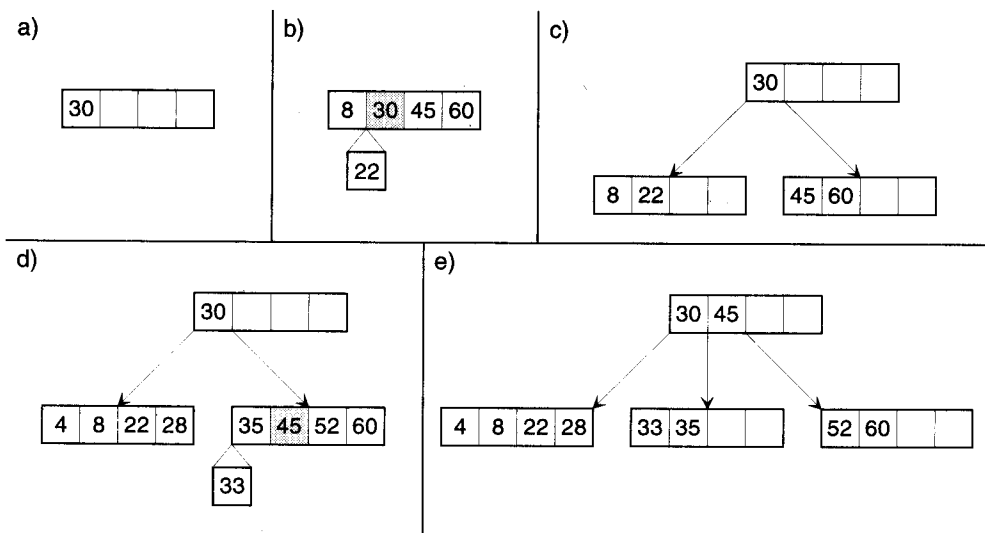


Figura 6.4 Inserción en un árbol B de orden 2 inicialmente vacío. En a) se muestra la inserción del primer elemento en el nodo raíz. b) Muestra sucesivas inserciones hasta llenar la página raíz, por lo que debe desdoblarse. c) Desdoblamiento de la página raíz. d) Más inserciones, llenando las páginas del segundo nivel; y e) nuevo desdoblamiento. (Continúa en la página siguiente).

Seguidamente se insertan las llaves 35, 4, 28, y 52 directamente en sus páginas (Figura 6.4.d.). La inserción del 33 debe realizarse detrás del 30 y delante del 35 pero esto sucede en una página llena. Obsérvese que no se puede insertar directamente en el nodo raíz ya que de esa forma apuntaría con el puntero de su izquierda a una página con elementos mayores que él rompiéndose el criterio de árbol de búsqueda. Por tanto, se hace necesaria una nueva división de página con el consiguiente ascenso de la llave central (el 45) tal y como se ilustra en la Figura 6.4.e.

La inserción del 13 se realiza de nuevo en una página llena como se muestra en la Figura 6.4.f. dando lugar a un nuevo desdoblamiento y ascenso de la correspondiente llave central que en este caso es el propio elemento a insertar (el 13) por lo que se muestra sombreado en la figura. La situación final tras este movimiento aparece en la Figura 6.4.g.

El 39 y el 41 se insertan directamente en la página correspondiente y al intentar insertar el 43, se comprueba que nuevamente le correspondería una página completamente llena (Figura 6.4.h). Esto provoca que la página se divida en dos, se suba el elemento central y se distribuyan las llaves, de la página llena, uniformemente en las páginas nuevas (Figura 6.4.i): 33 y 35 en la página de la izquierda y 41 y 43 en la nueva página de la derecha.

Los siguientes elementos son los que tienen como llave los números 24 y 25 que se insertan sin más entre los elementos de llaves 22 y 28 (Figura 6.4.j).

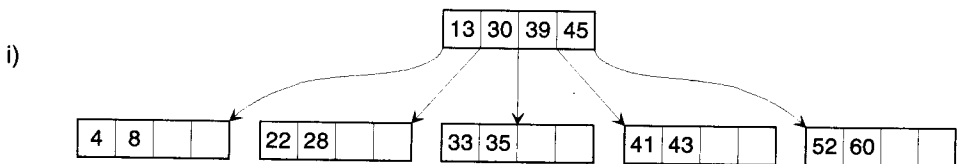
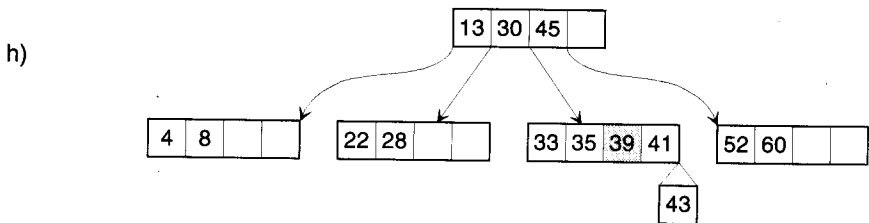
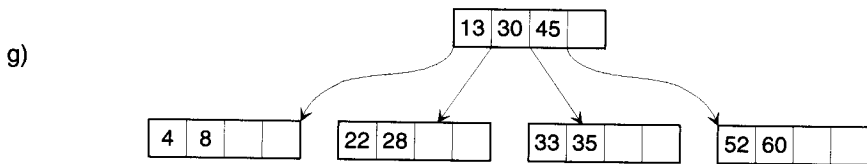
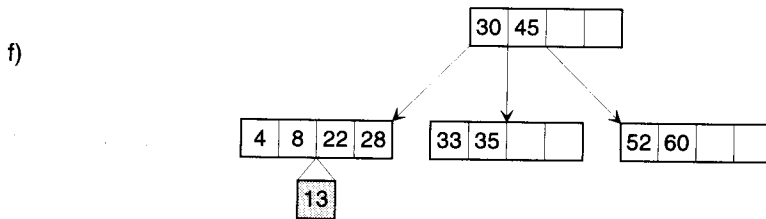
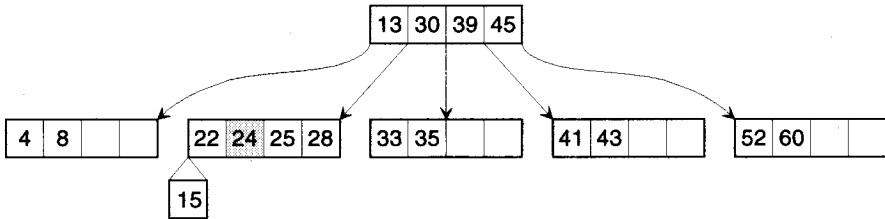


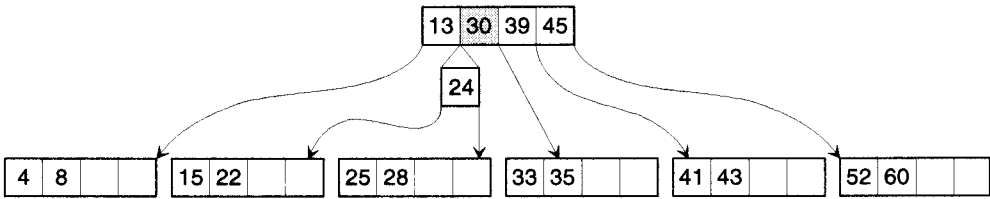
Figura 6.4. (cont.) f) Inserción del 13 en una página llena. g) Desdoblamiento y ascenso de la llave central (13). h) Inserción de elementos 39, 41 y nuevo desbordamiento con el 43. i) Desdoblamiento de página y ascenso del 39. (Continúa en la página siguiente).

Al elemento que viene a continuación (15) le corresponde también esta misma página, pero ya está llena (Figura 6.4.j), con lo que al insertarlo debe subirse el elemento central (24) dividiendo la página en dos y redistribuyéndose las llaves (Figura 6.4.k). Sin embargo, la página en la que debe insertarse el elemento que asciende también está llena (Figura 6.4.k), con lo que la inserción del elemento que sube requiere reorganizar nuevamente el árbol. De nuevo, se crean dos páginas nuevas, sube el elemento central (en este caso del nivel dos al nivel uno) y se redistribuyen las llaves (Figura 6.4.l). Como puede observarse la división de páginas se ha propagado desde las páginas hoja hasta la raíz dando lugar a un nuevo nivel.

j)



k)



l)

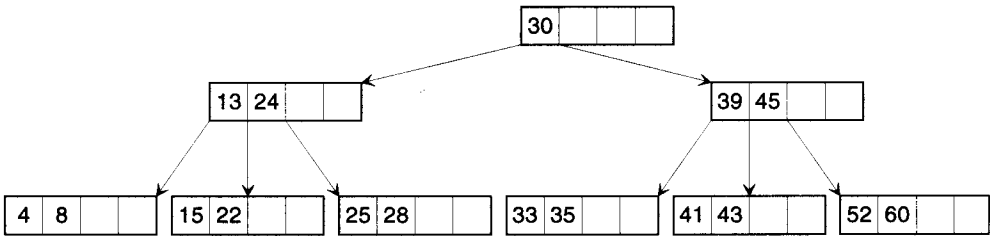


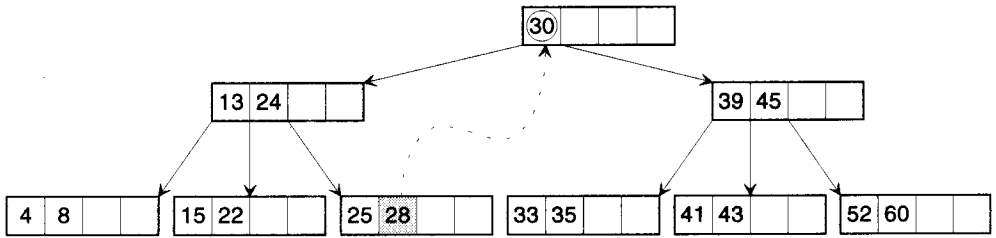
Figura 6.4. (cont.) j) Inserción de las llaves 24, 25 y 15. k) Desdoblamiento de página y ascenso de la llave central (24) a una página llena. l) Nuevo desdoblamiento y ascenso de otra llave central.

6.2.3 ELIMINACIÓN

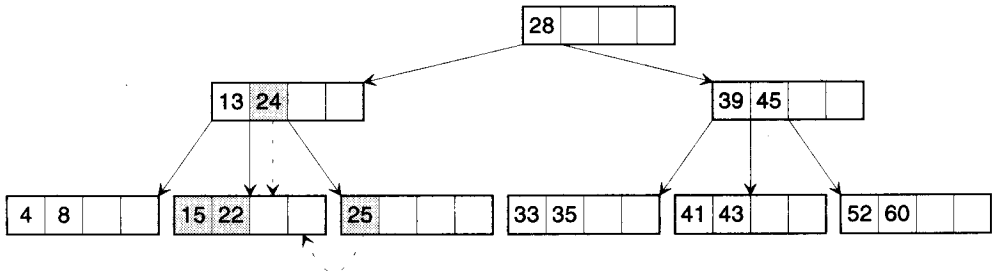
La eliminación en árboles B de orden 2 es similar a la de todo árbol de búsqueda. Una vez localizado el elemento a eliminar pueden darse dos casos.

- 1. El elemento a suprimir se encuentra en una página hoja. Entonces la eliminación es directa.
- 2. El elemento a suprimir no se encuentra en una página hoja. Entonces para que se siga cumpliendo la condición de árbol de búsqueda será necesario realizar una estrategia similar a la desarrollada en la eliminación en árboles binarios de búsqueda, sustituyéndolo por un elemento de una página hoja tal que se mantenga la característica de árbol de búsqueda. Así se tomaría el elemento de más a la derecha de la página de hoja del subárbol izquierdo, o el de más a la izquierda de la página de hoja del subárbol derecho.

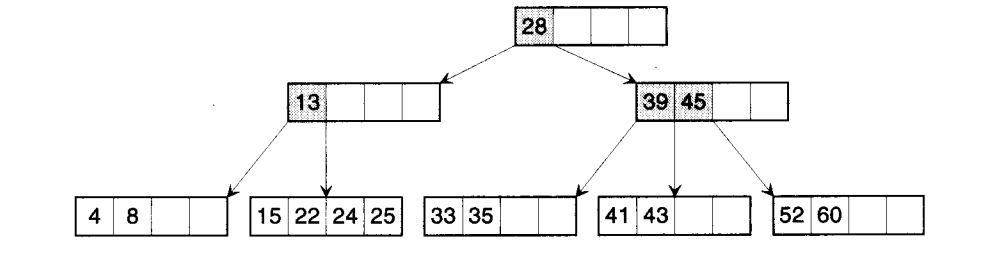
a)



b)



c)



d)

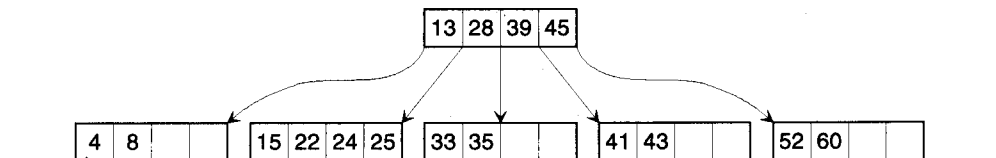


Figura 6.5 Eliminación del elemento con clave 30. a) Situación inicial en la que se muestra con un círculo el elemento a eliminar y sombreado el que le sustituye, en este caso el mayor de los menores. b) El intercambio anterior produce subocupación en la página que contiene el 25 por lo que se deben combinar dos páginas contiguas y el elemento entre ellas del nivel superior. c) Nueva subocupación de una página, por lo que se hace necesario combinar los elementos sombreados. d) Situación final donde se comprueba que se cumplen todas las condiciones impuestas a los árboles B.

Nuevamente, como en el caso de los árboles de búsqueda descritos en el capítulo anterior, la solución que sugiere el punto dos vuelve a ser la sustitución por el mayor de los menores o bien por el menor de los mayores.

En cualquier caso, es imprescindible que como mínimo cada página tenga n elementos. Si tras la eliminación este número es inferior, se produce subocupación, que puede ser resuelta mediante la estrategia habitual de rebalanceo de páginas, que consiste en tomar los elementos de dos páginas contiguas más el que las apunta desde la página padre, combinarlos en una sola página, y si existe sobreocupación se sube el elemento central.

Supongamos que en el árbol creado en el apartado anterior y cuya situación final se muestra en la Figura 6.4.1, se eliminan de forma consecutiva las llaves 30, 35 y 60.

La Figura 6.5 muestra el proceso de eliminación de la primera de estas tres llaves. En este caso se ha tomado el criterio de sustituir la llave eliminada por el mayor de los menores. O lo que es lo mismo por el elemento más a la derecha de su subárbol izquierdo. Este elemento sustituto resulta ser el 28. La Figura 6.5.a muestra este elemento sombreado y el elemento a eliminar se indica mediante un círculo. Pero el hecho de quitar el elemento de llave 28 de su página para pasarlo al nodo raíz sustituyendo al 30 provoca que esta página quede con un único elemento (el 25) con lo que no se cumplen las condiciones impuestas a los árboles B (menos de 2 llaves). Para solventar el problema deben unirse dos páginas contiguas, y esto requiere que la llave que está entre ellas (24) sea colocada en la misma página para que siga siendo árbol de búsqueda. La Figura 6.5.b muestra con fondo sombreado los elementos a combinar en una misma página y la Figura 6.5.c la situación tras esta recombinación donde se comprueba que se produce una nueva subocupación que requiere una nueva combinación de varios elementos en una misma página. En la Figura 6.5.c se muestra con fondo sombreado los elementos a combinar y la Figura 6.5.d ilustra la situación final en la que queda el árbol B tras la eliminación. En esta figura, se puede comprobar que se cumplen las condiciones de árbol B.

Veamos ahora la eliminación del siguiente elemento con llave 35. En este caso (Figura 6.6.a) se trata de un elemento que está en una página hoja o terminal, por lo que no es necesario sustituirlo por ningún otro, al contrario que en el caso anterior, y puede eliminarse directamente. No obstante hay que seguir verificando que se mantienen las condiciones de árbol B. Ahora, al eliminar el 35 nos encontramos nuevamente una subocupación en la página en la que estaba inicialmente este elemento. Esto requiere una combinación de dos páginas adyacentes y de la llave que, en un nivel superior, hay entre ellas. Siguiendo el criterio de combinarla con la página izquierda, se unen las llaves 15, 22, 24, 25, 28 y 33 en una misma página, pero al producirse sobreocupación se sube su elemento central y se divide esta página en dos redistribuyendo como es habitual. Los elementos empleados en la combinación se muestran con fondo sombreado en la Figura 6.6.a y en la Figura 6.6.b la situación final, donde se puede comprobar que nuevamente se cumplen las condiciones de árbol B.

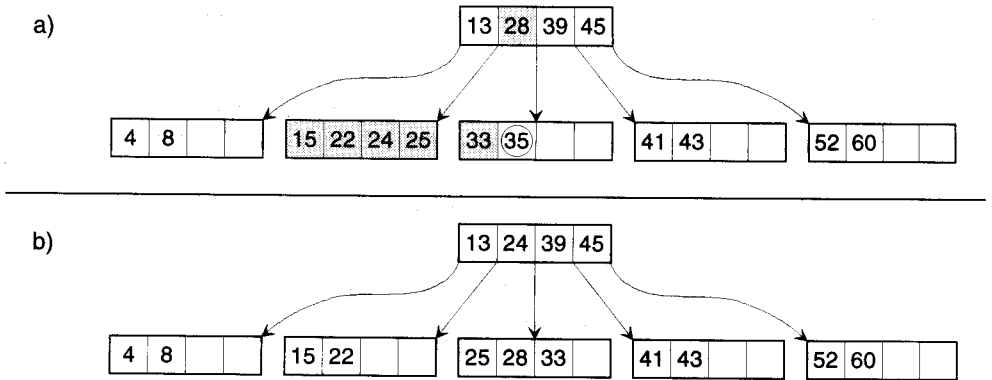


Figura 6.6 Eliminación del elemento de llave 35. a) Muestra el elemento a eliminar con un círculo y los elementos que se deben recombinar para solventar el problema de la subocupación que se produce en la página en la que estaba el nodo a eliminar. b) Situación final.

Por último, la eliminación del elemento 60 se ilustra en la Figura 6.7. Como en el caso anterior (eliminación del 35) no se requiere ninguna sustitución por tratarse de una llave perteneciente a una página hoja. No obstante, al eliminar el 60 se produce una subocupación de su página, ya que queda con un único elemento, lo que obliga a una combinación, como en los casos anteriores, entre dos páginas y la llave entre ellas (45) que está un nivel por encima. Estos elementos a combinar se muestran sombreados en la Figura 6.7.a. En la Figura 6.7.b se puede ver la situación final tras la eliminación donde se aprecia que se trata, nuevamente, de un árbol B.

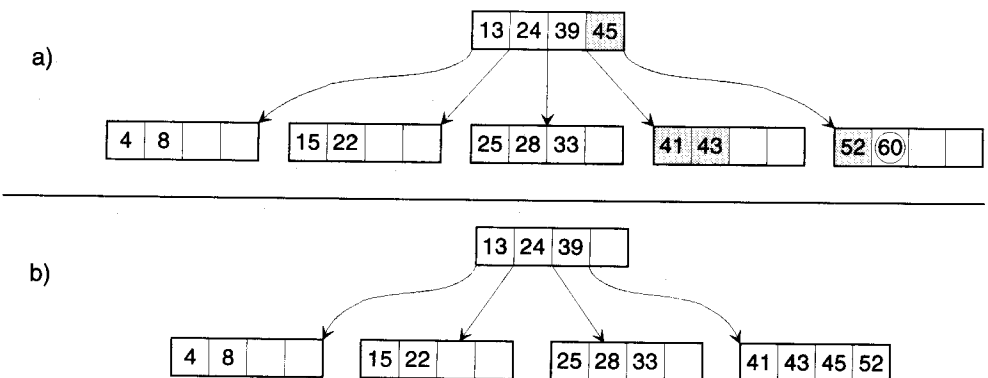


Figura 6.7 Eliminación del elemento con llave 60. a) Situación inicial mostrando el elemento que se elimina y los que se deben combinar en una misma página para solventar la subocupación. b) Situación final.

6.3 **ÁRBOLES B BINARIOS**

Un *árbol B Binario* (BB) es un árbol B de orden 1. Por tanto, consta de páginas (nodos) con una o dos llaves, y cada página contiene dos o tres punteros a los descendientes. Atendiendo a estas características estos árboles también son denominados *árboles 2-3*. En ellos, todas las páginas de hoja aparecen en el mismo nivel, y todas las páginas que no son de hoja tienen dos o tres descendientes, incluida la raíz.

Obsérvese que los árboles BB son multicamino ya que su grado es mayor que dos. Se utilizan fundamentalmente en memoria primaria. Los *apuntadores* pueden ser *verticales* u *horizontales*. La Figura 6.8 muestra la representación gráfica de los nodos (o páginas) en los árboles BB.

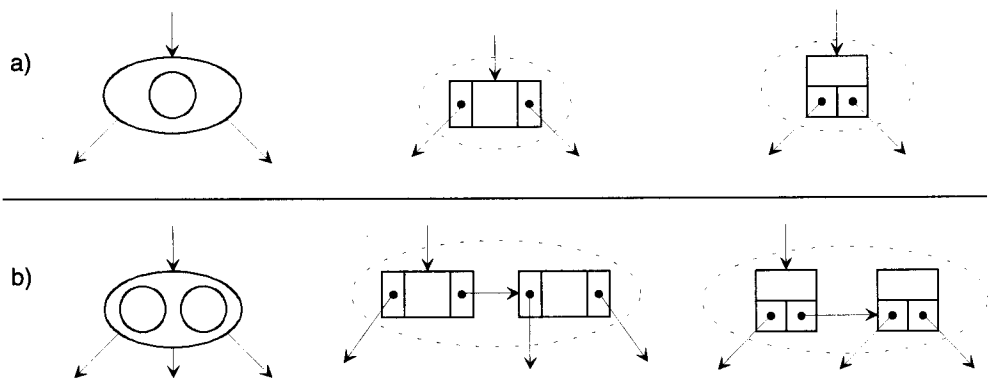


Figura 6.8 Distintas representaciones gráficas de las páginas en los árboles BB. a) Con una llave y dos punteros. b) Con dos llaves y tres punteros.

Los punteros verticales se utilizan para apuntar a los descendientes (páginas) y los horizontales apuntan a los hermanos (nodos de la misma página). Obsérvese que estos apuntadores horizontales también se presentaban implícitamente en los árboles B entre los elementos de una misma página.

En los árboles BB el apuntador horizontal tiene *un sentido* único, por convenio hacia la derecha. Así por ejemplo, la definición de un nodo de árbol BB de Bayer, expresada en Modula2, es la siguiente:

```
TYPE Ptr_Nodo = POINTER TO Nodo;
TYPE Nodo = RECORD llave: INTEGER;
    .....
    izq, dch: Ptr_Nodo;
    h: BOOLEAN(*rama horizontal a la derecha*)
END
```

Con esta estructura se garantiza una longitud de trayectoria máxima,

$$L_{max} = 2\lceil \log N \rceil.$$

La inserción en este tipo de árboles presenta cuatro posibles casos, representados en la Figura 6.9:

- a. Inserción por la derecha de un nodo sin hermano.
- b. Inserción por la derecha de un nodo con hermano.
- c. Inserción por la izquierda de un nodo sin hermano.
- d. Inserción por la izquierda de un nodo con hermano.

En el primer caso, al insertar crece el subárbol a la derecha de un nodo A , siendo A la única llave de su página. En este caso la inserción se realiza con un apuntador horizontal ya que de esta manera estarán al mismo nivel y la longitud de trayectoria será menor, y además se debe satisfacer que todas las hojas (a , b , c), estén al mismo nivel (Figura 6.9.a).

En el segundo caso (Figura 6.9.b) crece el subárbol a la derecha de un nodo A , teniendo A un hermano B . En este caso, para que todas las hojas estén al mismo nivel se sube el nodo insertado lo que lleva a una página con tres nodos, lo cual no está permitido. Para eliminar este problema se sube el elemento central apuntando verticalmente a los nodos izquierdo y derecho (Figura 6.9.b). Con ello se satisfacen todas las características de árbol BB.

En el tercer caso crece el subárbol a la izquierda de un nodo B , siendo B la única llave en su página. Para que todas las hojas se encuentren en el mismo nivel es necesario subir el nodo insertado al nivel inmediatamente superior, pero como no están definidos apuntadores horizontales hacia la izquierda se hace necesaria una *rotación* (Figura 6.9.c).

Por último, el cuarto caso (Figura 6.9.d), se da cuando crece el subárbol a la izquierda de un nodo B , teniendo B un hermano C . Ahora para que todos los nodos hoja estén al mismo nivel es necesario subir el nodo A , tras lo cual es necesario una rotación ya que no existen apuntadores hacia la izquierda al igual que en el caso anterior. Después de hacer esto tendríamos tres nodos en una misma página, por lo que debe realizarse una división de página, subiendo el elemento central, para que no se den dos apuntadores horizontales en la misma página. Todo esto es equivalente a bajar el nodo C un nivel.

Los árboles BB presentan claramente el inconveniente de que sólo tienen definido el apuntador horizontal en una dirección. Es decir, sólo uno de los dos punteros puede tener el atributo de horizontal, normalmente el derecho por convenio. Esta característica, aunque simple, resulta algo restrictiva y produce una asimetría en el crecimiento del árbol que sugiere pensar en un tipo de datos abstracto, similar al BB, pero con apuntadores en ambos sentidos. Éstos son los árboles de búsqueda simétricos, BBS.

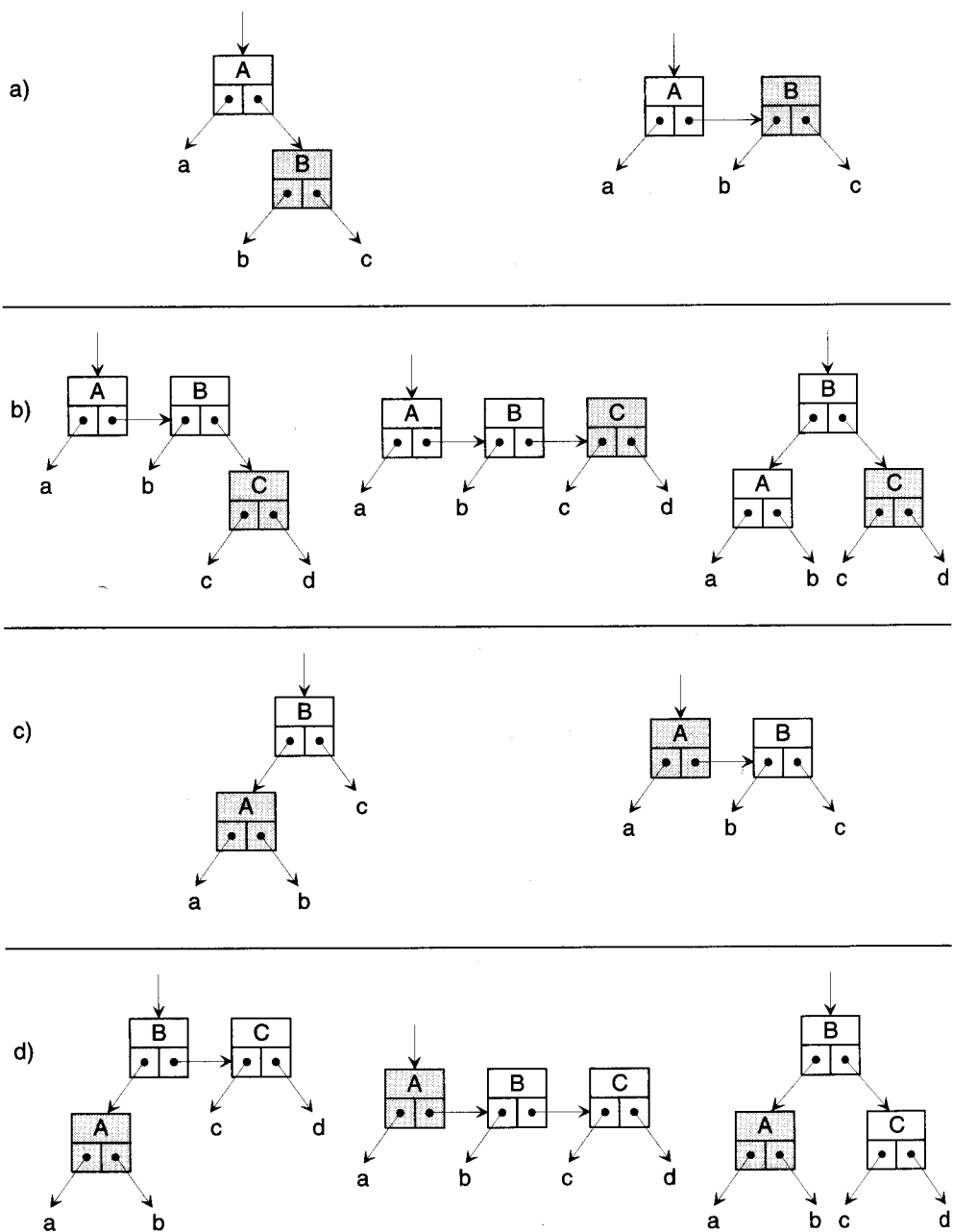


Figura 6.9 Inserción en árboles BB. a) Inserción de hermano. b) Tres nodos en la misma página que obliga a dividir en dos niveles. c) Inserción por la izquierda en un nodo sin hermano. d) Inserción por la izquierda de un nodo con hermano.

6.4 ÁRBOLES B BINARIOS SIMÉTRICOS (BBS)

Los *árboles B Binarios Simétricos* se definen con la idea de garantizar un crecimiento simétrico de los subárboles derecho e izquierdo del árbol. Se definen como árboles de búsqueda que satisfacen las siguientes propiedades:

1. Todo nodo contiene una llave, y como máximo dos apuntadores a subárboles.
2. Todo apuntador es horizontal o vertical. *No existen dos apuntadores consecutivos horizontales en ninguna trayectoria de búsqueda.*
3. *Todos los nodos terminales aparecen al mismo nivel.*

Esta definición evita la asimetría en el crecimiento de los árboles BB. En los árboles BBS es inmediato observar, a partir de sus propiedades, que la trayectoria más larga de búsqueda no es mayor que el doble de la altura del árbol, o lo que es lo mismo, un árbol BBS con n nodos no puede tener una altura superior a $\log n$. Por tanto, la longitud de trayectoria de búsqueda de un árbol BBS con n nodos es como máximo $2\lceil \log n \rceil$.

En la inserción en árboles BBS pueden darse cuatro casos en los que es necesario rebalanceo para que se satisfagan sus propiedades. Vamos a analizar cada uno de estos casos por separado.

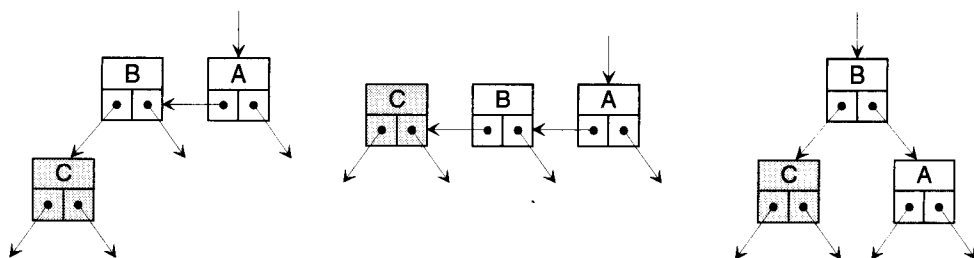


Figura 6.10 Inserción y rebalanceo LL en un árbol BBS.

En el primer caso, que se ilustra en la Figura 6.10, se inserta un nodo (C) a la izquierda del nodo padre (B) que a su vez es apuntado por un nodo hermano (A) por la izquierda. Es necesario subir un nivel el nodo insertado (C) para que todos los nodos terminales estén al mismo nivel. Pero entonces se encuentran dos apuntadores horizontales consecutivos en la misma trayectoria de búsqueda, hecho que se soluciona subiendo un nivel el nodo central. Los caracteres (A, B, C) se utilizan para identificar los nodos, pero no representan las llaves de los mismos. Este *rebalanceo se denomina LL*, por las mismas razones que en los árboles AVL.

En el segundo caso (Figura 6.11) se produce la situación simétrica. Se inserta un nodo (C) a la derecha del nodo padre (B) que a su vez es apuntado por un nodo hermano (A) por la derecha. Es necesario subir un nivel el nodo insertado (C) para

que todos los nodos terminales estén al mismo nivel. Pero entonces se encuentran dos apuntadores horizontales consecutivos en la misma trayectoria de búsqueda, hecho que se soluciona subiendo un nivel el nodo central (Figura 6.11). Este *rebalanceo se denomina RR*.

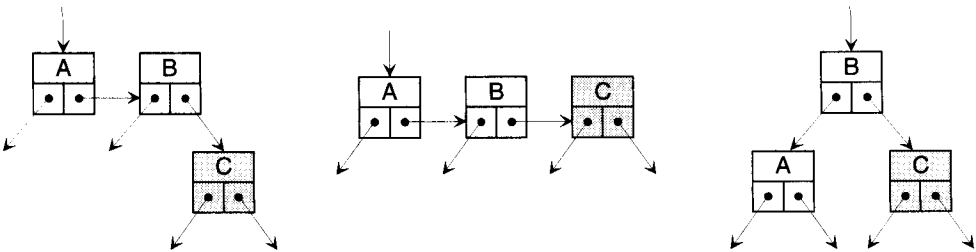


Figura 6.11 Inserción y rebalanceo RR en un árbol BBS.

Un tercer caso se da cuando se inserta un nodo (C) a la derecha del nodo padre (B) que a su vez es apuntado por la izquierda de un nodo hermano (A) (Figura 6.12.a). Es necesario subir el nodo insertado (C) un nivel para que todos los nodos terminales estén al mismo nivel. En esta situación el hermano izquierdo del nodo A debería ser C, pero entonces B no sería apuntado (Figura 6.12.b). Por ello deberá reasignarse el nodo A como hermano derecho de C (Figura 6.12.c), en cuyo caso habría dos apuntadores consecutivos en la misma trayectoria de búsqueda y se realizaría un rebalanceo RR. Este *rebalanceo se denomina LR* (Figura 6.12.d).

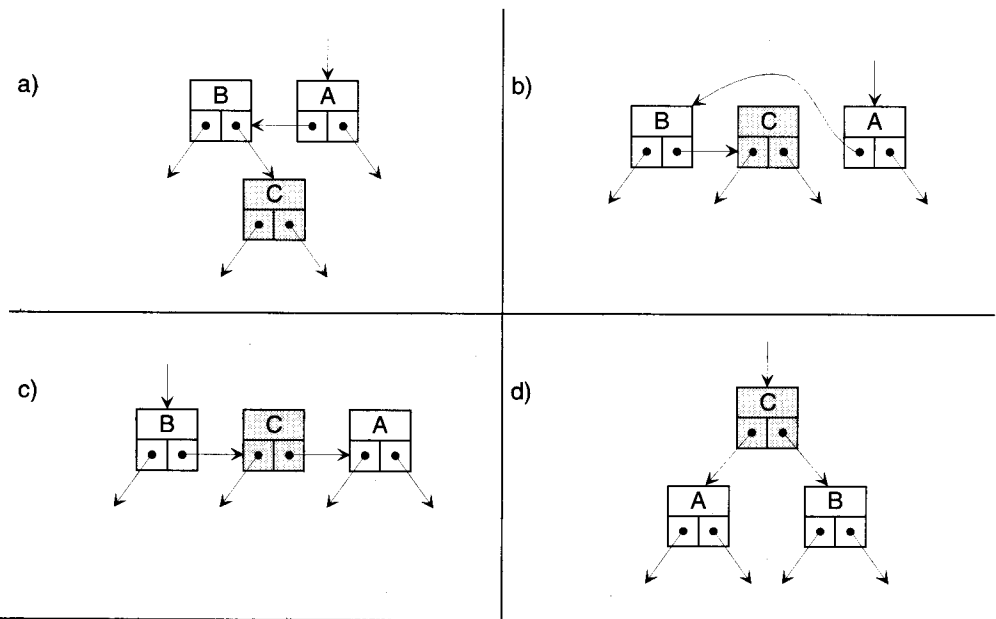


Figura 6.12 Inserción y rebalanceo LR en un árbol BBS.

El cuarto caso es simétrico al anterior y se da cuando se inserta un nodo (*C*) a la izquierda del nodo padre (*B*) que a su vez es apuntado por un nodo hermano (*A*) por la derecha (Figura 6.13.a). Es necesario subir el nodo insertado (*C*) un nivel para que todos los nodos terminales estén al mismo nivel (Figura 6.13.b). En esta situación el hermano derecho del nodo *A* debería ser *C*, pero entonces *B* no sería apuntado. Por ello, deberá reasignarse el nodo *A* como hermano izquierdo de *C*, en cuyo caso habría dos apuntadores consecutivos en la misma trayectoria de búsqueda (Figura 6.13.c) y se realizaría un rebalanceo LL. Este rebalanceo se denomina *RL*. La Figura 6.13.d muestra la situación final.

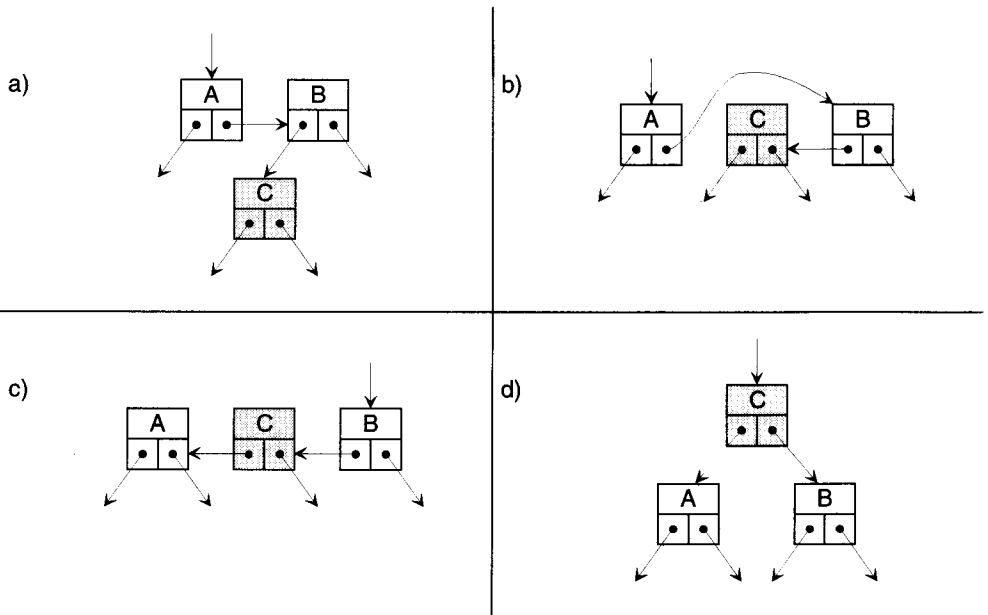


Figura 6.13 Inserción y rebalanceo RL en un árbol BBS.

El Programa 6.1 muestra el código en Modula2 de la operación de inserción en árboles BBS, con la siguiente declaración del nodo de un árbol BBS.

```

TYPE Ptr_Nodo=POINTER TO Nodo;
TYPE Nodo = RECORD
    llave: INTEGER;
    cont: CARDINAL;
    izq, dch: Ptr_Nodo;
    h_izq,h_dch: BOOLEAN
END;
```

Además se utiliza la variable entera *h* para indicar el crecimiento del árbol, análogamente a como se utilizó en los árboles AVL, con el siguiente significado:

1. $h = 0$: el subárbol p no requiere cambios en la estructura.
2. $h = 1$: el nodo p ha obtenido un hermano.
3. $h = 2$: el subárbol p ha aumentado de altura.

Como puede observarse los rebalanceos en los árboles BBS son muy similares a los desarrollados para árboles AVL. De hecho los AVL son un subconjunto de los BBS.

La desventaja de los árboles BBS frente a los AVL es que su longitud de trayectoria es mayor, pero son una alternativa a los AVL y suelen preferirse si las búsquedas son más frecuentes que las inserciones y las eliminaciones.

A continuación, se presentan algunos ejemplos que ilustran diversos crecimientos de los árboles BBS.

Programa 6.1 Inserción en árboles BBS.

```

PROCEDURE BBS(x: INTEGER; VAR p: Ptr_Nodo; VAR h: CARDINAL);
VAR p1, p2: Ptr_Nodo;
BEGIN
  IF p=NIL THEN      (* insertar *)
    ALLOCATE(p, SIZE(Nodo)); h:=2;
    WITH p^ DO
      llave:=x; cont:=1;
      izq:=NIL; dch:=NIL; h_izq:=FALSE; h_dch:=FALSE
    END
  ELSIF p^.llave>x THEN
    BBS(x, p^.izq, h);
    IF h>0 THEN      (* la rama izquierda ha crecido *)
      IF p^.h_izq THEN
        p1:=p^.izq; h:=2; p^.h_izq:=FALSE;
        IF p1^.h_izq THEN      (* LL *)
          p^.izq:=p1^.dch; p1^.dch:=p; p:=p1; p^.h_izq:=FALSE
        ELSIF p1^.h_dch THEN   (* LR *)
          p2:=p1^.dch; p1^.dch:=p2^.izq; p2^.izq:=p1;
          p^.izq:=p2^.dch; p2^.dch:=p; p:=p2;
          p1^.h_dch:=FALSE
        END
      ELSE h:=h-1;
      IF h>0 THEN p^.h_izq:=TRUE END
    END
  END
  ELSIF p^.llave<x THEN
    BBS(x, p^.dch, h);
    IF h>0 THEN      (* la rama derecha ha crecido *)
      IF p^.h_dch THEN
        p1:=p^.dch; h:=2; p^.h_dch:=FALSE;

```

```

IF p1^.h_dch THEN          (* RR *)
  p^.dch:=p1^.izq; p1^.izq:=p; p:=p1;p^.h_dch:=FALSE
ELSIF p1^.h_izq THEN      (* RL *)
  p2:=p1^.izq; p1^.izq:=p2^.dch; p2^.dch:=p1;
  p^.dch:=p2^.izq; p2^.izq:=p; p:=p2;
  p1^.h_izq:=FALSE
END
ELSE h:=h-1;
IF h>0 THEN p^.h_dch:=TRUE END
END
END
ELSE p^.cont:=p^.cont+1;  (*encontrado*)
END
END BBS;

```

6.4.1 EJEMPLOS DE CRECIMIENTO DE ÁRBOLES BBS

Supongamos que en un árbol BBS inicialmente vacío se insertan consecutivamente las llaves: 1, 2, 3, 4, 5, 6, 7.

El proceso se ilustra en la Figura 6.14. En primer lugar, se inserta el 1 como raíz. Seguidamente, el 2 queda como hermano derecho ya que al ser un árbol de búsqueda los mayores se sitúan a la derecha, y siendo hermano satisface la condición de que todos los nodos terminales están al mismo nivel. El 3 se inserta a la derecha del 1 y del 2, pero al colocarlo como hermano, para que todos los nodos terminales estén al mismo nivel, se dan dos apuntadores horizontales consecutivos en la misma trayectoria de búsqueda, con lo que es necesario subir el elemento central. Al insertar el 4 se coloca a la derecha del 2 y del 3, colocándose como hermano derecho de este último con el fin de que todos los nodos terminales estén al mismo nivel. Seguidamente el 5 se sitúa a la derecha de las llaves 2, 3 y 4, como hermano derecho del 4 para que todos los nodos terminales estén al mismo nivel. De nuevo, se dan dos apuntadores horizontales consecutivos en la misma dirección de búsqueda, entonces se aplica el mismo rebalanceo que antes pero ahora al subárbol implicado (de llaves 3, 4 y 5), subiendo el elemento central un nivel. Ahora el nodo de llave 4 es apuntado como hermano derecho del de llave 2. Análogamente, se inserta el 6 como hermano derecho del 5, y seguidamente el 7 como hermano derecho del 6 y de nuevo se produce la misma situación que requiere rebalanceo en el subárbol de llaves 5, 6 y 7. Al subir el elemento central el desbalanceo se propaga de manera que los nodos de llave 2, 4 y 6 tienen dos apuntadores consecutivos en la misma trayectoria de búsqueda. Aplicando el mismo rebalanceo, se sube el elemento central, que quedará como raíz. Obsérvese que en este ejemplo todos los rebalanceos son RR. En la Figura 6.14 se ilustran todos estos pasos y se indican mediante un recuadro a trazos las situaciones que requieren un rebalanceo. En esta figura, los nodos sombreados muestran el nodo que provoca el rebalanceo.

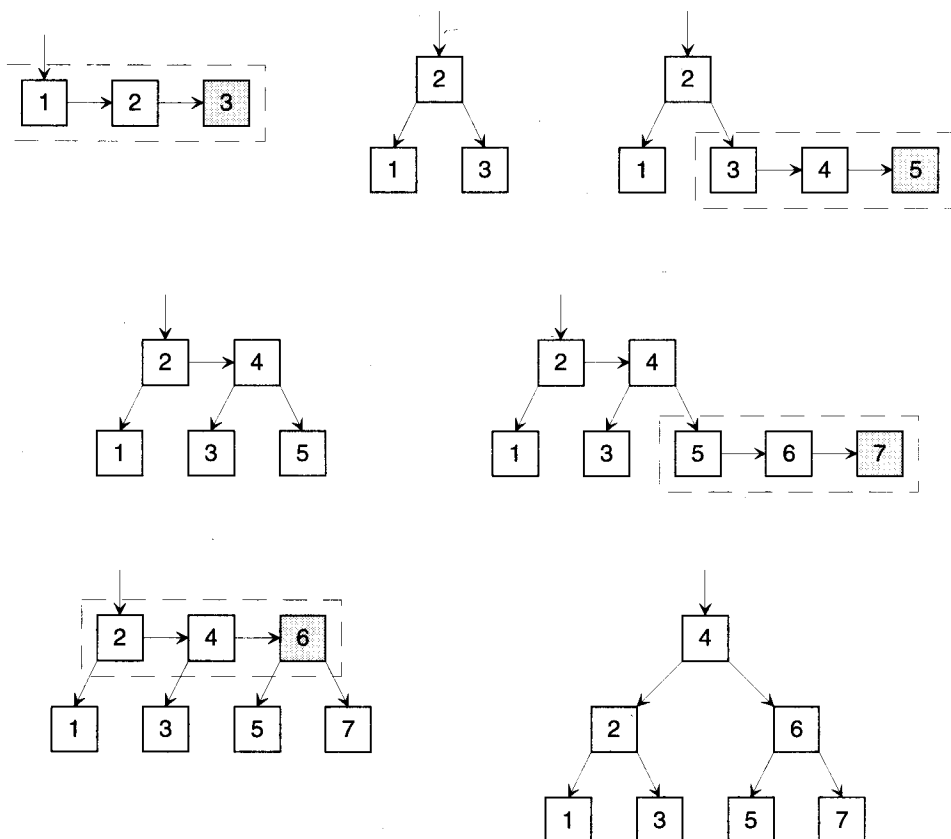


Figura 6.14 Inserción de la secuencia 1, 2, 3, 4, 5, 6, 7 en un árbol BBS. El nodo sombreado indica el nodo que provoca el rebalanceo.

Veamos ahora un ejemplo que ilustra los rebalanceos dobles: LR y RL.

Supongamos ahora que partiendo nuevamente de un árbol BBS inicialmente vacío se insertan consecutivamente las llaves: 7, 4, 9, 5, 8.

El proceso se ilustra en la Figura 6.15. En primer lugar, se inserta el elemento con llave 7 como nodo raíz. Seguidamente, el 4 es su hermano izquierdo (con lo que todos los nodos terminales están al mismo nivel), y análogamente al insertar el 9 se coloca como hermano derecho del 7. Obsérvese que *no* hay dos apuntadores horizontales consecutivos en la misma trayectoria de búsqueda, y por tanto, no es necesario rebalanceo. Seguidamente, entra el 5 a la izquierda del 7 y a la derecha del 4, con lo que el árbol deja de tener todas las hojas (nodos terminales) en el mismo nivel. En este caso hay que subir el 5 de nivel y luego subir el elemento central del subárbol desbalanceado (llaves 7, 4 y 5). Obsérvese que éste es un rebalanceo LR.

Finalmente, se inserta el 8 a la derecha del 5 y del 7 y a la izquierda de 9, creándose un nuevo nivel por lo que se hace necesario un rebalanceo. En este caso debe subirse el 8 un nivel y de nuevo subir el elemento central del subárbol implicado (llaves 7, 9 y 8). Obsérvese que éste es un rebalanceo RL.

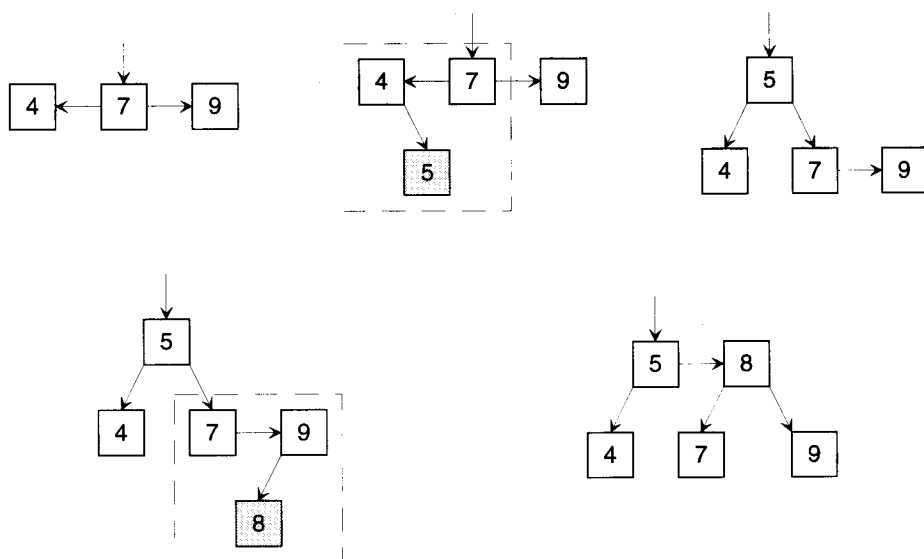


Figura 6.15 Inserción de la secuencia 7, 4, 9, 5, 8 en un árbol BBS. Como en el ejemplo anterior, el recuadro de trazos indica las situaciones que requieren rebalanceo y el nodo sombreado, es el que provoca el rebalanceo en cada caso.

6.5

CONCLUSIONES

En este capítulo se han presentado los TDA de árbol avanzados, que se caracterizan por ser sofisticaciones de las nociones fundamentales explicadas en los árboles binarios de búsqueda, AVL, etc. Como sucedía en el Capítulo 5 se pone de manifiesto la necesidad de establecer un criterio de equilibrio, y como puede observarse en todos ellos se precisan criterios de crecimiento que garanticen un comportamiento adecuado.

En primer lugar, se han presentado los árboles B multicamino, que combinan mediante el concepto de página una estructura no lineal, entre las páginas, con una lineal entre las llaves de la misma página. Con esta idea y mediante la definición de Bayer-McCreight se consigue que un árbol B con N elementos requiera en el peor caso $\log_n N$ accesos de página, y además el factor de utilización de memoria es siempre superior al 50 por ciento. Este tipo de árboles tienen especial aplicación en

memoria secundaria, siendo competidores de las tablas hash, y encontrando aplicación inmediata en sistemas de bases de datos [Silberschatz, 1998]. Obsérvese por otro lado que mientras en las funciones hash el coste es independiente de N , en los árboles es del orden $\log_n N$ en el peor caso.

Seguidamente, se explora la posibilidad de aplicar estas ideas a TDA para la organización en memoria primaria. Por ello se introducen en primer lugar los árboles B binarios. Una vez analizado su comportamiento, se observa que presentan una asimetría en su crecimiento, por lo que se definen los árboles B Binarios Simétricos, que evitan dicha asimetría, siendo de hecho una generalización de los árboles AVL.

CAPÍTULO 7

Programación orientada a objetos

7.1 INTRODUCCIÓN

A lo largo de este texto se han presentado diferentes Tipos de Datos Abstractos y los algoritmos básicos para el tratamiento y manipulación de los mismos. Recuérdese que la idea fundamental subyacente a todos ellos, y que motiva su elaboración, consistía en *abstraer* las estructuras de datos necesarias para almacenar los datos con el fin de mejorar la reutilización del software.

Esta misma idea es la que fundamentaba la programación estructurada, en la que las tareas repetidas se agrupaban en procedimientos, entendiendo por tareas un algoritmo o una secuencia de acciones que a partir de unos datos de entrada ofrece una solución o datos de salida. De este modo, una vez desarrollado un procedimiento, éste será utilizado en el mismo programa tantas veces como sea requerido atendiendo únicamente a sus entradas y a sus salidas, y además, podrá ser reutilizado en cualquier otro desarrollo que lo necesite.

En el contexto de la abstracción de datos la motivación es exactamente la misma. Así por ejemplo, definido el TDA y sus operaciones básicas asociadas, y analizadas detenidamente sus características y propiedades, se dispone de una herramienta de utilización inmediata cuando sea requerida en un problema dado, y podrá ser reutilizada tantas veces como sea necesario en cualquier aplicación que lo requiera.

La programación orientada a objetos tiene el mismo objetivo fundamental: la mejor reutilización del software. Sin embargo, no sólo se limita a la reutilización de procedimientos, como hace la programación estructurada, también denominada actualmente programación orientada a procedimientos, sino que extiende la idea a todo el software. Con ello permite que los sistemas de software se extiendan fácilmente para mejorar su funcionalidad y se puedan reutilizar en otros sistemas. Como puede observarse, los TDA presentados en este texto han sido desarrollados con este mismo objetivo. En ellos, y en la programación orientada a objetos, idealmente la extensión de una componente del software no requiere conocimientos específicos de sus detalles de implementación, gracias a lo cual los componentes de software se pueden empaquetar para ser incorporados fácilmente en el sistema, simplificando el desarrollo y mantenimiento del software.

Las ideas presentadas a lo largo del texto relacionadas con el concepto de tipo de datos abstracto, y los TDA presentados en los capítulos anteriores son conceptos clásicos y anteriores a la programación orientada a objetos. De hecho fueron desarrollados paralelamente al establecimiento de la programación estructurada, y se refieren especialmente al problema de clasificación. La programación orientada a objetos extiende la abstracción y el encapsulamiento a todo el desarrollo del software.

En este capítulo se presentan muy brevemente los conceptos básicos de la programación orientada a objetos (POO). No es objetivo del tema proporcionar una sólida formación en dicha programación, sino que el propósito es presentar los conceptos básicos y fundamentales de la misma, los cuales surgen de forma natural a partir de las ideas tratadas para los TDA.

7.2

CLASES Y OBJETOS

El concepto de *clase* es una de las claves en las que se fundamenta la programación orientada a objetos. Es un medio para *empaquetar* los Tipos de Datos Abstractos, y permite que *tanto los elementos de datos como las rutinas de acceso de un Tipo de Datos Abstracto se encapsulen juntos en una misma entidad*.

La clase controla el acceso a los elementos de datos. Un programa de aplicación *sólo* puede acceder a los elementos de datos mediante la interfaz determinada por las rutinas de acceso. Por tanto, es evidente la separación entre el nivel del programa de aplicación y el nivel de implementación de un tipo de datos abstracto dado.

Una clase contiene toda la información para construir *realizaciones separadas de ella misma*. A cada una de estas realizaciones separadas se le denomina **OBJETO**, es decir, un objeto es una materialización o realización de una clase. Cuando estos objetos representan objetos del mundo real se obtiene una aplicación natural del dominio del problema al dominio de su representación software, con lo que la elaboración y comprensión del código se facilita extraordinariamente.

Con el fin de aclarar ideas es importante insistir en la diferencia entre *clase* y *objeto*. La *clase* es la especificación para crear *objetos* y a partir de una única *clase* se pueden generar múltiples *objetos*. Estos últimos son las entidades que realmente se manipulan en el programa. La diferencia es análoga a la existente entre tipo de datos y su realización en variables, en el contexto de la programación orientada a procedimientos.

Cada objeto contiene su propio conjunto de elementos de datos denominados *variables miembro o datos miembro*. Así por ejemplo el objeto Alumno, de una clase denominada Alumnado que se define para una institución dada, podría tener como variables miembro Nombre, Apellido, Dirección, DNI, Nota, etc.

Una clase puede almacenar *información compartida* por todas las realizaciones de la clase en *variables de clase*. Por ejemplo, la clase Alumnado podría tener como variables de clase el *nombre de la institución, su dirección, su teléfono*, etc.

Tanto los miembros como las variables de clase se empaquetan tal que sólo pueden ser accedidos por las rutinas determinadas por la clase. Las rutinas de una clase son comunes a todos los objetos de la clase, y se denominan *funciones miembro*.

Como puede observarse, la definición de clase surge de forma inmediata y natural a partir del concepto de Tipo de Datos Abstracto, detenidamente expuesto a lo largo del texto. Así, definir las clases Pila, Cola, Arboles_AVL, etc., es sencillo y evidente atendiendo a las definiciones de los TDA y no olvidando sus operaciones básicas asociadas.

7.3

HERENCIA Y PASO DE MENSAJES

La herencia y el paso de mensajes constituyen las dos herramientas fundamentales de la programación orientada a objetos. La primera es la base para la definición adecuada de las clases, y la segunda es el método para la comunicación entre objetos.

La *herencia* permite crear una nueva clase, denominada *clase derivada*, como una extensión o particularización de una ya existente denominada *clase base*.

Mediante la *herencia*, los objetos de una clase pueden acceder a variables y funciones miembro sin necesidad de restablecer sus definiciones. Así, la clase derivada puede heredar tanto variables y funciones miembro como variables de clase. Un ejemplo típico aclaratorio del concepto de clase es el de los elementos de dibujo (figura 7.1). En general, se dibujan formas, que tienen un conjunto de características comunes, como por ejemplo el ser una sucesión de puntos. Partiendo de las formas, pueden establecerse por ejemplo, clases derivadas para aquellas que se realizan mediante líneas, con lo que se define la clase Línea, y para las que se caracterizan por responder a formas cuadráticas, definidas en la clase Elipse. A su vez, considerando la clase Línea como clase base se puede establecer la clase Polígono, y a partir

de esta última, las clases Rectángulo y Triángulo, y para definir los cuadrados basta con heredar de la clase Rectángulo definiendo la clase derivada Cuadrado.

En cada proceso de establecimiento de una clase derivada no es necesario modificar el código desarrollado para la clase base con el fin de obtener un TDA, que sería separado e independiente. Por ejemplo, cuando Cuadrado hereda de Rectángulo no es necesario modificar el código desarrollado para Rectángulo. Con ello las clases derivadas mantienen relación con la clase base que en parte las define y también con las clases que a su vez se derivan de ellas, con lo que la organización de la información en una clase es tal y como se ilustra en la figura 7.2.

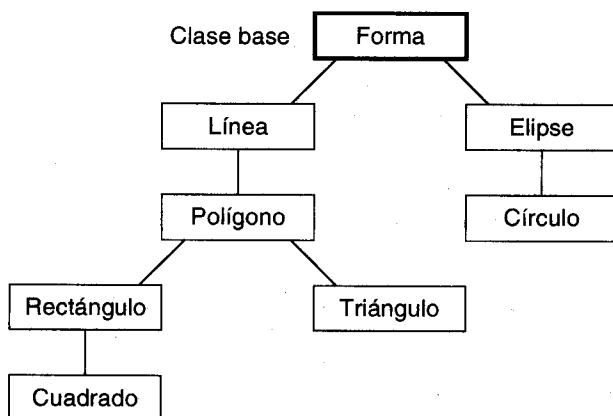


Figura 7.1 Ejemplo típico del concepto de herencia.

Los lenguajes de programación orientada a objetos deben soportar *herencia múltiple*, que consiste en que una clase derivada puede heredar de varias clases base distintas.

Por otro lado, una vez realizados los objetos es obvia la necesidad de establecer algún medio que permita la interacción entre ellos. El paso de mensajes es una forma de entender la comunicación entre objetos. Con esta técnica los objetos manipulan otros objetos enviando o recibiendo mensajes para realizar una determinada acción.

Los mensajes llaman a las funciones miembro en las clases de los objetos pudiendo provocar cambios en el estado de los propios objetos. Si una función miembro llamada *no se encuentra* en la clase del objeto entonces se *busca en la clase base* inmediata, y así sucesivamente por toda la jerarquía de clases *hasta la raíz*. Sólo si tampoco se encuentra en la raíz se produce un error, ya que se estaría actuando sobre una función miembro no definida para esa clase.

Es importante advertir la diferencia entre paso de mensaje y procedimiento. Un procedimiento responde a *cómo* realizar la acción mientras que un mensaje es una solicitud de un objeto para realizar una acción. La forma, (el cómo, el procedi-

miento), en que responde el objeto a un mensaje, *depende de* las funciones miembro de que dispone. Obsérvese la *abstracción* realizada con esta técnica *para la acción a realizar*. El *qué* hay que hacer se establece mediante el paso de mensaje, mientras que el *cómo* (detalle de implementación) es asunto de un procedimiento (función miembro) dado. Por ejemplo, el mensaje imprimir será realizado adecuadamente tanto para una clase Pila, como para una clase Cola, definidas adecuadamente. Es obvio que objetos de *diferentes clases* responden adecuadamente al *mismo mensaje*.

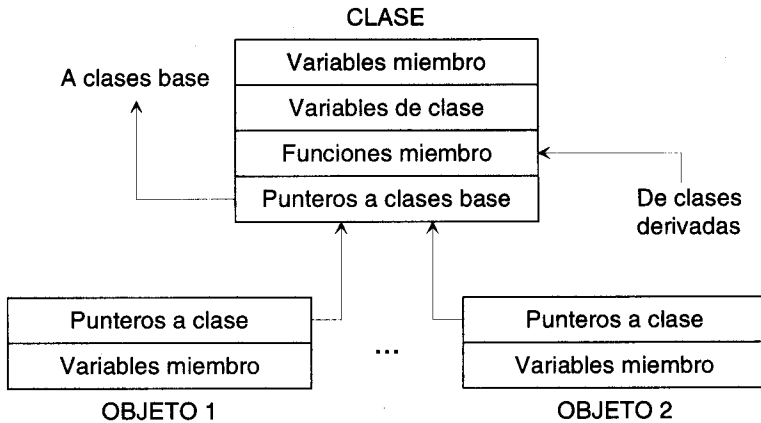


Figura 7.2 Organización de la información dentro de una estructura de clases.

7.4 ENLACE DINÁMICO Y POLIMORFISMO

Cuando un sistema de software determina cómo debe realizarse una operación sobre un objeto, se dice que *enlaza una operación específica de la operación*.

Si en tiempo de compilación se decide qué implementación se va a usar, entonces se dice que se realiza un *enlace estático*. La implementación de la operación es determinada por *la forma estática del objeto*.

El *enlace dinámico* permite decidir en tiempo de ejecución cómo se va a realizar la acción. En este caso, la implementación de la operación va a venir determinada por *la forma dinámica del objeto*, es decir, por la forma del objeto en el mismo instante de ejecución en que es requerida la operación a realizar, adaptándose automáticamente a los objetos sobre los que se aplica.

Como puede observarse el enlace dinámico es similar a la asignación dinámica de memoria, pero referida a las implementaciones de los procedimientos en lugar de a las de las estructuras de datos.

Como consecuencia directa del enlace dinámico aparece el concepto de *polimorfismo*. Si el término polimorfismo significa etimológicamente 'capacidad de

asumir varias formas', en la programación orientada a objetos indica la capacidad de *un mensaje* para responder en tiempo de ejecución a objetos de *diferentes clases*.

Si la función en la clase base *no se declara polimórfica*, entonces se realiza el *enlace estático*, y en tiempo de compilación se determina la función a utilizar. Entonces al asignar objetos de diferentes clases a este objeto de la clase base, cuando se requiera la función sobre el objeto de la clase base, siempre se realizará la función definida para la clase base, independientemente de su forma. Sin embargo, si se declara polimórfica el *enlace es dinámico* y entonces se chequea, en tiempo de ejecución, la forma del objeto antes de llamar a la función. Esto es debido a que al realizar la asignación *ha tomado la forma del objeto asignado*.

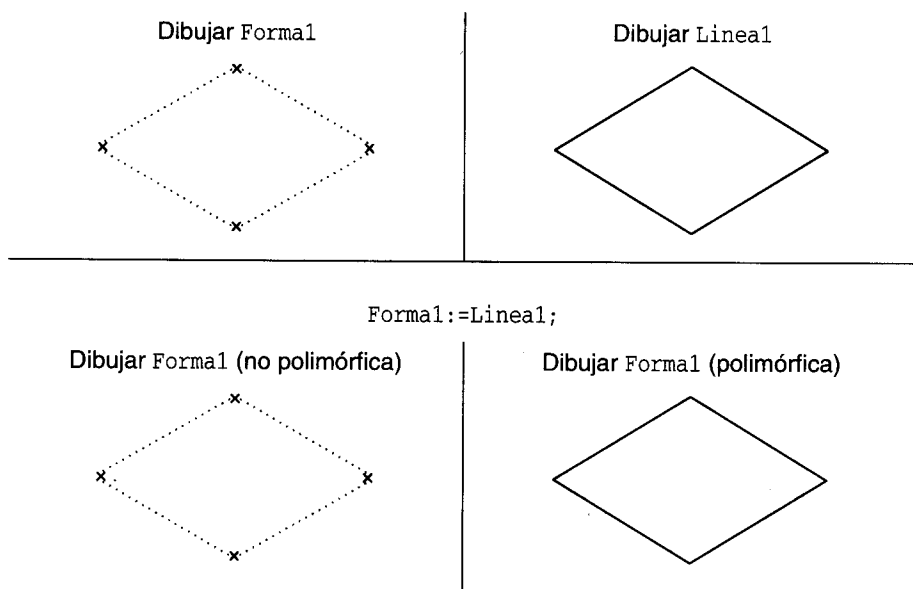


Figura 7.3 Polimorfismo.

Normalmente, una función de polimorfismo se declara en una clase base, y se sobrenombra en las clases derivadas. Entonces, si un objeto de la clase base se declara en un programa, al llamar a la función se realizará la función dada en la clase base, y si un *objeto de una clase derivada es asignado al objeto de la clase base*, al llamar a la función se realizará la función para la clase derivada (se analiza la forma del objeto, se determina que es de la clase derivada y entonces se utiliza su función). Obsérvese que el concepto de *función polimórfica es distinto a función sobrenombada*, y mediante él se consigue una mejor utilización de los recursos de software.

Por ejemplo, supóngase que con las clases mostradas en la Figura 7.1, para la clase Forma se define la función dibujar, que realiza la impresión en pantalla punto a punto. Y supóngase que para la clase derivada Linea se define la función dibujar y

se implementa uniendo los vértices de un segmento dado mediante un conjunto de sólo veinte puntos, y marcando los vértices con una 'x'.

Supongamos que a lo largo de un programa se encuentran definidos los objetos `Formal` de la clase `Forma` y `Lineal` de la clase `Linea`. En un momento dado se asignan los objetos (`Formal:=Lineal`) y seguidamente se decide dibujar `Formal`. Si la función `dibujar` no se declaró polimórfica en la clase `Forma`, que es la clase base de la clase `Linea`, el enlace es estático y se realiza la implementación punteada. Pero si ha sido declarada polimórfica entonces el enlace es dinámico, el objeto `Formal` *toma la forma* de `Lineal` al *realizar la asignación*, y cuando se requiere dibujar se chequea la forma y se realiza la definida para los objetos de la clase `Linea`. La figura 7.3 ilustra estas consideraciones.

7.5

CONCLUSIONES

En este capítulo se han presentado brevemente las ideas fundamentales en las que se basa la programación orientada a objetos. En primer lugar, se han mostrado los conceptos de clase y objeto. Es importante destacar que la implementación de la clase está oculta al resto del sistema, ya que los datos en un objeto sólo son accesibles a través de una interfaz externa bien definida y estas rutinas de interfaz se empaquetan con los datos. Por tanto, *el concepto de clase incorpora el encapsulado*, tanto de las estructuras de datos como de los algoritmos de manipulación de los mismos. Una clase *incorpora* directamente la implementación de un *Tipo de Datos Abstracto*. El encapsulado y el proceso de abstracción realizados en la programación orientada a objetos posibilita la implementación de Tipos de Datos Abstractos que realizan conjuntos de operaciones bien definidas *independientemente de la aplicación particular*.

Por otro lado, el concepto de herencia simplifica el diseño y reduce el tiempo de desarrollo al permitir añadir nuevas clases a otras ya desarrolladas sin modificarlas. Así, el programador que hace uso de las clases no necesita conocer sus detalles de implementación, sólo su *funcionalidad*.

Con todo ello la programación orientada a objetos requiere en primer lugar analizar las clases de objetos del sistema real y seguidamente se especifican las descripciones e interacciones entre ellos. Entonces se modelan los objetos del sistema físico mediante objetos del sistema software, tarea que es facilitada por las características de la programación orientada a objetos, para modelar sistemas más complejos, nuevos objetos, etc., de forma constructiva.

Como puede observarse tal y como se indicó en el Capítulo 1 la programación orientada a objetos generaliza la programación orientada a procedimientos de manera que el proceso de abstracción no sólo afecta a los algoritmos y a los datos por separado, sino que los contempla de forma conjunta.

Bibliografía

- [Aho, 1988] A. V. Aho, J. E. Hopcroft y J. D. Ullman. "Estructuras de Datos y Algoritmos. Ed. Addison-Wesley, 1988.
- [Dale, 1989] Neil Dale y Chip Weems. "Pascal". Ed. Mc Graw-Hill, 1989.
- [Dormido, 2000] S. Dormido, M.A. Canto, J. Mira y A. E. Delgado, "Estructura y tecnología de computadores". Ed. Sanz y Torres, 2000.
- [Enbody, 1988] R. J. Enbody y H. C. Du, "Dynamic Hashing Schemes", *Computing and Surveys*, Vol. 20, pág. 85-113 (1988).
- [Gilstad, 1960] R. L. Gilstad. "Polyphase Merge Sorting. An Advanced Technique". Proc. AFIPS Eastern Jt. Comp. Conf., Vol. 18, pág. 143-148 (1960).
- [Heileman, 1998] Gregory L. Heileman. "Estructuras de Datos, Algoritmos y Programación Orientada a Objetos". Ed. McGraw-Hill, 1997.
- [Knuth, 1998] Donald E. Knuth, "The art of computer programing". Ed. Addison-Wesley, 1998.

- [Lum, 1971] Lum, Yuen, y Dodd, "Key to Address Transform Techniques: a Fundamental Performance Study on Large Existing Formatted Files", *CACM*, Vol. 14, N° 4, Abril (1971).
- [Papernov, 1965] A. A. Papernov y G. V. Stasevich, "A Method of Information Sorting in Computer Memories" *Problems of Information Transmission*, Vol. 1, pág. 63-75, (1988).
- [Peña, 1998] R. Peña Marí. "Diseño de programas. Formalismo y abstracción". Ed. Prentice Hall, 1998.
- [Radke, 1970] C. E. Radke, "The Use of Quadratic Residue Research", *CACM*, Vol. 13, N° 2, Febrero (1970).
- [Sedgewick, 1986] R. Sedgewick, "A New Upper Bound for Shellsort", *Journal of Algorithms*, 7, pág. 159-173, (1986).
- [Shaffer, 1997] Clifford A. Shaffer. "Data Structures and Algorithm Analysis". Ed. Prentice Hall, 1997.
- [Weiss, 1997] Mark A. Weiss, "Data structures and algorithm analysis in C". Ed. Addison-Wesley, 1997.
- [Wirth, 1987] Niklaus Wirth, "Algoritmos y estructuras de datos". Ed. Prentice-Hall Iberoamericana, (1987)

Índice analítico

A

abstracción, 271

abstraer, 2, 267

acceso

 directo, 94, 129

 secuencial, 94

Adelson-Velskii, 219, 239

alfabeto, 8

algoritmo, 40, 227, 267

almacenamiento, 6, 41, 193

 ahorro, 17

 economía de, 42, 78, 191

altura, 196, 198

análisis, 40

 de árboles AVL, 239

 de árboles binarios, 214

 de las tablas hash, 139

análisis asintótico, 50, 56, 86

análisis de la clasificación

 por inserción binaria, 55

 por inserción directa, 50

 por intercambio directo, 64

 por montón, 81

 por partición, 86

 por selección directa, 59

 por vibración, 68

 Shell, 72

análisis de la mezcla

 balanceada múltiple, 116

 directa, 101

 natural, 109

anidamiento, 10

aplicación suprayectiva, 132

apuntador, 194

 horizontal, 256

 vertical, 256

árbol, 193

 2-3, 256

 AVL, 6, 219, 231

 B, 246

 B binario, 256, 266

 B binario simétrico, 259, 266

 balanceado, 219

 binario, 75, 195

 binario de búsqueda, 208

 de búsqueda, 6, 207, 208, 245

 de búsqueda balanceado, 218

 de expresión, 205

 extendido, 197

 multicamino, 245

 ordenado según el recorrido, 204

 perfectamente balanceado, 200, 215, 218

 ternario, 195

archivo 93

 de índices, 129

archivos indexados, 129, 141

arista, 195

ascendiente, 195
ascendiente directo, 195
ASCII, 8
asignación, 15
asignación de memoria
 dinámica, 12, 143, 144, 271
 estática, 143

B

balance, 220
bases de datos, 131, 142, 193, 246
Bayer, 256
Bayer-McCreight, 246, 265
bloque, 130, 132, 139
burbuja, 64
búsqueda, 45, 130, 193
 binaria, 53, 55
 dicotómica, 53, 130
 dirigida, 208
 en árboles, B 247
 en un árbol, 214

C

cabeza, 155
caso
 mejor, 42
 peor, 42
 promedio, 42
CDC-Científico, 8
celda, 132
cima, 157
cinta
 auxiliar, 103
 destino, 95
 fuente, 95
 magnética, 93
cintas
 auxiliares, 95
 temporales, 95
clase, 268
clase base, 269, 270
clase derivada, 269

clasificación, 41, 45
 basada en mezcla, 95
 directa, 47
 perfecta, 121
 polifásica, 116
 por inserción binaria, 53
 por inserción directa, 48
 por intercambio directo, 62
 por montón, 75
 por partición, 82
 por sacudida o vibración, 66
 por selección directa, 57
 rápida, 82
 satisfactoria, 118, 121
cola, 162, 163, 187
 circular, 166
 con vaciado, 191
 de impresión, 162
 llena, 167
 vacía, 167
colección, 143
colisión, 132, 133, 138
coma flotante, 7
comparación, 15
 de punteros, 16
comparaciones, 41
compilador, 46
comportamiento natural, 47
compromiso, 218, 241, 243
computador, 1
condiciones mutuamente excluyentes, 208,
 209, 215, 219
consistencia, 152
copiado de cabos, 96
coste, 6, 41
coste computacional, 40, 191
cota
 inferior, 42
 superior, 42
criterio
 de búsqueda, 219
 de equilibrio, 219, 243
 de organización, 207, 243
cuádruplos ordenados, 95
cualitativamente, 118

D

densidad de llaves, 133
 desbordamiento, 132, 133, 136
 descendiente directo, 195
 destino, 116
 DFT, 41
 dinámica, 181
 dirección física, 129, 130
 direcciones de memoria, 131
 disco duro, 93
 diseño, 40
 distribución, 105, 116
 distribución inicial, 122
 dividir, 95
 división
 de páginas, 249
 entera, 134

E

EBCDIC, 8
 eliminación, 210
 en árboles AVL, 234
 en árboles B, 252
 física, 130
 empaquetar, 268
 Enbody, 141
 encadenamiento, 138, 140
 encapsulado, 4, 273
 enlace, 144, 194
 dinámico, 271, 272
 doble, 150
 estático, 271
 equilibrado, 200
 equiprobable, 134
 estabilidad, 46
 estática, 180
 estructura
 dinámica, 157
 estática, 157
 estructura de datos, 3
 estructura iterativa, 27
 etapa, 94
 Euclides, 40, 239

Euler

 constante de, 61, 88, 218
 fórmula de, 218
 evaluación de expresiones, 179
 exploración
 cuadrática, 137, 140
 lineal, 136
 expresión, 205

F

factor de carga, 133
 fase, 94, 102
 de distribución, 109
 FFT, 41
 Fibonacci
 números de, 118, 119, 121, 240
 fichero, 93, 106, 193
 aleatorio, 129
 FIFO, 162
 Floyd, 75
 folding, 135
 fuente, 116
 fuentes disponibles, 115
 función
 de acceso, 143
 de dispersión, 136, 138
 de transformación, 132
 hash, 134, 136, 139
 miembro, 269
 polimórfica, 272
 sobrenombrada, 272

G

Gilstad, 116
 grado, 195, 198

H

hashing, 134
 herencia, 269
 herencia múltiple, 270
 heterogéneo, 10

Hibbard
 fórmula de, 216
 incrementos de, 74
 secuencias de, 75
 hijo, 195
 Hoare, 82
 hoja, 195, 246
 homogéneo, 10, 156, 163

I

implementación, 114, 122, 186, 268
 de colas, 164, 174
 de pilas, 157, 160
 dinámica, 190
 estática, 190
 implementación recursiva, 34
 imprimir una lista enlazada, 34
 incrementos decrecientes, 70
 indexado de cintas, 124
 índice
 denso, 129
 disperso, 130
 mapa de, 124
 infinitivo, 187
 inserción, 210
 en árboles, B 248
 inserción en una lista enlazada
 por el final, 23
 por la cabeza, 31
 según un criterio de orden, 28
 interfaz, 268
 invariantes, 40
 parámetros, 41

K

Knuth, 73, 75, 135

L

Lamé, 240
 Landis, 219, 239
 lenguaje

Algol, 20
 C, 4
 FORTRAN, 20
 Modula2, 143
 Pascal, 4
 SQL, 142
 librería, 46
 LIFO, 155
 lista enlazada, 138, 174, 189
 circular, 151
 con memoria, 150
 con puntero al final de la lista, 149
 doblemente, 144, 179
 multireferenciada, 151
 lista vacía, 30
 llave, 209
 longitud de trayectoria, 196, 197, 199, 215
 Lum, 139

M

mantenimiento, 268
 mapa de índice, 124
 máquina de estados, 187
 Matlab, 46
 media, 86
 mediana, 85
 memoria
 asignación dinámica, 18
 libre, 161
 primaria, 46, 130, 131, 141
 principal, 93
 secundaria, 93, 129, 131, 245
 mensaje, 272
 meter, 156, 158, 160, 175
 mezcla, 94, 97, 109, 116, 124, 141
 balanceada, 116
 balanceada múltiple, 109
 de dos fases, 97
 de tres cintas, 97
 de una fase, 100
 directa, 95, 100, 109
 directa balanceada, 100
 natural, 101
 movimientos, 41

N

natural, 47
 NIL, 16
 nivel, 196
 nodo, 18, 144, 194
 especial, 197
 interior, 195
 terminal, 195
 notación, 205
 infija, 180
 postfija, 182
 prefija, 182
 número
 armónico, 61, 140
 primo, 135, 137
 número promedio de exploraciones, 140

O

o exclusiva, 136
 objeto, 268
 offset, 129
 operaciones auxiliares, 50
 operador
 apuntador, 13
 operadores auxiliares, 156, 163
 operadores básicos, 156, 163
 ordenable, 207
 ordenación, 45
 externa, 46
 interna, 46
 ordenador, 1
 organización de datos, 191

P

página, 246
 palabra, 5
 Papernov, 74
 paréntesis, 205
 parentización, 179
 pares ordenados, 95
 pase, 94, 102
 picadillo, 134

pila, 155, 180
 con vaciado, 191
 pivote, 82
 plegado, 135
 por desplazamiento, 135
 por las fronteras, 136
 polimorfismo, 271
 postorden, 204
 precedencia, 180
 predecesor, 9
 predefinido, 10
 preorden, 204
 probabilidad, 52, 87, 101, 134, 139
 procedimientos recursivos, 203
 profundidad, 196
 programa, 40
 programación
 estructurada, 2, 267
 orientada a objetos, 268
 orientada a procedimientos, 2
 promedio, 141
 puntero, 12
 auxiliar, 27
 de escritura, 93
 de lectura, 93
 externo, 21, 149

Q

Quicksort, 82

R

Radke, 137
 raíz, 195, 246, 270
 rama, 195
 rebalanceo, 220
 doble, 264
 LL, 222, 229, 259
 LR, 225, 230, 260
 RL, 227, 233, 261
 RR, 227, 232, 260
 recuperación, 45
 recurrencia, 217

- recursión, 85, 194, 203
- recursiva, 209
- reglas de precedencia, 180, 205
- rehashing, 137, 140
- relación de orden, 46
- rendimiento, 101
- representación, 7
- resultados experimentales, 139
- reutilización, 267
- rotación, 257
 - doble, 225
 - simple, 222

S

- sacar, 156, 159, 160, 175
- sector, 129
- secuencia, 93
 - de cotejo, 8
 - de entrada, 106
- Sedgewick, 74
 - secuencias de, 75
- Shell, 70
- Silberschatz, 246, 266
- sincronizar, 162
- sinónimo, 132
- sistema de archivos, 130
- sistema operativo, 162, 193
- sobrecarga, 130, 136, 139
- subárbol
 - derecho, 195
 - descendiente, 210
 - izquierdo, 195
- subsecuencia
 - ficticia, 121, 123
 - máxima ordenada, 105
 - ordenada, 101, 102, 103, 109, 141
- sucesor, 9
- suma aritmética, 51, 59, 65, 90
- suma geométrica, 73
- suprimir en una lista enlazada
 - por el final, 27
 - por la cabeza, 25

T

- tabla circular, 136
- tabla de dispersión, 132
- tabla hash, 138, 141
- tabla hash dinámica, 142
- TDA, 3
 - árbol, 194
 - arreglo, 10
 - booleano, 7
 - carácter, 7
 - conjunto, 10
 - dinámicos lineales, 144, 191
 - entero, 7
 - lista, 11, 35
 - lista doblemente enlazada, 144
 - lista enlazada, 18
 - matriz, 11
 - puntero, 17
 - real, 7
 - registro, 10
 - secuencia, 11
- tiempo
 - de compilación, 143
 - de ejecución, 6, 13, 41, 189
- tipo base, 10
- tipo de datos, 3, 6
 - abstracto, 3, 7
 - compuesto, 10
 - definidos por el usuario, 10
 - escalar, 9
 - estructurado, 10
 - primitivo, 10
 - simple, 9
- tope, 155
- Transformada Discreta de Fourier, 41
- Transformada Rápida de Fourier, 41
- transformar llaves, 131
- truncamiento, 57

U

- uniforme, 134, 139
- uniformidad, 122

V

vaciar, 190

variable

 dinámica, 13

 estática, 13

variables de clase, 269

variables miembro, 269

verificación, 40

W

Weiss, 142

Williams, 75

Wirth, 141, 201, 202, 229

dientes minúsculas. En esta figura, las ramas o líneas de transición que no tienen etiqueta representan al resto de posibles caracteres no contemplados en el resto de ramas que parten del mismo nodo.

El Programa 4.18 realiza esta búsqueda sobre el texto de un fichero.

Programa 4.18 Programa para localizar e imprimir infinitivos en un fichero de texto.

```

MODULE infinitivo;
FROM InOut IMPORT Write,WriteString,WriteLn,WriteInt;
FROM FileSystem IMPORT File,Close,Reset,Lookup,ReadChar;
FROM colarray IMPORT TipoCola,IniciaCola,MeterCola,SacarCola,
ColaVacia,ColaLlena;
IMPORT ASCII;
TYPE
    TipoDatos = CHAR;
    TipoConj = SET OF TipoDatos;

VAR
    f                : File;
    ch,dato           : TipoDatos;
    cola              : TipoCola;
    estado,contador   : INTEGER;
    fin,penul,ulti    : TipoConj;
BEGIN
    fin := TipoConj{'.',',',' ',';',':',ASCII.EOL,ASCII.EOF};
    penul := TipoConj{'A','E','I','a','e','i'};
    ulti := TipoConj{'r','R'};
    Lookup(f,'c:\trabajo\infini.txt',FALSE); Reset(f);
    IniciaCola(col);
    contador := 0;
    WHILE ~f.eof DO
        ReadChar(f,ch);
        estado := 0;
        WHILE NOT (ch IN fin) DO
            IF NOT (ch IN fin) THEN MeterCola(col,ch) END;
            IF ch IN penul THEN estado:=1
            ELSIF (estado=1) & (ch IN ulti) THEN estado:=2;
            ELSIF NOT (ch IN fin) THEN estado:=0;
            END;
            ReadChar(f,ch);
        END;
        IF estado = 2 THEN (* Se ha encontrado un infinitivo *)
            WHILE NOT ColaVacia(col) DO
                (* Se imprime el infinitivo *)
            
```

Estructuras de Datos y Algoritmos

Hernández • Lázaro • Dormido • Ros

Este texto presenta los contenidos básicos de la materia denominada Estructuras de Datos y Algoritmos en el ámbito de los estudios de Informática, cubriendo los temas característicos de la misma: tipos de datos abstractos, estructuras de datos y algoritmos de manipulación, estructuras de información y ficheros.

Se introducen los Tipos de Datos Abstractos (TDA) fundamentales atendiendo al concepto de abstracción y encapsulamiento de datos, mostrando claramente la diferencia entre TDA e implementación del mismo. Esta metodología permite un desarrollo y mantenimiento del software sistemático y sencillo, facilitando su reutilización. Por otro lado, cada TDA es analizado, tanto desde el punto de vista de la economía de almacenamiento, como del costo computacional de los algoritmos de manipulación, permitiendo con ello establecer criterios que permiten compararlos y decidir cuál es el más adecuado según las características de una aplicación dada.

Tanto los contenidos como la estructura del texto se han desarrollado con el objetivo de favorecer el autoaprendizaje, presentando numerosos ejemplos ilustrativos de los conceptos introducidos, así como de posibles aplicaciones de los mismos.



El libro incorpora un CD-ROM en el que se encuentran todos los algoritmos del libro implementados tanto para Modula 2 como para Pascal estándar. También incluye algunos archivos ejecutables que muestran paso a paso de forma gráfica la creación y manipulación de las estructuras de árbol más complicadas de entender sobre el papel.

Sobre los autores

Los autores son profesores titulares del Departamento de Informática y Automática de la Universidad Nacional de Educación a Distancia, e imparten docencia desde hace varios años en las titulaciones en Ingeniería Técnica en Informática de Sistemas y en Ingeniería Técnica en Informática de Gestión de la Escuela Universitaria de Informática de la Universidad Nacional de Educación a Distancia.

Pearson
Educación

www.pearsoneducacion.com

ISBN 84-205-2980-X



9 788420 529806